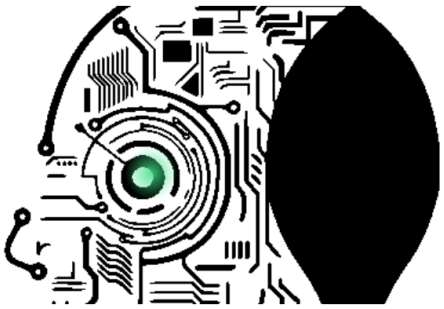


Modelos de IA

David Martínez Peña · CC BY-NC-SA 4.0

Modelos de IA (CE Inteligencia Artificial y Big Data)

IES Eduardo Primo Marques (Carlet)

David Martinez Peña

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Indice de contenidos

1. 🚀 Información importante 7
   1.1 🪧 Denominación del curso 7
   1.2 📋 Contenidos y temporalización 7
   1.3 🎯 Resultados de aprendizaje, pesos y calificación 8
   1.4 📝 Evaluación 9
   1.5 📜 Legislación vigente 9
2. UD00 10
   2.1 UD00 — Presentación del módulo y curso rápido de Docker 10
      1. Introducción 10
      2. Resultados de aprendizaje y objetivos 10
      3. Presentación del curso y del módulo (RA7-i) 11
      4. Cómo se evalúa el módulo 12
      5. Entorno de trabajo del curso 12
      6. Curso rápido de Docker (RA7-i) 16
      7. El Dockerfile: construir imágenes reproducibles (RA7-i) 21
      8. Docker Compose: varios servicios a la vez (RA7-i) 23
      9. El contenedor de prácticas de IA (RA7-i) 24
      10. Copias de seguridad de imágenes y contenedores (RA7-i) 27
      11. Puntos clave de la unidad 27
      12. Glosario 28
      13. FAQ 28
      14. Sesiones 29
      A. Anexo · chuleta de comandos 29
      B. Anexo · catálogo de contenedores para experimentar 30
      15. Recursos 31
      16. Evaluación 32
      17. Recuperación 32
   2.2 Diapositivas de la UD00 33
3. UD01 34
   3.1 UD01 — Caracterización de sistemas de IA 34
      1. Resultado de aprendizaje y criterios de evaluación 34
      2. Objetivos de la unidad 34
      3. Fundamentos de los sistemas inteligentes (RA1-a) 34
      4. IA, machine learning, deep learning e IA generativa (RA1-c) 38
      5. Campos de aplicación de la IA (RA1-b) 41

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

6. Nuevas formas de interacción y eficiencia operativa (RA1-d) 42
7. Beneficios, riesgos y ética (visión general) 45
8. Puntos clave de la unidad 45
9. Glosario 46
10. FAQ 47
11. Planificación sesión a sesión 48
12. Tabla final RA/CE 48
13. Recursos 48
14. Evaluación 49
15. Recuperación 49
3.2 Diapositivas de la UD01 50
4. UD02 51
4.1 UD02 — Modelos de IA y resolución de problemas 51
1. Introducción 51
2. Resultado de aprendizaje y criterios de evaluación 51
3. Objetivos de la unidad 52
4. Sistema de resolución de problemas (RA2-a) 52
5. Clasificación de modelos de IA (RA2-b) 57
6. Modelos de automatización de tareas (RA2-c) 58
7. Razonamiento impreciso: lógica difusa (RA2-d) 59
8. Sistemas basados en reglas (RA2-e) 61
9. Adecuación del modelo (RA2-f) 63
10. Caso de estudio: Robocode como sistema de resolución de problemas completo 64
11. Puntos clave de la unidad 64
12. Glosario 64
13. FAQ 65
14. Planificación sesión a sesión 66
15. Tabla final RA/CE 66
16. Recursos 66
17. Evaluación 67
18. Recuperación 67
4.2 Diapositivas de la UD02 68
4.3 Documentación de Robocode 69
UD02 · Comparativa Java vs Python para Robocode 69
UD02 · Robocode Tank Royale en Java 73
UD02 · Robocode Tank Royale en Python 84
5. UD05 101
5.1 UD05 — Sistemas expertos y controladores inteligentes 101

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

1. Introducción 101
2. Resultado de aprendizaje y criterios de evaluación 101
3. Objetivos de la unidad 102
4. Arquitectura y dinámica de los sistemas expertos (RA5-a) 102
5. Estructuras elementales de representación (RA5-a/b) 105
6. Representar y simular comportamientos con `experta` (RA5-b) 106
7. Sistemas híbridos reglas/datos (RA5-b) 108
8. Sistemas de razonamiento impreciso: lógica difusa (RA5-b/d) 109
9. Variación de características y dinámica (RA5-c) 115
10. Estrategias de control de un sistema experto (RA5-d) 116
11. Controladores inteligentes (RA5-e) 117
12. Aplicaciones y tendencias (contenidos del RA5) 120
13. Puntos clave de la unidad 121
14. Glosario 121
15. FAQ 122
16. Sesiones 123
17. Recursos 123
18. Evaluación 124
19. Recuperación 124

5.2 Diapositivas de la UD05 125

6. UD03 126

6.1 UD03 — Procesamiento del Lenguaje Natural 126

1. Introducción 126
2. Resultado de aprendizaje y criterios de evaluación 126
3. Objetivos de la unidad 127
4. Qué es el PLN (RA3-a) 127
5. El potencial (RA3-c) 129
6. La ambigüedad, el problema de fondo (RA3-c) 129
7. Desambiguación y etiquetado morfológico (RA3-a, RA3-c) 131
8. Las demás limitaciones (RA3-c) 132
9. Cuándo es factible aplicar PLN (RA3-d) 132
10. Quién hace un proyecto de PLN (RA3-b, RA3-e, RA3-f) 133
11. Las herramientas en Python (RA3-c, RA3-g) 134
12. Construir un sistema orientado a una tarea (RA3-g) 136
13. Puntos clave de la unidad 138
14. Glosario 139
15. FAQ 140
16. Sesiones 141

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

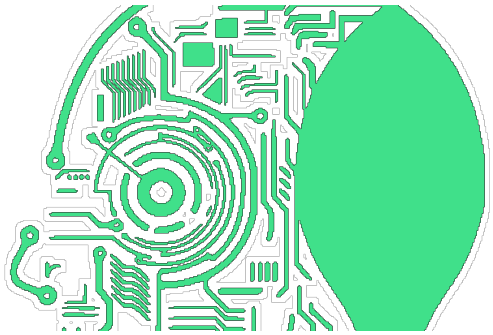
17. Recursos 141
18. Evaluación 142
19. Recuperación 142
6.2 Diapositivas de la UD03 143
7. UD04 144
7.1 UD04 — Análisis de sistemas robotizados 144
1. Introducción 144
2. Resultado de aprendizaje y criterios de evaluación 144
3. Objetivos de la unidad 145
4. Métodos y aplicaciones de la robótica (RA4-a) 145
5. Qué clase de problema resuelve la robótica 148
6. Modelado y control cinemático (RA4-a) 150
7. Los problemas y sus soluciones (RA4-b) 152
8. Percepción robótica (RA4-b) 154
9. Incertidumbre y aprendizaje (RA4-b) 155
10. Técnicas de programación de robots (RA4-c) 155
11. Humanos y robots (RA4-c, RA4-d) 157
12. Diseño e implementación de sistemas robotizados (RA4-d) 157
13. Practicar sin robot: los dos simuladores de la unidad 161
14. Puntos clave de la unidad 162
15. Glosario 162
16. FAQ 164
17. Sesiones 165
18. Recursos 165
19. Evaluación 166
20. Recuperación 166
7.2 Diapositivas de la UD04 167
8. UD06 168
8.1 UD06 — Aplicación de principios legales y éticos de la IA 168
1. Introducción 168
2. Resultado de aprendizaje y criterios de evaluación 168
3. Objetivos de la unidad 169
4. Riesgos y deontología de la IA (RA6-a) 169
5. Privacidad y protección de datos (RA6-b) 173
6. Cumplimiento estricto de la legalidad (RA6-c) 175
7. Security by design (RA6-d) 179
8. Privacy by design (RA6-e) 181
9. Sesgos de género y algorítmicos (RA6-f) 182

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

10. Puntos clave de la unidad 186
11. Glosario 187
12. FAQ 189
13. Sesiones 190
14. Recursos 190
15. Evaluación 191
16. Recuperación 191
8.2 Diapositivas de la UD06 193
9. 👋 Sobre mí 194
9.1 David Martínez Peña 194
9.2 📬 Contacto 194
10. Fuentes de información 195
10.1 Generales del curso 195
10.2 UD00 — Presentación y Docker 195
10.3 UD01 — Caracterización de sistemas de IA 195
10.4 UD02 — Modelos de IA y resolución de problemas 196
10.5 UD03 — Procesamiento del lenguaje natural 196
10.6 UD04 — Robótica 196
10.7 UD05 — Sistemas expertos y lógica difusa 196
10.8 UD06 — Riesgos, ética y legalidad de la IA 197

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

1. 🚀 Información importante



Modelos de IA

David Martínez Peña · CC BY-NC-SA 4.0

1.1 📛 Denominación del curso

📚 **Curso de especialización de Inteligencia Artificial y Big Data**

🤖 Modelos de Inteligencia Artificial (MIA) · Código **5071** · **90 h** · 4 ECTS

> **Curso 2026-2027**
>
> La docencia empieza el **1 de octubre de 2026** y el módulo termina el **28 de mayo de 2027**; junio se reserva a la **convocatoria ordinaria**. Las clases son de **lunes a jueves**: los viernes no hay clase y se aprovechan para las tareas y los talleres en casa.
>
> **Vacaciones**: Navidad del 22 de diciembre al 6 de enero · Pascua del 25 de marzo al 5 de abril.
>
> **Festivos que caen en día de clase**: 12 de octubre, 7 y 8 de diciembre, y **Fallas, 17 y 18 de marzo**. El 9 de octubre (Día de la Comunitat Valenciana) y el 19 de marzo caen en viernes, así que no afectan.
>
> Entre Fallas y Pascua, **marzo es el mes más interrumpido del curso**: tenlo en cuenta para planificar entregas.

1.2 📋 Contenidos y temporalización

Las unidades se imparten en este orden. El identificador `UDxx` va ligado a su resultado de aprendizaje (UD05 es siempre RA5), no al orden en que se imparte.

UD	Título	RA	Horas	Semanas	Fechas
UD00	Presentación y curso rápido de Docker	—	6	1-2	1 oct – 8 oct
UD01	Caracterización de sistemas de IA	RA1	12	3-6	12 oct – 6 nov
UD02	Modelos de IA y resolución de problemas	RA2	12	7-10	9 nov – 3 dic
UD05	Sistemas expertos	RA5	12	11-15	7 dic – 14 ene
UD03	Procesamiento del Lenguaje Natural	RA3	12	16-19	18 ene – 11 feb
UD04		RA4	12	20-23	15 feb – 11 mar

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

UD	Título	RA	Horas	Semanas	Fechas
	Análisis de sistemas robotizados				
UD06	Principios legales y éticos de la IA	RA6	6	24-28	15 mar – 23 abr
UD07	**Proyecto integrador (común al curso)**	RA7	15	29-33	26 abr – 28 may
Cierre	Presentaciones RA7 y prueba ordinaria (solo RA no superados)	—	3	34-35	1 jun – 18 jun
	Total		**90**	**30** nominales 33 con clase	

Por qué este orden

La UD05 (sistemas expertos) se imparte justo después de la UD02 porque se construye sobre las reglas y la lógica difusa que se ven allí. Y la UD06 (ética) ocupa el tramo de Fallas y Pascua, el más fragmentado del curso, porque es la unidad con menos carga de laboratorio.

El **contenido del módulo acaba a finales de abril**. Después viene el **proyecto intermodular (RA7)**, común a todo el curso de especialización: durante unas cinco semanas tu equipo trabaja el proyecto en las horas de todos los módulos.

1.3 🎯 Resultados de aprendizaje, pesos y calificación

RA	CE	Descripción	Nota mínima	Peso
RA1	4	Caracteriza sistemas de Inteligencia Artificial relacionándolos con la mejora de la eficiencia operativa de las organizaciones y empresas.	**5**	**14,55 %**
RA2	6	Utiliza modelos de sistemas de Inteligencia Artificial implementando sistemas de resolución de problemas.	**5**	**14,55 %**
RA3	7	Relaciona el procesamiento de lenguaje natural con sus aplicaciones determinando su potencial e identificando sus limitaciones.	**5**	**14,55 %**
RA4	4	Analiza sistemas robotizados, evaluando opciones de diseño e implementación.	**5**	**14,55 %**
RA5	5	Aplica sistemas expertos evaluando la influencia de los controladores inteligentes en el comportamiento del sistema.	**5**	**14,55 %**
RA6	6	Aplica principios legales y éticos al desarrollo de la Inteligencia Artificial integrándolos como parte del proceso.	**5**	**7,27 %**
RA7	9	Desarrolla un proyecto integrador que combine técnicas de IA y análisis	**5**	**20 %**

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

RA	CE	Descripción	Nota mínima	Peso
		de datos masivos para resolver un problema real o simulado, gestionando todo el ciclo de vida del proyecto.		
				100 %

Cada RA se califica **1 a 10, sin decimales** (Orden 8/2025, art. 5.1): **40 %** las entregas de la unidad + **60 %** la prueba escrita. Para superar el módulo hace falta **5 o más en cada RA**. Los ejercicios de autoevaluación y los notebooks guiados son práctica: no se entregan ni puntúan. El peso de cada entrega está en el libro de calificaciones de Moodle.

1.4 📝 Evaluación

- 🔍 La evaluación del módulo se realiza sobre los **Resultados de Aprendizaje (RA)** del currículo. Cada RA tiene asociados sus **criterios de evaluación (CE)**, que son los que determinan el grado de adquisición de las competencias del módulo.
- 📊 La nota final del módulo sale de la ponderación de los RA. Cada RA se evalúa de forma independiente, con **calificación de 1 a 10, sin decimales** (Orden 8/2025, art. 5.1).
- 📝 Cada RA tiene **una prueba escrita en Moodle** al cerrar su unidad, con preguntas de test y de desarrollo sobre el contenido de la unidad.
- ✅ Hay que obtener al menos un **5 en cada RA** para aprobar el módulo.
- 🔄 Si un RA queda por debajo de 5, se recupera con las actividades y pruebas diseñadas para **ese** resultado de aprendizaje. La convocatoria ordinaria de junio se dirige **solo a los RA no superados**: no hay prueba global del módulo.
- 🐳 La **UD00** (presentación y Docker) **no se califica**: el entorno de trabajo es un requisito. Sus **dos entregas** —los dos talleres— se marcan como **hecho / no hecho**, sin nota pero **obligatorias**: son la prueba de que tienes el entorno operativo, y sin ellos no se pueden hacer las prácticas de la UD02.
- ⏰ **Las entregas fuera de plazo tienen un máximo de 5.** El plazo de cada entrega se cierra en la fecha que indica Moodle. Si necesitas entregar después, **avísame** y te reabro esa tarea concreta un tiempo limitado; en ese caso la nota máxima del trabajo es **5 sobre 10**, y la rúbrica lo recoge de forma explícita. Vale para **todo el curso**, así que no se repite en cada unidad.
- 📁 En cada unidad, **«Práctica» es lo que no se entrega** —ejercicios de autoevaluación y notebooks guiados— y **«Entregas» es lo que sí**. Cada entrega dice en su página con qué se corrige: **rúbrica**, **apto / no apto** o **hecho / no hecho**. El **peso** de cada una está en el libro de calificaciones de Moodle, no en estas páginas.

> ⚠ **Importante**
>
> - ❗ Aprobar las evaluaciones parciales **no garantiza** aprobar el módulo.
> - 📌 Puedes aprobar las dos evaluaciones con buena nota, tener **un solo RA suspendido** y suspender el módulo.

1.5 📜 Legislación vigente

- 📄 RD 279/2021, de 20 de abril — establece el curso de especialización y fija los **RA y CE** del módulo.
- 📄 RD 497/2024, de 21 de mayo — modifica determinados reales decretos de cursos de especialización.
- 📄 Decreto 95/2026, de 9 de junio (DOGV) — currículo en la Comunitat Valenciana: fija el módulo en **90 h**.
- 🕒 Horario y organización del curso (GVA)

5 de octubre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

2. UD00

2.1 UD00 — Presentación del módulo y curso rápido de Docker

Unidad 0 · 6 h · semanas 1-2 (1 al 8 de octubre)

Unidad transversal de presentación y arranque del curso. **No tiene prueba escrita**: el entorno de trabajo es **requisito** para el resto del módulo, y el entregable de cada taller se marca como **hecho / no hecho**.

1. Introducción

Esta primera unidad tiene un doble objetivo. Por una parte, **situar el curso**: qué es el curso de especialización en Inteligencia Artificial y Big Data, qué se va a aprender en el módulo 5071 *Modelos de Inteligencia Artificial* y cómo se va a evaluar. Por otra, **dejar listo el entorno de trabajo**: sin herramientas fiables y reproducibles, cualquier práctica de IA se convierte en una lucha contra configuraciones que no funcionan. Por eso el núcleo técnico de la unidad es un **curso rápido de Docker**, la tecnología que nos permitirá ejecutar el mismo entorno de Python y Jupyter en cualquier equipo.

La idea que sostiene la unidad

La IA se hace con **entornos reproducibles**: si cada alumno tiene una configuración distinta, los resultados no son comparables. Docker resuelve ese problema empaquetando el entorno completo en un contenedor.

2. Resultados de aprendizaje y objetivos

La UD00 es **transversal** (no desarrolla un RA propio del módulo): sirve de nivelador tecnológico y de presentación. Las competencias que trabaja se asocian a la **ética y el cuidado del entorno de trabajo** (RA7-i) y a los **hábitos de trabajo** del curso.

Objetivo	Descripción
O1	Describir el curso de especialización IA y Big Data y el papel del módulo 5071 en él.
O2	Explicar cómo se evalúa el módulo (criterios, calificación, recuperación).
O3	Identificar el entorno de trabajo del curso: Aules, web, Python, Jupyter y Docker.
O4	Diferenciar contenedor, imagen, volumen y red, y explicar qué ventajas aporta Docker frente a las máquinas virtuales.
O5	Crear, ejecutar, detener y eliminar contenedores con `docker run`.
O6	Escribir un `Dockerfile` sencillo y levantarlo con `docker build`.
O7	Orquestar varios servicios con Docker Compose y conservar datos con volúmenes.
O8	Levantar un contenedor de prácticas con Jupyter y las bibliotecas del curso.

Nota sobre la tabla anterior.

¿Por qué no tiene RA propio?

El currículo del RD 279/2021 desarrolla 6 resultados de aprendizaje para el módulo 5071 (RA1-RA6). La UD00 es una **unidad de presentación y puesta en marcha** que prepara el terreno para esas unidades: sin ella, los RA1-RA6 perderían horas en configurar entornos.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

3. Presentación del curso y del módulo (RA7-i)

3.1 El curso de especialización en IA y Big Data

El **curso de especialización en Inteligencia Artificial y Big Data** es un título de **Grado Superior** (familia Informática y Comunicaciones) de **600 horas y 36 ECTS** establecido por el RD 279/2021. Su competencia general es *programar y aplicar sistemas inteligentes que optimizan la gestión de la información y la explotación de datos masivos, garantizando la seguridad y la ética*.

Se organiza en **cinco módulos**:

Código	Módulo	Horas en CV
5071	Modelos de Inteligencia Artificial	90
5072	Sistemas de aprendizaje automático	90
5073	Programación de Inteligencia Artificial	210
5074	Sistemas de Big Data	90
5075	Big Data aplicado	120

El acceso se realiza desde titulaciones de grado superior de la familia de informática y otras relacionadas (ASIR, DAM, DAW, telecomunicaciones, mecatrónica y robótica industrial).

Una frase para recordar

Este curso enseña a **construir sistemas inteligentes** (5071, 5072, 5073) y a **tratar datos a gran escala** (5074, 5075). El módulo 5071 es la base conceptual de los sistemas de IA.

3.2 El módulo 5071 *Modelos de Inteligencia Artificial*

El módulo 5071 se imparte a lo largo de todo el curso (**90 horas, 3 horas semanales**, 4 ECTS) y desarrolla **6 resultados de aprendizaje** (RA1-RA6), más el **RA7 de proyecto integrador transversal**:

RA	Qué aprenderás
RA1	Caracterizar sistemas de IA y relacionarlos con la eficiencia operativa
RA2	Utilizar modelos de IA para implementar sistemas de resolución de problemas
RA3	Relacionar el procesamiento del lenguaje natural (PLN) con sus aplicaciones
RA4	Analizar sistemas robotizados y evaluar su diseño e implementación
RA5	Aplicar sistemas expertos y valorar los controladores inteligentes
RA6	Aplicar principios legales y éticos (privacidad, sesgos, normativa)
RA7	Proyecto integrador transversal de todo el curso (definición → datos → modelos → Big Data → comunicación)

3.3 El proyecto integrador (RA7)

El **RA7** es un **proyecto común a todo el curso de especialización**: se desarrolla en **equipos** durante el bloque final (semanas 25-29) y en él **confluyen todos los módulos** del título. El alumnado define un problema, obtiene datos, aplica modelos de IA, integra soluciones de Big Data, comunica resultados y evalúa críticamente el trabajo realizado.

Es transversal por dos razones: 1. Lo **imparten todos los docentes** del curso, no solo el del 5071. 2. La **nota se consensúa** entre el equipo docente, valorando la contribución de cada miembro.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Implicación para este módulo

El contenido del 5071 (UD00-UD06) **finaliza a finales de abril** porque las últimas semanas del curso se dedican por completo al proyecto integrador. Desde el primer día, el proyecto te permitirá aplicar lo aprendido en este módulo.

4. Cómo se evalúa el módulo

La evaluación se rige por la **Orden 8/2025** de la Comunitat Valenciana (modificada por la **Orden 5/2026**). Lo que debes saber desde el primer día:

Los **pesos de cada RA, la nota mínima y el reparto 40/60** están en la página de información importante, que es la referencia única: si algún día cambian, cambian allí. Aquí van las reglas que conviene tener claras desde el primer día y que no dependen de los pesos:

Aspecto	Regla
Calificación por RA	De **1 a 10, sin decimales** (art. 5.1)
Evaluación continua	Se pierde al superar el **15 % de inasistencia** (asistencia ≥ 85 %)
Evaluaciones parciales	Dos parciales **informativos** (fin del 1.er y 2.º trimestre)
Recuperación	Programa individual por **cada RA no superado**; en junio, solo los RA pendientes
Esta unidad (UD00)	**No se califica**: es requisito. Sus **tres entregables** (los dos talleres y el notebook) se marcan **hecho / no hecho**

Nota sobre la tabla anterior.

Asistencia

Si superas el **15 % de faltas** perderás la evaluación continua del módulo (art. 7.3 Orden 8/2025). Es un criterio objetivo, no una decisión del profesor.

Evaluación inicial

Durante el primer mes realizaremos una **evaluación inicial o diagnóstica** (obligatoria antes de finalizar el segundo mes lectivo). No cuenta para la nota: sirve para conocer tus competencias previas y adaptar el ritmo del curso.

5. Entorno de trabajo del curso

Herramienta	Para qué la usaremos
Aules (Moodle de la GVA)	Entrega de tareas, pruebas, avisos y calificaciones
Web del curso (mkdocs)	Teoría, ejercicios, talleres y notebooks en el navegador
Python + Jupyter	Notebooks de prácticas con las bibliotecas de IA
Docker	Entornos reproducibles y el contenedor de prácticas
Git (opcional)	Control de versiones de tus proyectos

Sobre Aules

Aules es la **plataforma oficial de e-learning de la Conselleria de Educación**, basada en **Moodle**. Accederás con tu usuario (NIA) y desde ahí se gestionan las tareas del módulo.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

El stack de IA del curso

Las prácticas se basan en **Python** con estas bibliotecas:

`numpy`, `pandas`, `matplotlib`, `scikit-learn`, `scikit-fuzzy`, `nltk`, `spaCy`, `torch` y `experta`.

Este stack se ejecutará en un **contenedor de prácticas de IA** que montaremos en el taller 4 de esta unidad.

5.2 Instalar Docker

Antes de nada, Docker tiene que funcionar en tu máquina. Estos pasos están probados en el aula.

INSTALACIÓN EN UBUNTU

https://www.digitalocean.com/community/tutorials/how-to-install-and-use-docker-on-ubuntu-20-04-es

Requisitos previos:

- **apt-transport-https:** permite que el administrador de paquetes transfiera datos a través de https
- **ca-certificates:** permite que el navegador web y el sistema verifiquen los certificados de seguridad
- **curl:** transfiere datos (similar a wget)
- **software-properties-common:** agrega scripts para administrar el software

```
sudo apt-get install  curl apt-transport-https ca-certificates software-properties-common
```

Agregamos repositorio

```
# Primero clave GPG
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -

sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable"

sudo apt update

sudo apt install docker-ce

sudo systemctl status docker
```

Por defecto, el comando docker solo puede ser ejecutado por el usuario root o un usuario del grupo docker, que se crea automáticamente durante el proceso de instalación de Docker.

Para evitar escribir sudo al ejecutar el comando docker, agregue su nombre de usuario al grupo docker:

```
sudo usermod -aG docker ${USER}

# Cerramos y abrimos sesión de nuevo o ejecutamos
su - ${USER}

# Confirmamos los grupos de nuestro usuario
id -nG
```

INSTALACIÓN EN WINDOWS

https://www.youtube.com/watch?v=wMioaU5yk_w

DOCKER DESKTOP

¿Qué es Docker Desktop?



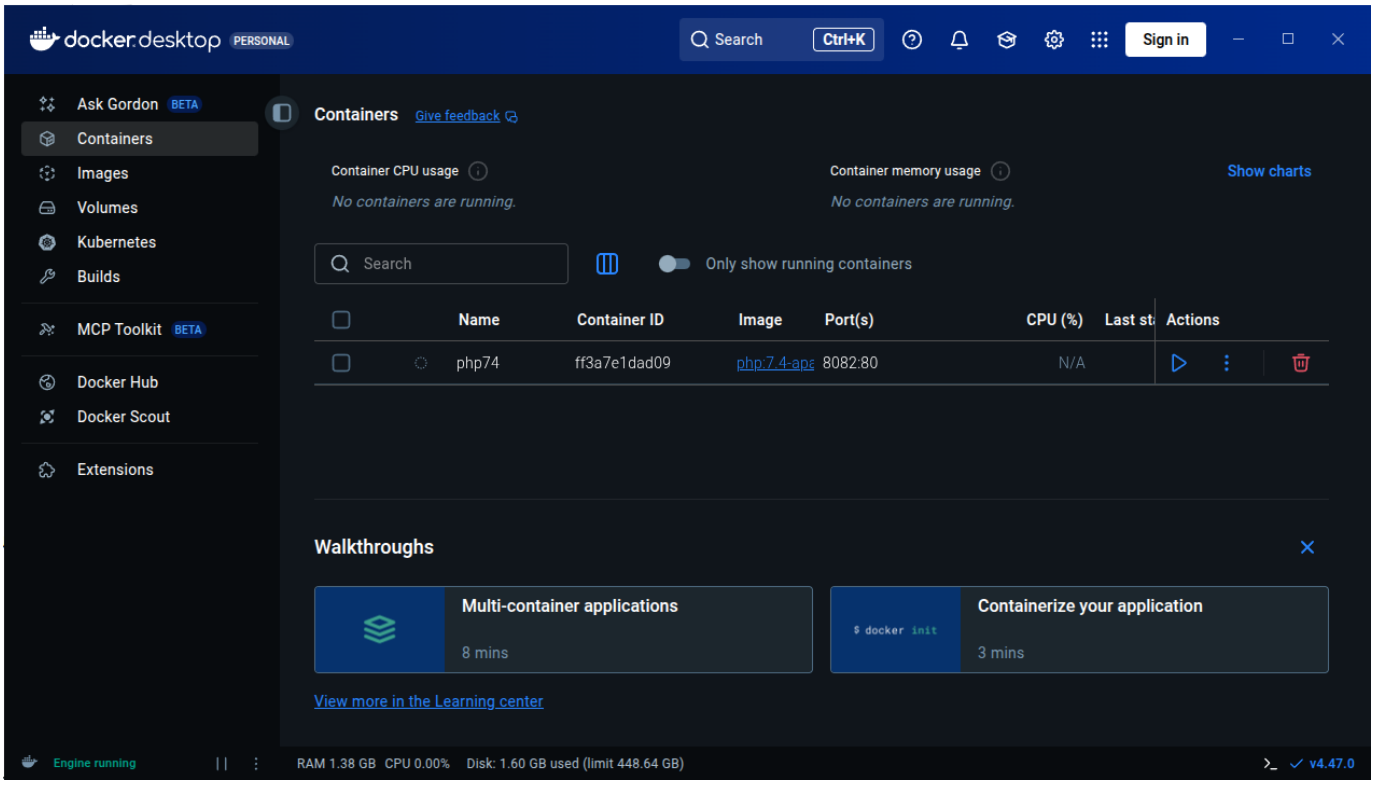
Es la aplicación oficial de Docker que te da una **interfaz gráfica (GUI)** para manejar contenedores, además de la línea de comandos.

¿Para qué sirve?

- **Gestión visual**: Ver contenedores, imágenes y volúmenes de forma gráfica

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- **Configuración fácil**: Ajustar recursos (CPU, RAM) con sliders
- **Monitorización**: Ver en tiempo real qué está pasando



Compatibilidad por Sistema Operativo

🪟 Windows

- **Windows 10/11** 64-bit (versiones Home, Pro, Enterprise, Education)
- **Requisitos importantes**:
- Habilitar **WSL 2** (Windows Subsystem for Linux)
- Virtualización activada en BIOS/UEFI
- **Windows Home** necesita WSL 2, **Pro/Enterprise** puede usar Hyper-V

🍎 macOS

- **macOS 12 Monterey** o superior
- **Tipos de chip**:
- **Apple Silicon** (M1, M2, M3, etc.)
- **Intel** con procesador de 2010 o más nuevo
- Necesita **macOS actualizado**

🐧 Linux (versión nativa)

- **Distribuciones compatibles**:
- Ubuntu 20.04 LTS o superior
- Debian 11 o superior
- Fedora 36 o superior
- Arch Linux (y derivados)
- **Requisitos**: kernel 5.10+, systemd, 64-bit

Guía Rápida de Instalación

Windows:

1. Descarga desde docker.com/products/docker-desktop
2. Ejecuta el instalador `.exe`

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

3. Sigue el asistente (marca "Use WSL 2" si tienes Windows Home)
4. Reinicia cuando termine
5. ¡Listo! Docker se inicia automáticamente

macOS:

1. Descarga desde la web oficial
2. Arrastra Docker.app a la carpeta Applications
3. Ejecuta desde Launchpad
4. Autoriza con contraseña del sistema
5. Espera a que configure todo (puede tardar unos minutos)

Linux (Ubuntu/Debian ejemplo):

```
# Paso 1: Descargar el .deb oficial (siempre la última versión)
wget https://desktop.docker.com/linux/main/amd64/docker-desktop-amd64.deb

# Paso 2: Instalar
sudo apt install ./docker-desktop-*.deb

# Paso 3: Iniciar
systemctl --user start docker-desktop
```

Problemas con Ubuntu 24.04

Si tienes problemas con Ubuntu 24.04, yo me he encontrado dos:

1. Problema de permisos

   ```
   ...
   S'estan processant els activadors per a desktop-file-utils (0.27-2build1)…
   N: La baixada es duu a terme fora de l'entorn segur com a root ja que el fitxer «/home/ubuntu/docker-desktop-4.27.2-amd64.deb» no és accessible per l'us
   uari «_apt». - pkgAcquire::Run (13: Permission denied)
   ```

   Lo he resuelto cambiando los permisos del deb:

   ```
   # Change ownership of the file to make it accessible
   sudo chown _apt:root /home/ubuntu/docker-desktop-4.27.2-amd64.deb
   # Or alternatively, change permissions to make it readable
   sudo chmod 644 /home/ubuntu/docker-desktop-4.27.2-amd64.deb
   ```

2. Lanzas Docker-desktop, aparece el icono, pero desaparece y la aplicación no inicia: Parece un problema con un cambio en Ubuntu 24.04 que he resuelto con la información de este post:

   ```
   sudo sysctl -w kernel.apparmor_restrict_unprivileged_userns=0
   systemctl --user restart docker-desktop
   ```

   En algunos casos parece que la solución anterior solo sirve hasta que reinicias, si es así, prueba esto también:

   Crea un nuevo fichero:

   ```
   sudo nano /etc/apparmor.d/opt.docker-desktop.bin.com.docker.backend
   ```

   Escribe dentro el siguiente contenido:

   ```
   abi <abi/4.0>,

   include <tunables/global>

   /opt/docker-desktop/bin/com.docker.backend flags=(default_allow) {
   userns,

   # Site-specific additions and overrides. See local/README for details.
   include if exists <local/opt.docker-desktop.bin.com.docker.backend>
   }
   ```

   Reinicia el servicio `apparmor.service`:

   ```
   sudo systemctl restart apparmor.service
   ```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Ventajas docker desktop (GUI) vs Línea de Comandos (CLI)

✅ **Ventajas de Docker Desktop:**

- **Más fácil para empezar** - Ideal para principiantes
- **Todo integrado** - No necesitas instalar nada más
- **Debugging visual** - Ves los logs y estados de un vistazo
- **Gestión de recursos** - Controlas CPU/RAM fácilmente

❌ **Desventajas:**

- **Más pesado** - Consume más recursos de tu PC
- **Menos flexible** - Algunas opciones avanzadas solo por comandos
- **Dependes de la GUI** - Si se cierra la app, pierdes la interfaz

🎯 **Conclusión:**

- **Empezad con Docker Desktop** para aprender sin frustraciones
- **Aprended también los comandos básicos** para ser más versátiles
- Usad **ambos**: la GUI para lo cotidiano y la terminal para lo avanzado

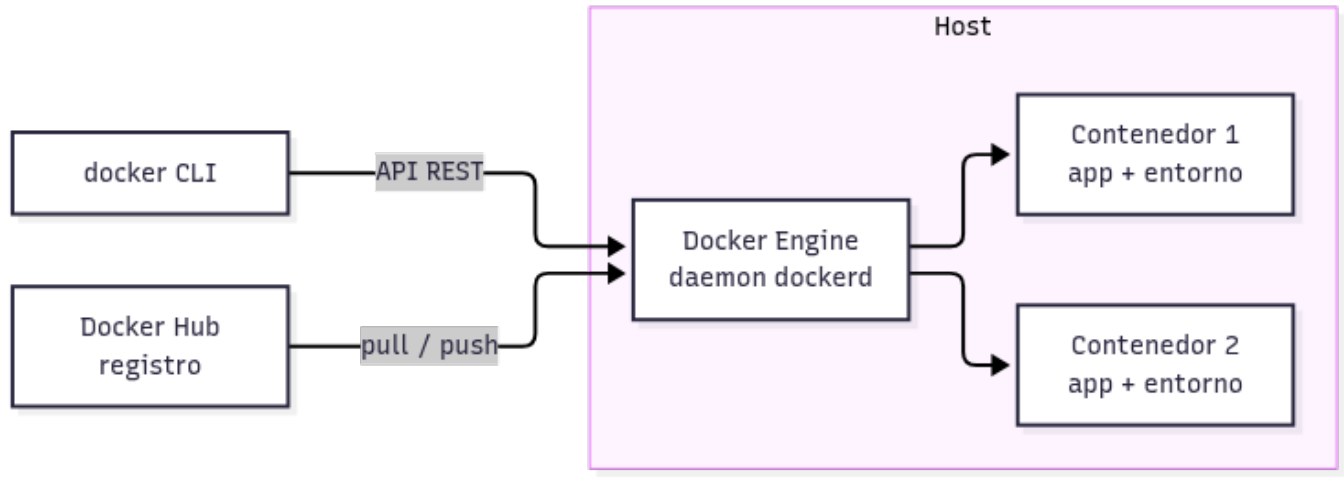
> ⚠ **Comprueba la instalación antes de seguir**
>
> `docker run hello-world` debe terminar sin errores. Si te responde `permission denied while trying to connect to the Docker daemon socket`, te falta el paso de añadir tu usuario al grupo `docker` **y cerrar y abrir sesión**.

6. Curso rápido de Docker (RA7-i)

6.1 El problema: entornos que no se reproducen

¿Cuántas veces funciona "en mi ordenador" y no en el del compañero? La causa es que los entornos difieren: versiones de Python, bibliotecas, sistema operativo, configuraciones. **Docker resuelve este problema** empaquetando la aplicación y todo lo que necesita en un **contenedor**.

> ✎ **Definición**
>
> **Docker** es una plataforma abierta para desarrollar, distribuir y ejecutar aplicaciones. Permite separar la aplicación de la infraestructura mediante contenedores: entornos aislados y ligeros que incluyen todo lo necesario para ejecutar la aplicación.



© 2025 David Martínez licensed under CC BY-NC-SA 4.0

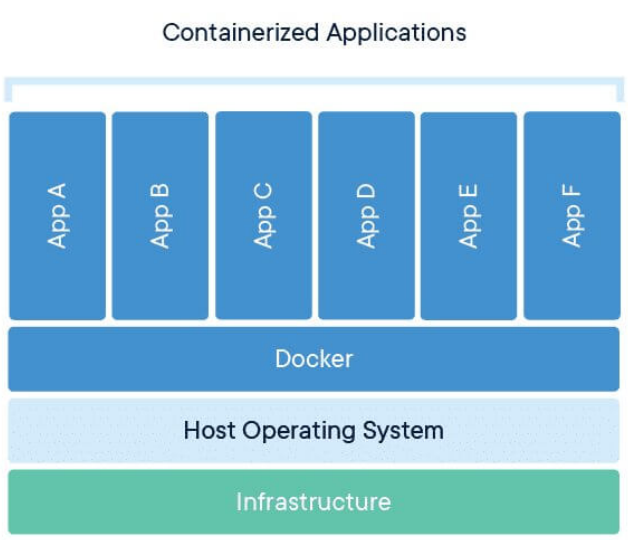
6.2 Contenedores frente a máquinas virtuales

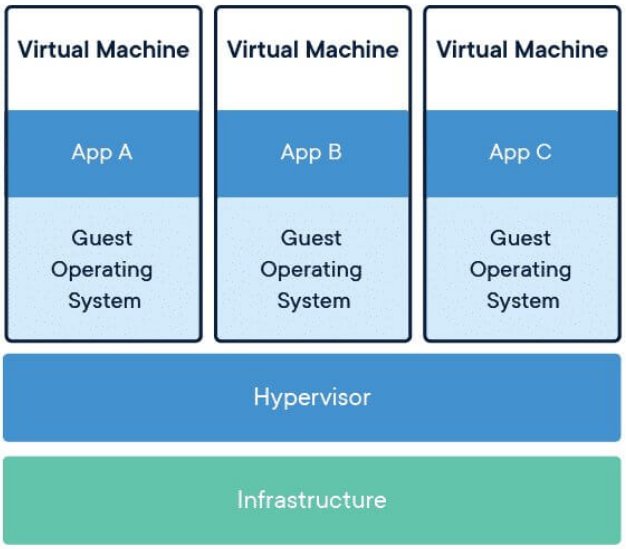
Característica	Máquina virtual	Contenedor
Aislamiento	Por hardware (hipervisor)	Por procesos (namespaces del kernel)
Sistema operativo	SO completo con su propio kernel	Comparte el kernel del host
Arranque	Minutos	Segundos
Tamaño	Gigabytes	Megabytes
Carga en un servidor	Decenas	Cientos/miles
Reproducibilidad	Media	Alta (imagen inmutable + capas)

Nota sobre la tabla anterior.

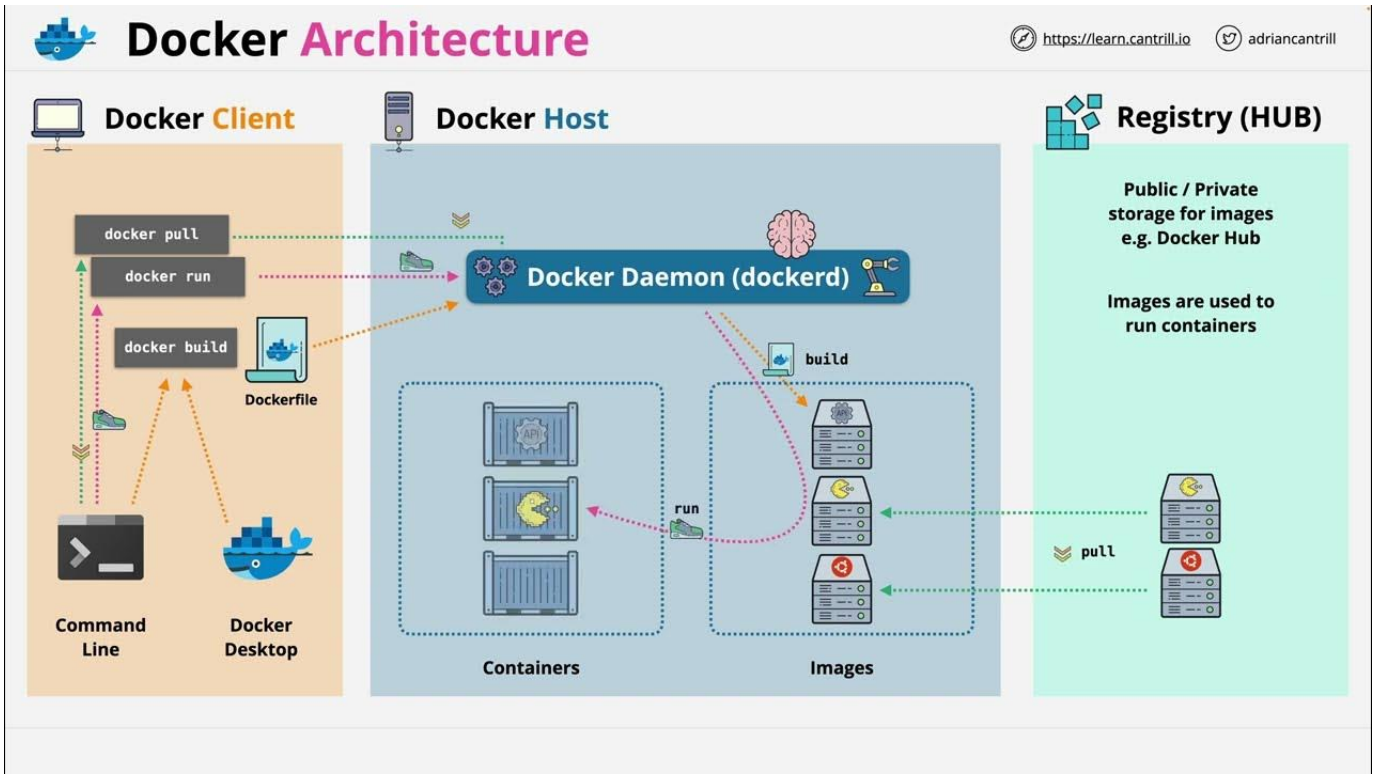
> **¿Y entonces las VMs son inútiles?**
>
> No. Las VMs siguen siendo necesarias cuando se requiere **aislamiento total de kernel** o se ejecutan sistemas operativos distintos. La práctica habitual es combinarlas: una VM con Docker encima. En este curso trabajaremos con contenedores porque son **ligeros y reproducibles**.

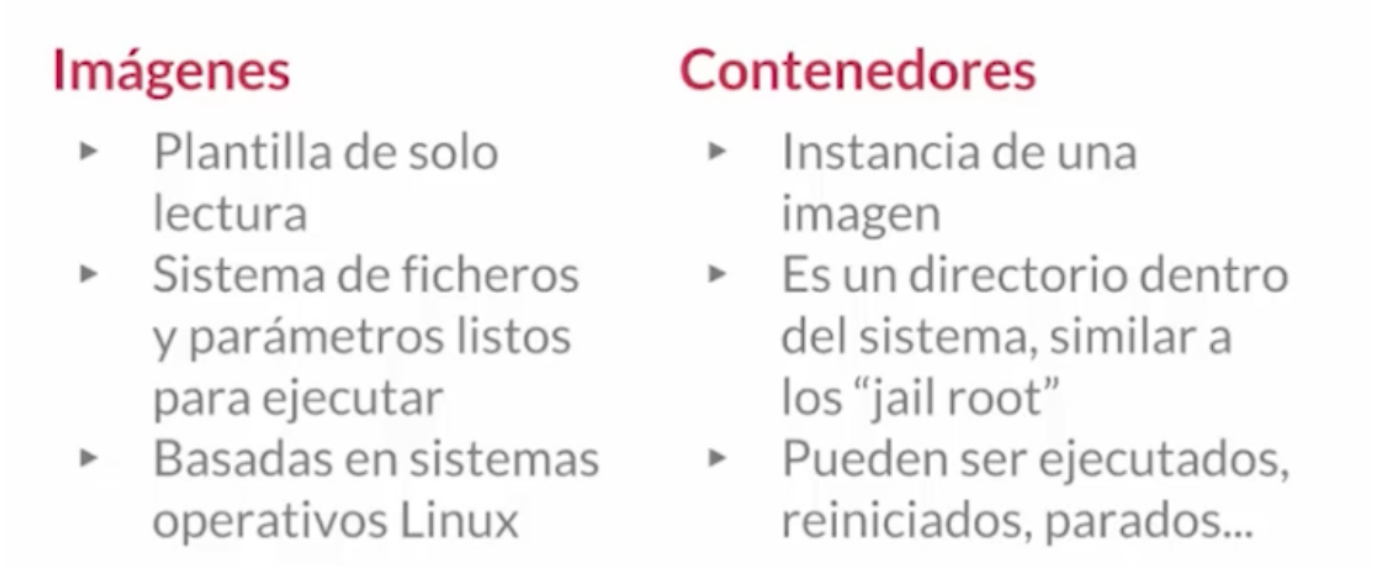
LA ARQUITECTURA DE DOCKER EN IMÁGENES





© 2025 David Martínez licensed under CC BY-NC-SA 4.0

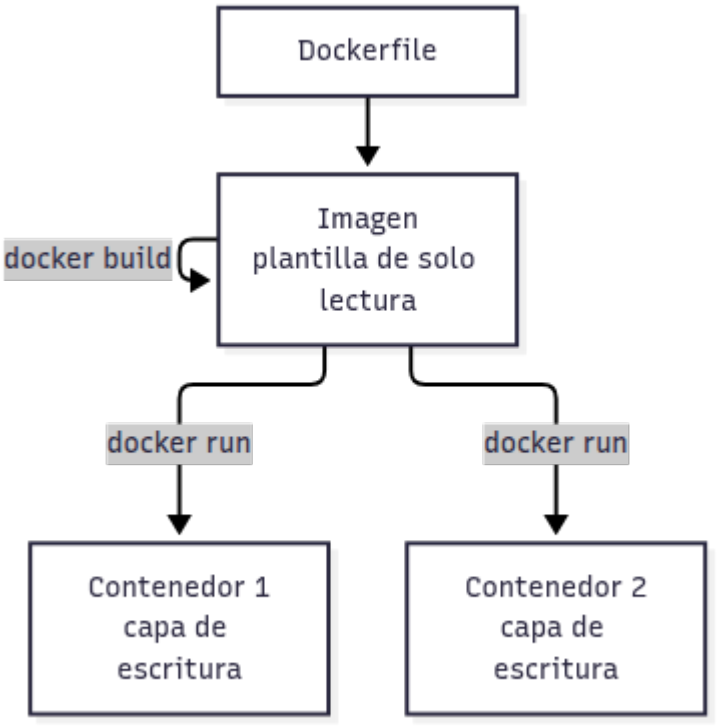




6.3 Imágenes y contenedores

- **Imagen**: plantilla **de solo lectura** (un paquete inmutable con el SO base, la aplicación y las dependencias). Se construye por **capas**.
- **Contenedor**: **instancia ejecutable** creada a partir de una imagen. Añade una capa de escritura por encima de la imagen.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



La regla de las capas

Cada instrucción del `Dockerfile` crea una **capa**. Al reconstruir una imagen, solo se regeneran las capas que han cambiado: por eso **el orden de las instrucciones importa** para aprovechar la caché.

6.4 El registro y el primer contenedor

Docker Hub es el registro público por defecto: se usa `pull` para **descargar** imágenes y `push` para **subirlas**. Nuestro primer comando será:

```
docker run hello-world
```

La primera vez, Docker descarga la imagen `hello-world`, crea un contenedor a partir de ella, ejecuta un mensaje de confirmación y lo detiene. Ese es exactamente el ciclo completo de Docker en una sola línea.

Comprueba tu instalación

Antes de seguir, verifica que Docker está instalado y el demonio en marcha:

```
docker --version
docker run hello-world
```

6.5 Mismo código, dos entornos: el caso de `experta`

El argumento definitivo a favor de los contenedores se ve mejor con una biblioteca que usaremos de verdad en este módulo: `experta`, el motor de reglas de la UD02 y la UD05.

Dato	Valor
Última versión	**1.9.4**, del 16/11/2019
Python que declara soportar	3.5 a **3.8**
Mantenimiento	**abandonado**; fallo abierto como *issue* #34 desde julio de 2023

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Dato	Valor
Causa	su dependencia `frozendict==1.2` usa `collections.Mapping`, que **desapareció en Python 3.10**

Monta dos contenedores con **el mismo código** y compara. Guardar la prueba en una variable deja claro que lo único que cambia entre los dos es la imagen:

```
# La prueba: instalar experta e importarla
PRUEBA="pip install -q experta && python -c 'import experta'"

docker run --rm python:3.9-slim  bash -c "$PRUEBA"   # Contenedor A: funciona tal cual
docker run --rm python:3.12-slim bash -c "$PRUEBA"   # Contenedor B: el mismo código revienta
```

El contenedor B falla con `AttributeError: module 'collections' has no attribute 'Mapping'`, y se arregla con tres líneas **antes** del import, las mismas que verás en los notebooks de la UD02:

```
import collections, collections.abc
for nombre in ("Mapping", "Iterable", "MutableMapping"):
    if not hasattr(collections, nombre):
        setattr(collections, nombre, getattr(collections.abc, nombre))

from experta import KnowledgeEngine   # ahora sí
```

Tres lecciones en un solo ejemplo

1. **El entorno importa**: mismo código, misma biblioteca, dos resultados. Sin fijar la versión de Python, «en mi máquina funciona» no significa nada.
2. **Las dependencias se abandonan**: `experta` no publica nada desde 2019. Elegir una biblioteca es apostar también por quien la mantiene.
3. **Se puede convivir con ello**: un parche documentado y versionado, y sigues adelante. Lo que no vale es descubrirlo el día de la entrega.

6.6 El ciclo de vida de un contenedor

Comando	Qué hace
`docker run`	Crea y arranca un contenedor nuevo (descarga la imagen si falta)
`docker ps`	Lista los contenedores en marcha
`docker ps -a`	Lista también los detenidos
`docker stop`	Detiene un contenedor (envía SIGTERM)
`docker start`	Reactiva un contenedor parado conservando sus cambios
`docker rm`	Elimina un contenedor (parado)
`docker rmi`	Elimina una imagen

Nota sobre la tabla anterior.

`docker run` no borra el contenedor

Cuando el proceso principal termina, el contenedor **se detiene pero no se elimina**. Cada `docker run` crea un contenedor nuevo; por eso conviene usar `--rm` en los contenedores desechables y limpiar con `docker rm` los que ya no se usen.

6.7 Flags clave de `docker run`

Flag	Significado	Ejemplo
`-it`	Interactivo + terminal (para entrar a un shell)	`docker run -it ubuntu bash`
`-d`	En segundo plano (detached)	`docker run -d -p 8080:80 nginx`

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Flag	Significado	Ejemplo
`-p`	Publica un puerto (host:contenedor)	`docker run -p 8888:8888 ...`
`-v`	Monta un volumen o bind mount	`docker run -v "$PWD":/app ...`
`--rm`	Elimina el contenedor al terminar	`docker run --rm hello-world`
`--name`	Asigna un nombre al contenedor	`docker run --name mi-python ...`

6.8 Qué ocurre dentro de `docker run`

Cuando ejecutas `docker run`, el demonio hace lo siguiente:

1. Comprueba si la imagen está en la caché local; si no, la **descarga** (`pull`).
2. **Crea el contenedor**: añade una capa de escritura sobre las capas de solo lectura.
3. **Monta el filesystem** (y los volúmenes o binds indicados) y crea la **interfaz de red**.
4. **Arranca el proceso principal** (el comando indicado, o el `CMD` de la imagen).
5. Cuando ese proceso termina, el contenedor **se detiene** (no se elimina).

6.9 Volúmenes y persistencia

Los contenedores son **efímeros**: al eliminarlos se pierden sus datos. Para conservarlos:

- **Volúmenes gestionados** (*named volumes*): los gestiona Docker (en `/var/lib/docker/volumes`). Son la opción recomendada para datos que no necesitas ver desde el host (cachés, datasets, bases de datos).
- **Bind mounts**: montan una **carpeta real del host** dentro del contenedor. Son ideales para el código y los notebooks del curso: editas en tu equipo y el contenedor ve los cambios al instante.

```
# bind mount: la carpeta practicas del host se ve en /app dentro del contenedor
docker run --rm -v "$PWD/practicas":/app -w /app python:3.12 python app.py
```

> **Bind mount vs volumen**
>
> Para **notebooks y código** del curso usa un **bind mount** (`-v ./notebooks:/home/jovyan/work`): son ficheros reales de tu equipo. Los **volúmenes nombrados** quedan para datos que no necesitas tocar desde el host.

7. El Dockerfile: construir imágenes reproducibles (RA7-i)

7.1 Instrucciones esenciales

Un `Dockerfile` es un **guion** que describe cómo construir la imagen. Siempre empieza por `FROM` (la imagen base).

Instrucción	Qué hace
`FROM`	Define la imagen base (obligatoria)
`WORKDIR`	Establece el directorio de trabajo
`COPY`	Copia ficheros del contexto de build a la imagen
`RUN`	Ejecuta un comando **durante el build** (instalar, compilar)
`ENV`	Define variables de entorno
`EXPOSE`	**Documenta** el puerto (no lo publica; hace falta `-p`)
`CMD`	Comando por defecto en runtime (sobrescribible en `docker run`)
`ENTRYPOINT`	Ejecutable fijo; se puede combinar con `CMD` para los argumentos

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
CMD ["python", "app.py"]
```

7.2 La caché de capas

Cada instrucción crea una capa. Docker **reutiliza las capas en caché** mientras las instrucciones y su contenido no cambien. Por eso se copia `requirements.txt` **antes** que el código: así las dependencias se instalan una vez y se cachean.

Buenas prácticas para un curso rápido

- Anclar la versión de la imagen base (`FROM python:3.12-slim`, no `FROM python`).
- Usar `.dockerignore` para excluir del build context `.git`, `*.pyc`, `.env`, datasets grandes.
- Unir `apt-get update && apt-get install` en un solo `RUN`.
- Un **proceso por contenedor** y contenedores **efímeros**.

7.3 Construir y ejecutar

```
docker build -t mi-app .       # construye la imagen desde el Dockerfile del directorio actual
docker run -p 8000:8000 mi-app # ejecuta la imagen publicando el puerto 8000
```

7.4 Gestionar las imágenes que acumulas

Es un instalador donde podemos incorporar nuestra aplicación. Es el punto de inicio para crear contenedores. Hay imágenes oficiales de por ejemplo Ubuntu, Apache, etc, que fueron creadas por sus creadores oficiales.

Página oficial para imágenes: https://hub.docker.com

Vamos a utilizar la siguiente imagen para pruebas:

https://hub.docker.com/_/hello-world

Para ejecutar este contenedor “hello-word” escribimos en la terminal:

```
docker run hello-world
```

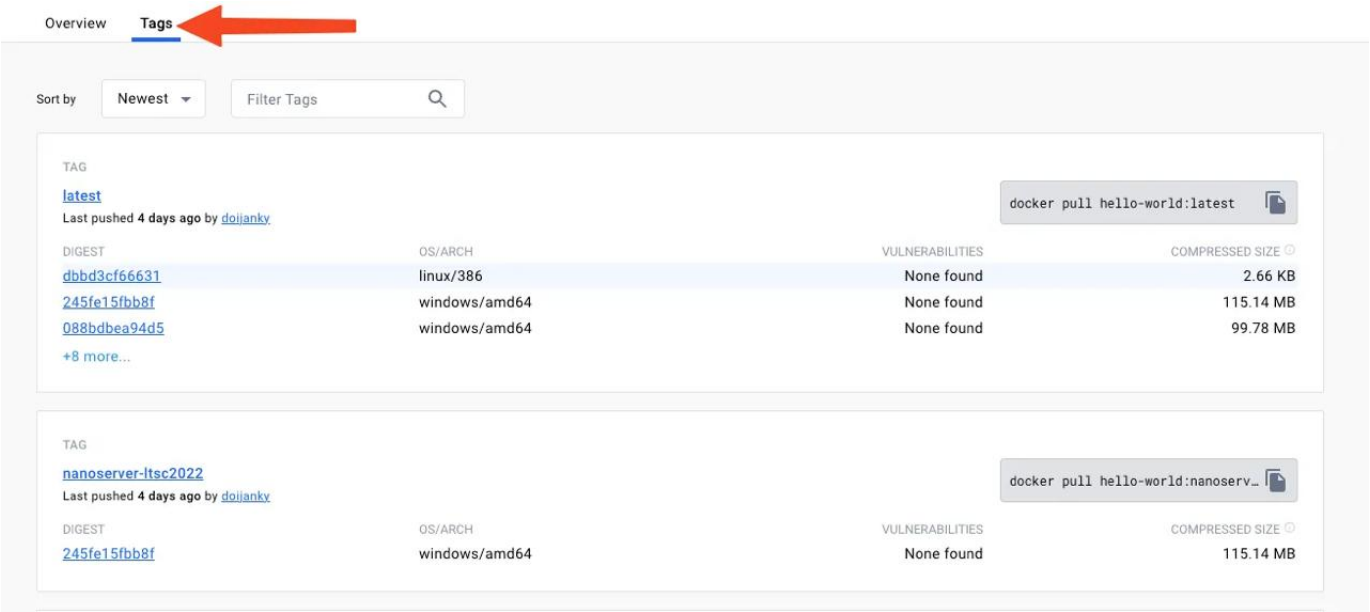
Una vez ejecutado, ya dispondremos de la imagen descargada, podemos ver todas las imágenes que tenemos descargadas con:

```
docker images

REPOSITORY                          TAG           IMAGE ID       CREATED        SIZE
hello-world                         latest        ee301c921b8a   9 months ago   9.14kB
```

Desde la página web de docker hub, podemos ver diferentes versiones de la misma imagen en la pestaña “TAGS”

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Podemos descargar una imagen específica y ejecutarla:

```
docker run hello-world:linux

docker images
REPOSITORY                          TAG           IMAGE ID       CREATED        SIZE
hello-world                         latest        ee301c921b8a   9 months ago   9.14kB
hello-world                         linux         ee301c921b8a   9 months ago   9.14kB

docker run hello-world:linux
```

Para eliminar una imagen utilizamos el parámetro `rmi` por ejemplo:

```
docker pull alpine

docker images
REPOSITORY                          TAG           IMAGE ID       CREATED        SIZE
alpine                              latest        ace17d5d883e   3 weeks ago    7.73MB
hello-world                         latest        ee301c921b8a   9 months ago   9.14kB
hello-world                         linux         ee301c921b8a   9 months ago   9.14kB

# Eliminamos, opción 1
docker rmi ace17d5d883e

# Eliminamos, opción 2
docker rmi alpine

# Eliminamos, opción 3
docker rmi ace1
```

También se pueden buscar imágenes desde consola.

```
docker search ubuntu

NAME                                DESCRIPTION                                     STARS     OFFICIAL
ubuntu                              Ubuntu is a Debian-based Linux operating sys…   16888     [OK]
websphere-liberty                   WebSphere Liberty multi-architecture images …   298       [OK]
open-liberty                        Open Liberty multi-architecture images based…   64        [OK]
neurodebian                         NeuroDebian provides neuroscience research s…   106       [OK]
ubuntu-debootstrap                  DEPRECATED; use "ubuntu" instead                52        [OK]
ubuntu-upstart                      DEPRECATED, as is Upstart (find other proces…   115       [OK]
ubuntu/nginx                        Nginx, a high-performance reverse proxy & we…   112
ubuntu/squid                        Squid is a caching proxy for the Web. Long-t…   83
ubuntu/cortex                       Cortex provides storage for Prometheus. Long…   4
ubuntu/prometheus                   Prometheus is a systems and service monitori…   56
ubuntu/apache2                      Apache, a secure & extensible open-source HT…   70
...
```

8. Docker Compose: varios servicios a la vez (RA7-i)

Compose orquesta **múltiples contenedores** (p. ej. la aplicación + una base de datos) mediante un fichero `compose.yaml`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```yaml
services:
  jupyter:
    image: quay.io/jupyter/scipy-notebook
    ports:
      - "8888:8888"
    volumes:
      - ./notebooks:/home/jovyan/work
    environment:
      - JUPYTER_TOKEN=cursoia
```

Comando	Qué hace
`docker compose up -d`	Crea y arranca los servicios en segundo plano
`docker compose down`	Detiene y elimina contenedores (y la red por defecto)
`docker compose down -v`	Además borra los volúmenes nombrados (¡borra datos!)
`docker compose config`	Muestra la configuración resuelta sin arrancar nada
`docker compose logs -f`	Sigue los logs de los servicios
`docker compose exec`	Ejecuta un comando dentro de un contenedor en marcha

Para ordenar el arranque se usa `depends_on` (con `condition: service_healthy` y un `healthcheck` para esperar a que un servicio esté listo).

> **`down -v` borra datos**
>
> `docker compose down -v` elimina también los **volúmenes nombrados**. Úsalo solo cuando quieras empezar de cero; tus notebooks se perderían.

9. El contenedor de prácticas de IA (RA7-i)

El entorno del curso se levantará con la imagen `quay.io/jupyter/scipy-notebook`, que ya incluye numpy, scipy, scikit-learn, pandas, matplotlib y otras bibliotecas científicas. Se descarga de `quay.io`, el registro actual del proyecto Jupyter: las imágenes `jupyter/*` de Docker Hub están congeladas desde 2023.

> **¿Por qué `python:3.12-slim` y no Alpine?**
>
> La variante `-slim` usa **Debian (glibc)**; la `-alpine` usa **musl libc**. Bibliotecas como scikit-learn y torch **solo publican wheels para glibc**: en Alpine habría que compilar desde fuente (torch puede tardar 30-60 min). Con `-slim` todo el stack se instala con wheels precompilados. Los Jupyter Docker Stacks científicos también se construyen sobre Ubuntu (glibc).

El contenedor de prácticas:

- Publica el puerto **8888** (Jupyter).
- Monta tu carpeta de prácticas en `/home/jovyan/work` (bind mount) para que los notebooks sean ficheros reales de tu equipo.
- Fija el token con `JUPYTER_TOKEN` para que la URL sea determinista.

```
docker compose up -d
# abre http://localhost:8888 y usa el token cursoia
docker compose exec jupyter bash   # para entrar dentro
docker compose down               # cuando termines
```

Lo pondrás en marcha paso a paso en el **Taller 4** de esta unidad.

9.1 Construir la imagen con un Dockerfile propio

Otra forma de levantar un contenedor con docker-compose es utilizar un Dockerfile para generar la imagen (en lugar de usar una de DockerHub)

Necesitamos una carpeta con la siguiente estructura:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
.
├── docker-compose.yml
├── Dockerfile
└── notebooks
    └── rockpaperscissors.ipynb
```

El fichero `Dockerfile` tiene el siguiente contenido:

```
#versión de python compatible con experta
FROM python:3.6
#libreria de python para sistemas expertos
RUN pip3 install experta
#libreria para usar notebooks en python
RUN pip3 install notebook
#creamos la carpeta de trabajo
WORKDIR /home/MIA2526
#ejecutamos el jupyter notebook en el puerto 8888
CMD jupyter notebook --allow-root --ip=0.0.0.0 --port=8888 --no-browser
```

Ahora, para el `docker-compose.yml` tendremos:

```
services:
  experta:
    container_name: experta #bautizamos a nuestro contenedor
    build: . #con esta linea le indicamos que busque el Dockerfile, en lugar de buscar una imagen en un repositorio
    ports:
      - "8888:8888" #publicamos el puerto para que sea accesible desde fuera
    volumes:
      - ./notebooks:/home/MIA2526 #aquí asignamos la carpeta local notebooks con la carpeta de trabajo del contenedor
```

Y por último necesitamos el notebook para hacer pruebas, en nuestro caso es un juego de piedra, papel o tijeras:

rockpaperscissors.ipynb este fichero debemos guardarlo en la carpeta `notebooks`

Ahora con toda la estructura lista y desde la raíz de la carpeta (donde estan el docker-compose.yml y el Dockerfile) ejecutamos:

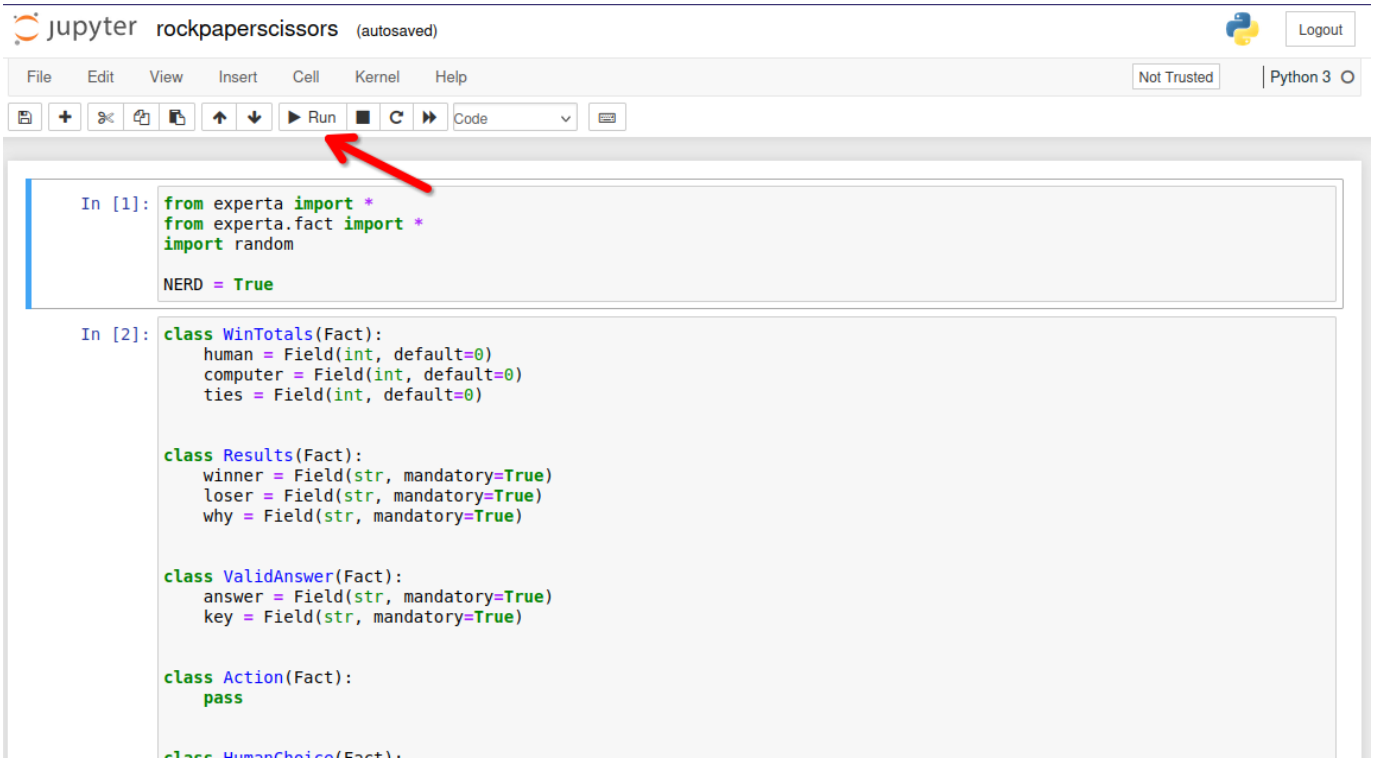
```
$ docker-compose up
[+] Running 1/1
 ✔ Container experta  Recreated                                                                    0.1s
Attaching to experta
experta  | [I 16:30:08.568 NotebookApp] Writing notebook server cookie secret to /root/.local/share/jupyter/runtime/notebook_cookie_secret
experta  | [I 16:30:08.784 NotebookApp] Serving notebooks from local directory: /home/MIA2324
experta  | [I 16:30:08.784 NotebookApp] Jupyter Notebook 6.4.10 is running at:
experta  | [I 16:30:08.784 NotebookApp] http://e0ec65ac1d84:8888/?token=825b1ba76502787821bc045496e3429bca02ba09720a835b
experta  | [I 16:30:08.784 NotebookApp]  or http://127.0.0.1:8888/?token=825b1ba76502787821bc045496e3429bca02ba09720a835b
experta  | [I 16:30:08.784 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
experta  | [C 16:30:08.787 NotebookApp]
experta  |
experta  |     To access the notebook, open this file in a browser:
experta  |         file:///root/.local/share/jupyter/runtime/nbserver-7-open.html
experta  |     Or copy and paste one of these URLs:
experta  |         http://e0ec65ac1d84:8888/?token=825b1ba76502787821bc045496e3429bca02ba09720a835b
experta  |      or http://127.0.0.1:8888/?token=825b1ba76502787821bc045496e3429bca02ba09720a835b
```

Ahora podemos hacer click directamente sobre el enlace a http://127.0.0.1:8888/?token=bebf660273e8e168c7fec90978ed56fb50db6b08d915cb14 donde veremos nuestro jupyter notebook:

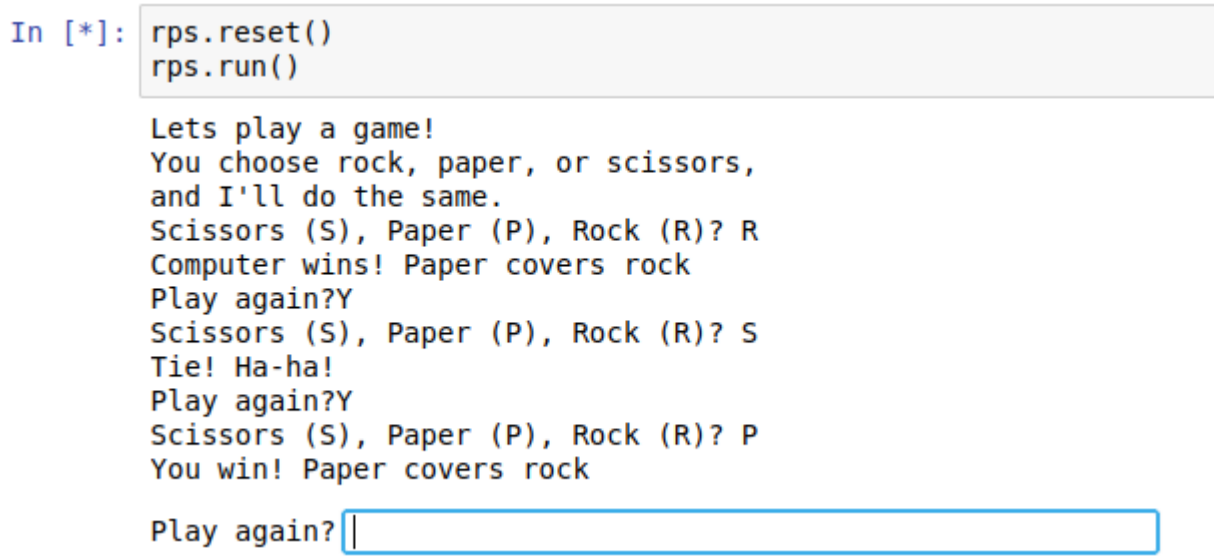


Si hacemos click sobre el notebook `rockpaperscissors.ipynb` podremos ver:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Después de pulsar varias veces (para ir ejecutando todas las celdas) podremos ver como evoluciona nuestro juego:



Para detener el contenedor que hemos lanzado con `docker-compose`, solo hemos de pulsar `^ Ctrl` + `C`

> **Actualizado respecto al material antiguo**
>
> El `Dockerfile` de partida fijaba `python:3.6`, fuera de soporte desde 2021. La versión del curso usa **`python:3.12-slim`** e instala las dependencias desde `requirements-ia.txt`. Los ficheros listos para usar están en `entorno/`, junto a esta unidad. Con Python 3.12, `experta` necesita el parche del apartado 6.5.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

10. Copias de seguridad de imágenes y contenedores (RA7-i)

Copias de contenedores

Ya estén encendidos o apagados, podemos realizar respaldos de seguridad de los contenedores. Utilizando la opción "*export*" empaquetará el contenido, generando un fichero con extensión "*.tar*" de la siguiente manera:

```
docker export -o fichero-resultante.tar nombre-contenedor
```

o

```
docker export nombre-contenedor > fichero-resultante.tar
```

Restauración de copias de seguridad de contenedores

Hay que tener en cuenta, antes de nada, que no es posible restaurar el contenedor directamente, de forma automática. En cambio, sí podemos crear una imagen, a partir de un respaldo de un contenedor, mediante el parámetro "*import*" de la siguiente manera:

```
docker import fichero-backup.tar nombre-nueva-imagen
```

Copias de imágenes

Aunque no tiene mucho sentido por que se bajan muy rápido, también tenemos la posibilidad de realizar copias de seguridad de imágenes. El proceso se realiza al utilizar el parámetro '*save*', que empaquetará el contenido y generará un fichero con extensión "*tar*", así:

```
docker save nombre_imagen > imagen.tar
```

o

```
docker save -o imagen.tar nombre_imagen
```

Restaurar copias de seguridad de imágenes

Con el parámetro 'load', podemos restaurar copias de seguridad en formato '.tar' y de esta manera recuperar la imagen.

```
docker load -i fichero.tar
```

11. Puntos clave de la unidad

- El curso de especialización IA y Big Data dura **600 horas / 36 ECTS** y el módulo 5071 se imparte en **90 horas** en la Comunitat Valenciana.
- El módulo 5071 desarrolla **6 RA** más el **RA7 de proyecto integrador transversal** (común a todo el curso, semanas 25-29, nota consensuada).
- **La normativa** exige alcanzar **todos los RA** del módulo; el centro lo concreta en **cada RA ≥ 5**. La nota de cada RA combina **40 % las entregas + 60 % la prueba escrita**; se pierde la evaluación continua superando el **15 % de inasistencia**.
- El entorno de trabajo usa **Aules** (Moodle), la web del curso, **Python + Jupyter** y **Docker**.
- Un **contenedor** es una instancia ejecutable de una **imagen**; Docker aporta aislamiento por procesos, ligereza y reproducibilidad frente a las máquinas virtuales.
- Los contenedores son **efímeros**: los datos se conservan con **volúmenes** o **bind mounts**.
- Un `Dockerfile` define la imagen por capas; el **orden de las instrucciones** aprovecha la caché.
- **Compose** orquesta varios servicios con un solo fichero.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

12. Glosario

Término	Definición
Contenedor	Instancia ejecutable de una imagen con su propia capa de escritura
Imagen	Plantilla de solo lectura que define el entorno (SO base + app + dependencias)
Capa	Cada instrucción del Dockerfile; las capas se reutilizan en caché
Dockerfile	Fichero que describe cómo construir una imagen
Registro	Repositorio de imágenes (p. ej. Docker Hub)
Daemon (dockerd)	Proceso de Docker que hace el trabajo pesado
CLI (docker)	Cliente de línea de comandos que envía órdenes al daemon
Bind mount	Montaje de una carpeta real del host dentro del contenedor
Volumen	Almacenamiento gestionado por Docker para persistir datos
Compose	Herramienta para orquestar varios contenedores con un fichero YAML
Servicio	Un contenedor definido dentro de un `compose.yaml`
Healthcheck	Comprobación del estado de un servicio para ordenar el arranque
RA (resultado de aprendizaje)	Capacidad que el alumnado debe demostrar al terminar un módulo
CE (criterio de evaluación)	Evidencia observable que permite comprobar un RA
FE (formación en empresa)	Fase en empresa del alumnado (opcional en este curso, según centro)

13. FAQ

¿Docker es una máquina virtual?

No. Las VMs virtualizan **hardware** (cada una con su kernel); los contenedores virtualizan el **sistema operativo** (comparten el kernel del host) y aíslan por procesos. Por eso son más ligeros y arrancan en segundos.

¿Pierdo mis datos si elimino un contenedor?

Sí, a menos que los guardes fuera: en un **volumen** o en un **bind mount**. Por eso en las prácticas montamos tu carpeta de trabajo dentro del contenedor.

¿Por qué `EXPOSE` no me deja abrir el navegador?

Porque `EXPOSE` solo **documenta** el puerto. Para que sea accesible hay que **publicarlo** con `-p` (o `ports:` en Compose).

¿Qué pasa si `docker run` me pide descargar una imagen muy grande?

Solo ocurre la **primera vez**. Después, Docker reutiliza la imagen desde la caché local. Si la imagen es demasiado grande, se puede empezar con variantes más ligeras (`-slim`) y anclar la versión.

¿El RA7 cuenta para la nota del módulo 5071?

Sí. El RA7 aporta el **20 %** de la nota final del módulo (RA1-RA6 el 80 %). Además, como es transversal, se evalúa de forma **consensuada** por todo el equipo docente del curso.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

¿Puedo instalar Docker en cualquier equipo?

Docker funciona en Linux, Windows y macOS. En este curso lo usaremos en el aula y para los talleres; si tu equipo no lo soporta, el entorno gestionado del aula será el plan B.

14. Sesiones

La unidad son **6 h en dos semanas** (1-8 de octubre), a 3 h por semana.

Semana	Fechas	Horas	Contenido	Evidencia
1	1-2 oct	3	Presentación del curso, del módulo y de la evaluación · instalación de Docker · conceptos: imagen, contenedor, registro	Docker funcionando (`docker run hello-world`)
2	5-8 oct	3	`docker run` y volúmenes · Dockerfile y Compose · contenedor de prácticas de IA	`T03` y `T04` (hecho / no hecho) y entorno del curso levantado

Nota sobre la tabla anterior.

Si el horario del módulo son 2 h + 1 h

Cada semana se parte en dos sesiones. El **bloque de 2 h** es el único que admite trabajo con contenedores (descargas, construcción de imágenes, resolución de errores); la sesión de **1 h** se dedica a los conceptos, al repaso y a cerrar el taller. La primera semana es corta: el curso empieza el jueves 1 de octubre.

A. Anexo · chuleta de comandos

Referencia rápida para consultar en clase, no para memorizar.

GESTIÓN DE IMÁGENES

```
docker image
docker history
docker inspect
docker save/load
docker rmi
```

GESTIÓN DE CONTENEDORES

```
docker attach
docker exec
docker inspect
docker kill
docker logs
docker pause/unpause
docker port
docker ps
docker rename
docker start/stop/restart
docker rm
docker run
docker stats
docker top
docker update
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

EJEMPLO

```
# Ver los contenedores que tenemos
docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

# Ver las imagenes que tenemos
docker images
REPOSITORY                          TAG       IMAGE ID       CREATED        SIZE

# Crear un contenedor con una imagen básica de debian
# Como no tenemos ninguna imagen de debian, la descarga y la ejecuta
docker run debian
# Intentamos ver el contenedor en ejecución, no aparece nada porque ya se ha cerrado
docker ps
# Podemos verlo con
docker ps -a
CONTAINER ID   IMAGE     COMMAND       CREATED              STATUS                          PORTS     NAMES
09b14daab800   debian    "bash"        2 seconds ago        Exited (0) 1 second ago                   pensive_wozniak

# Ejecutar un comando en un contenedor
docker run debian /bin/echo "Hello World"
Hello World

# Información
docker inspect debian
```

Crear contenedor interactivo y con nombre

https://jolthgs.wordpress.com/2019/09/25/create-a-debian-container-in-docker-for-development/

Para que docker no se invente un nombre como “pensive_wozniak” (comando anterior) podemos definir el nombre que queremos.

Utilizaremos una de las imágenes de: https://hub.docker.com/_/debian/tags

```
# Obtenemos la imagen, en el apartado anterior la hemos ejecutado directamente con "run", esto
# la obtiene implícitamente. En este caso la vamos a descargar.
$ docker pull debian:13-slim

# --name
# -h hostname que tendrá el contenedor
# -e codificación de caracteres
# -it modo interactivo
# /bin/bash -l  la shell que se ejecutará
$ docker run --name debian-mini -h equipo1 -e LANG=C.UTF-8 -it debian:13-slim /bin/bash -l

--- Estamos dentro del contenedor ---
# Una vez dentro del contenedor podemos actualizarlo e instalar los paquetes que creamos necesarios
apt update && apt upgrade --yes && apt install sudo locales --yes
# Configurar timezone
dpkg-reconfigure tzdata

# Vamos a nuestra home y creamos un archivo
cd
echo "hola" > prueba.txt

# Salimos del contenedor
exit (o control + d)

--- Ahora volvemos a la consola de nuestro equipo ---
$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

$ docker ps -a
CONTAINER ID   IMAGE            COMMAND          CREATED          STATUS                      PORTS     NAMES
8c6b29c818ee   debian:13-slim   "/bin/bash -l"   44 seconds ago   Exited (0) 11 seconds ago             debian-mini
```

B. Anexo · catálogo de contenedores para experimentar

MONITORIZACIÓN Y GESTIÓN

- **Netdata** - Monitorización de sistemas en tiempo real con dashboard web completo
- **Glances** - Monitorización del sistema via web que muestra información de CPU, memoria, red y procesos
- **Portainer** - Interfaz web para gestionar y administrar contenedores Docker
- **Uptime Kuma** - Monitor de disponibilidad para verificar el estado de contenedores y servicios

PROXY Y RED

- **Nginx Proxy Manager** - Proxy inverso con interfaz web para gestionar hosts virtuales y certificados SSL/TLS
- **Acestream** - Motor P2P para streaming de video, útil para integrar canales en Jellyfin

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- **Libreddit** - Interfaz alternativa para Reddit libre de publicidad y tracking (modo lectura)
- **Invidious** - Interfaz alternativa para YouTube que respeta la privacidad

MULTIMEDIA Y ENTRETENIMIENTO

- **Jellyfin** - Servidor multimedia libre para organizar y streaming de películas, series y música
- **Soulseek** - Cliente para la red P2P Soulseek especializada en compartir música
- **youtube-dl** - Herramienta para descargar videos y audio de YouTube y otros sitios
- **Photoprism** - Servicio de gestión y visualización de fotos personales con funciones de IA

PRODUCTIVIDAD Y ORGANIZACIÓN

- **Nextcloud** - Plataforma de colaboración y almacenamiento en la nube auto-alojado
- **Linkding** - Marcador social para guardar y organizar enlaces web
- **GitBucket** - Plataforma Git auto-alojada similar a GitHub
- **SearXNG** - Metabuscador privado que agrega resultados de múltiples motores de búsqueda

SEGURIDAD Y CONTRASEÑAS

- **Vaultwarden** - Implementación alternativa del gestor de contraseñas Bitwarden, más ligera y eficiente

AUTOMATIZACIÓN DEL HOGAR

- **Home Assistant** - Plataforma de domótica de código abierto para automatizar y controlar dispositivos del hogar
- **Homebridge** - Puente que permite integrar dispositivos no compatibles con el ecosistema Apple HomeKit
- **Camera.UI** - Aplicación para gestionar y visualizar cámaras de seguridad con integración HomeKit

UTILIDADES Y MANTENIMIENTO

- **Homepage** - Dashboard personalizable como página de inicio para acceder a todos los servicios
- **Watchtower** - Servicio que actualiza automáticamente los contenedores Docker cuando hay nuevas versiones disponibles

> **Uso de este catálogo**
>
> Sirve para elegir imagen en la fase 4 del Taller 3. Antes de proponer una, comprueba en Docker Hub que sigue mantenida: hay imágenes populares con años sin actualizar.

15. Recursos

- Diapositivas
- **Práctica** — se hace, no se entrega ni puntúa:
  - Ejercicios de autoevaluación
  - Notebooks guiados — `N01`, entorno Python para IA
- **Entregas** — se entregan en Moodle y se califican **hecho / no hecho**: no llevan nota, pero son requisito para las prácticas de la UD02:
  - T01 · Control de versiones con GitHub
  - T02 · Documentar con Markdown
  - T03 · Verificación del entorno y primer contenedor
  - T04 · Contenedor de prácticas de IA
- Los notebooks se abren desde **Práctica** y **Entregas**, con descarga y apertura en Colab.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Referencias de la unidad

Documentación oficial de Docker:

- What is Docker?
- What is a container?
- Dockerfile reference
- Compose Quickstart
- Building best practices

Normativa y plataforma:

- RD 279/2021 — currículo del curso de especialización
- Orden 8/2025 de la Comunitat Valenciana — evaluación
- Aules GVA — plataforma del centro

16. Evaluación

- **Entregas de la unidad**: el informe de los talleres 1 y 2, en Moodle. Se califican **hecho / no hecho**; son requisito para las prácticas de la UD02, pero **no puntúan** ni tienen ítem en el libro de calificaciones.
- **Evaluación inicial**: diagnóstica, sin nota, antes del segundo mes lectivo.
- **La normativa exige alcanzar todos los RA** del módulo para superarlo (art. 5.1 de la Orden 8/2025: la calificación del módulo está «*en función de la consecución de los RA*»; y las Instrucciones 26-27, que impiden calificar positivamente un módulo con RA no superados). El centro concreta ese mandato exigiendo **≥ 5 en cada RA**.

17. Recuperación

Si no superas la tarea de esta unidad, se activará un **programa de recuperación individual** (art. 14.4 Orden 8/2025) con actividades y criterios de evaluación específicos para el RA correspondiente.

Volver al índice

19 de octubre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

2.2 Diapositivas de la UD00



Diapositivas PDF

Cómo se manejan

Flechas o clic para avanzar · **F** para verlas a pantalla completa · **P** para las notas del ponente.

24 de agosto de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

3. UD01

3.1 UD01 — Caracterización de sistemas de IA

Unidad 1 · 12 h · semanas 3-6 (12 de octubre al 6 de noviembre)

1. Resultado de aprendizaje y criterios de evaluación

RA1 — Caracteriza sistemas de Inteligencia Artificial relacionándolos con la mejora de la eficiencia operativa de las organizaciones y empresas.

CE	Criterio de evaluación
RA1-a	Se han identificado los principios fundamentales de los sistemas inteligentes.
RA1-b	Se ha recopilado información sobre campos donde se aplica Inteligencia Artificial.
RA1-c	Se han identificado las técnicas básicas a utilizar en el entorno de la IA.
RA1-d	Se han identificado nuevas formas de interacciones en los negocios que mejoran la eficiencia operativa.

Nota sobre la tabla anterior.

Bloque de contenidos oficial (RD 279/2021, anexo I)

El currículo para este RA dice textualmente:

Caracterización de sistemas de Inteligencia Artificial:

- *Principios de los sistemas inteligentes.*
- *Campos de aplicación.*
- *Técnicas de la Inteligencia Artificial.*
- *Nuevas formas de interacción.*

2. Objetivos de la unidad

Objetivo	Descripción
O1	Explicar qué es un sistema inteligente y cuáles son sus principios fundamentales.
O2	Distinguir las escuelas de pensamiento y las clasificaciones de la IA (débil/fuerte, Russell-Norvig, Hintze).
O3	Distinguir IA, aprendizaje automático, aprendizaje profundo e IA generativa.
O4	Enumerar los principales campos de aplicación de la IA con ejemplos reales.
O5	Distinguir las técnicas básicas de la IA y saber qué problema resuelve cada una.
O6	Relacionar las nuevas formas de interacción (asistentes, chatbots, visión, voz) con la mejora de la eficiencia operativa.
O7	Identificar en casos prácticos qué técnica usar y qué mejora operativa aporta.

3. Fundamentos de los sistemas inteligentes (RA1-a)

3.1 ¿Qué es la inteligencia artificial?

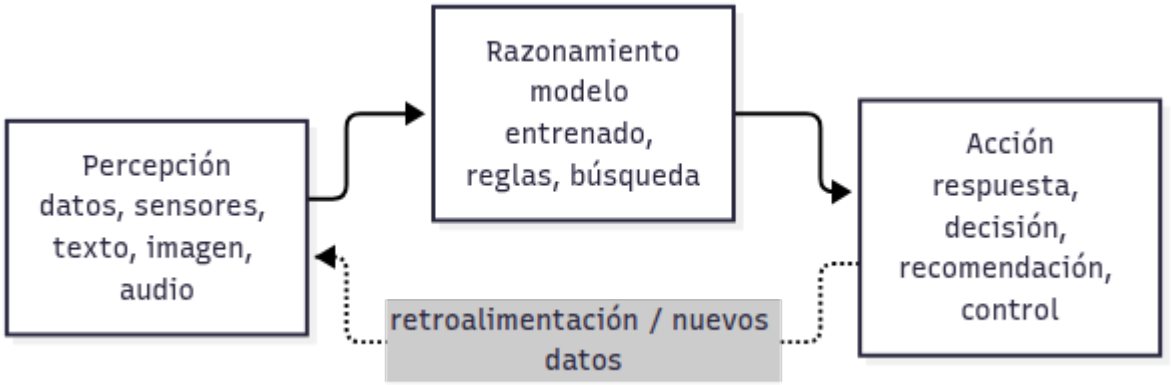
La **inteligencia artificial (IA)** es la tecnología que permite a las máquinas **simular el aprendizaje, la comprensión, la resolución de problemas, la toma de decisiones y la creatividad** humanas. Las aplicaciones dotadas de IA pueden ver e

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

identificar objetos, entender y responder al lenguaje, aprender de la experiencia, recomendar decisiones e incluso **actuar de forma autónoma** (p. ej. un coche autónomo).

3.2 El ciclo percepción → razonamiento → acción

Un **sistema inteligente** percibe su entorno, razona sobre esa información y actúa para conseguir un objetivo:



- **Percepción**: captar datos del mundo (texto, imagen, audio, sensores, registros de negocio).
- **Razonamiento**: procesar los datos para obtener conocimiento o tomar decisiones (un modelo entrenado, un conjunto de reglas, una búsqueda).
- **Acción**: actuar sobre el entorno o sobre las personas (responder, recomendar, controlar un proceso).

3.3 Características de un sistema inteligente

Característica	Descripción
Autonomía	Opera sin supervisión humana constante
Adaptación	Aprende de los datos y mejora con la experiencia
Toma de decisiones	Recomienda o actúa basándose en datos, no solo en reglas fijas

Nota sobre la tabla anterior.

> **IA basada en reglas vs. IA que aprende**
>
> La IA **más elemental** puede ser un conjunto de reglas `si... entonces...` programadas a mano (un termostato que enciende la calefacción si baja de 18 ºC es, formalmente, un sistema de IA basado en reglas). Pero cuando la tarea se complica, definir todas las reglas es imposible: ahí entra el **aprendizaje automático**, que deduce los patrones de los datos.

3.4 IA débil frente a IA fuerte

La clasificación más simple de la IA es **según la tarea que resuelve**: hay tareas concretas y acotadas (jugar al ajedrez) y tareas abiertas que exigen contexto, ética o creatividad (gestionar una cocina sin intervención humana). De ahí surgen dos categorías:

- **IA débil o estrecha (narrow/weak)**: diseñada para **una tarea o un conjunto limitado de tareas**. Es **reactiva** (no actúa si no se la activa), **no flexible** (colapsa ante lo no previsto), queda **limitada por lo que programó una persona** y **no tiene conciencia**: computa, no razona en sentido humano. Los asistentes virtuales (Siri, Alexa) son el ejemplo típico — operan dentro de un rango de respuestas definido en su base de datos y, fuera de él, dan respuestas inadecuadas o directamente no responden. **Es toda la IA que existe hoy.**
- **IA general o fuerte (AGI/strong)**: igualaría o superaría la inteligencia humana en **cualquier** tarea intelectual, sería **proactiva**, **flexible** (aprendería una tarea nueva a partir de una parecida) y se **autorregularía**. **Es puramente teórica**: ningún sistema actual se aproxima, más allá de la ficción (T-800, Wall-E, J.A.R.V.I.S.).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

> ⚠ **La IA débil también tiene riesgos**
>
> Precisamente por no considerar un contexto amplio ni reglas sociales o éticas, la IA débil ejecuta su tarea con eficacia y sin matices: no evalúa consecuencias como lo haría una persona. Es una tecnología incompleta y potencialmente peligrosa si se usa sin prudencia, o si quien la programa busca causar daño — el motivo de fondo de la UD06.

3.5 Escuelas de pensamiento y clasificaciones de la IA

Además de por la tarea, la IA se puede clasificar por **cómo llega a sus resultados** (la escuela de pensamiento) o por **qué capacidades tiene** (Russell-Norvig, Hintze). Ninguna de las tres es «la correcta»: son lentes distintas sobre el mismo campo.

DOS ESCUELAS: CONVENCIONAL Y COMPUTACIONAL

	IA convencional (simbólico-deductiva)	**IA computacional (subsimbólico-inductiva)**
Cómo razona	Análisis formal y estadístico explícito	Aprendizaje interactivo a partir de datos empíricos
Técnicas	Razonamiento basado en casos, sistemas expertos, redes bayesianas	Redes neuronales, máquinas de vectores soporte, sistemas difusos, computación evolutiva
Qué originó	La «automatización» clásica (reglas + estadística)	El **aprendizaje automático** actual

Con el auge del *machine learning*, buena parte de lo que hacía la escuela convencional se ha ido llevando al campo computacional — no son compartimentos estancos.

CLASIFICACIÓN DE RUSSELL Y NORVIG (1995)

En *Artificial Intelligence: A Modern Approach* — el libro de texto de IA más usado en el mundo —, Stuart Russell y Peter Norvig proponen cuatro categorías según el **origen del comportamiento inteligente**:

Categoría	**Enfoque**	**Ejemplo de aplicación**
Sistemas cognitivos	Piensan como humanos: emulan el proceso de decisión	Modelos cognitivos, aprendizaje
Test de Turing	Actúan como humanos, sin pasar por el razonamiento	Robótica, actuadores en el mundo físico
Leyes del pensamiento	Piensan con lógica formal, sin excepciones	Sistemas expertos (aproximaciones acotadas)
Agentes racionales	Actúan racionalmente, sin razonamiento lógico explícito	Agentes de software actuales

Las dos últimas exigen una capacidad de cómputo que, para el caso general, todavía es inalcanzable.

LOS TESTS DE TURING Y DE LOVELACE: ¿DEMUESTRAN QUE UN SISTEMA ES INTELIGENTE?

La fila «Test de Turing» de la tabla anterior remite a una propuesta muy concreta. **Alan Turing** planteó en 1950, en *Computing Machinery and Intelligence*, el **juego de la imitación**: un interrogador humano conversa por escrito con una persona y con una máquina, sin saber cuál es cuál; si tras el interrogatorio no logra distinguirlas de forma fiable, la máquina **pasa el test**. Es una prueba **conductual**: no exige que la máquina piense, solo que actúe de forma indistinguible de como lo haría una persona.

Casi un siglo antes, **Ada Lovelace** ya había puesto en duda que eso bastara. En 1843, en la «Nota G» que acompañaba su traducción de un trabajo sobre la máquina analítica de Babbage, escribió que *«la máquina analítica no tiene pretensión de originar nada; puede hacer cualquier cosa que sepamos ordenarle que ejecute»* — para Lovelace, una máquina solo repite lo que alguien programó, nunca **origina** algo genuinamente nuevo. El propio Turing cita esta «objeción de Lovelace» en su artículo de 1950 y responde que una máquina sí podría «sorprendernos», sin zanjar el debate.

Esa objeción, no la conversación, es lo que en 2001 formalizaron Selmer Bringsjord, Paul Bello y David Ferrucci en el **test de Lovelace**: un sistema lo supera si produce un resultado que su propio programador **no puede explicar cómo se generó** a partir del código que escribió — un test sobre **creatividad y originalidad**, no sobre saber conversar. En 2014, Mark Riedl propuso una versión más manejable, el **test de Lovelace 2.0**: pedir al sistema artefactos creativos concretos (un relato, un poema, una imagen) que cumplan unos requisitos dados por un evaluador humano, que juzga si el resultado los satisface — más fácil de repetir y de comparar entre sistemas que el criterio original.

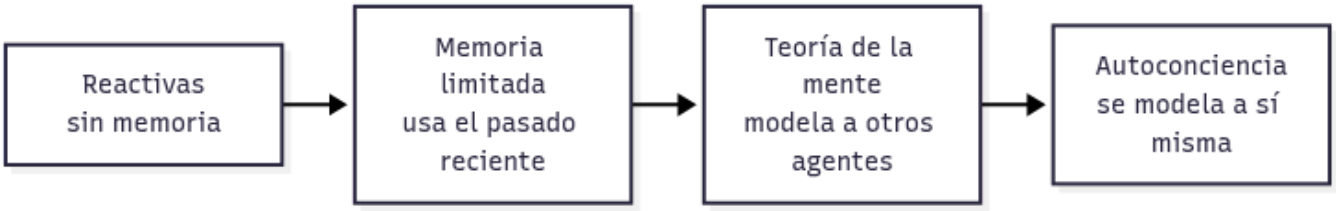
© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Por qué importan los dos, no solo el de Turing

El test de Turing mide si una máquina puede **imitar** una conversación humana. El de Lovelace mide algo distinto: si puede **originar** algo que no es una consecuencia previsible de su programación. Superar el primero no implica superar el segundo — y es justo la pregunta de fondo al hablar de IA fuerte (§3.4): ¿la creatividad de una IA generativa es real, o solo una recombinación muy sofisticada de lo que ya ha visto?

CLASIFICACIÓN DE HINTZE (2016)

Arend Hintze (Universidad de Michigan) propuso una clasificación por **capacidades**, de la más simple a la más avanzada — la más citada para explicar hacia dónde evoluciona la IA:



- **Máquinas reactivas**: sin memoria ni capacidad de usar experiencia pasada. **Deep Blue** (IBM, venció a Kasparov en 1997) es el ejemplo perfecto: evalúa el tablero en tiempo real, sin ningún concepto de lo ocurrido antes.
- **Memoria limitada**: usan observaciones recientes para decidir, sin guardarlas a largo plazo. Los **vehículos autónomos** actuales encajan aquí: memorizan velocidad y trayectoria de otros coches para decidir un cambio de carril, pero esa información es transitoria.
- **Teoría de la mente** *(teórica)*: formaría representaciones sobre lo que piensan y sienten otros agentes — la base de la interacción social. Ningún sistema actual llega a esto.
- **Autoconciencia** *(teórica)*: el sistema se representaría a sí mismo, conocería sus propios estados internos. Es el paso final, y el más lejano, del desarrollo de la IA.

Cómo no perderse entre tres clasificaciones

Si te preguntan «¿qué tipo de IA es esto?», identifica primero la **tarea** (débil casi siempre, hoy), después la **escuela** (¿reglas explícitas o aprendida de datos?) y por último, si aplica, el **nivel de Hintze** (¿tiene memoria de lo que ha visto?). Las tres respuestas pueden convivir para el mismo sistema.

3.6 Breve historia de la IA

Año	Hito
1950	Alan Turing publica *Computing Machinery and Intelligence* y propone el **Test de Turing**
1956	Dartmouth: John McCarthy acuña el término **inteligencia artificial**
1958	Frank Rosenblatt publica el **perceptrón**, primera red neuronal que aprende (concebido en 1957; máquina Mark I Perceptron construida entre 1958 y 1960)
1997	Deep Blue (IBM) vence al campeón de ajedrez Kasparov
2011	Watson (IBM) gana en *Jeopardy!*; emerge la ciencia de datos
2015	Google libera **TensorFlow** como código abierto, democratizando las herramientas de deep learning
2016	AlphaGo (DeepMind) vence en Go (más de 14,5 billones de jugadas tras 4 movimientos)
2017	**Vaswani et al. publican *Attention Is All You Need*** y proponen la arquitectura **Transformer**, basada solo en mecanismos de atención: elimina la recurrencia y las convoluciones. Es la base técnica de los LLM y de la IA generativa posterior
2022	Los **grandes modelos de lenguaje (LLM)**, p. ej. ChatGPT, cambian la industria
2024-26	Modelos **multimodales** y **agentes de IA** autónomos

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

3.7 Tipos de IA según su forma

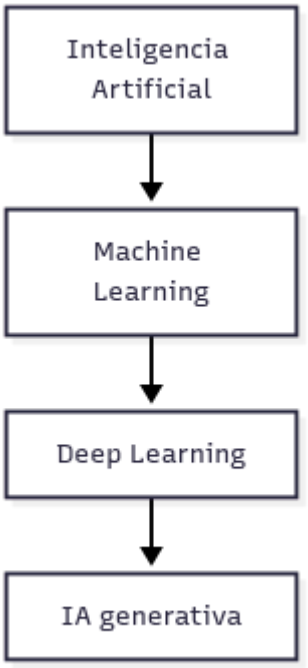
La IA se presenta en dos formas generales:

- **IA de software**: programas que procesan información sin cuerpo físico (asistentes virtuales, buscadores, análisis de imágenes, reconocimiento de voz y rostro, motores de traducción).
- **IA "encarnada" (embodied)**: sistemas con presencia física que interactúan con el mundo (robots, coches autónomos, drones, internet de las cosas).

Muchas aplicaciones combinan ambas: un coche autónomo usa visión y decisiones de software dentro de un vehículo físico.

4. IA, machine learning, deep learning e IA generativa (RA1-c)

La relación entre estos términos se representa como cajas anidadas:



Regla para recordar

Todo machine learning es IA, pero no toda IA es machine learning. Y todo deep learning es machine learning, y la IA generativa es una parte del deep learning.

4.1 Aprendizaje automático (machine learning, ML)

El ML crea **modelos** entrenando un algoritmo sobre datos para hacer predicciones o tomar decisiones **sin ser programado explícitamente** para cada caso. Su objetivo central es la **generalización**: que el modelo acierte con datos **nuevos**, no solo con los de entrenamiento.

En lugar de programar reglas manuales (filtro de spam por criterios a mano), el modelo **aprende** de ejemplos etiquetados y deduce los criterios por sí mismo.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

CÓMO FUNCIONA UN MODELO DE ML, PASO A PASO



1. **Representar los datos numéricamente**: cada ejemplo se convierte en un vector de *features* (características). P. ej., una casa se representa como `[superficie, habitaciones, edad]`.
2. **Entrenar**: el algoritmo ajusta sus **parámetros** (los pesos de la fórmula) para que el error entre su salida y la respuesta correcta sea mínimo. Se usa una **función de pérdida** y un optimizador (gradiente descendente).
3. **Evaluar**: se comprueba el rendimiento con **datos que el modelo no ha visto** (test), no solo con los de entrenamiento, para asegurar que **generaliza** y no memoriza (sobreajuste).
4. **Inferir**: en producción, el modelo predice sobre datos nuevos (a esto se le llama *AI inference*).

> **Ejemplo con números**
>
> Para predecir el precio de una casa: `Precio = A·superficie + B·habitaciones − C·edad + base`. El objetivo del ML es encontrar los valores de `A`, `B`, `C` y `base` que minimizan el error con las ventas conocidas.

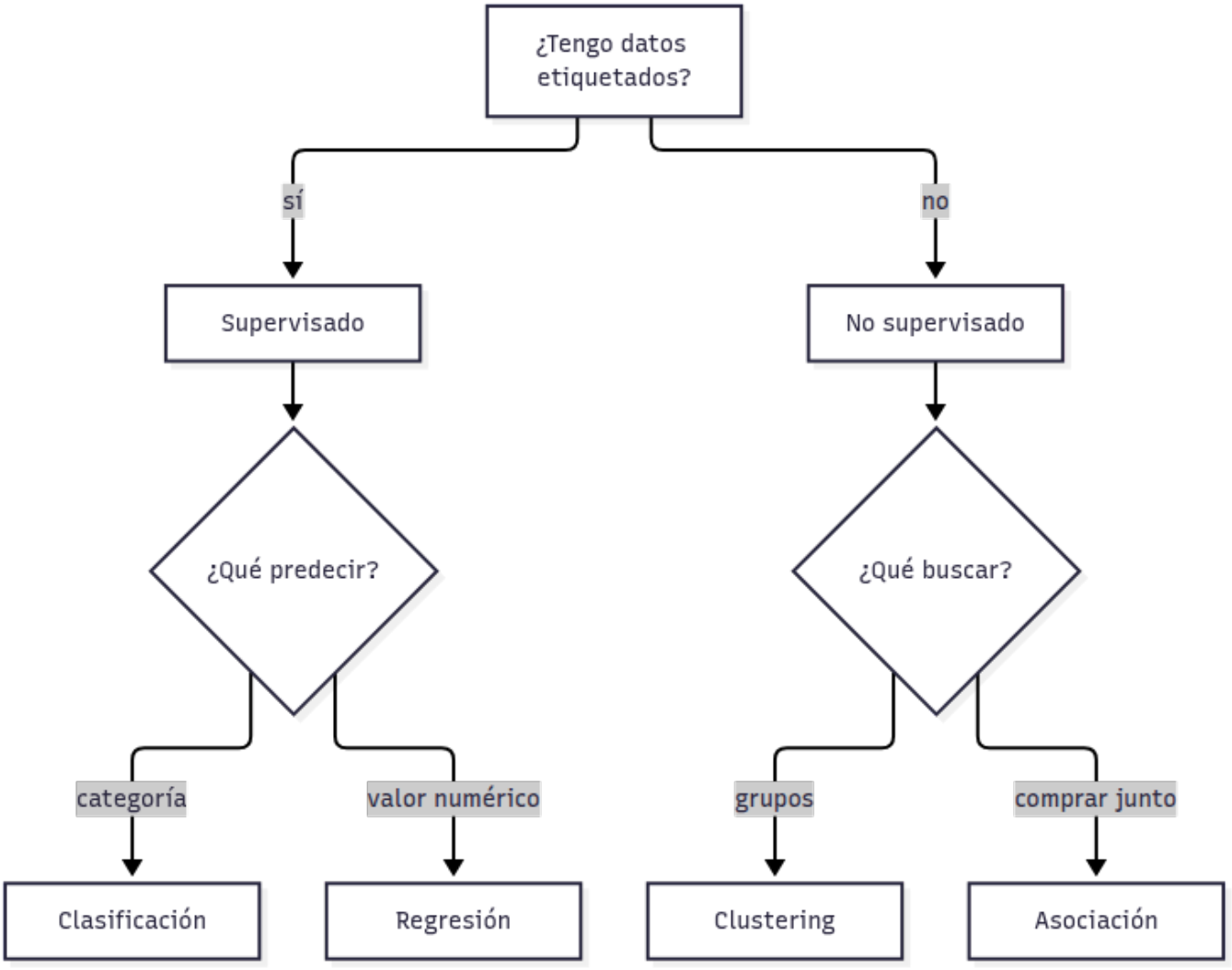
4.2 Tipos de aprendizaje

Tipo	Datos	Objetivo	Ejemplos
Supervisado	Etiquetados (con respuesta)	Predecir la respuesta de datos nuevos	Clasificación (spam/no spam), regresión (precio)
No supervisado	Sin etiquetas	Encontrar estructura oculta	Clustering (segmentar clientes), asociación, reducción de dimensionalidad
Refuerzo (RL)	Interacción con el entorno	Maximizar recompensa por prueba-error	Robots, juegos, control
Semi-supervisado	Muchos sin etiquetar + pocos etiquetados	Combinar ambos	Cuando etiquetar es caro
Auto-supervisado	Sin etiquetas humanas	Aprender de la propia estructura (p. ej. ocultar palabras y predecirlas)	Entrenamiento de LLM

- **Supervisado** → clasificación (categorías) y regresión (valores continuos). Algoritmos típicos: árboles de decisión, k-vecinos (KNN), naive Bayes, SVM, regresión logística, random forest.
- **No supervisado** → clustering (k-means, DBSCAN), detección de anomalías, PCA.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

4.2.1 ¿QUÉ ALGORITMO ELEGIR?



Un vistazo a los algoritmos más usados (todos disponibles en `scikit-learn`):

Algoritmo	Tipo	Cómo funciona (resumen)	Ejemplo de uso
k-vecinos (KNN)	Supervisado	Clasifica según los k ejemplos más cercanos	Recomendar producto similar
Árbol de decisión	Supervisado	Serie de preguntas `si/entonces` aprendidas de los datos	Decidir aprobar un préstamo
Naive Bayes	Supervisado	Probabilidades con supuesto de independencia	Clasificar spam / análisis de sentimiento
Regresión logística	Supervisado	Probabilidad de pertenecer a una clase	Predecir abandono de cliente
Random forest	Supervisado	Combinación de muchos árboles	Mantenimiento predictivo
k-means	No supervisado	Agrupa en k grupos por cercanía al centro	Segmentar clientes
DBSCAN	No supervisado	Agrupa por densidad, detecta anomalías	Detectar transacciones atípicas

Nota sobre la tabla anterior.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Primer contacto con scikit-learn

Todos estos algoritmos comparten la misma interfaz `fit` / `predict`, y se evalúan dividiendo los datos en **entrenamiento y prueba** (`train_test_split`). Por ejemplo, para clasificar especies con el dataset *Iris*:

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

modelo = DecisionTreeClassifier()
modelo.fit(X_train, y_train)          # entrena con los datos etiquetados
precision = modelo.score(X_test, y_test)  # evalúa con datos nuevos
print("Precisión:", precision)
```

Este mismo patrón (`fit` → `score`) se repetirá en las prácticas de la UD02 y siguientes; en esta unidad solo necesitas reconocerlo.

4.3 Aprendizaje profundo (deep learning, DL)

El DL usa **redes neuronales artificiales con muchas capas** (entrada, capas ocultas, salida). Cada conexión tiene un **peso** y cada neurona una **activación no lineal**: la red puede modelar patrones muy complejos. Se entrena con **backpropagation + gradiente descendente** y necesita **muchos datos y GPU**.

- **CNN**: redes convolucionales, especializadas en imágenes (visión).
- **RNN/LSTM**: redes recurrentes, pensadas para secuencias (texto, series temporales).
- **Transformers** (2017): arquitectura con **mecanismo de atención**; base de los LLM y de casi toda la IA moderna.
- **Mamba** (2023): arquitectura alternativa basada en *state space models*.

Ventajas del DL: aprende representaciones complejas sin extraer características a mano. Inconvenientes: necesita muchos datos y cómputo, y es menos explicable.

4.4 IA generativa

La **IA generativa** (gen AI) crea contenido nuevo (texto, imágenes, vídeo, audio) a partir de un **prompt**. Funciona en tres fases:

1. **Entrenamiento**: se crea un **modelo de base** (foundation model) entrenado con enormes volúmenes de datos (texto de internet, imágenes...). Los **LLM** (ChatGPT, GPT-4, BERT, Llama, Gemini) son el ejemplo más común.
2. **Ajuste (tuning)**: se adapta a la tarea con *fine-tuning* o *RLHF* (refuerzo con feedback humano).
3. **Generación y evaluación**: se genera y se mejora de forma continua; técnicas como **RAG** conectan el modelo a fuentes de datos externas para mayor precisión.

Agentes de IA / agentic AI: programas autónomos que **diseñan su propio flujo de trabajo** y usan herramientas (apps, servicios) para lograr objetivos. Es el paso natural después de la IA generativa: no solo responden, **actúan**.

5. Campos de aplicación de la IA (RA1-b)

Campo	Ejemplos de aplicación	Beneficio típico
Industria y logística	Mantenimiento predictivo, optimización de rutas, control de calidad visual	Menos paradas, menos costes
Salud	Diagnóstico asistido por imagen, triaje de pacientes, descubrimiento de fármacos	Mayor precisión, menos errores
Finanzas y banca	Detección de fraude, scoring crediticio, atención al cliente	Menos pérdidas, más velocidad
Comercio y retail	Recomendación de productos, previsión de demanda, chatbots de venta	Más ventas, menos stock roto
Marketing	Segmentación de clientes, análisis de sentimiento, personalización	Campañas más rentables
Educación	Tutoría adaptativa, corrección automática, análisis de abandono	Menos abandono, más personalización

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Campo	Ejemplos de aplicación	Beneficio típico
Administración	Automatización de trámites, asistentes virtuales	Menos tiempos y costes
Transporte	Conducción asistida, gestión del tráfico, mantenimiento de flotas	Seguridad y eficiencia
Recursos humanos	Cribado de CV, emparejado con ofertas, entrevistas asistidas	Menos tiempo de contratación

5.1 La IA en la vida cotidiana

Muchas tecnologías que usas a diario son IA, aunque no lo parezca (Parlamento Europeo):

Uso cotidiano	Cómo interviene la IA
Compras online y publicidad	Recomendaciones personalizadas según búsquedas y compras; optimización de inventario y logística
Buscadores	Aprenden de los datos para devolver resultados relevantes
Asistentes de voz	Responden, recomiendan y organizan rutinas
Traducción automática	Traduce texto y voz; subtítulos automáticos
Hogar y ciudad inteligente	Termostatos que aprenden del comportamiento; regulación del tráfico
Coches	Asistentes de seguridad, navegación
Ciberseguridad	Detecta y combate ciberataques reconociendo patrones
Desinformación	Detecta noticias falsas analizando fuentes y lenguaje
Agricultura	Monitoriza animales, riego y fertilizantes para reducir costes e impacto ambiental
Administración pública	Alertas tempranas de catástrofes, trámites automatizados

5.2 Casos de estudio con beneficio medible

Caso 1 · Detección de fraude en banca (RA1-b, RA1-c, RA1-d) Un banco entrena un modelo de ML con **datos históricos etiquetados** (transacción fraudulenta o no). El modelo analiza en tiempo real importe, ubicación, horario y hábitos de cada cliente. *Antes*: revisión manual posterior a la pérdida. *Después*: alerta inmediata y bloqueo preventivo. **Mejora operativa**: menos pérdidas económicas y menor fricción para clientes legítimos.

Caso 2 · Mantenimiento predictivo en industria (RA1-b, RA1-c) Sensores en máquinas envían datos (temperatura, vibración, horas de uso). Un modelo de ML predice cuándo fallará un equipo antes de que ocurra. *Antes*: averías inesperadas y paradas de línea. *Después*: sustitución programada. **Mejora operativa**: menos paradas, menos costes de reparación, mayor disponibilidad.

Caso 3 · Atención al cliente con PLN (RA1-c, RA1-d) Un **chatbot** con PLN responde consultas frecuentes (estado de pedido, devoluciones) y deriva las complejas a personas. *Antes*: colas y horario limitado. *Después*: atención 24/7 e inmediata. **Mejora operativa**: menor coste por consulta, más satisfacción, agentes dedicados a casos difíciles.

Caso 4 · Salud: detección precoz (RA1-b) Un programa con IA analiza llamadas de emergencia para **reconocer un paro cardíaco** durante la llamada, más rápido que un operador humano; también se usa visión para detectar infecciones en TAC pulmonar. **Mejora operativa**: menor tiempo de respuesta y mayor precisión en el diagnóstico.

6. Nuevas formas de interacción y eficiencia operativa (RA1-d)

6.1 Nuevas interacciones

Interacción	Qué es	Ejemplo
Asistente virtual	Responde por voz o texto a peticiones del usuario	Siri, Alexa, asistentes corporativos
Chatbot	Conversación automatizada en web/ mensajería	Chat de soporte de una tienda online

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Interacción	Qué es	Ejemplo
Interacción por voz	Transcribe y analiza audio	Transcripción de reuniones, análisis de llamadas
Interacción por visión	Lee e interpreta imágenes/vídeo	Lectura de documentos, control de accesos, inspección visual
Agentes autónomos	Ejecutan tareas y usan herramientas por sí solos	Reservar vuelo, tramitar una incidencia

EL PLN DETRÁS DE LAS INTERACCIONES POR TEXTO Y VOZ

El **procesamiento del lenguaje natural (PLN)** es la técnica de IA que permite entender y generar lenguaje humano — se estudia a fondo en la UD03. Detrás de un chatbot, un asistente de voz o un análisis de opiniones hay varias tareas de PLN:

Tarea de PLN	Qué hace	Ejemplo
Análisis de sentimiento	Determina si un texto es positivo, negativo o neutro	Valorar opiniones de clientes
Reconocimiento de entidades (NER)	Identifica nombres, lugares, fechas	Extraer `María`, `Madrid`, `05/05` de un texto
Clasificación de texto	Asigna un texto a una categoría	Spam, tipo de incidencia, idioma
Traducción automática	Convierte texto de un idioma a otro	Documentos, subtítulos
Resumen de texto	Condensa documentos largos	Resumir informes y artículos
Reconocimiento de voz	Convierte audio en texto	Transcripción de llamadas

Un *pipeline* de PLN típico es: **preprocesado** (tokenización, minúsculas, eliminación de palabras vacías, lematización) → **extracción de características** (bolsa de palabras, TF-IDF, embeddings) → **modelo** (ML o redes profundas, p. ej. *transformers* como BERT) → **salida** (clasificación, traducción, respuesta). En el notebook de la unidad verás un ejemplo de sentimiento con `scikit-learn`.

6.2 Eficiencia operativa

La IA mejora la **eficiencia operativa** cuando reduce **costes, tiempos o errores** en un proceso. Indicadores que suelen mejorar:

Caso	Antes (sin IA)	Después (con IA)
Atención al cliente	Colas, horario limitado	Chatbot 24/7, respuesta inmediata
Inspección de calidad	Revisión manual, errores	Visión artificial: más rápida y constante
Previsión de demanda	Roturas de stock o excedentes	Predicción con ML: menos pérdidas
Detección de fraude	Revisión posterior	Detección en tiempo real

6.3 CÓMO MEDIR LA MEJORA (KPIS)

Para saber si una solución de IA merece la pena, se compara el proceso **antes y después** con indicadores objetivos:

Indicador	Qué mide
Tiempo de ciclo	Cuánto tarda una operación (respuesta, inspección, trámite)
Coste unitario	Coste por consulta, por pedido, por transacción
Tasa de error	Errores humanos o del proceso por cada N operaciones
Disponibilidad	Horas al día en que el servicio está operativo
Rendimiento	Operaciones por unidad de tiempo o por empleado

Según el tipo de proceso, se usan KPIs específicos que conviene conocer por su nombre, porque aparecen en informes y ofertas de las empresas de IA:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Ámbito	KPI	Qué mide
Atención al cliente	**FCR** (*first contact resolution*)	% de consultas resueltas en el primer contacto
Atención al cliente	**AHT** (*average handling time*)	Tiempo medio de gestión de una consulta
Atención al cliente	**Containment rate**	% de consultas cerradas por la IA sin intervención humana
Industria	**OEE** (*overall equipment effectiveness*)	Disponibilidad × rendimiento × calidad de una máquina
Industria	**MTBF** (*mean time between failures*)	Tiempo medio entre dos averías
Operaciones	**Cycle time**	Tiempo total de una operación de principio a fin
Operaciones	**Coste por documento**	Coste de procesar cada documento (contrato, factura, email)

Nota sobre la tabla anterior.

> **KPI ≠ mejora anecdótica**
>
> Decir "el chatbot funciona muy bien" no sirve para justificar una inversión. Lo que vale es un KPI comparado: "*la tasa de resolución en el primer contacto pasó de X a Y*" o "*el tiempo medio de gestión bajó de 5 minutos a 2*". En el ejemplo siguiente se hace exactamente eso.

Ejemplo resuelto: un centro de atención recibe 1.000 consultas/día, cada una cuesta 3 € y tarda 5 min. Un chatbot resuelve el 60 % de las consultas en 10 segundos y a un coste de 0,10 €.

- Coste diario antes: 1.000 × 3 € = **3.000 €**.
- Coste diario después: 600 × 0,10 € + 400 × 3 € = 60 € + 1.200 € = **1.260 €**.
- **Reducción de coste ≈ 58 %** y de tiempo medio de atención de 5 min a ~2 min.

Este tipo de análisis (identificar indicador → estimar antes/después → decidir) es exactamente lo que se practica en el taller 3.

6.4 De la técnica al beneficio

Técnica de IA	Ejemplo de uso	Mejora operativa típica
Clasificación (supervisado)	Priorizar incidencias, cribar CV	Menos tiempo y errores de clasificación
Regresión (supervisado)	Prever demanda, precios	Menos stock roto, mejor fijación de precios
Clustering (no supervisado)	Segmentar clientes	Campañas más rentables
Detección de anomalías	Fraude, averías	Menos pérdidas y paradas
PLN	Chatbots, sentimiento, resúmenes	Atención 24/7, menos coste por consulta
Visión artificial	Inspección, lectura de documentos	Menos errores y tiempo de inspección
Robótica	Almacén, fabricación	Mayor velocidad y seguridad
Sistemas expertos	Diagnóstico técnico, rutas	Conocimiento experto disponible 24/7
IA generativa / agentes	Redacción, tramitación	Automatización de tareas repetitivas

Nota sobre la tabla anterior.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

De identificar a implementar

En esta unidad aprenderás a **identificar** oportunidades (qué técnica encaja y qué mejora aporta). **Implementar** modelos y sistemas es el trabajo de las unidades siguientes.

6.5 Ejemplo guiado: de un problema de negocio a una solución de IA

Para cerrar el bloque, recorremos juntos los pasos que repetirás en el taller 3. El caso es completamente ficticio para no depender de datos externos.

Problema: una tienda online recibe 200 reclamaciones de devolución al día por email. Hoy, dos personas leen cada correo, lo clasifican a mano y lo enrutan. Tardan unos 8 minutos por correo y el error de clasificación ronda el 15 %.

Paso 1 · Elegir el KPI. Medimos el **tiempo de ciclo** (8 min/reclamación), la **tasa de error** (15 %) y el **coste** (2 personas × jornada completa).

Paso 2 · Identificar la técnica. Clasificar texto en categorías (devolución, cambio, defecto, consulta) es una tarea de **PLN + clasificación supervisada** (p. ej. naive Bayes o un árbol de decisión sobre textos vectorizados).

Paso 3 · Estimar el antes/después. Con un clasificador que resuelve el 70 % de los correos en 30 segundos y el resto lo revisa una persona:

- Tiempo medio por correo: 0,70 × 0,5 min + 0,30 × 8 min ≈ **2,8 min** (frente a 8).
- Error de clasificación objetivo: **< 5 %** (frente al 15 %).
- Las dos personas pasan de clasificar a **tratar solo los casos complejos**.

Paso 4 · Decidir y comunicar. El responsable presenta la mejora con los KPIs antes/después y señala los riesgos (correos ambiguos, privacidad de los datos de los clientes según el RGPD).

El papel del ejemplo guiado

Este mismo esquema —*problema → KPI → técnica → antes/después → decisión*— es el que define qué es **caracterizar un sistema de IA** (RA1): no hace falta entrenar todavía el modelo, solo identificar correctamente qué técnica encaja y qué mejora operativa aporta.

7. Beneficios, riesgos y ética (visión general)

- **Beneficios** (para valorar la eficiencia operativa): automatización de tareas repetitivas, más y más rápida información de los datos, mejor toma de decisiones, menos errores humanos, disponibilidad 24×7, menor riesgo físico.
- **Riesgos** (se estudian a fondo en UD06): datos con sesgos o manipulación, modelos robados o alterados, fallos operativos (*model drift*), problemas de privacidad y ética. Un ejemplo relevante: la formación de los modelos puede reforzar sesgos de género si los datos de entrenamiento los contienen.
- **Marco normativo**: el **AI Act** (Reglamento UE 2024/1689) clasifica la IA por riesgo y entra en vigor por fases: las prácticas prohibidas desde **2/2/2025**, las sanciones desde **2/8/2025** y la aplicación general desde **2/8/2026**. Un modelo de IA de uso general con **riesgo sistémico** se presume cuando el cómputo acumulado de su entrenamiento supera los $\mathbf{10^{25}}$ **FLOPS** (art. 51). El **RGPD** limita además el uso de datos personales, lo que obliga a equilibrar el entrenamiento masivo con la **minimización de datos**. Todo esto se trabaja en la UD06.
- **IA responsable**: explicabilidad, equidad, robustez, rendición de cuentas, privacidad y cumplimiento normativo (RGPD, AI Act).

8. Puntos clave de la unidad

- Un sistema inteligente **percibe, razona y actúa**, y se caracteriza por autonomía, adaptación y toma de decisiones.
- La IA se clasifica por **tarea** (débil/fuerte), por **escuela** (convencional/computacional) y por **capacidades** (Russell-Norvig, Hintze) — son lentes complementarias, no excluyentes.
- **IA > ML > DL > IA generativa**: todo ML es IA, pero no toda IA es ML.
- Aprendizaje **supervisado** (con etiquetas: clasificación/regresión), **no supervisado** (patrones sin etiquetas: clustering), **por refuerzo** (recompensas), **semi** y **auto-supervisado**.
- El **deep learning** (redes profundas) domina visión, lenguaje y generación, pero requiere datos y cómputo.
- La IA ya está en la vida cotidiana (compras, buscadores, asistentes, traducción, coches, ciberseguridad).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- Las **nuevas interacciones** (chatbots, voz, visión, agentes) mejoran la **eficiencia operativa** reduciendo coste, tiempo o error.

9. Glosario

Término	Definición
IA (inteligencia artificial)	Tecnología que simula capacidades humanas (aprender, razonar, decidir)
Sistema inteligente	Sistema que percibe, razona y actúa para lograr un objetivo
IA débil / fuerte	Orientada a una tarea concreta (toda la actual) frente a IA general al nivel humano (teórica)
IA convencional / computacional	Escuela simbólico-deductiva (reglas explícitas) frente a subsimbólica-inductiva (aprende de datos)
Test de Turing	Un interrogador conversa con una persona y una máquina; si no las distingue, la máquina lo supera
Test de Lovelace	El sistema origina un resultado que su propio programador no puede explicar a partir del código
Teoría de la mente / autoconciencia	Niveles teóricos de la clasificación de Hintze, aún sin sistemas reales
ML (machine learning)	Técnicas que aprenden patrones de los datos sin programación explícita
Deep learning	Subconjunto del ML basado en redes neuronales profundas
IA generativa	IA que crea contenido nuevo (texto, imagen, audio)
LLM	Modelo de lenguaje grande (p. ej. ChatGPT) entrenado con enormes volúmenes de texto
Modelo	Resultado del entrenamiento de un algoritmo sobre datos
Entrenamiento	Proceso de ajustar los parámetros del modelo con datos
Generalización	Capacidad del modelo de acertar con datos nuevos
Supervisado	Aprendizaje con ejemplos etiquetados
No supervisado	Aprendizaje que encuentra estructura sin etiquetas
Refuerzo (RL)	Aprendizaje por prueba-error con recompensas
Clasificación	Tarea supervisada de predecir categorías
Regresión	Tarea supervisada de predecir valores continuos
Clustering	Agrupación no supervisada de datos similares
PLN	Procesamiento del lenguaje natural (texto y voz)
Visión por computador	IA que interpreta imágenes y vídeo
Robot	Sistema físico que percibe y actúa en su entorno
Sistema experto	IA basada en reglas de un experto humano
Agente de IA	Programa autónomo que usa herramientas para lograr objetivos
Prompt	Instrucción de texto que se da a un modelo generativo
Eficiencia operativa	Reducción de costes, tiempos o errores en un proceso

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

10. FAQ

¿Todo lo que usa datos es machine learning?

No. Hay IA que no es ML (sistemas basados en reglas, búsquedas heurísticas). El ML es la familia de técnicas que aprende patrones de los datos; todo ML es IA, pero no toda IA es ML.

¿En qué se diferencian IA débil e IA fuerte, y por qué toda la IA actual es débil?

La IA débil resuelve una tarea concreta y no se adapta más allá de lo programado; la fuerte igualaría a una persona en cualquier tarea. La fuerte es un objetivo teórico: ningún sistema actual generaliza tan ampliamente como para considerarse IA general.

¿Qué diferencia hay entre clasificación y regresión?

La **clasificación** predice una categoría (spam/no spam); la **regresión** predice un valor numérico continuo (precio de una vivienda).

¿Cuándo conviene usar aprendizaje profundo y cuándo no?

Conviene con **muchos datos y tareas complejas** (imágenes, audio, texto). Con datos pequeños o problemas sencillos, un modelo clásico (árbol, KNN, regresión logística) suele bastar y es más explicable.

¿Un chatbot entiende de verdad lo que le digo?

Los chatbots modernos con PLN/LLM **procesan estadísticamente el lenguaje** y generan respuestas plausibles, pero no tienen comprensión ni intenciones humanas; pueden equivocarse y conviene verificar sus respuestas.

¿La IA va a sustituir a las personas?

La IA **automatiza tareas** concretas, no profesiones enteras. Lo habitual es que cambie el trabajo: las personas se centran en casos complejos y la IA en tareas repetitivas. La pregunta de fondo es de organización y ética, no solo técnica.

¿Qué es la IA generativa y en qué se diferencia de la IA predictiva?

La IA **predictiva** estima un resultado (probabilidad, categoría, valor). La IA **generativa** crea contenido nuevo (texto, imagen, audio) a partir de un prompt, usando modelos como los LLM o los modelos de difusión.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

11. Planificación sesión a sesión

Semana	Horas	Contenido	CE	Evidencia / actividad
3	3	Fundamentos; escuelas de pensamiento y clasificaciones (débil/fuerte, Russell-Norvig, Hintze)	RA1-a	Ejercicios bloque A, `N02`
4	3	Campos de aplicación de la IA	RA1-b	Ejercicios bloque C
5	3	Técnicas de la IA (ML, PLN, visión, robótica, sistemas expertos)	RA1-c	Ejercicios bloques B y D, `N03`
6	3	Nuevas interacciones, eficiencia operativa y KPIs; entrega	RA1-d	Ejercicios bloques E y F, `N04` y `N05`, notebook demo

12. Tabla final RA/CE

CE	Dónde se trabaja	Con qué se evalúa
RA1-a	§3	Ejercicios bloque A, `N02`, `N05`, prueba del RA1
RA1-b	§5	Ejercicios bloque C, `N02`, prueba del RA1
RA1-c	§4, §6.1	Ejercicios bloques B y D, `N03`, prueba del RA1
RA1-d	§6	Ejercicios bloques E y F, `N04`, prueba del RA1

13. Recursos

- Diapositivas
- **Práctica** — se hace, no se entrega ni puntúa:
  - Ejercicios de autoevaluación
  - Notebooks guiados — el notebook demo del profesor
- **Entregas** — las cuatro se corrigen con rúbrica; qué se entrega:
  - N02 · mapa de sistemas inteligentes · N03 · técnicas en casos reales · N04 · nuevas interacciones · N05 · línea del tiempo
- Los notebooks se abren desde **Práctica** y **Entregas**, con descarga y apertura en Colab.

Referencias de la unidad

- IBM · ¿Qué es la IA?
- IBM · ¿Qué es el machine learning?
- IBM · ¿Qué es el PLN?
- scikit-learn · Aprendizaje supervisado
- Parlamento Europeo · ¿Qué es la IA y cómo se usa?
- AI Act (UE 2024/1689)
- Russell y Norvig · AIMA (sitio oficial)
- Arend Hintze · «Understanding the four types of AI» (The Conversation, 2016)
- Vaswani et al. · Attention Is All You Need (2017)

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

14. Evaluación

Peso	Instrumento
40 % actividades	Los **4 talleres**, cada uno con su rúbrica en la tarea de Moodle. Pesan lo mismo, así que es su media
60 % prueba escrita	Prueba del RA1 en Moodle: preguntas de test y de desarrollo sobre el contenido de la unidad

- **La normativa exige alcanzar todos los RA** del módulo para superarlo (art. 5.1 de la Orden 8/2025: la calificación del módulo está *«en función de la consecución de los RA»*; y las Instrucciones 26-27, que impiden calificar positivamente un módulo con RA no superados). El centro concreta ese mandato exigiendo **≥ 5 en cada RA**.

15. Recuperación

Actividades del programa de recuperación individual por RA (art. 14.4 Orden 8/2025): repetir el análisis del caso real con un caso distinto y las pruebas de autoevaluación de esta unidad.

Volver al índice

26 de octubre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

3.2 Diapositivas de la UD01



Diapositivas PDF

Cómo se manejan

Flechas o clic para avanzar · **F** para verlas a pantalla completa · **P** para las notas del ponente.

24 de agosto de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

4. UD02

4.1 UD02 — Modelos de IA y resolución de problemas

Unidad 2 · 12 h · semanas 7-10 (9 de noviembre al 3 de diciembre)

1. Introducción

En la UD01 aprendiste a **identificar** sistemas de IA y a relacionarlos con la eficiencia operativa. En esta unidad damos un paso más: no solo reconocer la IA, sino **formalizar y resolver problemas con modelos de IA**. Para ello recorremos un camino progresivo:

1. Primero, **qué debe cumplir un sistema** que resuelve problemas con IA, y **cómo se representa un problema** para que una máquina pueda resolverlo (espacio de estados y algoritmos de búsqueda).
2. Después, **cómo se clasifican los modelos de IA** (por aprendizaje, por análisis, por conocimiento frente a datos).
3. A continuación, **qué se puede automatizar** y con qué tecnología (RPA frente a IA, agentes).
4. Luego, las dos familias de razonamiento que tocarás en el laboratorio: **lógica difusa** y **sistemas basados en reglas**.
5. Por último, **cómo elegir el modelo adecuado** para cada problema (CE f).

El núcleo práctico usa dos librerías de Python que ya están en el stack del curso: `scikit-fuzzy` (lógica difusa) y `experta` (sistemas basados en reglas). La unidad se cierra con **Robocode Tank Royale**: programar un bot que compite en un campo de batalla es, literalmente, implementar un sistema de resolución de problemas de principio a fin.

> **Hilo conductor de la unidad**
>
> *Antes de programar IA hay que saber formalizar un problema y elegir bien el modelo. Lo haremos paso a paso: modelar → clasificar → automatizar → razonar (difuso y por reglas) → decidir → implementar.*

2. Resultado de aprendizaje y criterios de evaluación

RA2 — Utiliza modelos de sistemas de Inteligencia Artificial implementando sistemas de resolución de problemas.

CE	Criterio de evaluación
RA2-a	Se han determinado los requisitos básicos a implementar en un sistema de resolución de problemas.
RA2-b	Se han clasificado modelos de Inteligencia Artificial.
RA2-c	Se han caracterizado los modelos de automatización de tareas.
RA2-d	Se han caracterizado los modelos de razonamiento impreciso.
RA2-e	Se han caracterizado los modelos de sistemas basados en reglas.
RA2-f	Se ha valorado la adecuación de los modelos a la implementación del sistema de resolución de problemas.

Nota sobre la tabla anterior.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Bloque de contenidos oficial (RD 279/2021, anexo I)

El currículo para este RA dice textualmente:

Utilización de modelos de Inteligencia Artificial:

- *Requisitos básicos de un sistema de resolución de problemas.*

Modelos de sistemas de Inteligencia Artificial:

- *Automatización de tareas.*
- *Sistemas de razonamiento impreciso.*
- *Sistemas basados en reglas.*

3. Objetivos de la unidad

Objetivo	Descripción
O1	Enumerar los requisitos básicos de un sistema de resolución de problemas de IA.
O2	Formalizar un problema como espacio de estados (estado inicial, acciones, transición, objetivo).
O3	Aplicar búsqueda en anchura, en profundidad y A* y saber cuándo usar cada una.
O4	Clasificar modelos de IA por paradigma de aprendizaje, por tipo de análisis y por base de conocimiento/datos.
O5	Diferenciar RPA de IA y describir la automatización inteligente y los agentes software.
O6	Explicar la lógica difusa: conjuntos difusos, funciones de pertenencia, reglas, Mamdani/Sugeno y defuzzificación.
O7	Implementar un control difuso sencillo con `scikit-fuzzy`.
O8	Describir el ciclo reconocer-actuar de un sistema basado en reglas y el papel de CLIPS y de `experta`.
O9	Implementar un sistema basado en reglas con `experta` (con el parche de compatibilidad).
O10	Elegir el modelo adecuado a un problema según datos, explicabilidad, coste y tiempo real.
O11	Implementar un sistema de resolución de problemas completo (Robocode) dotando de comportamiento a un bot.

4. Sistema de resolución de problemas (RA2-a)

4.1 Cinco requisitos de un sistema de resolución de problemas

Antes de formalizar un problema concreto, conviene fijar **qué debe cumplir** cualquier sistema de IA que pretenda resolver problemas de forma efectiva y útil:

1. **Representación del problema**: elegir una estructura de datos y un modelo que reflejen fielmente el dominio. En PLN, por ejemplo, convertir texto a vectores numéricos; en un puzzle, una matriz de fichas (§4.2).
2. **Razonamiento y toma de decisiones**: aplicar técnicas lógicas, de aprendizaje o de búsqueda heurística para llegar a una solución a partir de la información disponible.
3. **Aprendizaje y adaptabilidad**: mejorar el rendimiento con la experiencia — aprendizaje automático o por refuerzo cuando el problema lo permite.
4. **Eficiencia computacional**: usar algoritmos y estructuras que den respuestas rápidas y escalables (por eso importa tanto BFS/DFS/A*, §4.3).
5. **Interacción con las personas usuarias**: una interfaz comprensible que facilite la comunicación entre el sistema y quien lo usa.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

> **No son independientes**
>
> Un sistema puede cumplir muy bien 4 requisitos y fallar estrepitosamente por el quinto: un algoritmo de búsqueda óptimo (requisito 4) que nadie sabe usar porque no tiene interfaz (requisito 5) no resuelve el problema en la práctica.

4.2 El espacio de estados

Muchos problemas de IA se resuelven convirtiéndolos en una **búsqueda**: existe un conjunto de configuraciones posibles (los **estados**) y unas **acciones** que pasan de un estado a otro. Ese grafo de estados conectados por acciones es el **espacio de estados**.



Un **sistema de resolución de problemas** necesita, formalmente (base AIMA):

1. **Estado inicial**: la configuración de partida.
2. **Acciones aplicables**: qué movimientos se pueden hacer en cada estado.
3. **Modelo de transición**: función que, dado un estado y una acción, produce el siguiente estado.
4. **Test de objetivo**: cómo saber que hemos llegado a la solución.
5. **Coste de camino**: el coste de cada acción (para encontrar la solución más barata).

Además, se elige la **representación** (cómo codificar cada estado) y la **estrategia de búsqueda** (completa, óptima, memoria).

> **Regla para recordar**
>
> Un problema de IA bien planteado es aquel del que sabes responder: *¿cuál es el estado inicial?, ¿qué acciones puedo aplicar?, ¿cuándo he terminado?* Si no sabes responderlas, el problema no está bien formalizado.

4.3 Ejemplos clásicos de representación

Antes de comparar algoritmos de búsqueda (§4.4) hay que entender **qué pide resolver** cada problema clásico: no todos buscan lo mismo, y por eso no todos se resuelven igual de bien con el mismo algoritmo.

4.3.1 EL PROBLEMA DE LAS JARRAS (8-5-3)

Enunciado: tienes jarras sin marcas de capacidad 8, 5 y 3 litros, una fuente de agua ilimitada, y quieres medir una cantidad exacta (p. ej. 4 litros) usando solo tres acciones: **llenar** una jarra hasta el tope, **vaciar** una jarra por completo, o **verter** el contenido de una jarra en otra hasta que la que recibe se llene o la que vierte se vacíe.

- **Estado**: triple `[jarra8, jarra5, jarra3]` con el contenido actual de cada jarra.
- **Acciones**: todas las combinaciones de llenar/vaciar/verter entre pares de jarras.
- **Objetivo**: de `[8,0,0]` a `[4,4,0]` en 7 pasos (el ejemplo del diagrama de §4.2).

> **Por qué encaja con BFS**
>
> El espacio de estados es pequeño (como mucho `(8+1)×(5+1)×(3+1)` combinaciones) y lo que interesa es el **camino más corto** en número de trasvases, con coste unitario por acción: justo lo que **BFS** garantiza. No hay estados "prohibidos" que podar, así que no hace falta backtracking.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

4.3.2 MISIONEROS Y CANÍBALES

Enunciado: 3 misioneros y 3 caníbales deben cruzar un río en una barca que como máximo lleva 2 personas. En **ninguna** orilla, en ningún momento, puede haber más caníbales que misioneros si hay al menos un misionero presente (o los misioneros son devorados).

- **Estado**: vector `(m,c,barca)` — misioneros y caníbales en la orilla de partida, y en qué orilla está la barca.
- **Acciones**: mover la barca con 1 o 2 personas de una orilla a otra.
- **Restricción**: se descartan los estados donde los caníbales superan a los misioneros en cualquier orilla con misioneros.
- **Objetivo**: de `(3,3,1)` a `(0,0,0)` (solución óptima: 11 cruces).

> **Por qué aparece el backtracking**
>
> Muchos movimientos posibles llevan a un estado que **viola la restricción**: ese estado se descarta antes de expandirlo, sin seguir explorando esa rama. Esa poda inmediata de ramas inválidas es la esencia del **backtracking**, propio de DFS. BFS también puede resolverlo (el espacio es pequeño), pero aquí la restricción es el elemento pedagógico clave, no la memoria.

4.3.3 EL PROBLEMA DE LAS 8 REINAS

Enunciado: colocar 8 reinas en un tablero de ajedrez 8×8 de modo que ninguna ataque a otra (ni misma fila, ni misma columna, ni misma diagonal).

- **Representación**: un vector de 8 posiciones donde el índice es la columna y el valor es la fila de la reina en esa columna — así ya queda garantizado "una reina por columna" sin tener que comprobarlo en cada paso.
- **Tamaño del espacio**: elegir 8 casillas cualesquiera de las 64 sin ninguna restricción da 4.426.165.368 combinaciones; exigir una reina por columna lo reduce a las `8! = 40.320` permutaciones de filas; de esas, solo **92** cumplen además la restricción de filas y diagonales.

> **No es un problema de «camino más corto»**
>
> Aquí no hay un camino que se recorra paso a paso con un coste que minimizar: **cualquier** configuración válida es una solución igual de buena que otra. Es un problema de **satisfacción de restricciones (CSP)**, no de búsqueda de ruta óptima. Por eso el algoritmo natural es **backtracking** puro: coloca una reina en cada columna, comprueba las restricciones con las reinas ya puestas y retrocede en cuanto ninguna fila es válida para la siguiente columna. BFS o A* no aportan nada aquí porque no hay "coste de camino" que optimizar.

4.3.4 EL 15-PUZZLE

Enunciado: una caja 4×4 con 15 fichas numeradas y un hueco; se mueve la ficha adyacente al hueco hasta llegar al orden objetivo.

- **Representación**: matriz 4×4 con las fichas (o el hueco) en cada casilla.
- **Tamaño**: `16!/2 ≈ 10,46·10¹²` estados (solo la mitad son alcanzables); las soluciones óptimas pueden necesitar hasta 80 movimientos.

> **Por qué aquí la heurística es imprescindible, no opcional**
>
> Con un espacio de **10 billones** de estados, BFS y DFS son inviables: BFS se queda sin memoria guardando un nivel entero y DFS puede perderse en ramas sin fondo. **A*** con una heurística admisible (número de fichas mal colocadas, o distancia de Manhattan, §4.4) es la única opción realista: guía la búsqueda hacia el objetivo en lugar de explorar a ciegas, y sigue garantizando la solución óptima.

4.3.5 ¿QUÉ ALGORITMO CONVIENE A CADA PROBLEMA?

Problema	Qué hay que encontrar	Algoritmo recomendado	Por qué
Jarras	Camino más corto (coste unitario)	**BFS**	Espacio pequeño; importa el número de pasos
Misioneros y caníbales	Camino evitando estados prohibidos	**BFS** (o DFS con poda)	Hay que descartar estados inválidos antes de expandirlos

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Problema	Qué hay que encontrar	Algoritmo recomendado	Por qué
8-reinas	Cualquier configuración válida (CSP)	**Backtracking (DFS)**	No hay coste de camino que optimizar; basta una solución
15-puzzle	Camino óptimo en un espacio enorme	**A*** con heurística	BFS/DFS inviables por tamaño; hace falta una estimación que guíe la búsqueda

Nota sobre la tabla anterior.

> **La explosión combinatoria**
>
> Estos problemas "de juguete" sirven para medir algo esencial: el número de estados crece **exponencialmente** con el tamaño (factor de ramificación `b` elevado a profundidad `d`). Un espacio de estados real (rutas de reparto, planificación de tareas) es enorme: por eso hace falta una **estrategia de búsqueda** y, cuando se puede, una **heurística**.

4.4 Algoritmos de búsqueda: BFS, DFS y A*

BFS son las siglas de *breadth-first search* (búsqueda en anchura) y **DFS** las de *depth-first search* (búsqueda en profundidad).

Criterio	Búsqueda en anchura (BFS)	Búsqueda en profundidad (DFS)	A*
Estructura	Cola (FIFO)	Pila (LIFO)	Cola de prioridad por `f = g + h`
Completa	Sí	No (en espacios infinitos)	Sí
Óptima	Sí (coste unitario)	No	Sí (heurística admisible)
Memoria	O(b^d)	O(b·d)	O(b^d)
Cuándo usar	Camino más corto en número de pasos	Exploración exhaustiva, backtracking	Ruta óptima con costes variados y buena heurística

4.4.1 BFS PASO A PASO

BFS explora por niveles: primero todos los estados a 1 acción del inicio, luego todos los que están a 2 acciones, etc. Encuentra el camino más corto en pasos, pero consume mucha memoria (guarda todos los estados del nivel actual en la cola).

```
BFS(inicio, es_objetivo):
    frontera ← Cola([inicio])        # FIFO
    visitados ← {inicio}
    mientras frontera no vacía:
        estado ← frontera.sacar_primero()
        si es_objetivo(estado): devolver camino_hasta(estado)
        para cada accion aplicable a estado:
            siguiente ← transicion(estado, accion)
            si siguiente ∉ visitados:
                visitados.añadir(siguiente)
                frontera.meter(siguiente)
    devolver fracaso
```

> **Traza sobre las jarras (§4.3.1)**
>
> - **Nivel 0**: `[8,0,0]` (inicio).
> - **Nivel 1**: todos los estados alcanzables con **una** acción, p. ej. `[3,5,0]` (verter la jarra8 en la jarra5), `[8,0,3]` (llenar la jarra3), `[5,0,3]` …
> - **Nivel 2**: todos los alcanzables con una acción más desde el nivel 1, y así sucesivamente.
> - BFS no se detiene hasta **vaciar un nivel entero** antes de pasar al siguiente: por eso garantiza que `[4,4,0]` se encuentra en el **menor número de pasos** posible (7, en este caso), pero también por eso necesita guardar en memoria todos los estados de cada nivel.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

4.4.2 DFS Y BACKTRACKING PASO A PASO

DFS baja hasta el fondo de una rama antes de probar otra: usa poca memoria (solo el camino actual), pero puede no encontrar la solución óptima, o no terminar en espacios infinitos sin control de visitados. Es la base del ***backtracking***.

```
DFS(estado, es_objetivo, es_valido):
    si es_objetivo(estado): devolver [estado]
    para cada accion aplicable a estado:
        siguiente ← transicion(estado, accion)
        si es_valido(siguiente):                # no visitado / no viola restricciones
            camino ← DFS(siguiente, es_objetivo, es_valido)
            si camino ≠ fracaso: devolver [estado] + camino
            # si no, se retrocede (backtrack) y se prueba la siguiente acción
    devolver fracaso
```

Traza sobre las 8-reinas (§4.3.3)

1. Columna 1: coloca la reina en fila 1. Válido (no hay más reinas aún) → avanza.
2. Columna 2: prueba fila 1 (misma fila que la reina 1 → inválido), prueba fila 2 (misma diagonal → inválido), prueba fila 3 (válido) → avanza.
3. Columna 3: ninguna fila es válida con las dos reinas ya puestas → **retrocede** (backtrack) a la columna 2 y prueba la siguiente fila disponible.
4. El proceso continúa probando-avanzando-retrocediendo hasta colocar las 8 reinas (una de las 92 soluciones) o agotar las posibilidades de la columna 1.

Es exactamente la poda que describía §4.3.2 para misioneros y caníbales: un estado inválido se descarta **sin expandirlo**, y el algoritmo retrocede al último punto con alternativas.

4.4.3 A* PASO A PASO

A* combina el coste acumulado `g(n)` con una **heurística** `h(n)` (una estimación de lo que falta): `f(n) = g(n) + h(n)`. En vez de una cola FIFO o una pila, usa una **cola de prioridad** que siempre extrae el estado con menor `f`. Con una heurística admisible (que no sobreestime), A* es completo y óptimo. Se usa mucho en videojuegos, mapas y planificación de rutas.

```
A*(inicio, es_objetivo, h):
    frontera ← ColaPrioridad([(h(inicio), inicio)])  # ordenada por f = g + h
    g ← {inicio: 0}
    mientras frontera no vacía:
        estado ← frontera.sacar_minimo_f()
        si es_objetivo(estado): devolver camino_hasta(estado)
        para cada accion aplicable a estado:
            siguiente ← transicion(estado, accion)
            g_tentativo ← g[estado] + coste(accion)
            si siguiente ∉ g o g_tentativo < g[siguiente]:
                g[siguiente] ← g_tentativo
                f ← g_tentativo + h(siguiente)
                frontera.meter((f, siguiente))
    devolver fracaso
```

Traza sobre el 15-puzzle (§4.3.4)

Con `h` = distancia de Manhattan: desde el estado inicial se calculan `g=0` (sin moverse aún) y `h` (suma de las distancias de cada ficha a su casilla objetivo). En cada paso, A* expande el estado de la frontera con **menor** `f = g+h`, no necesariamente el más reciente ni el más antiguo: eso es lo que le permite ignorar ramas que claramente se alejan del objetivo (h alto) aunque BFS las habría explorado igual que las demás.

Regla práctica (Red Blob Games)

"*Usa el algoritmo más simple que puedas*": si todos los costes son iguales y solo quieres el camino más corto en pasos, BFS basta. Si los costes varían, Dijkstra. Si apuntas a un único objetivo, prefiere A* con la heurística más simple posible (p. ej. Manhattan en rejillas).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

5. Clasificación de modelos de IA (RA2-b)

5.1 Por paradigma de aprendizaje

Criterio	Supervisado	No supervisado	Refuerzo
Datos	Etiquetados (ground truth)	Sin etiquetas	Estados, acciones, recompensas
Objetivo	Predecir la salida correcta	Descubrir patrones	Maximizar recompensa acumulada
Tareas	Clasificación, regresión	Clustering, asociación, reducción de dim.	Control, juegos, robótica
Ejemplos	Spam, precio de un coche	Segmentación de clientes (k-means)	Robot que aprende a andar, AlphaGo
Algoritmos (scikit-learn)	Regresión logística, SVM, árboles	k-means, DBSCAN, PCA	(fuera del stack estándar)

Nota sobre la tabla anterior.

> **Recordatorio de la UD01**
>
> **Todo ML es IA, pero no toda IA es ML.** Los sistemas basados en reglas o en lógica difusa son IA, pero no aprenden de datos. En esta unidad estudiamos precisamente esos modelos "no-ML".

5.2 Por tipo de análisis

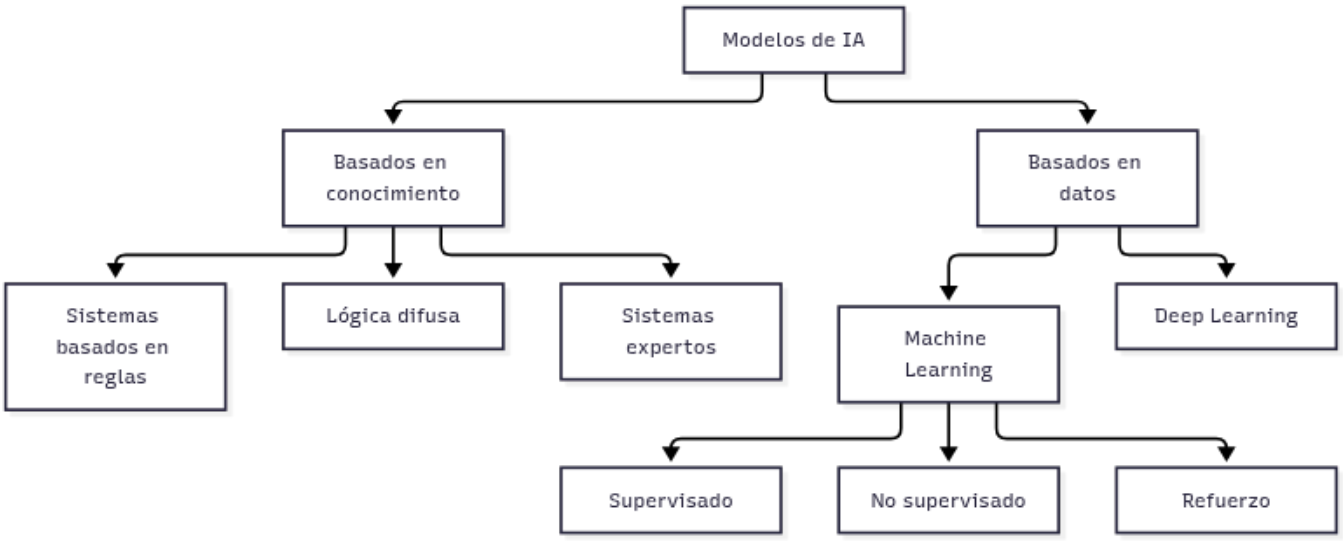
Fase	Pregunta	Ejemplo
Descriptivo	¿Qué pasó?	Informe de ventas
Predictivo	¿Qué pasará?	Previsión de demanda
Prescriptivo	¿Qué hacer?	Qué precio fijar y qué ocurre si lo cambio

El **prescriptivo** es el que aporta más valor: no solo predice, **recomienda o ejecuta** la decisión óptima (Gartner lo llama "la última frontera" de la analítica).

5.3 Basados en conocimiento frente a basados en datos

Criterio	Basados en conocimiento	Basados en datos (ML/DL)
Conocimiento	Explícito (reglas IF-THEN)	Implícito (aprendido de los datos)
Ejemplos	Sistemas expertos, lógica difusa, termostato	Árboles, regresión, redes neuronales
Ventajas	Interpretable, fácil de mantener, explica el razonamiento	Flexible, escala a tareas complejas
Limitaciones	Frágil en tareas complejas; adquisición de conocimiento difícil	Requiere datos y cómputo; caja negra
Requisito	Experto + ingeniero del conocimiento	Datos etiquetados suficientes y representativos

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Dos formas de resolver el mismo problema

Un termostato con reglas `SI temperatura < 18 ENTONCES calefacción ON` es IA basada en conocimiento (reglas). Un sistema que aprende de datos históricos qué temperatura poner según la hora y la estación es ML. La elección depende del problema (lo veremos en el CE f).

6. Modelos de automatización de tareas (RA2-c)

6.1 RPA frente a IA

RPA son las siglas de *robotic process automation* (automatización robótica de procesos): un **software robot** que repite, en la interfaz de un programa, los mismos clics, pulsaciones y copia-pega que haría una persona —rellenar un formulario, mover datos entre dos aplicaciones—, siguiendo siempre el mismo flujo predefinido. No "entiende" lo que hace ni aprende de ello: por eso no es IA, aunque muchas veces se las confunda.

Criterio	RPA	IA
Qué hace	**Hace**: replica tareas humanas repetitivas en la interfaz (GUI, formularios)	**Piensa/aprende**: reconoce patrones, decide, mejora con los datos
Base	Procesos (*process-driven*), reglas predefinidas	Datos (*data-driven*), modelos
Se adapta	No: los flujos hay que mantenerlos si cambia el sistema	Sí: aprende de la experiencia
Ejemplo	Un robot que copia datos de un email a un ERP	Un modelo que clasifica si el email es urgente o no

Nota sobre la tabla anterior.

RPA no es IA

La distinción de IBM es clara: RPA **automatiza procesos** replicando acciones en pantalla; la IA **razona sobre datos**. Se complementan: la IA da decisión y el RPA ejecuta. Cuando se combinan IA + BPM (orquestación de flujos) + RPA (ejecución) hablamos de **automatización inteligente** (IA).

6.2 Agentes software

Un **agente de IA** es un programa que percibe su entorno y actúa para lograr un objetivo. Se ordenan de más simple a más avanzado:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Tipo de agente	Cómo decide	Ejemplo
Reflejo simple	Reglas `si... entonces...` directas	Termostato
Reflejo con modelo	Mantiene un estado interno del mundo	Robot aspiradora que "recuerda" la habitación
Basado en objetivos	Busca y planifica para alcanzar una meta	Navegador que elige una ruta
Basado en utilidad	Maximiza una utilidad (coste, tiempo, riesgo)	Ruta que minimiza peajes y tiempo
Con aprendizaje	Aprende de la experiencia	Motor de recomendación

6.3 Tareas cognitivas automatizables

La IA automatiza **tareas cognitivas** que antes requerían juicio humano:

- **Extracción**: leer documentos (OCR), extraer entidades (NER), capturar datos de facturas.
- **Clasificación**: spam, incidencias, prioridades. Los **sistemas de reconocimiento de voz** (Google Speech-to-Text, Microsoft Azure Speech Service) son otro caso de automatización de tareas: convierten habla en texto y automatizan la transcripción o los comandos de voz.
- **Generación**: redactar respuestas, resúmenes o informes (IA generativa).

Caso con cifra (ilustrativo)

En automatización de back-office, un sistema que extrae datos de emails y mensajería puede reducir el cierre contable mensual de varios días a uno, con menor tasa de error. Las cifras concretas dependen de cada despliegue; en la práctica se mide con KPIs (tiempo de ciclo, coste por documento, tasa de error) como vimos en la UD01. Otro ejemplo habitual es la **detección de fraude**: algoritmos de aprendizaje automático que analizan patrones en transacciones financieras para señalar operaciones sospechosas, reduciendo la carga de revisión manual.

7. Razonamiento impreciso: lógica difusa (RA2-d)

7.1 De lo booleano a lo difuso

La lógica clásica solo admite **verdadero/falso** (0/1). La **lógica difusa** (Zadeh, 1965) permite grados de verdad **entre 0 y 1**: algo puede ser *parcialmente verdadero*. Modela la **vaguedad** del lenguaje humano (la probabilidad, en cambio, modela la incertidumbre). Es la familia de modelos que se usa cuando los datos son **incompletos o inciertos** y aun así hay que decidir algo razonable.

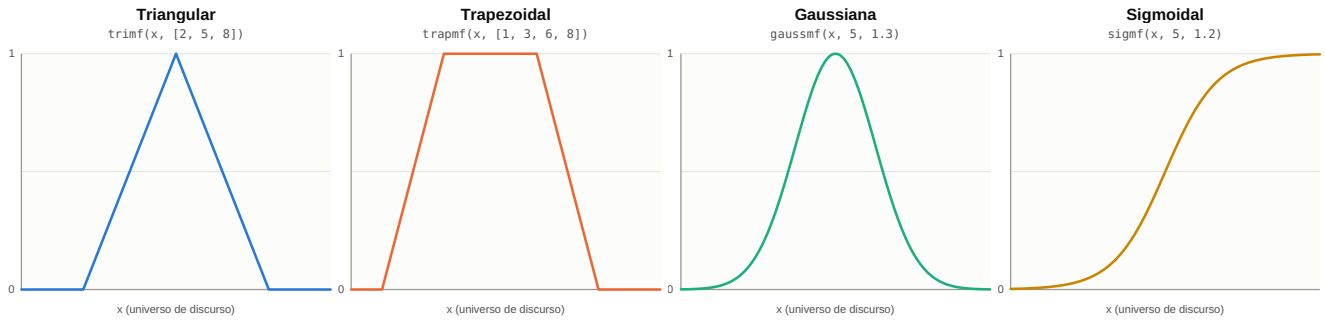
Variable	Valor clásico	Valor difuso
Temperatura	"18 ºC es frío" (sí/no)	"18 ºC es frío con 0,7"
Velocidad	Rápida o no	"Rápida con 0,6, media con 0,4"

7.2 Conjuntos difusos y funciones de pertenencia

Un **conjunto difuso** asigna a cada valor un **grado de pertenencia** $\mu(x) \in [0,1]$ mediante una **función de pertenencia**. Las más usadas:

Función	Forma	Uso típico
Triangular (`trimf`)	Pico en un punto	Variables simples (p. ej. "media")
Trapezoidal (`trapmf`)	Meseta central	Intervalos ("normal")
Gaussiana (`gaussmf`)	Campana suave	Variables continuas suaves
Sigmoidal (`sigmf`)	Escalón suave	Extremos ("frío", "caliente")

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



> **Diseño de variables**
>
> No importa tanto la forma exacta como el **número y la posición** de las funciones: se recomiendan entre **3 y 7 curvas** por variable, solapadas para que no existan "huecos".

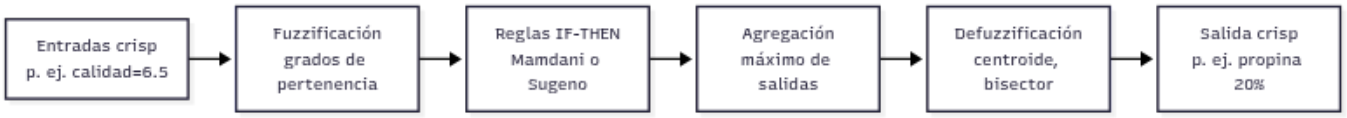
7.3 Operaciones difusas

Zadeh definió las operaciones básicas sobre grados de pertenencia:

- **AND (intersección)**: mínimo de los grados (`μ_A ∧ μ_B = min(μ_A, μ_B)`).
- **OR (unión)**: máximo (`μ_A ∨ μ_B = max(μ_A, μ_B)`).
- **NOT (complemento)**: `1 − μ`.

En `scikit-fuzzy`, las reglas usan por defecto `fmin` / `fmax` para AND/OR (se pueden cambiar a producto/suma difusa).

7.4 El sistema de inferencia difuso (FIS)



1. **Fuzzificar**: convertir cada entrada numérica en grados de pertenencia.
2. **Aplicar reglas** `SI calidad es mala O servicio es pobre ENTONCES propina es baja`.
3. **Agregar**: combinar la salida de todas las reglas.
4. **Defuzzificar**: obtener un número nítido. Métodos típicos: **centroide** (centro de gravedad), **bisector**, **MOM/SOM/LOM**.

Hay dos familias de sistemas: **Mamdani** (la salida es un conjunto difuso que luego se defuzzifica; la implementa `scikit-fuzzy`) y **Sugeno/TSK** (la salida de cada regla es una función `z = f(x,y)`, más eficiente para control).

7.5 Aplicaciones reales

- **Metro de Sendai (Japón, 1987)**: control difuso de aceleración y frenado.
- **Autofocus de Canon**: 12 entradas (claridad y velocidad del objetivo), **13 reglas**, solo **1,1 KB** de memoria.
- **ABS** (frenos): estima el agarre con lógica difusa sobre la temperatura del freno.
- **Electrodomésticos**: lavadoras Hitachi (peso/suciedad → programa), vacuolas Matsushita.
- **Control industrial**: hornos de cemento (Dinamarca, 1976), aires acondicionados Mitsubishi (calienta/enfría 5× más rápido con un 24 % menos de consumo).
- **Control de tráfico**: ajuste de los tiempos de los semáforos en función del flujo vehicular en tiempo real, para reducir la espera de los vehículos.
- **Apoyo al diagnóstico médico**: evaluación de síntomas cuando los resultados de las pruebas no son concluyentes, para dar una valoración preliminar que considere varios factores a la vez.

7.6 Práctica con `scikit-fuzzy`

`pip install -U scikit-fuzzy` (versión 0.5.0, 2024). Ejemplo clásico: el **problema de la propina** — la propina depende de la calidad del servicio y de la comida.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```python
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl

# 1. Variables (universo de discurso)
calidad = ctrl.Antecedent(np.arange(0, 11, 1), 'calidad')
servicio = ctrl.Antecedent(np.arange(0, 11, 1), 'servicio')
propina = ctrl.Consequent(np.arange(0, 26, 1), 'propina')

# 2. Funciones de pertenencia
calidad.automf(3, names=['mala', 'aceptable', 'excelente'])
servicio.automf(3, names=['pobre', 'aceptable', 'bueno'])
propina['baja'] = fuzz.trimf(propina.universe, [0, 5, 10])
propina['media'] = fuzz.trimf(propina.universe, [10, 15, 20])
propina['alta'] = fuzz.trimf(propina.universe, [15, 20, 25])

# 3. Reglas
regla1 = ctrl.Rule(calidad['mala'] | servicio['pobre'], propina['baja'])
regla2 = ctrl.Rule(servicio['aceptable'], propina['media'])
regla3 = ctrl.Rule(servicio['bueno'] | calidad['excelente'], propina['alta'])

# 4. Sistema y simulación
sistema = ctrl.ControlSystem([regla1, regla2, regla3])
sim = ctrl.ControlSystemSimulation(sistema)
sim.input['calidad'] = 6.5
sim.input['servicio'] = 9.8
sim.compute()
print(f"Propina sugerida: {sim.output['propina']:.2f}%")
# Salida esperada: Propina sugerida: ~20% (el valor exacto depende del universo)
```

> **Compatibilidad de la librería**
>
> `scikit-fuzzy` es una librería **semi-mantenida** (última versión 2024). Para los ejercicios del curso funciona sin problemas, pero conviene fijar la versión (`scikit-fuzzy==0.5.0`) en el entorno reproducible del contenedor.

> **Este ejemplo, ya resuelto y explicado paso a paso**
>
> El notebook guiado `N01` · Lógica difusa: el problema de la propina recorre este mismo ejemplo con las tres fases comentadas y las figuras de cada variable. No se entrega ni puntúa: ábrelo para repasarlo antes de `N02` y `N03` .

8. Sistemas basados en reglas (RA2-e)

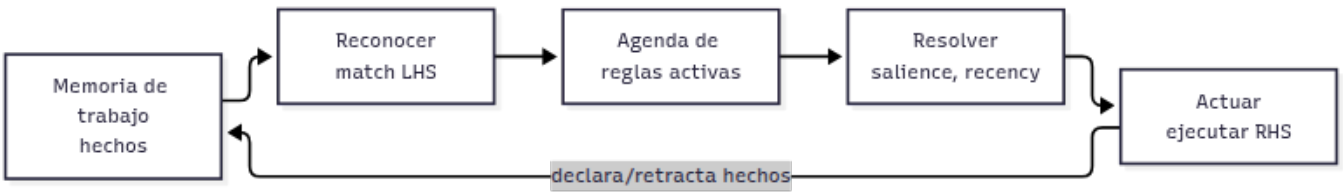
8.1 El ciclo reconocer-actuar

Un **sistema de producción** (o sistema basado en reglas) tiene:

- **Memoria de trabajo**: los hechos conocidos en cada momento.
- **Base de conocimiento**: las reglas `SI <condiciones> ENTONCES <acciones>` .
- **Motor de inferencia**: ejecuta el ciclo **reconocer-actuar**.

El ciclo se repite hasta que no queden reglas aplicables:

1. **Reconocer (match)**: se comparan los hechos de la memoria de trabajo con las condiciones de las reglas; las reglas cuyas condiciones se cumplen van a la **agenda**.
2. **Resolver (resolve)**: si hay varias reglas activas, se elige una según prioridad (*salience*), frescura de los hechos o especificidad.
3. **Actuar (act)**: se ejecuta la parte `ENTONCES` , que modifica la memoria de trabajo (declarar/retractar hechos) y el ciclo se repite.



Los motores industriales usan el **algoritmo RETE** (Forgy, 1974) para no reevaluar todas las reglas cada vez: construye una red de filtrado en memoria.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

8.2 Encadenamiento hacia delante y hacia atrás

- **Hacia delante (forward chaining, data-driven)**: parte de los hechos y deduce hechos nuevos. Ideal para monitorización, planificación y entornos dinámicos. Es el que usan CLIPS y `experta`.
- **Hacia atrás (backward chaining, goal-driven)**: parte de una **meta** y busca qué hechos la sustentan (preguntando al usuario solo lo necesario). Ideal para diagnóstico (MYCIN, Prolog).

8.3 CLIPS y `experta`

CLIPS (*C Language Integrated Production System*) es el motor de reglas de referencia, desarrollado por la NASA (1985-1996) y de dominio público desde 1996. `experta` es una librería de Python **inspirada en CLIPS** (fork de `pyknow`) que implementa un motor RETE con sintaxis nativa de Python.

8.4 Ejemplos cotidianos de sistemas basados en reglas

Antes de los grandes sistemas expertos históricos (§8.6), los sistemas basados en reglas están en aplicaciones que usas a diario:

- **Sistemas de recomendación**: plataformas de comercio electrónico como Amazon sugieren productos relacionados aplicando reglas sobre el historial de compras y el comportamiento del usuario.
- **Soporte al diagnóstico médico**: en asistencia remota, reglas basadas en los síntomas declarados dan una recomendación preliminar antes de que la persona sea atendida por un profesional.

Se usan mucho porque el razonamiento es **transparente**: las reglas son explícitas y una persona puede leer exactamente por qué el sistema tomó esa decisión — al contrario que una red neuronal.

8.5 Práctica con `experta`

> ⚠ **Compatibilidad con Python 3.10+ (obligatorio)**
>
> `experta` (1.9.4, de 2019) fija `frozendict==1.2`, que usa `collections.Mapping`, eliminado en Python 3.10. Sin parche, `from experta import *` falla. Dos soluciones: - **En el código** (más didáctico): añadir el monkey-patch antes del import. - **En el entorno** (más limpio): `pip install "frozendict>=2.3"` + `pip install --no-deps experta` + `pip install schema==0.6.7`.

```python
# PARCHE para Python 3.10+ (issue #34 de experta)
import collections
import collections.abc
if not hasattr(collections, 'Mapping'):
    collections.Mapping = collections.abc.Mapping
    collections.Iterable = collections.abc.Iterable
    collections.MutableMapping = collections.abc.MutableMapping

from experta import *

class DiagnosticoVehiculo(KnowledgeEngine):
    @DefFacts()
    def inicio(self):
        yield Fact(accion="diagnosticar")

    @Rule(Fact(accion="diagnosticar"), salience=10)
    def arrancar(self):
        print("Iniciando diagnóstico del vehículo...")
        self.declare(Fact(bateria="descargada"))
        self.declare(Fact(luces="no_encienden"))

    @Rule(Fact(bateria="descargada"), Fact(luces="no_encienden"))
    def bateria(self):
        self.declare(Fact(causa="bateria_descargada"))

    @Rule(Fact(causa="bateria_descargada"))
    def resultado(self):
        print("CAUSA PROBABLE: Batería descargada")

engine = DiagnosticoVehiculo()
engine.reset()   # activa DefFacts e InitialFact
engine.run()     # ejecuta el ciclo reconocer-actuar
```

Salida esperada:

```
Iniciando diagnóstico del vehículo...
CAUSA PROBABLE: Batería descargada
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

8.6 Sistemas expertos reales

Sistema	Origen	Qué hacía	Dato relevante
MYCIN	Stanford (1972)	Diagnóstico de infecciones y antibióticos (backward chaining + factores de certeza)	~500-600 reglas; acierto del 65-70 % frente a facultativos
XCON/R1	CMU/DEC (1978)	Configuración de ordenadores VAX	Ahorró a DEC del orden de 25 M$/año
SID	DEC (años 80)	Diseño de puertas de la CPU del VAX 9000	Generó el **93 %** de las puertas lógicas
Dendral	Stanford (años 70)	Identificación de moléculas orgánicas	Primer sistema experto "completo"

Nota sobre la tabla anterior.

> **Cuándo usar reglas y cuándo ML**
>
> Usa **reglas** cuando el dominio esté acotado, la explicabilidad sea exigible (auditoría, normativa) o no haya datos. Usa **ML** cuando haya datos abundantes y patrones complejos. Una práctica habitual es el **híbrido**: reglas como guardarraíl/validación y ML para la decisión fina.

9. Adecuación del modelo (RA2-f)

9.1 Criterios para elegir modelo

Criterio	Pregunta guía
Datos disponibles	¿Hay datos? ¿Etiquetados? ¿Representativos?
Explicabilidad	¿Necesito justificar la decisión (auditoría, normativa)?
Coste	¿Cuánto cuesta el cómputo, el etiquetado y el mantenimiento?
Precisión requerida	¿Cuánto mejora frente a una solución no-ML (baseline)?
Tiempo real / latencia	¿La decisión debe ser instantánea (RPA/reglas) o admite espera?

Nota sobre la tabla anterior.

> **La regla de oro: empieza simple**
>
> - **Google (problem framing)**: "El ML es una herramienta especializada; no quieras una solución compleja cuando una más simple funciona". Primero optimiza la solución **no-ML** y úsala como *benchmark*.
> - **scikit-learn**: su mapa de selección te hace probar primero estimadores simples y avanzar ("Try next") solo si no alcanzas el objetivo.
> - **Azure ML**: "No tengas miedo a una competición en paralelo entre varios algoritmos".
> - **No Free Lunch** (Wolpert y Macready, 1997): ningún algoritmo es mejor en promedio sobre todos los problemas → la elección depende de las suposiciones sobre tu problema.

9.2 Secuencia práctica recomendada

1. **Reglas o heurística** (explicable, sin datos).
2. **Árbol de decisión / regresión logística** (datos tabulares pequeños).
3. **Ensamble** (Random Forest / XGBoost) si hace falta precisión.
4. **Deep learning** solo si hay datos masivos y el problema lo exige.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Evidencia para «empezar simple»

Un estudio con 45 datasets tabulares de tamaño medio (~10.000 muestras) mostró que los modelos basados en árboles siguen siendo estado del arte, además de más rápidos de entrenar que el deep learning. No siempre hace falta una red neuronal.

10. Caso de estudio: Robocode como sistema de resolución de problemas completo

Los cinco requisitos del §4.1 y la elección de modelo del §9 dejan de ser teoría en cuanto se programa un bot de **Robocode Tank Royale** (el taller `T02` de la unidad):

- **Representación**: el estado del bot (posición, energía, rumbo del radar) y del campo de batalla se traduce a variables que el programa puede leer en cada turno.
- **Razonamiento**: decidir hacia dónde disparar o moverse es exactamente el problema de elegir modelo del §9 — puedes usar reglas fijas («si el enemigo está a menos de 100 px, dispara»), lógica difusa sobre la distancia y el ángulo, o una combinación.
- **Eficiencia**: el bot decide en tiempo real, turno a turno; una lógica demasiado costosa computacionalmente pierde el combate por lentitud, no por mala estrategia.
- **Adecuación del modelo (CE f)**: no hay datos de entrenamiento previos sobre "cómo gana este bot en concreto", así que el punto de partida natural es la regla o heurística (§9.2, paso 1), no el aprendizaje automático.

11. Puntos clave de la unidad

- Un sistema de resolución de problemas de IA debe cumplir **5 requisitos**: representación, razonamiento, aprendizaje/adaptabilidad, eficiencia computacional e interacción con el usuario.
- Un problema se formaliza con **estado inicial, acciones, transición, objetivo y coste**; la búsqueda recorre el **espacio de estados**.
- **BFS** es completa y óptima en pasos pero usa mucha memoria; **DFS** usa poca memoria pero no es óptima; **A*** es óptimo con heurística admisible.
- Los modelos de IA se clasifican por **aprendizaje** (supervisado/no supervisado/refuerzo), por **análisis** (descriptivo/predictivo/prescriptivo) y por **base** (conocimiento vs datos).
- **RPA "hace" y la IA "piensa"**: la automatización inteligente combina IA + BPM + RPA.
- La **lógica difusa** modela la vaguedad con grados de pertenencia [0,1], funciones de pertenencia, reglas Mamdani/Sugeno y defuzzificación por centroide.
- Los **sistemas basados en reglas** ejecutan el ciclo **reconocer-actuar** con un motor RETE; `experta` es su port Python (requiere parche en Py3.10+).
- Para elegir modelo (**CE f**): valora **datos, explicabilidad, coste, precisión y tiempo real** y **empieza por el modelo más simple**.

12. Glosario

Término	Definición
Espacio de estados	Grafo de configuraciones posibles conectadas por acciones
Estado	Configuración concreta de un problema en un momento dado
Búsqueda en anchura (BFS)	Algoritmo que explora el espacio por niveles (cola)
Búsqueda en profundidad (DFS)	Algoritmo que explora hasta el fondo (pila)
A*	Búsqueda con heurística: f = g + h (coste acumulado + estimación)
Heurística	Estimación del coste restante hasta el objetivo
Backtracking	Exploración con retroceso (base de DFS)
RPA	Automatización robótica de procesos: replica tareas en la interfaz
Automatización inteligente	Combinación de IA (decisión) + BPM (orquestación) + RPA (ejecución)
Agente de IA	Programa que percibe y actúa para lograr un objetivo

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Término	Definición
Lógica difusa	Lógica con grados de verdad en [0,1] (Zadeh, 1965)
Función de pertenencia	Asigna a cada valor su grado de pertenencia μ(x)
Variable lingüística	Variable con términos difusos (frío, templado, caliente)
FIS	Sistema de inferencia difuso (fuzzificar → reglas → defuzzificar)
Mamdani	Inferencia difusa con salida difusa defuzzificada
Sugeno/TSK	Inferencia con salida funcional, más eficiente para control
Defuzzificación	Conversión del resultado difuso a número nítido (centroide...)
Sistema de producción	Sistema basado en reglas con memoria de trabajo y motor de inferencia
Ciclo reconocer-actuar	match → resolve → act hasta agotar la agenda
RETE	Algoritmo de emparejamiento eficiente de reglas
Salience	Prioridad numérica de una regla
Forward chaining	Encadenamiento hacia delante (data-driven)
Backward chaining	Encadenamiento hacia atrás (goal-driven), propio del diagnóstico
CLIPS	Motor de reglas de la NASA (1985-1996), referencia histórica
No Free Lunch	Teorema: ningún algoritmo es mejor en promedio sobre todos los problemas

13. FAQ

¿Un problema «de juguete» como el de las jarras sirve para algo real?

Sí: es la misma técnica que usan los planificadores de rutas, los motores de búsqueda de rutas en logística o los sistemas de planificación de tareas. Si aprendes a representar estados y acciones, sabes formalizar problemas reales.

¿A* siempre encuentra la solución más corta?

Con una heurística **admisible** (que nunca sobreestime el coste restante), sí. Si la heurística sobreestima, A* pierde la garantía de optimalidad (pero suele ser más rápido).

¿La lógica difusa es lo mismo que la probabilidad?

No. La difusa modela la **vaguedad** (pertenencia imprecisa: "hace calor") y la probabilidad modela la **incertidumbre** ("hay un 60 % de que llueva"). Son matemáticas distintas.

¿`experta` no funciona en Python moderno?

La librería (2019) tiene un problema de dependencias con Python 3.10+. Con el parche `collections.Mapping = collections.abc.Mapping` (o instalando `frozendict>=2.3` y `--no-deps`) funciona bien en el entorno del curso.

¿Un sistema basado en reglas aprende de los datos?

No: las reglas las escribe un experto. No aprende solo, pero es **totalmente explicable** (puede justificar cada decisión), algo que el ML no siempre puede.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

¿Cuándo conviene usar RPA en lugar de un modelo de ML?

Si la tarea es **repetitiva y con reglas claras** (copiar datos entre sistemas, rellenar formularios), RPA. Si exige **juicio, patrones o lenguaje**, IA/ML. Y a menudo se combinan.

¿Por qué el entregable de la unidad es un juego (Robocode) y no un dataset?

Porque un bot de combate obliga a decidir en tiempo real con información incompleta — es un sistema de resolución de problemas completo y compacto, más cercano a control que a clasificación, y permite comparar en la práctica una solución por reglas con una por lógica difusa (§10).

14. Planificación sesión a sesión

Semana	Horas	Contenido	CE	Evidencia / actividad
7	3	Requisitos de un SRP; espacio de estados y representación; BFS/ DFS/A*	RA2-a	Ejercicios bloque A
8	3	Clasificación de modelos; automatización de tareas	RA2-b, RA2-c	Ejercicios bloques B y C
9	3	Lógica difusa (teoría + `N01` guiado y `N02` scikit-fuzzy)	RA2-d	`N01`, `N02`
9-10	3	Sistemas basados en reglas (teoría + `N03` experta); CE f	RA2-e, RA2-f	`N03`
9-10	—	Taller `T01`: preparar el entorno para Robocode (GitHub y Markdown ya se hicieron en la UD00)	—	Taller `T01`
10	3	Robocode Tank Royale: entrega y evaluación	RA2-a, RA2-f	`T02` Robocode

15. Tabla final RA/CE

CE	Dónde se trabaja	Con qué se evalúa
RA2-a	§4	Ejercicios bloque A, Robocode
RA2-b	§5	Ejercicios bloque B
RA2-c	§6	Ejercicios bloque C
RA2-d	§7	Ejercicios bloque D, `N02`
RA2-e	§8	Ejercicios bloque E, `N03`
RA2-f	§9, §10	Ejercicios bloque F, Robocode

16. Recursos

- Diapositivas
- **Práctica** — se hace, no se entrega ni puntúa:
  - Ejercicios de autoevaluación
  - Notebooks guiados: `N01` · lógica difusa de la propina
- **Entregas**:
  - con rúbrica: `N02` · control difuso · `N03` · sistema de reglas · `T02` · Robocode Tank Royale, la práctica de cierre del RA2

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

  - de **hecho / no hecho**, requisito sin nota: T01 · preparar el entorno (GitHub y Markdown se entregan en la UD00)
- **Documentación de Robocode**: comparativa Java/Python · tutorial Java · tutorial Python
- Los notebooks se abren desde **Práctica** y **Entregas**, con descarga y apertura en Colab.

Referencias de la unidad

- Red Blob Games · A*
- Wikipedia · A*
- scikit-fuzzy (pythonhosted)
- experta (readthedocs)
- CLIPS
- IBM · RPA
- IBM · Automatización inteligente
- scikit-learn · Elegir el estimador
- arXiv · Tree-based vs DL en tabulares
- Robocode Tank Royale · documentación oficial

17. Evaluación

Peso	Instrumento
40 % actividades	`T02` **Robocode** y los notebooks `N02` y `N03`, cada uno con su rúbrica. Robocode es el de más peso, con diferencia. El taller `T01` (y los de GitHub/Markdown de la UD00) se entregan como **hecho / no hecho** y no puntúan
60 % prueba escrita	Prueba del RA2 en Moodle: preguntas de test y de desarrollo sobre el contenido de la unidad

- **La normativa exige alcanzar todos los RA** del módulo para superarlo (art. 5.1 de la Orden 8/2025: la calificación del módulo está *«en función de la consecución de los RA»*; y las Instrucciones 26-27, que impiden calificar positivamente un módulo con RA no superados). El centro concreta ese mandato exigiendo **≥ 5 en cada RA**.

18. Recuperación

Actividades del programa de recuperación individual por RA (art. 14.4 Orden 8/2025): repetir la resolución de un problema de búsqueda con un caso distinto y las pruebas de autoevaluación de la unidad.

Volver al índice

3 de noviembre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

4.2 Diapositivas de la UD02



Diapositivas PDF

Cómo se manejan

Flechas o clic para avanzar · **F** para verlas a pantalla completa · **P** para las notas del ponente.

24 de agosto de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

4.3 Documentación de Robocode

UD02 · Comparativa Java vs Python para Robocode

Robocode Tank Royale (RCTR) ofrece API oficial para múltiples lenguajes. En esta unidad podéis elegir entre **Java (JVM)** y **Python**. A continuación se detallan las diferencias, ventajas e inconvenientes de cada opción.

¿Por qué dos lenguajes?

RCTR v0.34.0 introdujo la API multilingüe. El proyecto Robocode nació en Java, pero la comunidad y la evolución del software han traído soporte para Python, .NET y TypeScript. Esto permite elegir el lenguaje que mejor se adapte a vuestro perfil y a los objetivos del curso.

Tabla comparativa

Aspecto	Java	Python
Entorno requerido	JDK 11-23 + IntelliJ IDEA + Maven	Python 3.10+ + VS Code + pip
Dependencias	`pom.xml` (Maven) o JAR manual	`pip install robocode-tank-royale`
Sintaxis API	`camelCase()` — `setTurnLeft()`, `onScannedBot()`	`snake_case()` — `set_turn_left()`, `on_scanned_bot()`
Tipado	Estático (obligatorio)	Dinámico (opcional, con type hints)
Proyecto starter	ZIP preempaquetado (`IABDBotMaven.zip`)	Manual: 4 ficheros (`.py` + `.json` + `.cmd` + `.sh`)
Conf. servidor	Variables de entorno en configuración de ejecución de IntelliJ	Variables de entorno `SERVER_SECRET` y `SERVER_URL` en terminal
Curva aprendizaje	Más pronunciada (JDK, Maven, IDE pesado)	Más suave (pip, editor ligero)
Rendimiento batalla	Compilado JIT (mayor velocidad)	Interpretado (suficiente para RCTR, pero menor rendimiento)
Ecosistema IA/ML	Limitado para IA aplicada	Rico: scikit-learn, PyTorch, TensorFlow, etc.
Documentación comunitaria	RoboWiki mayoritariamente en Java	RoboWiki en Java (traducción a Python necesaria)
Uso en CE IA&BD	Transversal (programación genérica)	**Alineado** (Python es el lenguaje de referencia en IA)
API RCTR	Madura (existe desde el origen)	Estable (v0.34.0+, en evolución)

Ejemplos de código comparativos

CLASE BOT E IMPORTS

Java

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
import dev.robocode.tankroyale.botapi.*;
import dev.robocode.tankroyale.botapi.events.*;

public class MiBot extends Bot {
    MiBot() {
        super(BotInfo.fromFile(getConfigPath()));
    }

    private static String getConfigPath() {
        String configPath = "src/main/java/MiBot.json";
        java.io.File configFile = new java.io.File(configPath);
        if (!configFile.exists()) {
            configPath = "MiBot.json";
        }
        return configPath;
    }
}
```

Python

```python
from robocode_tank_royale.bot_api import Bot, BotInfo

class MiBot(Bot):
    def __init__(self):
        super().__init__(BotInfo.from_file("MiBot.json"))
```

MÉTODO `run()` Y BUCLE PRINCIPAL

Java

```java
public void run() {
    setAdjustRadarForBodyTurn(true);
    setAdjustGunForBodyTurn(true);
    setAdjustRadarForGunTurn(true);

    while (isRunning()) {
        setTurnRadarLeft(Double.POSITIVE_INFINITY);
        forward(100);
        turnGunLeft(360);
        back(100);
        turnGunLeft(360);
    }
}
```

Python

```python
def run(self):
    self.set_adjust_radar_for_body_turn(True)
    self.set_adjust_gun_for_body_turn(True)
    self.set_adjust_radar_for_gun_turn(True)

    while self.is_running:  # type: ignore[attr-defined]  # pylint: disable=no-member
        self.set_turn_radar_left(float("inf"))
        self.forward(100)
        self.turn_gun_left(360)
        self.back(100)
        self.turn_gun_left(360)
```

EVENT HANDLER `onScannedBot`

Java

```java
public void onScannedBot(ScannedBotEvent e) {
    fire(1);
}
```

Python

```python
def on_scanned_bot(self, scanned_bot_event: ScannedBotEvent):
    del scanned_bot_event
    self.fire(1)
```

DISPARO PREDICTIVO (TRIGONOMÉTRICO)

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Java

```java
public void onScannedBot(ScannedBotEvent e) {
    double enemyDistance = distanceTo(e.getX(), e.getY());
    double firePower = Math.min(500 / enemyDistance, 3);
    double bulletSpeed = 20 - firePower * 3;
    long time = (long) (enemyDistance / bulletSpeed);

    double enemyVelocity = e.getSpeed();
    double enemyDirection = e.getDirection();
    double deltaX = enemyVelocity * Math.cos(Math.toRadians(enemyDirection));
    double deltaY = enemyVelocity * Math.sin(Math.toRadians(enemyDirection));
    double futureX = e.getX() + (deltaX * time);
    double futureY = e.getY() + (deltaY * time);

    setTurnGunLeft(gunBearingTo(futureX, futureY));
    if (getGunTurnRemaining() <= 0 && getGunHeat() == 0) {
        fire(firePower);
    }
}
```

Python

```python
def on_scanned_bot(self, e: ScannedBotEvent):
    enemy_distance = self.distance_to(e.x, e.y)
    fire_power = min(500 / enemy_distance, 3)
    bullet_speed = 20 - fire_power * 3
    time = int(enemy_distance / bullet_speed)

    enemy_velocity = e.speed
    enemy_direction = e.direction
    delta_x = enemy_velocity * math.cos(math.radians(enemy_direction))
    delta_y = enemy_velocity * math.sin(math.radians(enemy_direction))
    future_x = e.x + (delta_x * time)
    future_y = e.y + (delta_y * time)

    self.set_turn_gun_left(self.gun_bearing_to(future_x, future_y))
    if self.gun_turn_remaining <= 0 and self.gun_heat == 0:
        self.fire(fire_power)
```

Ventajas e inconvenientes

JAVA

Ventajas:

- Documentación histórica amplia (RoboWiki, ejemplos clásicos)
- Proyecto starter preempaquetado (descomprimir y abrir en IntelliJ)
- Rendimiento compilado mayor en cálculos intensivos
- API muy madura y estable

Inconvenientes:

- Requerimientos de entorno pesados (JDK + IntelliJ)
- Curva de aprendizaje mayor si no se conoce Java/Maven
- Menos alineado con el ecosistema de IA y Big Data del curso
- Configuración de ejecución dependiente del IDE

PYTHON

Ventajas:

- Instalación sencilla (`pip install`)
- Editor ligero (VS Code) y rápido de configurar
- Sintaxis limpia y legible
- Alineado con el resto del curso (ecosistema IA/BD)
- Mejor integración con librerías de IA si se quiere ampliar el bot
- Variables de entorno simples (terminal o script)

Inconvenientes:

- Documentación histórica en Java (hay que traducir conceptualmente)
- Proyecto starter manual (4 ficheros) en lugar de ZIP preempaquetado
- Rendimiento interpretado (normalmente no perceptible en RCTR)

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Recomendación

Para la naturaleza de este curso de especialización en **Inteligencia Artificial y Big Data**, se recomienda el uso de **Python** como lenguaje principal. Python es la herramienta estándar de la industria en IA/ML, y su sintaxis clara permite centrarse en las estrategias del bot en lugar de la infraestructura del lenguaje.

La opción Java queda disponible para alumnado con experiencia previa en Java o interés particular, pero la vía Python será la que reciba soporte preferente por parte del profesorado.

Elegid vuestro camino

- **Noveles en programación o provenientes de Python** → seguid la guía Python
- **Experiencia previa en Java o interés en JVM** → seguid la guía Java
- **Dudas** → consultad al profesorado

28 de agosto de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

UD02 · Robocode Tank Royale en Java

Preparación del entorno

- Al menos java 11 (Yo estoy usando Java 23 (OpenJDK) y la versión 0.34.0 de Tank Royale)

```
java -version
```

- Descargar la última versión desde releases: https://github.com/robocode-dev/tank-royale/releases
- Ejecutar el `jar` descargado:

```
java -jar robocode-tankroyale-gui-x.y.z.jar
```

- Descarga y descomprime los Bots de ejemplo: https://github.com/robocode-dev/tank-royale/releases
- Configura la gui para acceder a la carpeta de robots: `Config` -> `Bot Root Directories` (del menú)
- [opcional] Añadir sonidos al gui. Descarga y descomprime la carpeta `sounds.zip` de: https://github.com/robocode-dev/sounds/releases

```
[directorio padre]
├── robocode-tankroyale-gui-x.y.z.jar
└── sounds/  <-- carpeta de sonidos
    ├── bots_collision.wav
    ...
    └── wall_collision.wav
```

- Necesitaras IntelliJ para poner todo en funcionamiento y para programar tu Bot.

> **Consejo**
>
> Juega un poco por tu cuenta con el entorno GUI, explora las opciones, tipos de batallas, crea alguna ronda de las mismas, inicializa Bots, añádelos, juega con la velocidad de juego, etc.

Robocode con Maven

Robocode tankroyale está disponible como artefacto del repositorio de maven, si estas familiarizado con el uso de maven puede simplificar bastante el trabajo y actualizaciones de dependencias, te dejo el enlace por si quieres investigar por tu cuenta, ya que escapa del objetivo de esta asignatura: https://central.sonatype.com/search?q=g:dev.robocode.tankroyale&smo=true

Mi primer Bot

PROYECTO INTELLIJ

Para acelerar el proceso, he preparado un proyecto en IntelliJ con toda la arquitectura básica que necesitará tu Bot. Descarga y descomprime este paquete y abre el proyecto con el IDE IntelliJ.

> **Sin maven**
>
> Si no quieres usar maven y deseas montar el proyecto por tu cuenta sigue estas indicaciones:
>
> También necesitaras la libreria (API) de Robocode Tank Royale que puedes descargar desde aquí: https://github.com/robocode-dev/tank-royale/releases. Descarga el archivo: `robocode-tankroyale-bot-api-x.y.z.jar`
>
> La estructura debería quedar de la siguiente manera:
>
> ```
> [directorio padre]
> ├── lib
> |   └──robocode-tankroyale-gui-x.y.z.jar
> └── IABDBot  <-- Proyecto de IntelliJ
>     ├── src
>     ...
>     └── IABDBot.iml
> ```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Atención

Asegúrate de entender el funcionamiento del Bot IABDBot, tiene comentarios donde se explican partes importante del Bot y que daremos por conocidas en los siguientes puntos.

Es muy importante que entiendas la diferencia entre movimientos bloqueantes y simultáneos, así como las condiciones y los eventos disponibles.

Ojo!

Gracias a este fragmento, tu bot funcionará correctamente desde IntelliJ y desde el gui de RoboCode:

```
// Constructor, que carga el fichero de configuración
IABDBot() {
    super(BotInfo.fromFile(getConfigPath()));
}

// Este método obtiene la ruta del fichero de configuración y se asegura que funcione desde IntelliJ y desde la gui de Robocode
private static String getConfigPath() {
    String miBot = "IABDBot";
    String configPath = "src/main/java/" + miBot + ".json";
    java.io.File configFile = new java.io.File(configPath);
    if (!configFile.exists()) {
        configPath = miBot + ".json";
    }
    return configPath;
}
```

ÚLTIMAS NOVEDADES DEL PROYECTO

- Se pueden grabar los combates en un fichero dentro de la carpeta `recordings` con la extensión `battle.gz` que luego podemos reproducir en diferido. Esto nos permite revisar situaciones y estudiar estrategias de mejora.

CONFIGURACIÓN DEL SERVIDOR (GUI) Y EL PROYECTO INTELLIJ PARA EJECUTAR EN LOCAL O REMOTO

El servidor permite conexiones en remoto/local a través de un password que se genera automáticamente la primera vez que lanzas la GUI. Este password lo puedes encontrar/modificar en el archivo `server.properties`, en el ejemplo siguiente la clave de acceso es **CEIABDEPM2025**

```
bots-secrets=CEIABDEPM2025
[...]
```

Ojo

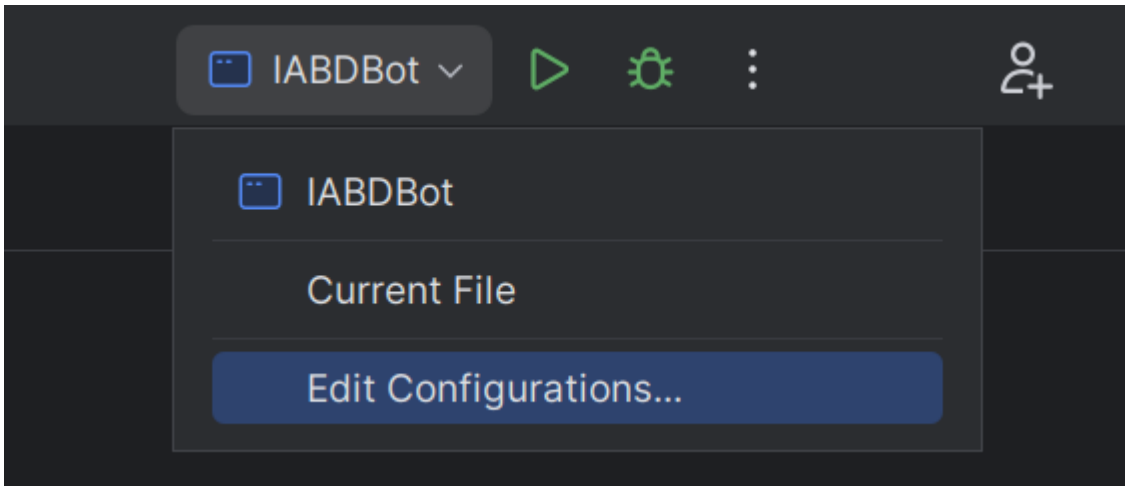
Con las últimas versiones de la gui el uso de contraseñas en el servidor está inicialmente deshabilitado, si queremos habilitarlo lo podemos hacer desde el propio interfaz: `Config/Server Options/Enable server secrets` o bien desde el fichero de configuración `server.properties` añadiendo:

```
server-secrets-enabled=true
```

Otra información que necesitaras será la IP del servidor al que quieres conectar, normalmente será `localhost` (tu host local), y cuando sea necesario hacer pruebas el profesor te facilitará su IP.

Ahora solo queda configurar nuestro proyecto de IntelliJ para configurar estas variables en el momento de ejecutar el Bot. Accede a la configuración de las ejecuciones en la parte superior derecha (una vez abierta la clase `IABDBot.java`):

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Puedes ver que en el apartado `Environment Variables` tienes el siguiente valor:

```
SERVER_SECRET=CEIABDEPM2025;SERVER_URL=ws://localhost:7654
```

Como puedes imaginar **SERVER_SECRET** hace referencia a la clave del servidor, y **SERVER_URL** a la dirección del servidor y es donde deberás reemplazar (según el caso) localhost por la IP que te indique el profesor. Así podrás ejecutar tu Bot directamente desde IntelliJ y aparecerá en el Servidor para poder añadirlo a las batallas.

Atención

Ojo, el servidor debe estar inicializado antes de lanzar el bot de lo contrario obendrás un error similar a este:

```
Exception in thread "main" dev.robocode.tankroyale.botapi.BotException: Could not create web socket for URL: ws://localhost:7654
    at dev.robocode.tankroyale.botapi.internal.BaseBotInternals.connect(BaseBotInternals.java:268)
    at dev.robocode.tankroyale.botapi.internal.BaseBotInternals.start(BaseBotInternals.java:254)
    at dev.robocode.tankroyale.botapi.BaseBot.start(BaseBot.java:114)
    at IABDBot.main(IABDBot.java:11)

Process finished with exit code 1
```

Si el Bot encuentra el servidor y la contraseña es correcta deberia aparecer esto al inicializarlo en IntelliJ:

```
Connected to: ws://localhost:7654
```

Consejo

Prueba por tu cuenta mezclando en las batallas Bots de ejemplo, con tu Bot o incluso el de algún compañero/a.

¿Cómo mejoro mi Bot?

Dividiremos todo lo que debemos conocer en 4 apartados:

CONOCIENDO EL CAMPO DE BATALLA

Sigue atentamente la documentación de RCTR sobre la **anatomía** de los Bots

Puntos importantes:

- Si el radar no se mueve, no detecta
- Inicialmente el robot, el cañón y el radar se mueven conjuntamente (pero se puede cambiar)
- El Bot se simplifica a un cuadrado de 36x36 unidades.
- Para las colisiones (con Bots o paredes) se simula como un círculo de 18 unidades de radio.

A continuación revisa las **coordenadas y ángulos**

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Puntos importantes:

- Este apartado es totalmente diferente al de Robocode Original.

Estudia también las **físicas**

Puntos importantes:

- Se frena más rápido que se acelera.
- Cuanto más rápido vas, más lento giras.
- El cañón gira máximo 20º por turno.
- El radar gira máximo 45º por turno.
- La potencia de tiro influye en el daño provocado y los puntos conseguidos, pero también en el calentamiento del cañón.
- Inicialmente el cañón está caliente (3 unidades), al principio has de esperar a que se enfrie para disparar.
- El atropello da más puntos que los disparos (pero te resta energía)
- Si chocas con una pared, pierdes puntos.
- La energía que te sobra en una ronda no da puntos (modo kamikaze con el último robot?)

Por último te en cuenta las **puntuaciones**

Puntos importantes:

- Consigues puntos si:
- tu disparo golpea a otro Bot (sino, te resta)
- eres el que mata al Bot
- cada vez que otro Bot muere, pero tu sigues vivo
- eres el último robot vivo
- atropellas a otro Bot
- si matas por atropello a otro Bot
- El último Bot que quede vivo no tiene porqué ser el ganador.

Eventos própios

Definimos una condición, y cuando esta se cumple se dispara un evento:

```
// Esta condición se dispara cuando el bot completa su giro
public static class TurnCompleteCondition extends Condition {

    private final IBot bot;

    public TurnCompleteCondition(IBot bot) {
        this.bot = bot;
    }

    @Override
    public boolean test() {
        // El método test() es el que debemos reescribir para definir el resultado de nuestra condición.
        return bot.getTurnRemaining() == 0;
    }
}
```

Podriamos usar esta condición en un fragmento similar a este:

```
waitFor(new TurnCompleteCondition());
```

Podriamos usar funciones Lambda de la siguiente manera:

```
waitFor(new Condition(() -> getTurnRemaining() == 0);
```

El resultado de los dos fragmentos seria el mismo.

PERSONALIZAR MI BOT

Podemos establecer colores para el cuerpo, torreta de cañón, el radar, el arco de escaneo y de la bala:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
Color verde = Color.fromRgb(0,255,0);
setBodyColor(Color.KHAKI);
setTurretColor(Color.KHAKI);
setRadarColor(Color.KHAKI);
setScanColor(Color.KHAKI);
setBulletColor(Color.KHAKI);
```

Puedes encontrar la lista completa de colores disponibles en la API de Robocode TankRoyale.

TÉCNICAS DE ESCANEO (RADAR)

Sentidos de nuestro Bot:

Sentido del **tacto**, tu Bot sabe cuando:

- golpea una pared (`onHitWall`),
- es alcanzado por un disparo (`onHitByBullet`),
- es alcanzado por otro Bot (`onHitBot`).

Sentido de la **vista**, tu Bot sabe cuándo ha visto otro robot, pero sólo si lo escanea (`onScannedBot`)

Otros sentidos, tu robot también sabe cuándo ha muerto (`onDeath`), cuándo ha muerto otro robot (`onRobotDeath`).

Además también es consciente de sus balas y sabe cuando una bala ha sido disparada (`onBulletFired`) ha alcanzado a un oponente (`onBulletHit`), cuando una bala golpea una pared (`onBulletWall`) o cuando una bala golpea a otra bala (`onBulletHitBullet`).

Configurar correctamente tu Bot con:

```
setAdjustRadarForBodyTurn(true);//true if the radar must adjust/compensate for the body's turn;
                                //false if the radar must turn with the body turning (default).
setAdjustGunForBodyTurn(true);  //true if the gun must adjust/compensate for the bot's turn;
                                //false if the gun must turn with the bot's turn.
setAdjustRadarForGunTurn(true); //true if the radar must adjust/compensate for the gun's turn;
                                //false if the radar must turn with the gun's turn.
```

Que el radar siempre de vueltas:

```
setTurnRadarLeft(Double.POSITIVE_INFINITY); //degrees - is the amount of degrees to turn left. If negative, the radar will turn right.
```

Revisa el API (`rescan()` i `setRescan()`)

Fijar el radar en un enemigo (one on one radar):

- Escaneo con multiplicador

```
public void onScannedBot(ScannedBotEvent e) {
    double factor=1.9;
    setTurnRadarLeft(factor * radarBearingTo(e.getX(), e.getY()));
}
```

Segun el factor:

- `1.0` - Bloqueo de radar delgado. Debe llamar a scan() para evitar perder el bloqueo. Que Dios te ayude si alguna vez te saltas un turno.
- `1.9` - El arco del radar comienza amplio y se estrecha lentamente tanto como sea posible mientras se mantiene en el objetivo.
- `2.0` : el arco del radar recorre un ángulo fijo. El ángulo exacto elegido depende de las posiciones del enemigo y del radar cuando se detecta al enemigo por primera vez. El ángulo se incrementará si es necesario para mantener el bloqueo.
- Arco de radar Ancho

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
public void onScannedBot(ScannedBotEvent e) {
    //set the wide to add to the scan.
    double wide=36.0;
    // Absolute angle towards target
    double angleToEnemy = radarBearingTo(e.getX(), e.getY());
    // Distance we want to scan from middle of enemy to either side
    // The 36.0 is how many units from the center of the enemy robot it scans.
    double extraTurn = Math.min(Math.toDegrees(Math.atan(36.0 / distanceTo(e.getX(), e.getY()))), getMaxRadarTurnRate());

    // Adjust the radar turn so it goes that much further in the direction it is going to turn
    // Basically if we were going to turn it left, turn it even more left, if right, turn more right.
    // This allows us to overshoot our enemy so that we get a good sweep that will not slip.
    if (angleToEnemy < 0)
        angleToEnemy -= extraTurn;
    else
        angleToEnemy += extraTurn;

    //Turn the radar
    setTurnRadarLeft(angleToEnemy);
}
```

- Radar Oscilante (variable global que sepa la dirección y nos ayude a cambiarla de vez en cuando)
- Escaneo más inteligente (seguir al cercano? según la situación?) ...
- Nos interesa mantener una lista de enemigos? `e.getScannedBotId()` ? y por ejemplo fijar el radar en el más débil?...

TÉCNICAS DE DESPLAZAMIENTO (MOVIMIENTO)

Pensamiento lateral

Si has jugado baloncesto antes, sabes que si quieres defender a alguien que sostiene el balón, debes maximizar tu movimiento lateral enfrentándote siempre a él (de frente). Lo mismo ocurre con tu robot.

Para conseguir esta posición debemos hacer algo similar a esto:

```java
turnLeft(bearingTo(e.getX(), e.getY()) + 90);
```

que siempre colocará tu robot perpendicular (90 grados) a tu enemigo.

¿Hacia adelante o hacia atrás?

Cuando te enfrentas a un oponente, la idea de "adelante" y "atrás" (del primer Bot) se vuelven algo obsoletas. Probablemente estés pensando más en términos de "atacar a la izquierda" o "atacar a la derecha". Para realizar un seguimiento de la dirección del movimiento, simplemente declare una variable como hicimos para oscilar el radar.

```java
class MyRobot extends Bot {
    private byte moveDirection = 1;
```

luego, cuando quieras mover tu robot, simplemente puedes decir:

```java
setForward(100 * moveDirection);
```

Puedes cambiar de dirección cambiando el valor de moveDirection de 1 a -1 así:

```java
moveDirection *= -1;
```

Cambiar de dirección

El enfoque más intuitivo para cambiar de dirección es simplemente cambiar la dirección del movimiento cada vez que golpeas una pared o golpeas a otro robot de esta manera:

```java
public void onHitWall(HitWallEvent e) { moveDirection *= -1; }
public void onHitBot(HitBotEvent e) { moveDirection *= -1; }
```

Sin embargo, descubrirás que si haces eso, terminarás presionando obstinadamente contra un robot que te embiste desde un costado (como un perro en celo). Esto se debe a que se llama a `onHitRobot()` tantas veces que moveDirection sigue cambiando y nunca te alejas.

Un mejor enfoque es simplemente probar para ver si su robot se ha detenido. Si es así, probablemente significa que has golpeado algo y querrás cambiar de dirección. Puedes hacerlo con el código:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
if (getSpeed() == 0)
    moveDirection *= -1;
```

Ponlo en tu método `doMove()` (o en cualquier otro lugar donde estés manejando el movimiento) y podrás manejar todos los eventos de impacto con las paredes u otros Bots.

¿Bailamos?

Dando vueltas (Círculos)

Puedes rodear a tu enemigo simplemente usando las técnicas anteriores:

```
public class IABDBot extends Bot {
    double moveDirection=1;
    private double ultimoEnemigoVisto;
    ...
@Override
    public void run() {
        while (isRunning()) {
            doMove();
            go();
        }

    public void onScannedBot(ScannedBotEvent e) {
        ...
        ultimoEnemigoVisto=bearingTo(e.getX(), e.getY());
        ...
    }

    public void doMove() {
        // switch directions if we've stopped
        if (getSpeed() == 0)
            moveDirection *= -1;
        // circle our enemy
        setTurnLeft(ultimoEnemigoVisto+90);
        setForward(1000 * moveDirection);
    }
```

Objetivo: rodea a tu enemigo usando el código de movimiento anterior, como un tiburón rodeando a su presa en el agua.

Evita Ametrallamiento

Un problema que puedes notar con el anterior tipo de movimiento es que es presa fácil para los objetivos predictivos porque sus movimientos son tan... predecibles.

Para evadir las balas de manera más efectiva, debes moverte de lado a lado o "atacar". Una buena forma de hacerlo es cambiar de dirección después de un cierto número de "tics", así:

```
public void doMove() {
    // switch directions if we've stopped
    if (getSpeed() == 0)
        moveDirection *= -1;
    // circle our enemy
    setTurnLeft(ultimoEnemigoVisto+90);
    if (getTurnNumber() % 20 == 0) {
        moveDirection *= -1;
        setForward(1000 * moveDirection);
    }
}
```

Mira como se balancea hacia adelante y hacia atrás usando el código de movimiento anterior. Observa lo bien que esquiva las balas. Pero ojo, porque puede afectar a tu precisión de disparo.

Cada vez más cerca

Notarás que tanto el primero como este último tipo de movimientos tienen otro problema: se atascan fácilmente en las esquinas y terminan golpeándose contra las paredes. Un problema adicional es que si su enemigo está lejos, disparan mucho pero no aciertan mucho.

Para hacer que tu robot se acerque a tu enemigo, simplemente modifica el código para que se gire ligeramente hacia su enemigo, así:

```
setTurnLeft(lastScannedBearing + (90 - (15 * moveDirection)));
```

15 es un factor en grados que puedes modificar y ajustar. Prueba!

Modificando el primero conseguimos una variación que utiliza el código anterior para lanzarse en espiral hacia su enemigo.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Mientras que con el segundo que utiliza el código anterior para acercarse cada vez más. También es bastante bueno para evadir balas.

Fíjate que ninguno de los Bots anteriores queda atrapado en una esquina por mucho tiempo.

Evitando las paredes

Un problema con todos los Bots anteriores es que golpean mucho las paredes y golpearlas agota tu energía. Una mejor estrategia sería detenerse antes de chocar contra las paredes. ¿Pero cómo?

Agregar un evento personalizado

Lo primero que debemos hacer es decidir qué tan cerca permitiremos que nuestro robot llegue a las paredes:

```
public class WallAvoider extends Bot {
    ...
    private int wallMargin = 60;
```

A continuación, agregamos un evento personalizado que se activará cuando se cumpla una determinada condición dentro del método `run()`:

```
public void run() {
    ...
    // No te acerques mucho a las paredes
    addCustomEvent(new Condition("TooCloseToWalls") {
        public boolean test() {
            // El método test() es el que debemos reescribir para definir el resultado de nuestra condición.
            return (
                // cerca de la pared izquierda
                (getX() <= wallMargin ||
                 // cerca de la pared derecha
                 getX() >= getArenaWidth() - wallMargin ||
                 // cerca de la pared inferior
                 getY() <= wallMargin ||
                 // cerca de la pared superior
                 getY() >= getArenaHeight() - wallMargin)
            );
        }
    });
    ...
}
```

Ten en cuenta que estamos creando una clase interna anónima con esta llamada. Necesitamos hacer override del método `test()` para devolver un valor booleano cuando ocurra nuestro evento personalizado.

Gestionar el evento personalizado

Lo siguiente que debemos hacer es manejar el evento, que se puede hacer así:

```
    public void onCustomEvent(CustomEvent e) {
        if (e.getCondition().getName().equals("TooCloseToWalls")) {
            // switch directions and move away
            moveDirection *= -1;
            forward(100 * moveDirection);
        }
    }
```

Sin embargo, el problema con ese enfoque es que este evento podría dispararse una y otra vez, provocando que cambiemos rápidamente de un lado a otro, sin alejarnos nunca. O si ya aparecemos cerca de la pared quedamos atrapados.

Para evitar este "sacudida de la muerte" deberíamos tener una variable que indique que estamos manejando el evento. Podemos declarar otro así:

```
public class WallAvoider extends Bot {
    ...
    private int tooCloseToWall = 0;
```

Luego maneje el evento de manera un poco más inteligente:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
public void onCustomEvent(CustomEvent e) {
    if (e.getCondition().getName().equals("TooCloseToWalls")) {
        if (tooCloseToWall <= 0) {
            // Si no estábamos ya cerca de las paredes, ahora lo estamos
            tooCloseToWall += wallMargin;
            setMaxSpeed(0); // Para!!!
        }
    }
}
```

Manejo de los dos modos

Hay dos últimos problemas que debemos resolver. En primer lugar, tenemos un método `doMove()` donde colocamos todo nuestro código de movimiento normal. Si estamos tratando de alejarnos de una pared, no queremos que se llame a nuestro código de movimiento normal, creando (una vez más) la "sacudida de la muerte". En segundo lugar, queremos volver eventualmente al movimiento "normal", por lo que eventualmente deberíamos tener el "tiempo de espera" de la variable `tooCloseToWall`.

Podemos resolver ambos problemas con la siguiente implementación de `doMove()`:

```
public void doMove() {
    ...

    // if we're close to the wall, eventually, we'll move away
    if (tooCloseToWall > 0) tooCloseToWall--;

    // switch directions if we've stopped
    if (getSpeed() == 0) {
        setMaxSpeed(8);
        moveDirection *= -1;
        setForward(1000 * moveDirection);
    }
}
```

Con todo el código anterior evitamos chocar contra las paredes. Observa cómo se desliza suavemente hacia los lados pero nunca (bueno, rara vez) los golpea.

Bot multimodo

Además de los colores que elijas, la mayor parte de la personalidad de tu robot está en su código de movimiento. Por otra parte, situaciones diferentes requieren tácticas diferentes. Usando el ejemplo de evitar paredes, es posible que desees codificar tu bot para que cambie los "modos" según ciertos criterios. Podrías pensar en algo como esto:

```
public class MultiModeBot extends Bot {
    private final static int MODO_RONDAR=0;
    private final static int MODO_ESQUIVAR=1;
    private final static int MODO_ASESINO=2;
    private int modoRobot=MODO_RONDAR;
    ...
}

public void onBotDeath(BotDeathEvent e) {
    ...
    if (getEnemyCount() > 10) {
        // Un gran número de enemigos sugiere movimientos fluidos
        modoRobot=MODO_RONDAR;
    } else if (getEnemyCount() > 1) {
        // esquivar es la mejor táctica para grupos pequeños
        modoRobot=MODO_ESQUIVAR;
    } else if (getEnemyCount() == 1) {
        // Si solo queda un robot, persíguelo
        modoRobot=MODO_ASESINO;
    }
    ...
}

public void doMove() {
    switch (modoRobot){
        case MODO_RONDAR:
            //ronda
            break;
        case MODO_ESQUIVAR:
            //esquiva
            break;
        case MODO_ASESINO:
            //asesina
            break;
    }
    ...
}
```

Los detalles se dejan (como siempre) como ejercicio para el alumnado.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

TÉCNICAS DE ATAQUE (DISPARO)

Técnicas básicas

- Separa el movimiento del cañón del movimiento del Bot (`setAdjustGunForBodyTurn`)
- Técnica de apuntado básica: `setTurnGunLeft` o `setTurnGunRight` junto con `gunBearingTo`

Fórmula de cálculo de potencia de fuego

Otro aspecto importante al disparar es calcular la potencia de fuego de tu bala. La documentación del método `fire()` explica que puedes disparar una bala en el rango de `0.1` a `3.0`. Como probablemente ya habrás concluido, es una buena idea disparar balas de baja potencia cuando tu enemigo está lejos y balas de alta resistencia cuando está cerca.

```
public void smartFire(double distance){
    if ((distance >200) || (getEnergy() <15)){
        fire(1);
    } else if (distance > 50){
        fire(2);
    } else {
        fire(3);
    }
}
```

Podrías usar una serie de declaraciones if-else-if-else para determinar la potencia de fuego, en función de si el enemigo está a 50 unidades, a 200, etc (como en el fragmento anterior). Pero tales construcciones son demasiado rígidas. Después de todo, el rango de posibles valores de potencia de fuego cae a lo largo de un continuo, no de bloques discretos. Un mejor enfoque es utilizar una fórmula. He aquí un ejemplo:

```
setFire(400/distanceTo(e.getX(), e.getY()));
```

Con esta fórmula, a medida que aumenta la distancia del enemigo, la potencia de fuego disminuye. Asimismo, a medida que el enemigo se acerca, la potencia de fuego aumenta. Los valores superiores a 3 se reducen a 3, por lo que nunca dispararemos una bala mayor que 3, pero probablemente deberíamos reducir el valor de todos modos (solo para estar seguros) de esta manera:

```
setFire(Math.min(500/distanceTo(e.getX(), e.getY()), 3));
```

Evitar disparos prematuros

Una situación que encontraras es que tu Bot dispare antes de haber girado el arma hacia el objetivo. Para evitar disparos prematuros, usa el método `getGunTurnRemaining()` para ver qué tan lejos está tu cañón del objetivo y no dispare hasta que esté cerca.

Además, no puedes disparar si el arma está "caliente" desde el último disparo y llamar a `fire()` o `setFire()` solo desperdiciará un turno. Podemos probar si el arma está fría llamando a `getGunHeat()`.

Apuntando de manera predictiva

O... "Usar la trigonometría para impresionar a tus amigos y destruir a tus enemigos".

Si quisiéramos poder golpear a un robot que recorre las paredes siempre fallaríamos, necesitamos poder predecir dónde estará en el futuro, pero ¿cómo podemos hacerlo?

$$ Distancia = Velocidad * Tiempo $$ Usando la anterior formula podemos calcular cuánto tiempo tardará una bala en llegar allí.

- **Distancia**: se puede encontrar llamando a `distanceTo(e.getX(), e.getY())`
- **Velocidad**: según la documentación de RCTR, una bala viaja a una velocidad de: $$ 20 - potenciaDeFuego * 3 $$
- **Tiempo**: podemos calcular el tiempo resolviendo: $$ Tiempo = {Distancia \over Velocidad} $$

El siguiente código lo hace:

```
// calcular la potencia basado en la distancia
double enemyDistance = distanceTo(e.getX(), e.getY());
double firePower = Math.min(500 / enemyDistance, 3);
// calcular la velocidad de la bala
double bulletSpeed = 20 - firePower * 3;
// calculamos el tiempo que necesita la bala para impactar al enemigo
long time = (long) (enemyDistance / bulletSpeed);
```

Obteniendo futuras coordenadas X,Y

A continuación, debemos calcular la posición futura de nuestro enemigo:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
// Calcular la velocidad actual de tu enemigo
double enemyVelocity = e.getSpeed();
// Calcular la dirección actual del enemigo
double enemyDirection = e.getDirection();
// Descomponer el desplazamiento en las coordenadas X e Y
// Componente X del desplazamiento COSENO
double deltaX = enemyVelocity * Math.cos(Math.toRadians(enemyDirection));
// Componente Y del desplazamiento SENO
double deltaY = enemyVelocity * Math.sin(Math.toRadians(enemyDirection));

// Calcular las coordenadas futuras
double futureX = e.getX() + (deltaX * time);
double futureY = e.getY() + (deltaY * time);
```

Girando el arma al punto previsto

Ahora debemos apuntar nuestro cañon al lugar previsto de impacto:

```java
setTurnGunLeft(gunBearingTo(futureX, futureY));
```

Disparando!

Por último disparamos nuestro cañon

```java
if (getGunTurnRemaining() <= 0 && getGunHeat() == 0) {
    fire(firePower);
}
```

Mejor aún...!

Aquí te dejo algunas mejoras para que las valores:

- ¿Debemos esperar siempre a que el arma esté completamente en la dirección calculada?
- Cuando nuestro enemigo se acerca a una pared, nuestros disparos impactan en ella.
- ¿Puedo saber a quien disparo si mi previsión de impacto falla mucho?

Investigación y desarrollo propio

A partir de aquí el trabajo será individual de cada alumno (puede solaparse con el paso 3). Deberéis investigar/prever a vuestros adversarios (los conocidos, y los de vuestros compañeros). Aplicar las técnicas que consideréis más útiles para intentar quedar lo más arriba posible en la tabla de clasificación.

19 de octubre de 2025

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

UD02 · Robocode Tank Royale en Python

Antes de empezar

Leed primero la Comparativa Java vs Python para entender las diferencias y elegir vuestro camino.

Esta guía es la versión **Python** de la actividad Robocode. Si buscáis la versión Java, id a Robocode en Java.

Preparación del entorno

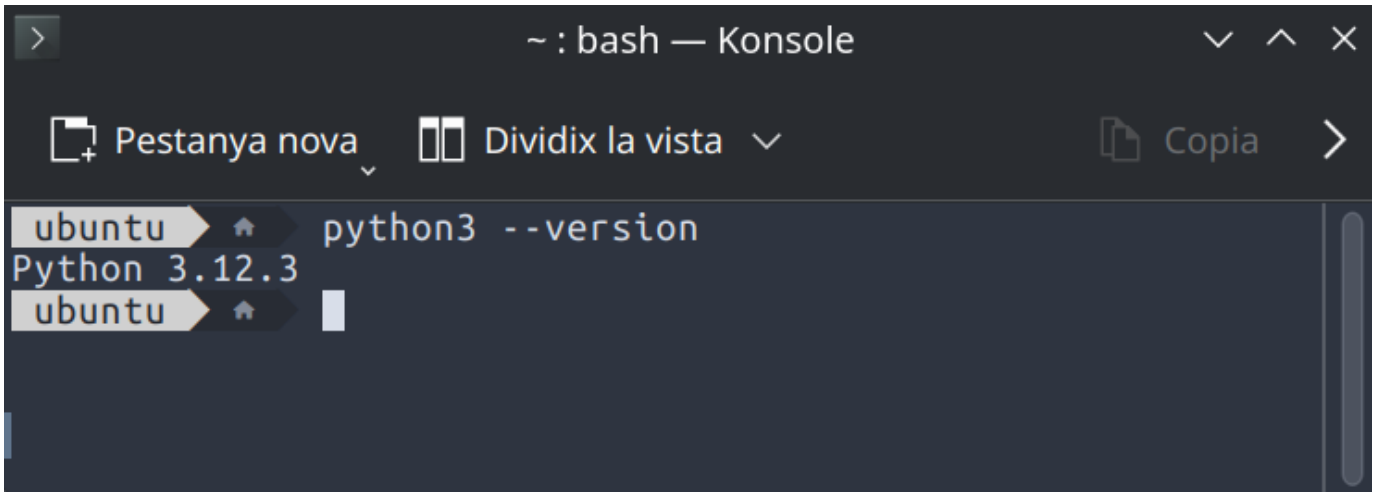
REQUISITOS PREVIOS

- **Python 3.10 o superior** instalado en el sistema
- **VS Code** (Visual Studio Code) u otro editor de código
- Conexión a Internet para instalar dependencias

COMPROBACIÓN DE PYTHON

Abrid un terminal y ejecutad:

```
python3 --version
```



> **Windows**
>
> Si usas Windows, el comando es `python --version` en lugar de `python3 --version`.

Si no tenéis Python instalado, descargadlo desde https://www.python.org/downloads/ o usad el gestor de paquetes de vuestro sistema:

Linux (Debian/Ubuntu)

```
sudo apt install python3 python3-pip python3-venv
```

Linux (Fedora)

```
sudo dnf install python3 python3-pip
```

macOS

```
brew install python3
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Windows

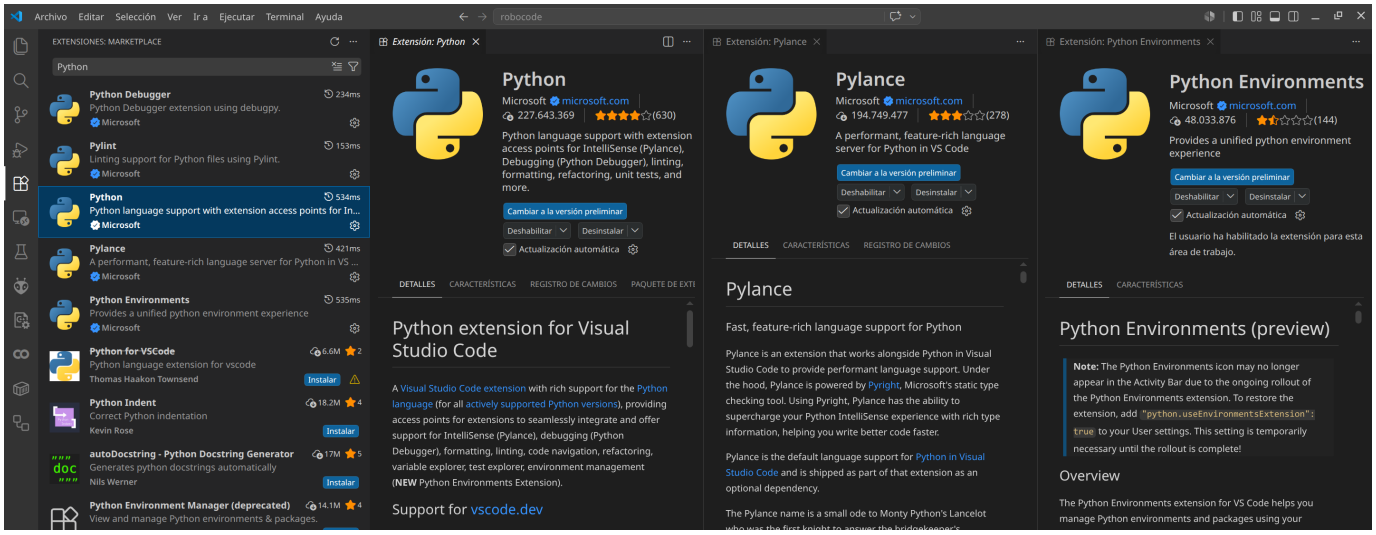
Descargad el instalador desde https://www.python.org/downloads/ y marcad la opción "Add Python to PATH"

VS CODE Y EXTENSIONES

Si aún no tenéis VS Code, descargadlo desde https://code.visualstudio.com/

Una vez instalado, añadid las siguientes extensiones:

1. **Python** (de Microsoft) — todo el soporte para Python: linting, debugging, IntelliSense
2. **Pylance** (de Microsoft) — completado de código y tipado avanzado
3. *Opcional*: **Python Environments** (de Microsoft)



INSTALACIÓN DE LA API DE ROBOCODE

La API de Python para Robocode Tank Royale se distribuye como paquete pip:

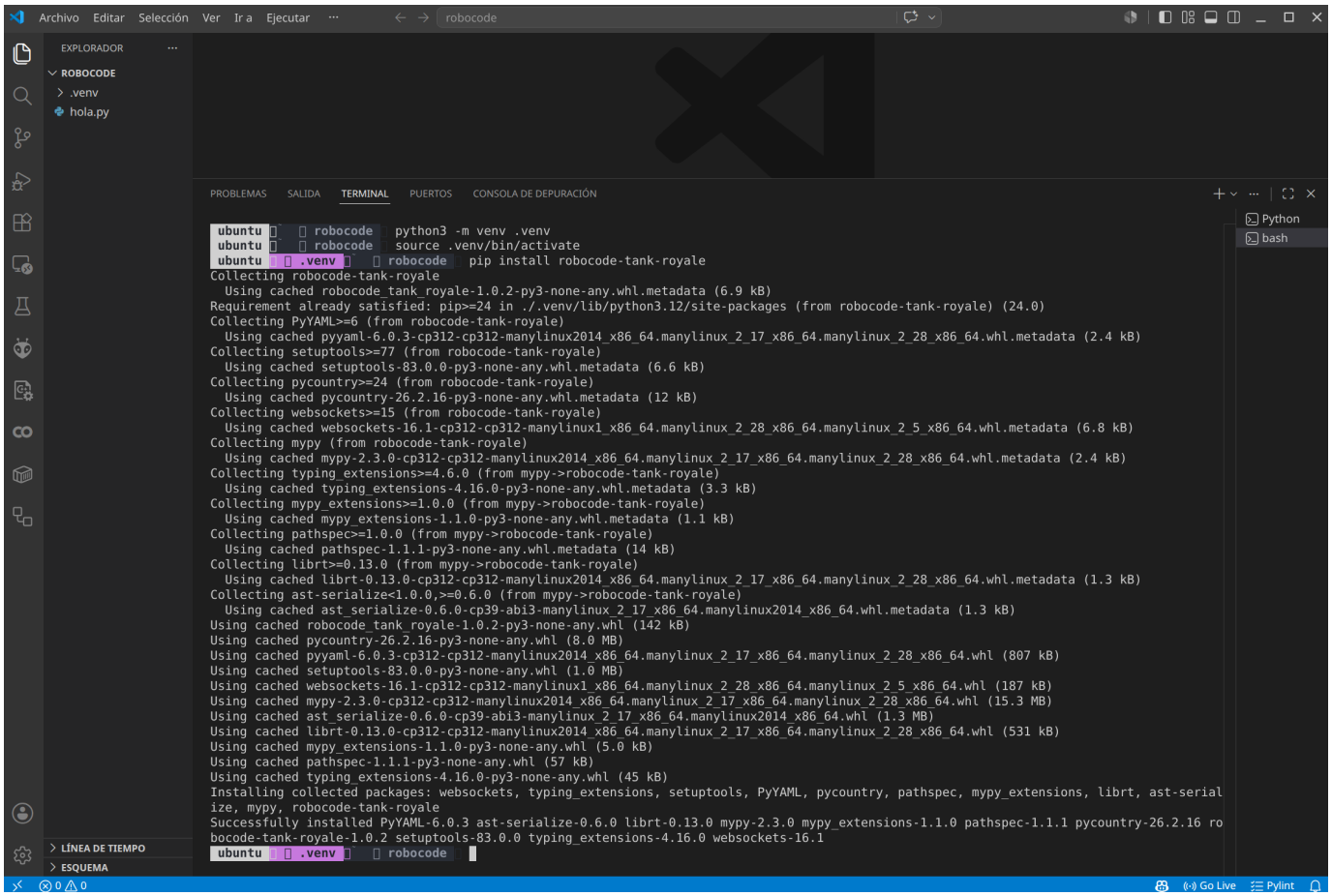
```
pip install robocode-tank-royale
```

O si trabajáis con entornos virtuales:

```
python3 -m venv .venv
# Linux/macOS:
source .venv/bin/activate
# Windows PowerShell:
# .venv\Scripts\Activate.ps1
# Windows CMD:
# .venv\Scripts\activate.bat

pip install robocode-tank-royale
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0



La GUI de Robocode Tank Royale

DESCARGA Y EJECUCIÓN DE LA GUI

La GUI (interfaz gráfica) de Robocode es **independiente del lenguaje**. El proceso es idéntico tanto si usáis Java como Python:

1. Id a https://github.com/robocode-dev/tank-royale/releases
2. Descargad la última versión del fichero `robocode-tankroyale-gui-x.y.z.jar`
3. Ejecutadlo:

```
java -jar robocode-tankroyale-gui-x.y.z.jar
```

> **Java necesario para la GUI**
>
> La GUI requiere Java 11 o superior para ejecutarse, independientemente de que vuestro bot esté escrito en Python. Si no tenéis Java, ved el Taller 1: preparar el entorno o instalad OpenJDK:
>
> ```
> sudo apt install default-jdk   # Linux
> ```

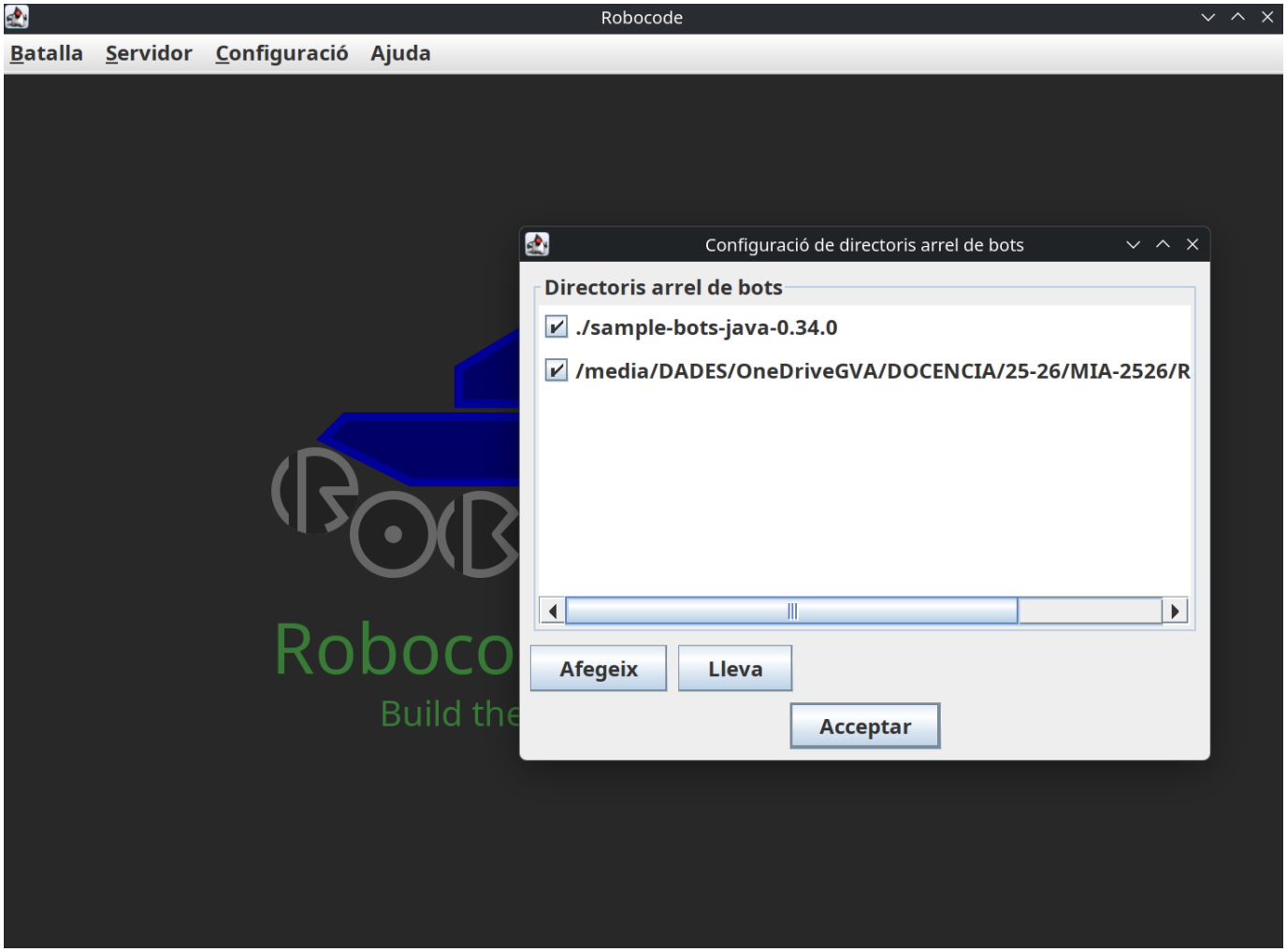
CONFIGURACIÓN INICIAL DE LA GUI

Una vez abierta la GUI:

1. Descargad y descomprimid los **Bots de ejemplo** desde la misma página de releases
2. Id a `Config` → `Bot Root Directories` y añadid la carpeta donde habéis descomprimido los bots de ejemplo
3. *Opcional*: descargad y descomprimid `sounds.zip` de los releases de sonidos en el mismo directorio que el JAR

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
[directorio padre]
├── robocode-tankroyale-gui-x.y.z.jar
├── sounds/
│   ├── bots_collision.wav
│   ...
│   └── wall_collision.wav
└── bots/              <-- carpeta de bots de ejemplo
```



Consejo

Jugad un poco con el entorno GUI, explorad las opciones, tipos de batallas, cread alguna ronda, inicializad bots, añadidlos, ajustad la velocidad de juego, etc.

Vuestro primer Bot en Python

ESTRUCTURA DEL PROYECTO

Cread una carpeta para vuestro bot (por ejemplo, `MiBot`) con la siguiente estructura:

```
MiBot/
├── MiBot.py
├── MiBot.json
├── MiBot.cmd          (Windows)
└── MiBot.sh           (Linux/macOS)
```

FICHERO DE CONFIGURACIÓN JSON

El bot necesita un fichero `.json` con la información del bot. Cread `MiBot.json`:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
{
  "name": "MiBot",
  "version": "1.0",
  "authors": ["Vuestro Nombre"],
  "description": "Mi primer bot con Python",
  "homepage": "",
  "countryCodes": ["es"],
  "platform": "Python",
  "programmingLang": "Python 3.x"
}
```

CÓDIGO BÁSICO DEL BOT

Cread `MiBot.py` con el siguiente contenido:

```
import os
from robocode_tank_royale.bot_api import Bot, BotInfo
from robocode_tank_royale.bot_api.events import ScannedBotEvent, HitByBulletEvent

class MiBot(Bot):
    def __init__(self):
        config_path = os.path.join(os.path.dirname(__file__), "MiBot.json")
        super().__init__(BotInfo.from_file(config_path))

    def run(self):
        while self.is_running:  # type: ignore[attr-defined]  # pylint: disable=no-member
            self.forward(100)
            self.turn_gun_left(360)
            self.back(100)
            self.turn_gun_left(360)

    def on_scanned_bot(self, scanned_bot_event: ScannedBotEvent):
        del scanned_bot_event
        self.fire(1)

    def on_hit_by_bullet(self, hit_by_bullet_event: HitByBulletEvent):
        bearing = self.calc_bearing(hit_by_bullet_event.bullet.direction)
        self.turn_right(90 - bearing)

def main():
    bot = MiBot()
    bot.start()

if __name__ == "__main__":
    main()
```

VS Code: Pylint/Pylance no encuentran el paquete

Si después de seleccionar el intérprete del venv (`Ctrl+Shift+P` → "Python: Select Interpreter" → `./.venv/bin/python`) los errores persisten, es porque Pylint usa su propio intérprete y Pylance necesita el fichero `.vscode/settings.json` .

Cread la carpeta `.vscode/` dentro de la carpeta de vuestro bot (`/home/ubuntu/robocode/MiBot/`) y añadid el siguiente fichero:

```
{
    "python.defaultInterpreterPath": "${workspaceFolder}/.venv/bin/python",
    "python.linting.pylintEnabled": false,
    "python.linting.enabled": false,
    "python.analysis.autoImportCompletions": true
}
```

Si no tenéis Pylint instalado en el venv (solo lo tiene el sistema), VS Code puede ejecutar el Pylint del sistema que no ve los paquetes instalados en el venv. Opciones: (1) desactivar Pylint (`"python.linting.enabled": false`) y confiar solo en Pylance, o (2) instalar Pylint en el venv: `pip install pylint` .

Los subrayados rojos desaparecerán al recargar la ventana (`Ctrl+Shift+P` → "Developer: Reload Window").

SCRIPTS DE INICIO

Windows (**`MiBot.cmd`**)

```
@echo off
python MiBot.py %*
```

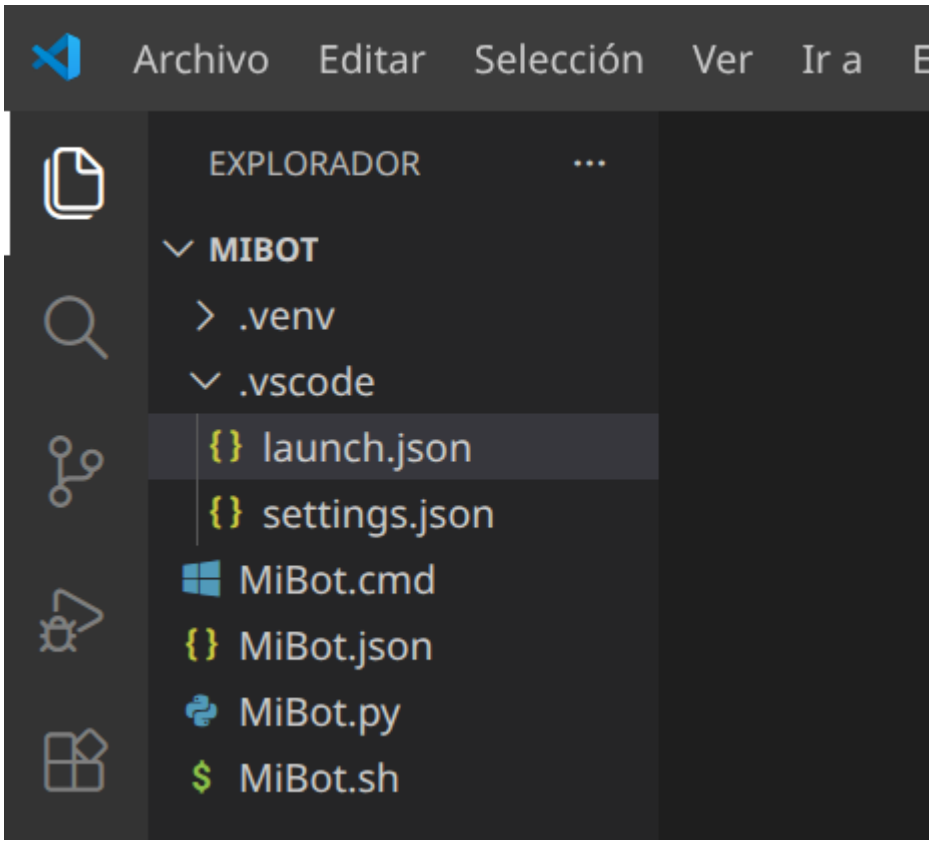

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Linux/macOS (`MiBot.sh`)

```
#!/usr/bin/env sh
python3 "$(dirname "$0")/MiBot.py" "$@"
```

Hacedlo ejecutable:

```
chmod 755 MiBot.sh
```



PRUEBA DEL BOT

1. Iniciad la GUI de Robocode (`java -jar robocode-tankroyale-gui-x.y.z.jar`)
2. Abrid un terminal en el directorio de vuestro bot
3. Ejecutad:

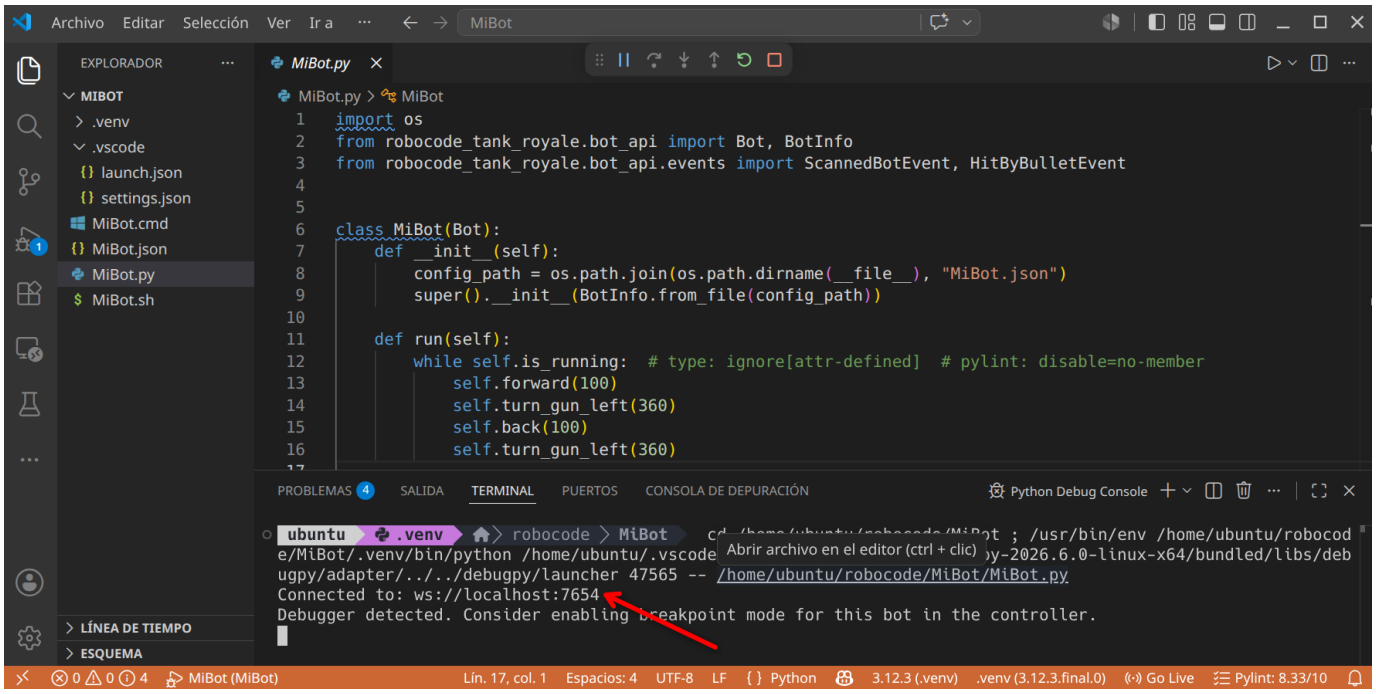
```
python3 MiBot.py
```

Si todo va bien, veréis un mensaje similar a:

```
Connected to: ws://localhost:7654
```

Y vuestro bot aparecerá en la lista de bots disponibles en la GUI.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Configuración del servidor y secrets

SECRETS DEL SERVIDOR

El servidor de Robocode puede requerir una contraseña (secret) para conectarse. Esta contraseña se genera automáticamente la primera vez que lanzáis la GUI.

Para habilitar los secrets:

1. Id a `Config` → `Server Options` → `Enable server secrets` en la GUI
2. O editad `server.properties` y añadid:

```
server-secrets-enabled=true
bots-secrets=CEIABDEPM2025
```

CONECTAR EL BOT CON SECRET

Estableced la variable de entorno `SERVER_SECRET` antes de ejecutar el bot:

Linux/macOS

```
export SERVER_SECRET=CEIABDEPM2025
python3 MiBot.py
```

Windows CMD

```
set SERVER_SECRET=CEIABDEPM2025
python MiBot.py
```

Windows PowerShell

```
$Env:SERVER_SECRET = "CEIABDEPM2025"
python MiBot.py
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

CONECTARSE A UN SERVIDOR REMOTO

Si el profesor indica una IP remota, podéis especificarla con `SERVER_URL`:

Linux/macOS

```
export SERVER_URL=ws://192.168.1.100:7654
export SERVER_SECRET=CEIABDEPM2025
python3 MiBot.py
```

Windows CMD

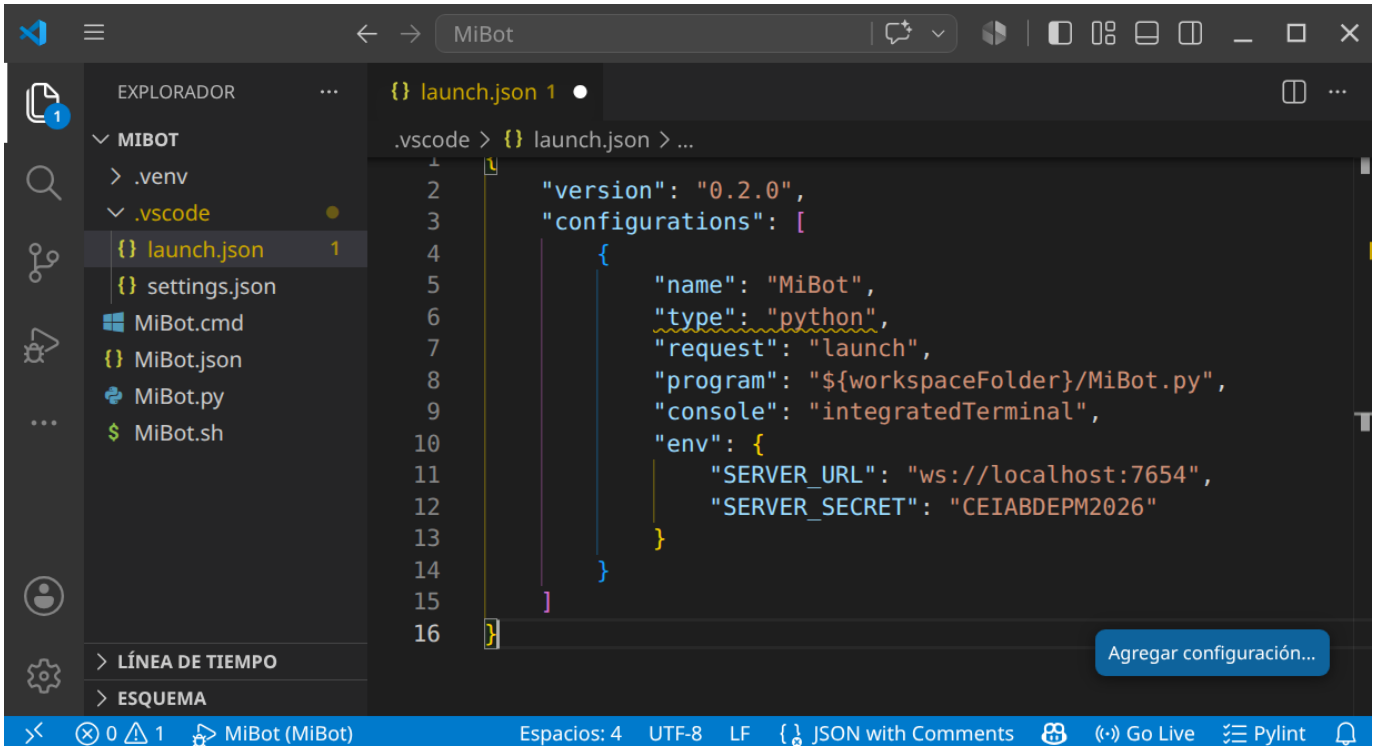
```
set SERVER_URL=ws://192.168.1.100:7654
set SERVER_SECRET=CEIABDEPM2025
python MiBot.py
```

USO CON VS CODE (LAUNCH.JSON)

Para mayor comodidad, podéis configurar VS Code para ejecutar el bot con las variables de entorno definidas en el fichero `.vscode/launch.json`:

Cread la carpeta `.vscode/` dentro del proyecto y el fichero `launch.json`:

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "MiBot",
            "type": "python",
            "request": "launch",
            "program": "${workspaceFolder}/MiBot.py",
            "console": "integratedTerminal",
            "env": {
                "SERVER_URL": "ws://localhost:7654",
                "SERVER_SECRET": "CEIABDEPM2025"
            }
        }
    ]
}
```



Ahora podéis ejecutar el bot directamente desde VS Code pulsando `F5` o yendo a `Run` → `Start Debugging`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

El servidor debe estar en marcha

El servidor (la GUI) debe estar inicializado **antes** de lanzar el bot. Si no, obtendréis un error:

```
Could not create web socket for URL: ws://localhost:7654
```

API de Python: diferencias clave con Java

La API de Python utiliza **snake_case** en lugar de **camelCase**. Aquí tenéis una tabla de correspondencia:

Java	Python
`setTurnLeft(degrees)`	`set_turn_left(degrees)`
`setForward(dist)`	`set_forward(dist)`
`setTurnRadarLeft(degrees)`	`set_turn_radar_left(degrees)`
`setAdjustRadarForBodyTurn(bool)`	`set_adjust_radar_for_body_turn(bool)`
`distanceTo(x, y)`	`distance_to(x, y)`
`bearingTo(x, y)`	`bearing_to(x, y)`
`gunBearingTo(x, y)`	`gun_bearing_to(x, y)`
`getX()`	`self.x` (propiedad)
`getY()`	`self.y` (propiedad)
`getSpeed()`	`self.speed` (propiedad)
`getEnergy()`	`self.energy` (propiedad)
`getTurnRemaining()`	`self.turn_remaining` (propiedad)
`getGunHeat()`	`self.gun_heat` (propiedad)
`isRunning()`	`self.is_running` (propiedad)
`onScannedBot(e)`	`on_scanned_bot(self, e)`
`e.getX()`	`e.x` (propiedad)
`e.getY()`	`e.y` (propiedad)
`e.getSpeed()`	`e.speed` (propiedad)
`e.getDirection()`	`e.direction` (propiedad)
`Double.POSITIVE_INFINITY`	`float("inf")`
`Math.min(a, b)`	`min(a, b)`
`Math.cos(angle)`	`math.cos(angle)`
`Math.toRadians(degrees)`	`math.radians(degrees)`

¿Cómo mejoro mi Bot?

Dividiremos todo lo que debemos conocer en 4 apartados:

CONOCIENDO EL CAMPO DE BATALLA

Sigue atentamente la documentación de RCTR sobre la **anatomía** de los Bots

Puntos importantes:

- Si el radar no se mueve, no detecta
- Inicialmente el robot, el cañón y el radar se mueven conjuntamente (pero se puede cambiar)

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- El Bot se simplifica a un cuadrado de 36x36 unidades.
- Para las colisiones (con Bots o paredes) se simula como un círculo de 18 unidades de radio.

A continuación revisa las **coordenadas y ángulos**

Puntos importantes:

- Este apartado es totalmente diferente al de Robocode Original.

Estudia también las **físicas**

Puntos importantes:

- Se frena más rápido que se acelera.
- Cuanto más rápido vas, más lento giras.
- El cañón gira máximo 20º por turno.
- El radar gira máximo 45º por turno.
- La potencia de tiro influye en el daño provocado y los puntos conseguidos, pero también en el calentamiento del cañón.
- Inicialmente el cañón está caliente (3 unidades), al principio has de esperar a que se enfríe para disparar.
- El atropello da más puntos que los disparos (pero te resta energía)
- Si chocas con una pared, pierdes puntos.
- La energía que te sobra en una ronda no da puntos (modo kamikaze con el último robot?)

Por último ten en cuenta las **puntuaciones**

Puntos importantes:

- Consigues puntos si:
- tu disparo golpea a otro Bot (sino, te resta)
- eres el que mata al Bot
- cada vez que otro Bot muere, pero tu sigues vivo
- eres el último robot vivo
- atropellas a otro Bot
- si matas por atropello a otro Bot
- El último Bot que quede vivo no tiene porqué ser el ganador.

Eventos propios

Definimos una condición, y cuando esta se cumple se dispara un evento:

```
from robocode_tank_royale.bot_api.events import Condition

class TurnCompleteCondition(Condition):
    def __init__(self, bot):
        super().__init__()
        self.bot = bot

    def test(self):
        return self.bot.turn_remaining == 0
```

Podríamos usar esta condición en un fragmento similar a este:

```
self.wait_for(TurnCompleteCondition(self))
```

Podríamos usar funciones lambda de la siguiente manera:

```
self.wait_for(Condition(lambda: self.turn_remaining == 0))
```

El resultado de los dos fragmentos sería el mismo.

PERSONALIZAR MI BOT

Podemos establecer colores para el cuerpo, torreta de cañón, el radar, el arco de escaneo y de la bala:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
verde = Color.from_rgb(0, 255, 0)
self.set_body_color(Color.from_rgb(0, 0, 0))
self.set_turret_color(Color.from_rgb(255, 0, 0))
self.set_radar_color(Color.from_rgb(255, 0, 0))
self.set_scan_color(Color.from_rgb(255, 255, 0))
self.set_bullet_color(Color.from_rgb(255, 255, 255))
```

Puedes encontrar la lista completa de colores disponibles en la API de Robocode TankRoyale.

TÉCNICAS DE ESCANEO (RADAR)

Sentidos de nuestro Bot:

Sentido del **tacto**, tu Bot sabe cuando:

- golpea una pared (`on_hit_wall`),
- es alcanzado por un disparo (`on_hit_by_bullet`),
- es alcanzado por otro Bot (`on_hit_bot`).

Sentido de la **vista**, tu Bot sabe cuándo ha visto otro robot, pero sólo si lo escanea (`on_scanned_bot`)

Otros sentidos, tu robot también sabe cuándo ha muerto (`on_death`), cuándo ha muerto otro robot (`on_bot_death`).

Además también es consciente de sus balas y sabe cuando una bala ha sido disparada (`on_bullet_fired`) ha alcanzado a un oponente (`on_bullet_hit`), cuando una bala golpea una pared (`on_bullet_wall_hit`) o cuando una bala golpea a otra bala (`on_bullet_hit_bullet`).

Configurar correctamente tu Bot con:

```
self.set_adjust_radar_for_body_turn(True)   # True si el radar debe ajustar/compensar el giro del cuerpo;
                                            # False si el radar gira con el cuerpo (por defecto).
self.set_adjust_gun_for_body_turn(True)     # True si el cañón debe ajustar/compensar el giro del cuerpo;
                                            # False si el cañón gira con el cuerpo (por defecto).
self.set_adjust_radar_for_gun_turn(True)    # True si el radar debe ajustar/compensar el giro del cañón;
                                            # False si el radar gira con el cañón (por defecto).
```

Que el radar siempre de vueltas:

```
self.set_turn_radar_left(float("inf"))  # grados - cantidad de grados a girar a la izquierda. Si es negativo, gira a la derecha.
```

Revisa el API (`rescan()` y `set_rescan()`)

Fijar el radar en un enemigo (one on one radar):

- Escaneo con multiplicador

```
def on_scanned_bot(self, e: ScannedBotEvent):
    factor = 1.9
    self.set_turn_radar_left(factor * self.radar_bearing_to(e.x, e.y))
```

Según el factor:

- `1.0` - Bloqueo de radar delgado. Debe llamar a scan() para evitar perder el bloqueo. Que Dios te ayude si alguna vez te saltas un turno.
- `1.9` - El arco del radar comienza amplio y se estrecha lentamente tanto como sea posible mientras se mantiene en el objetivo.
- `2.0` : el arco del radar recorre un ángulo fijo. El ángulo exacto elegido depende de las posiciones del enemigo y del radar cuando se detecta al enemigo por primera vez. El ángulo se incrementará si es necesario para mantener el bloqueo.
- Arco de radar Ancho

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```python
def on_scanned_bot(self, e: ScannedBotEvent):
    wide = 36.0
    # Ángulo absoluto hacia el objetivo
    angle_to_enemy = self.radar_bearing_to(e.x, e.y)
    # Distancia que queremos escanear desde el centro del enemigo a cada lado
    extra_turn = min(
        math.degrees(math.atan(wide / self.distance_to(e.x, e.y))),
        self.max_radar_turn_rate
    )
    # Ajustamos el giro del radar para que vaya más allá en la dirección que ya gira
    if angle_to_enemy < 0:
        angle_to_enemy -= extra_turn
    else:
        angle_to_enemy += extra_turn
    # Giramos el radar
    self.set_turn_radar_left(angle_to_enemy)
```

- Radar Oscilante (variable global que sepa la dirección y nos ayude a cambiarla de vez en cuando)
- Escaneo más inteligente (seguir al cercano? según la situación?)
- Nos interesa mantener una lista de enemigos? `e.scanned_bot_id`? y por ejemplo fijar el radar en el más débil?

TÉCNICAS DE DESPLAZAMIENTO (MOVIMIENTO)

Pensamiento lateral

Si has jugado baloncesto antes, sabes que si quieres defender a alguien que sostiene el balón, debes maximizar tu movimiento lateral enfrentándote siempre a él (de frente). Lo mismo ocurre con tu robot.

Para conseguir esta posición debemos hacer algo similar a esto:

```python
self.turn_left(self.bearing_to(e.x, e.y) + 90)
```

que siempre colocará tu robot perpendicular (90 grados) a tu enemigo.

¿Hacia adelante o hacia atrás?

Cuando te enfrentas a un oponente, la idea de "adelante" y "atrás" (del primer Bot) se vuelven algo obsoletas. Probablemente estés pensando más en términos de "atacar a la izquierda" o "atacar a la derecha". Para realizar un seguimiento de la dirección del movimiento, simplemente declara una variable como hicimos para oscilar el radar.

```python
class MiBot(Bot):
    def __init__(self):
        super().__init__(BotInfo.from_file("MiBot.json"))
        self.move_direction = 1
```

luego, cuando quieras mover tu robot, simplemente puedes decir:

```python
self.set_forward(100 * self.move_direction)
```

Puedes cambiar de dirección cambiando el valor de move_direction de 1 a -1 así:

```python
self.move_direction *= -1
```

Cambiar de dirección

El enfoque más intuitivo para cambiar de dirección es simplemente cambiar la dirección del movimiento cada vez que golpeas una pared o golpeas a otro robot de esta manera:

```python
def on_hit_wall(self, e: HitWallEvent):
    self.move_direction *= -1

def on_hit_bot(self, e: HitBotEvent):
    self.move_direction *= -1
```

Sin embargo, descubrirás que si haces eso, terminarás presionando obstinadamente contra un robot que te embiste desde un costado (como un perro en celo). Esto se debe a que se llama a `on_hit_bot` tantas veces que `move_direction` sigue cambiando y nunca te alejas.

Un mejor enfoque es simplemente probar para ver si tu robot se ha detenido. Si es así, probablemente significa que has golpeado algo y querrás cambiar de dirección. Puedes hacerlo con el código:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
if self.speed == 0:
    self.move_direction *= -1
```

Ponlo en tu método que maneje el movimiento y podrás manejar todos los eventos de impacto con las paredes u otros Bots.

¿Bailamos?

Dando vueltas (Círculos)

Puedes rodear a tu enemigo simplemente usando las técnicas anteriores:

```
class MiBot(Bot):
    def __init__(self):
        super().__init__(BotInfo.from_file("MiBot.json"))
        self.move_direction = 1
        self.ultimo_enemigo_visto = 0

    def run(self):
        while self.is_running:  # type: ignore[attr-defined]  # pylint: disable=no-member
            self.do_move()
            self.go()

    def on_scanned_bot(self, e: ScannedBotEvent):
        self.ultimo_enemigo_visto = self.bearing_to(e.x, e.y)

    def do_move(self):
        if self.speed == 0:
            self.move_direction *= -1
        self.set_turn_left(self.ultimo_enemigo_visto + 90)
        self.set_forward(1000 * self.move_direction)
```

Objetivo: rodea a tu enemigo usando el código de movimiento anterior, como un tiburón rodeando a su presa en el agua.

Evita Ametrallamiento

Un problema que puedes notar con el anterior tipo de movimiento es que es presa fácil para los objetivos predictivos porque sus movimientos son tan... predecibles.

Para evadir las balas de manera más efectiva, debes moverte de lado a lado o "atacar". Una buena forma de hacerlo es cambiar de dirección después de un cierto número de "tics", así:

```
def do_move(self):
    if self.speed == 0:
        self.move_direction *= -1
    self.set_turn_left(self.ultimo_enemigo_visto + 90)
    if self.turn_number % 20 == 0:
        self.move_direction *= -1
        self.set_forward(1000 * self.move_direction)
```

Mira como se balancea hacia adelante y hacia atrás usando el código de movimiento anterior. Observa lo bien que esquiva las balas. Pero ojo, porque puede afectar a tu precisión de disparo.

Cada vez más cerca

Notarás que tanto el primero como este último tipo de movimientos tienen otro problema: se atascan fácilmente en las esquinas y terminan golpeándose contra las paredes. Un problema adicional es que si tu enemigo está lejos, disparan mucho pero no aciertan mucho.

Para hacer que tu robot se acerque a tu enemigo, simplemente modifica el código para que se gire ligeramente hacia su enemigo, así:

```
self.set_turn_left(self.ultimo_enemigo_visto + (90 - (15 * self.move_direction)))
```

15 es un factor en grados que puedes modificar y ajustar. ¡Prueba!

Modificando el primero conseguimos una variación que utiliza el código anterior para lanzarse en espiral hacia su enemigo.

Mientras que con el segundo que utiliza el código anterior para acercarse cada vez más. También es bastante bueno para evadir balas.

Fíjate que ninguno de los Bots anteriores queda atrapado en una esquina por mucho tiempo.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Evitando las paredes

Un problema con todos los Bots anteriores es que golpean mucho las paredes y golpearlas agota tu energía. Una mejor estrategia sería detenerse antes de chocar contra las paredes. ¿Pero cómo?

Agregar un evento personalizado

Lo primero que debemos hacer es decidir qué tan cerca permitiremos que nuestro robot llegue a las paredes:

```
class WallAvoider(Bot):
    def __init__(self):
        super().__init__(BotInfo.from_file("WallAvoider.json"))
        self.wall_margin = 60
        self.too_close_to_wall = 0
        self.move_direction = 1
```

A continuación, agregamos un evento personalizado que se activará cuando se cumpla una determinada condición dentro del método `run()`:

```
def run(self):
    # ... inicialización ...
    # No te acerques mucho a las paredes
    self.add_custom_event(Condition("TooCloseToWalls", lambda: (
        self.x <= self.wall_margin or
        self.x >= self.arena_width - self.wall_margin or
        self.y <= self.wall_margin or
        self.y >= self.arena_height - self.wall_margin
    )))
    # ... resto del bucle ...
```

Necesitamos hacer override del método `test()` para devolver un valor booleano cuando ocurra nuestro evento personalizado.

Gestionar el evento personalizado

Lo siguiente que debemos hacer es manejar el evento, que se puede hacer así:

```
def on_custom_event(self, e: CustomEvent):
    if e.condition.name == "TooCloseToWalls":
        self.move_direction *= -1
        self.set_forward(100 * self.move_direction)
```

Sin embargo, el problema con ese enfoque es que este evento podría dispararse una y otra vez, provocando que cambiemos rápidamente de un lado a otro, sin alejarnos nunca. O si ya aparecemos cerca de la pared quedamos atrapados.

Para evitar este "sacudida de la muerte" deberíamos tener una variable que indique que estamos manejando el evento:

```
def on_custom_event(self, e: CustomEvent):
    if e.condition.name == "TooCloseToWalls":
        if self.too_close_to_wall <= 0:
            self.too_close_to_wall += self.wall_margin
            self.set_max_speed(0)
```

Manejo de los dos modos

Hay dos últimos problemas que debemos resolver. En primer lugar, tenemos un método `do_move()` donde colocamos todo nuestro código de movimiento normal. Si estamos tratando de alejarnos de una pared, no queremos que se llame a nuestro código de movimiento normal, creando (una vez más) la "sacudida de la muerte". En segundo lugar, queremos volver eventualmente al movimiento "normal", por lo que eventualmente deberíamos tener el "tiempo de espera" de la variable `too_close_to_wall`.

Podemos resolver ambos problemas con la siguiente implementación de `do_move()`:

```
def do_move(self):
    if self.too_close_to_wall > 0:
        self.too_close_to_wall -= 1
    if self.speed == 0:
        self.set_max_speed(8)
        self.move_direction *= -1
        self.set_forward(1000 * self.move_direction)
```

Con todo el código anterior evitamos chocar contra las paredes. Observa cómo se desliza suavemente hacia los lados pero nunca (bueno, rara vez) los golpea.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Bot multimodo

Además de los colores que elijas, la mayor parte de la personalidad de tu robot está en su código de movimiento. Por otra parte, situaciones diferentes requieren tácticas diferentes. Usando el ejemplo de evitar paredes, es posible que desees codificar tu bot para que cambie los "modos" según ciertos criterios. Podrías pensar en algo como esto:

```
MODO_RONDAR = 0
MODO_ESQUIVAR = 1
MODO_ASESINO = 2

class MultiModeBot(Bot):
    def __init__(self):
        super().__init__(BotInfo.from_file("MultiModeBot.json"))
        self.modo_robot = MODO_RONDAR

    def on_bot_death(self, e: BotDeathEvent):
        if self.enemy_count > 10:
            self.modo_robot = MODO_RONDAR
        elif self.enemy_count > 1:
            self.modo_robot = MODO_ESQUIVAR
        elif self.enemy_count == 1:
            self.modo_robot = MODO_ASESINO

    def do_move(self):
        if self.modo_robot == MODO_RONDAR:
            # ronda
            pass
        elif self.modo_robot == MODO_ESQUIVAR:
            # esquiva
            pass
        elif self.modo_robot == MODO_ASESINO:
            # asesina
            pass
```

Los detalles se dejan (como siempre) como ejercicio para el alumnado.

TÉCNICAS DE ATAQUE (DISPARO)

Técnicas básicas

- Separa el movimiento del cañón del movimiento del Bot (`set_adjust_gun_for_body_turn`)
- Técnica de apuntado básica: `set_turn_gun_left` o `set_turn_gun_right` junto con `gun_bearing_to`

Fórmula de cálculo de potencia de fuego

Otro aspecto importante al disparar es calcular la potencia de fuego de tu bala. La documentación del método `fire()` explica que puedes disparar una bala en el rango de `0.1` a `3.0`. Como probablemente ya habrás concluido, es una buena idea disparar balas de baja potencia cuando tu enemigo está lejos y balas de alta resistencia cuando está cerca.

```
def smart_fire(self, distance):
    if distance > 200 or self.energy < 15:
        self.fire(1)
    elif distance > 50:
        self.fire(2)
    else:
        self.fire(3)
```

Podrías usar una serie de declaraciones if-elif-else para determinar la potencia de fuego, en función de si el enemigo está a 50 unidades, a 200, etc (como en el fragmento anterior). Pero tales construcciones son demasiado rígidas. Después de todo, el rango de posibles valores de potencia de fuego cae a lo largo de un continuo, no de bloques discretos. Un mejor enfoque es utilizar una fórmula. He aquí un ejemplo:

```
self.set_fire(400 / self.distance_to(e.x, e.y))
```

Con esta fórmula, a medida que aumenta la distancia del enemigo, la potencia de fuego disminuye. Asimismo, a medida que el enemigo se acerca, la potencia de fuego aumenta. Los valores superiores a 3 se reducen a 3, por lo que nunca dispararemos una bala mayor que 3, pero probablemente deberíamos reducir el valor de todos modos (solo para estar seguros) de esta manera:

```
self.set_fire(min(500 / self.distance_to(e.x, e.y), 3))
```

Evitar disparos prematuros

Una situación que encontraras es que tu Bot dispare antes de haber girado el arma hacia el objetivo. Para evitar disparos prematuros, usa la propiedad `gun_turn_remaining` para ver qué tan lejos está tu cañón del objetivo y no dispare hasta que esté cerca.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Además, no puedes disparar si el arma está "caliente" desde el último disparo y llamar a `fire()` o `set_fire()` solo desperdiciará un turno. Podemos probar si el arma está fría llamando a `gun_heat`.

Apuntando de manera predictiva

O... "Usar la trigonometría para impresionar a tus amigos y destruir a tus enemigos".

Si quisiéramos poder golpear a un robot que recorre las paredes siempre fallaríamos, necesitamos poder predecir dónde estará en el futuro, pero ¿cómo podemos hacerlo?

\[Distancia = Velocidad \times Tiempo \]

Usando la anterior fórmula podemos calcular cuánto tiempo tardará una bala en llegar allí.

- **Distancia**: se puede encontrar llamando a `distance_to(e.x, e.y)`
- **Velocidad**: según la documentación de RCTR, una bala viaja a una velocidad de: $$ 20 - potenciaDeFuego \times 3 $$
- **Tiempo**: podemos calcular el tiempo resolviendo: $$ Tiempo = {Distancia \over Velocidad} $$

El siguiente código lo hace:

```
# calcular la potencia basado en la distancia
enemy_distance = self.distance_to(e.x, e.y)
fire_power = min(500 / enemy_distance, 3)
# calcular la velocidad de la bala
bullet_speed = 20 - fire_power * 3
# calculamos el tiempo que necesita la bala para impactar al enemigo
time = int(enemy_distance / bullet_speed)
```

Obteniendo futuras coordenadas X, Y

A continuación, debemos calcular la posición futura de nuestro enemigo:

```
import math

# Calcular la velocidad actual de tu enemigo
enemy_velocity = e.speed
# Calcular la dirección actual del enemigo
enemy_direction = e.direction
# Descomponer el desplazamiento en las coordenadas X e Y
# Componente X del desplazamiento COSENO
delta_x = enemy_velocity * math.cos(math.radians(enemy_direction))
# Componente Y del desplazamiento SENO
delta_y = enemy_velocity * math.sin(math.radians(enemy_direction))

# Calcular las coordenadas futuras
future_x = e.x + (delta_x * time)
future_y = e.y + (delta_y * time)
```

Girando el arma al punto previsto

Ahora debemos apuntar nuestro cañón al lugar previsto de impacto:

```
self.set_turn_gun_left(self.gun_bearing_to(future_x, future_y))
```

¡Disparando!

Por último disparamos nuestro cañón:

```
if self.gun_turn_remaining <= 0 and self.gun_heat == 0:
    self.fire(fire_power)
```

Mejor aún...!

Aquí te dejo algunas mejoras para que las valores:

- ¿Debemos esperar siempre a que el arma esté completamente en la dirección calculada?
- Cuando nuestro enemigo se acerca a una pared, nuestros disparos impactan en ella.
- ¿Puedo saber a quién disparo si mi previsión de impacto falla mucho?

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Investigación y desarrollo propio

A partir de aquí el trabajo será individual de cada alumno (puede solaparse con el paso 3). Deberéis investigar/prever a vuestros adversarios (los conocidos, y los de vuestros compañeros). Aplicar las técnicas que consideréis más útiles para intentar quedar lo más arriba posible en la tabla de clasificación.

Algunos recursos adicionales:

- Documentación oficial de la API Python
- Tutorial oficial "My First Bot for Python"
- The Book of Robocode — estrategias avanzadas
- RoboWiki — conocimiento comunitario (código en Java, conceptos universales)

27 de agosto de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

5. UD05

5.1 UD05 — Sistemas expertos y controladores inteligentes

Unidad 5 · 12 h · semanas 11-15 (7 de diciembre al 14 de enero)

Continúa las reglas y la lógica difusa de RA2, justo antes de que la fragmentación de Navidad parta el curso. Se evalúa con **seis entregas prácticas** y la prueba escrita del RA5.

1. Introducción

Todo el curso ha sido **cómo construir** sistemas de IA. Esta unidad da un paso más: los **sistemas expertos**, que codifican el conocimiento de un experto humano en reglas y lo utilizan para **diagnosticar, asesorar y controlar**. Y lo conectamos con el **control**, porque un sistema experto puede actuar como **controlador inteligente** de un proceso físico, compitiendo con los controladores clásicos (PID).

Antes de construir nada hay que situar **qué es el conocimiento** y **cómo se representa** para que una máquina pueda usarlo — esa es la primera pregunta de la inteligencia artificial simbólica, y la respondemos con la jerarquía DIKW y el continuo de representaciones del §5.

El recorrido de la unidad:

1. **Arquitectura y dinámica** de los sistemas expertos (base de conocimiento, motor de inferencia, ciclo reconocer-actuar).
2. **Estructuras elementales** de representación del conocimiento (reglas, marcos, lógica, ontologías, redes semánticas).
3. **Representar y simular comportamientos** con la librería `experta`, en dominios muy diversos: diagnóstico médico, clasificación, control de procesos.
4. **Sistemas híbridos reglas/datos**: cuando las reglas se combinan o se extraen del aprendizaje automático.
5. **Sistemas de razonamiento impreciso**: la lógica difusa, para trabajar con lo que no es blanco o negro.
6. **Cómo influye la variación de las características** en la dinámica del sistema (sensibilidad y robustez).
7. **Estrategias de control** de un sistema experto.
8. **Controladores inteligentes** (difuso, por reglas, redes neuronales, MPC) y su influencia en el comportamiento del sistema.
9. **Aplicaciones y tendencias** (MYCIN, XCON, BRMS, neuro-simbólico, IA explicable).

Hilo conductor de la unidad

Un sistema experto convierte el conocimiento de un experto en reglas que pueden diagnosticar, asesorar y controlar. Primero situamos qué es ese conocimiento y cómo se representa; después lo construimos y simulamos, con reglas puras, híbridas o difusas; luego estudiamos por qué a veces falla; y por último lo usamos como controlador, comparándolo con el PID clásico.

2. Resultado de aprendizaje y criterios de evaluación

RA5 — Aplica sistemas expertos evaluando la influencia de los controladores inteligentes en el comportamiento del sistema.

CE	Criterio de evaluación	Bloque
RA5-a	Se ha descrito la dinámica y las estructuras elementales de los sistemas expertos.	§4-5
RA5-b	Se han determinado las destrezas necesarias para representar y simular comportamientos básicos de sistemas de muy diversos ámbitos.	§6-8
RA5-c	Se ha razonado cómo influye la variación de las características de los sistemas en su dinámica de actuación.	§9
RA5-d	Se han desarrollado estrategias de control definiendo los objetivos y las especificaciones de la respuesta del sistema.	§10
RA5-e		§11

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

CE	Criterio de evaluación	Bloque
	Se han relacionado los controladores inteligentes con el comportamiento del sistema.	

Nota sobre la tabla anterior.

Bloque de contenidos oficial (RD 279/2021, anexo I)

El currículo llama a este bloque, textualmente, *«Sistemas Expertos»*, y le asigna estos contenidos:

- *Dinámica de los sistemas expertos.*
- *Estructuras elementales de los sistemas expertos.*
- *Representar y simular comportamientos básicos.*
- *Estrategias de control de un sistema experto.*
- *Aplicaciones de sistemas expertos.*
- *Tendencias en sistemas expertos.*

Son **seis contenidos para cinco criterios de evaluación**, y no encajan uno a uno: los dos últimos (aplicaciones y tendencias) no tienen CE propio y se reparten entre todos, mientras que RA5-c (variación de las características) **no tiene contenido explícito** en el anexo y lo desarrolla el centro (art. 3.3 del Decreto 95/2026).

3. Objetivos de la unidad

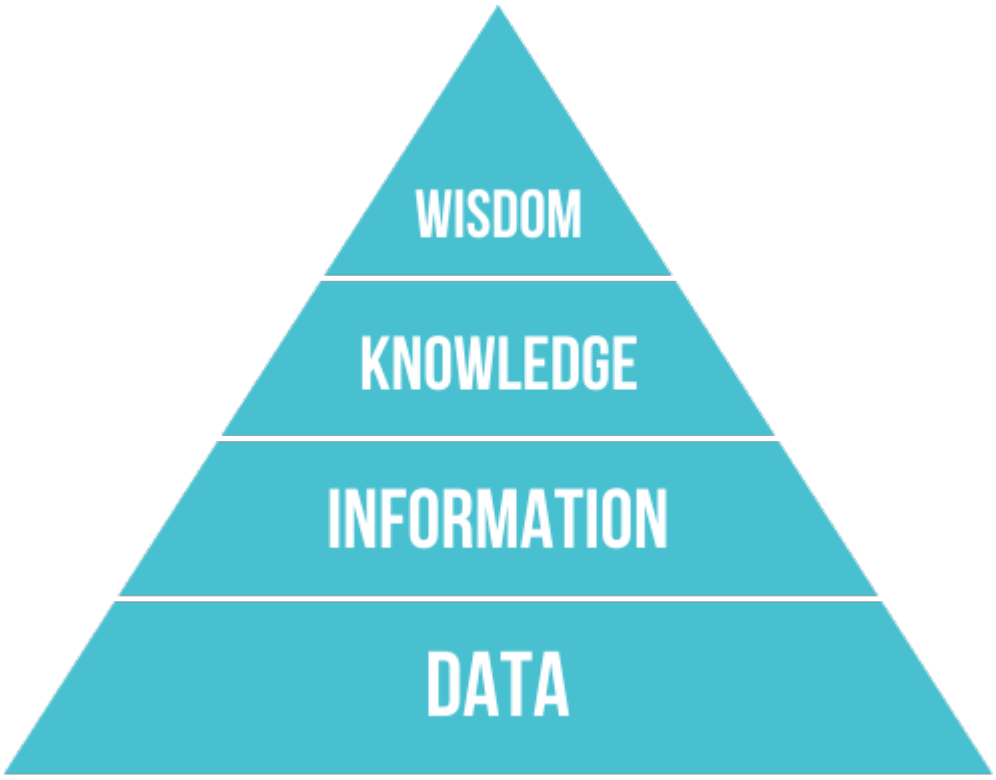
Objetivo	Al terminar la unidad serás capaz de…
O1	**Situar** el conocimiento en la jerarquía DIKW y **describir** la arquitectura de un sistema experto.
O2	**Diferenciar** los modos de representación del conocimiento (reglas, frames, lógica, ontologías, redes semánticas).
O3	**Representar y simular** un comportamiento básico con `experta`, en un dominio real.
O4	**Combinar** reglas con datos cuando el conocimiento experto no basta por sí solo.
O5	**Aplicar** lógica difusa a un problema con incertidumbre.
O6	**Explicar** cómo influye la variación de reglas, hechos y umbrales en la dinámica (sensibilidad/robustez).
O7	**Desarrollar** estrategias de control (salience, control de meta) con sus especificaciones.
O8	**Relacionar** los controladores inteligentes (difuso, por reglas, ANN, MPC) con el comportamiento del sistema y compararlos con un PID.
O9	Conocer aplicaciones reales y tendencias de los sistemas expertos.

4. Arquitectura y dinámica de los sistemas expertos (RA5-a)

4.1 Del dato al conocimiento: la jerarquía DIKW

Antes de representar el conocimiento hay que distinguirlo de sus vecinos. La **jerarquía DIKW** (*Data, Information, Knowledge, Wisdom*) los ordena:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Nivel	Qué es	Ejemplo
Datos	Hechos o valores registrados, independientes de quien los lee	«Un reloj inteligente registra la temperatura corporal»
Información	Los datos interpretados por un agente; subjetiva	«La temperatura corporal es 37 ºC»
Conocimiento	Información integrada en un modelo del mundo	«Si la temperatura supera 37 ºC, la persona tiene fiebre»
Sabiduría	Meta-conocimiento: cuándo y cómo aplicar el conocimiento	«Si tiene fiebre, debe tomar paracetamol»

Nota sobre la tabla anterior.

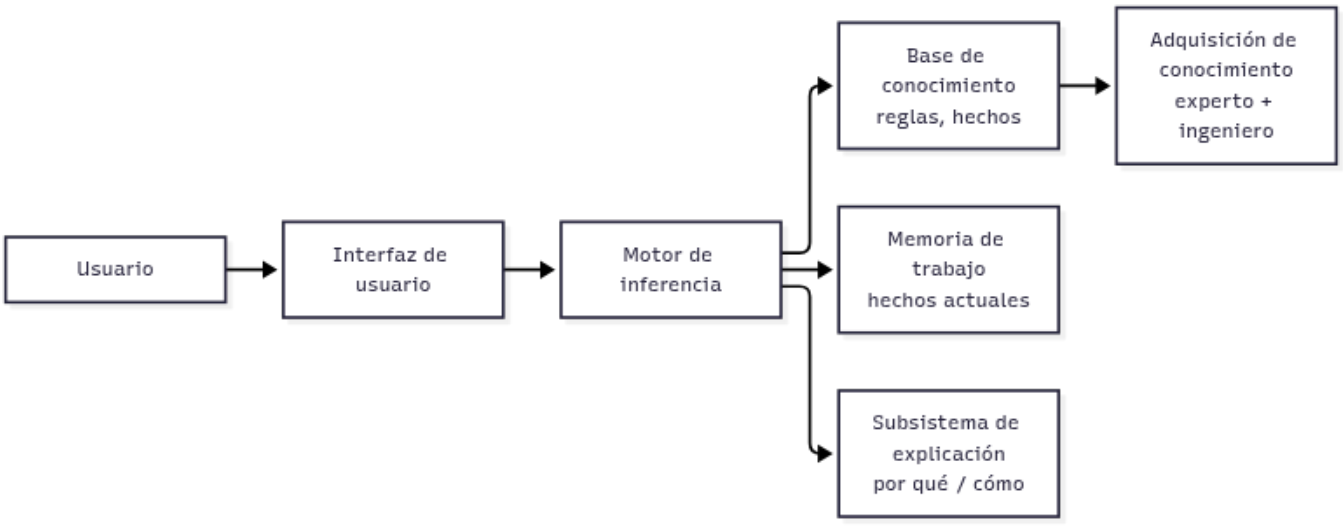
Por qué importa esta distinción

Un sistema experto no almacena datos ni información: almacena **conocimiento** (reglas) y, en los más avanzados, un poco de **sabiduría** (metarreglas que deciden cuándo aplicar otras reglas — el «control de meta» del §10.1). Confundir los niveles es el error más común al empezar a diseñar uno: se acumulan datos en vez de codificar reglas.

4.2 ¿Qué es un sistema experto?

Un **sistema experto** es un programa de IA que **emula el razonamiento de un experto humano** en un dominio concreto: codifica su conocimiento en una **base de conocimiento** y utiliza un **motor de inferencia** para aplicar ese conocimiento a los hechos del problema. Comenzaron su desarrollo en los años 70 y fueron muy populares esa década y la siguiente: se consideran los primeros sistemas de IA capaces de obtener resultados con utilidad práctica real.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



4.3 Componentes

Componente	Función
Interfaz de usuario	Vía de las consultas: pregunta datos, muestra resultados y alerta de datos erróneos. Incluye un **módulo de comunicaciones** (con otros sistemas, útil en automatización industrial)
Base de conocimiento (KB)	Reglas y hechos del dominio, formalizados por el **ingeniero de conocimiento** junto al experto. Puede organizarse como listas, objetos relacionados, cálculo de predicados o redes semánticas
Memoria de trabajo (base de hechos)	Hechos actuales: los que aporta el usuario o los sensores, más los deducidos durante el razonamiento. Guarda el histórico de estados de la consulta
Motor de inferencia	Evalúa qué reglas se cumplen, resuelve conflictos y ejecuta las acciones. Determina el orden de las acciones y la interacción entre las partes del sistema
Subsistema de explicación	Justifica el razonamiento («por qué» y «cómo» llegó a esa conclusión), mostrando las reglas usadas. Ayuda también al ingeniero de conocimiento a **depurar** el motor y al experto a **verificar** la coherencia de la base
Adquisición de conocimiento	Herramientas para incorporar nuevo conocimiento sin perfil técnico (el famoso *bottleneck*: conseguir que el experto vuelque lo que sabe es la parte más lenta de construir el sistema)

Nota sobre la tabla anterior.

> **La explicación marca la diferencia**
>
> La capacidad de **explicar** su razonamiento es lo que distingue a un sistema experto de un modelo de ML: en dominios regulados (medicina, finanzas) la justificación es obligatoria. MYCIN fue el pionero de esta *explicabilidad*, mostrando las reglas de inferencia que empleó — aunque, para consultas complejas, enumerar todas las reglas puede resultar tedioso para el usuario.

4.4 El ciclo de ejecución (dinámica)

El motor de inferencia opera en un **bucle reconocer-actuar**:

1. **Reconocer (match)**: se comparan los hechos de la memoria de trabajo con las condiciones (LHS) de las reglas; las reglas activas van a la **agenda**.
2. **Resolver (resolve)**: si hay varias reglas activas, se elige una según la estrategia de control (salience, recency, especificidad — ver §10.1).
3. **Actuar (act)**: se ejecuta el consecuente (RHS), que declara/modifica/retira hechos, y el ciclo se repite hasta agotar la agenda.

El objetivo de fondo es hacer que la lógica sea **explícita** en vez de estar enterrada en código que solo revisa un informático: con reglas legibles, el propio experto del dominio puede revisar y mantener el sistema.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

4.5 Encadenamiento

- **Hacia delante (forward, data-driven)**: parte de los hechos y deduce conclusiones. Ideal para monitorización, control de procesos en tiempo real y planificación.
- **Hacia atrás (backward, goal-driven)**: parte de una **meta** y busca qué hechos la sustentan, preguntando al usuario solo lo necesario. Ideal para diagnóstico (MYCIN, Prolog).
- **Encadenamiento mixto**: combina los dos anteriores.
- Otros mecanismos de razonamiento: **búsqueda heurística** (el proceso de inferencia recorre un árbol) y **herencia** (en entornos orientados a objetos, un objeto hijo hereda propiedades del padre).

Las dos reglas de inferencia clásicas

Antes de forward/backward chaining está la lógica que los sustenta: **Modus Ponens** («si P implica Q, y P es verdad, entonces Q es verdad») y **Modus Tollens** («si P implica Q, y Q no es cierto, entonces P no es cierto»). El encadenamiento hacia delante aplica Modus Ponens repetidamente; el encadenamiento hacia atrás busca qué P sustentaría un Q dado.

4.6 Incertidumbre: factores de certeza

MYCIN introdujo los **factores de certeza (CF)** para manejar la incertidumbre: cada regla tiene un CF y se combinan al encadenar. Alternativas modernas: lógica difusa (§8), teoría de Dempster-Shafer o redes bayesianas (Heckerman & Shortliffe, 1992).

Combinar factores de certeza

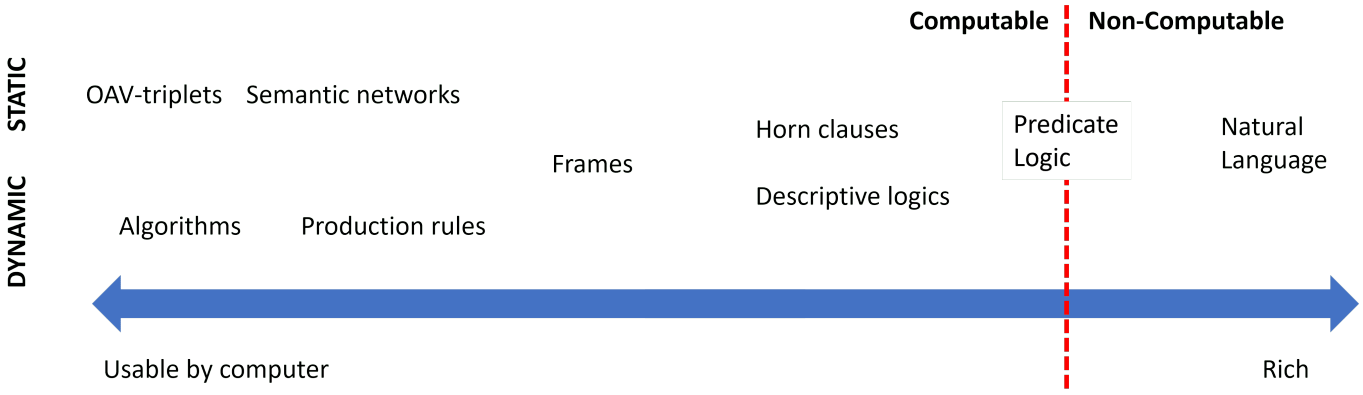
Una regla dice «SI hay fiebre alta ENTONCES sospecha de meningitis **con CF = 0,6**». Otra regla independiente aporta «SI rigidez de nuca ENTONCES sospecha de meningitis **con CF = 0,4**». MYCIN combina ambos apoyos para dar una confianza conjunta mayor que cada una por separado (combinación de evidencia). Si en cambio hubiera evidencia en contra, los CF se descuentan. De ahí nace el concepto de **acumulación de evidencia**, que luego se formalizó con redes bayesianas.

Esta gestión explícita de la **incertidumbre** es una diferencia clave frente a un sistema de reglas «duro» (todo verdadero o falso): permite razonar con conocimiento parcial y explicar el nivel de confianza de una conclusión. La lógica difusa del §8 resuelve el mismo problema de otra forma: en vez de un factor de certeza sobre una regla booleana, deja que el propio hecho sea **parcialmente verdadero**.

5. Estructuras elementales de representación (RA5-a/b)

5.1 Un continuo, no una lista cerrada

Representar el conocimiento es uno de los problemas fundamentales de la IA: hay que hacerlo de forma **entendible** para las máquinas, **útil** para resolver problemas y **eficiente** de procesar. Las representaciones forman un **continuo**:



En un extremo están las representaciones **simples** (algoritmos): eficientes para el ordenador pero poco flexibles. En el otro, las **flexibles** (texto natural): muy expresivas pero no directamente utilizables por una máquina. Entre medias viven las que usamos en esta unidad.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

5.2 Las representaciones, con sus ventajas y límites

Representación	Estructura	Inferencia	Ventajas	Limitaciones
Pares atributo-valor (tripletes objeto-atributo-valor)	Lista de nodos y aristas de un grafo: «el perro es un animal, tiene cuatro patas…»	Recorrido y coincidencia de atributos	Muy simple de construir a partir de descripciones	Poco expresiva para relaciones complejas
Reglas de producción	`SI ... ENTONCES ...`	Forward/ backward + resolución de conflictos	Modular, legible, explicable	Difícil modelar jerarquías; lenta con bases grandes
Representaciones jerárquicas	Árbol: nodos = conceptos, aristas = relaciones (Animales → Vertebrados → Mamíferos → Perros)	Recorrido del árbol, herencia	Natural para taxonomías	Rígida si un concepto pertenece a varias ramas
Marcos (frames)	Registros con ranuras (slots) y valores por defecto	Herencia de clases, disparadores	Taxonomías y conocimiento estructurado	Conflicto con herencia múltiple
Lógica formal	Predicados de primer orden (propuesta por Aristóteles hace más de 2000 años)	Resolución, unificación, deducción	Rigor matemático	Explosión combinatoria; poco utilizable directamente por máquinas (salvo un subconjunto, como en Prolog)
Redes semánticas	Grafos dirigidos con etiquetas	Búsqueda en grafos, propagación	Intuitiva para relaciones	Semántica ambigua
Ontologías	Clases, propiedades, axiomas (OWL)	Razonadores (Pellet, HermiT)	Interoperabilidad, reutilización	Curva de aprendizaje alta

Nota sobre la tabla anterior.

Lo mismo en cuatro formas distintas

El hecho «si la temperatura supera 37 ºC, la persona tiene fiebre» se puede escribir como **regla de producción** (`SI temperatura > 37 ENTONCES fiebre`), como **lógica proposicional** (\(p \rightarrow q\), con \(p\): «tiene fiebre», \(q\): «debe tomar paracetamol»), como **jerarquía** (Síntomas → Fiebre → Paracetamol) o como **par atributo-valor** (`persona.temperatura = 38.2`, evaluado por una regla externa). Elegir la representación es elegir **qué es fácil de hacer** con ese conocimiento después.

Regla práctica de esta unidad

Usaremos sobre todo **reglas de producción** (con `experta`) y, en el §8, **lógica difusa** (con `scikit-fuzzy`). Las ontologías y las redes semánticas se mencionan como alternativas; los **frames** y la **lógica formal** se tratan a nivel conceptual.

6. Representar y simular comportamientos con `experta` (RA5-b)

Para **simular un comportamiento** se escribe la base de conocimiento en `experta` y se ejecuta el ciclo de inferencia. Recordamos el parche obligatorio para Python 3.10+ (la librería es de 2019 y `collections.Mapping` desapareció en esa versión):

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```python
import collections, collections.abc
if not hasattr(collections, 'Mapping'):
    collections.Mapping = collections.abc.Mapping
    collections.Iterable = collections.abc.Iterable
    collections.MutableMapping = collections.abc.MutableMapping

from experta import *

class DiagnosticoPc(KnowledgeEngine):
    @DefFacts()
    def inicio(self):
        yield Fact(accion="diagnosticar")

    @Rule(Fact(accion="diagnosticar"), salience=10)
    def arrancar(self):
        print("Diagnóstico del PC...")
        self.declare(Fact(luz_encendida=True))
        self.declare(Fact(sonido="pitidos_cortos"))

    @Rule(Fact(luz_encendida=True), Fact(sonido="pitidos_cortos"))
    def ram(self):
        self.declare(Fact(causa="problema_ram"))

    @Rule(Fact(causa="problema_ram"))
    def resultado(self):
        print("DIAGNÓSTICO: Fallo de memoria RAM.")

engine = DiagnosticoPc()
engine.reset()
engine.run()
```

Salida esperada:

```
Diagnóstico del PC...
DIAGNÓSTICO: Fallo de memoria RAM.
```

Otro ámbito: clasificación de animales (sistema de producción clásico)

El mismo motor `experta` simula un sistema de clasificación con reglas de un zoólogo:

```python
class Animales(KnowledgeEngine):
    @DefFacts()
    def inicio(self):
        yield Fact(analizar=True)

    @Rule(Fact(analizar=True), salience=10)
    def datos(self):
        self.declare(Fact(pelo=True), Fact(carnivoro=True), Fact(color="leonado"))
        self.declare(Fact(manchas="oscuras"))

    @Rule(Fact(pelo=True))
    def mamifero(self):
        self.declare(Fact(mamifero=True))

    @Rule(Fact(mamifero=True), Fact(carnivoro=True), Fact(color="leonado"),
          Fact(manchas="oscuras"))
    def guepardo(self):
        print("IDENTIFICADO: guepardo")
```

Así se comprueba que un sistema experto **representa y simula el comportamiento** de un dominio sin programar la lógica imperativamente: el motor aplica las reglas hasta llegar a la conclusión.

6.1 «Muy diversos ámbitos»: los notebooks de la unidad

El CE b exige simular comportamientos en **dominios distintos**, no repetir el mismo ejemplo. Esta unidad trae varios notebooks ejecutados de verdad, en dominios reales (la lista completa está en las notebooks guiados y las entregas):

Notebook	Dominio	Qué simula
`UD05_N01_experta_primeros_pasos.ipynb`	Introducción	Primeros pasos con `experta`: hechos, reglas, `DefFacts`
`UD05_N02_piedra_papel_tijera.ipynb`	Juego	Piedra, papel o tijera decidido por reglas
`UD05_N03_clasificacion_animales.ipynb`	Zoología	El ejemplo de arriba, completo y ejecutado
`N11` · **lesión de rodilla**	Medicina	Diagnóstico de una lesión a partir de síntomas, con `experta`

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Notebook	Dominio	Qué simula
`UD05_N04_reglas_desde_datos_titanic.ipynb`	Datos históricos	Reglas **extraídas** de datos, no escritas a mano (§7)
`N12` · **valor de mercado**	Deporte	Estimación híbrida reglas + aprendizaje automático (§7)
`N13` · **centrocampistas** · `N14` · **quemador de gas**	Deporte · industria	Lógica difusa (§8) y control de un proceso real (§10-11)

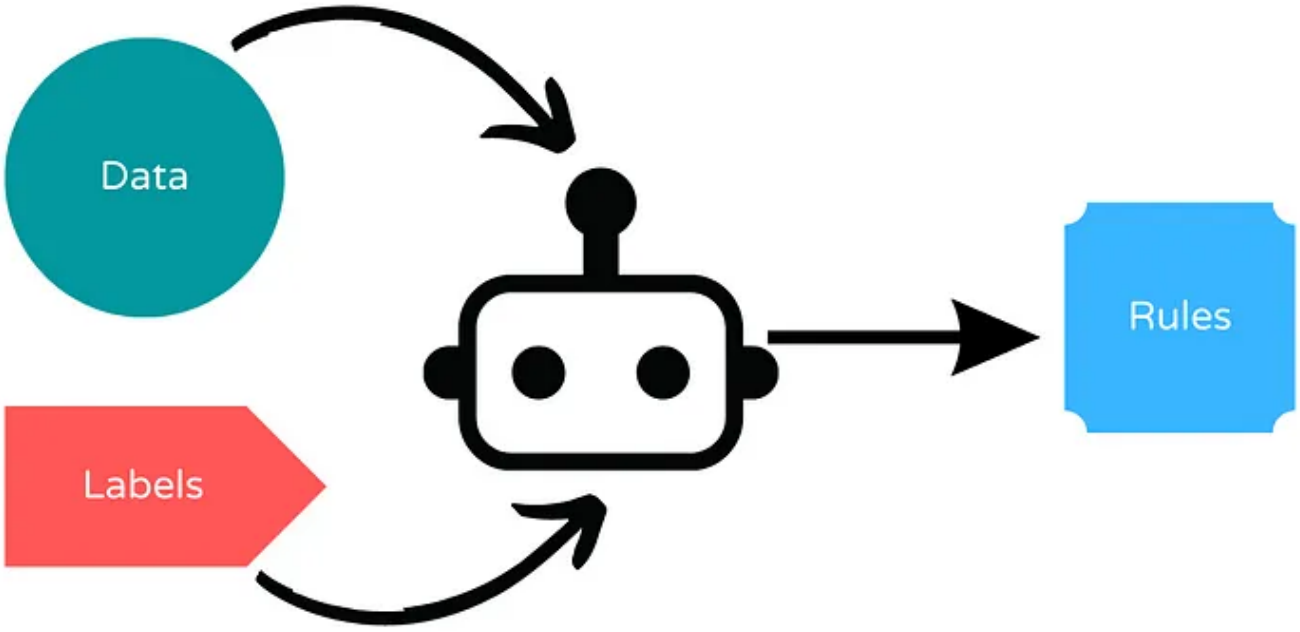
Nota sobre la tabla anterior.

> **Por qué importa la diversidad**
>
> Un sistema experto no es una técnica de un solo dominio: la **misma arquitectura** (base de conocimiento + motor de inferencia) sirve para diagnosticar una rodilla, valorar a un futbolista o regular un quemador de gas. Lo que cambia es el conocimiento que se codifica, no el motor que lo aplica.

7. Sistemas híbridos reglas/datos (RA5-b)

Un sistema experto puro necesita que **alguien escriba las reglas a mano**. Pero a veces el conocimiento del dominio ya está en los datos, o conviene mezclar las dos fuentes. Hay dos enfoques:

- **Deducir reglas a partir de datos**: un algoritmo genera reglas legibles a partir de un conjunto de entrenamiento, en vez de que las escriba una persona. Facilita la **interpretación** del razonamiento, porque el resultado sigue siendo `SI ... ENTONCES ...` en vez de una caja negra.
- **Integrar reglas definidas por el usuario con aprendizaje automático**: el experto pone las reglas de partida y el aprendizaje automático las **mejora** con datos.



© 2025 David Martínez licensed under CC BY-NC-SA 4.0

7.1 Bibliotecas del segundo enfoque

Biblioteca	Qué hace
Human-Learn	Permite **definir y dibujar** reglas propias como si fueran un clasificador de scikit-learn (`FunctionClassifier`), y combinarlas con aprendizaje automático
skope-rules	Analiza los datos y **deduce** reglas de clasificación; permite auditarlas e interpretarlas
FIGS (`imodels`)	Genera **reglas fáciles de interpretar** combinando varios árboles pequeños («sumas de árboles»), en vez de un único árbol grande difícil de leer
spaCy	Reglas para **extraer información de texto**: útil cuando no hay suficientes datos etiquetados o para casos muy específicos

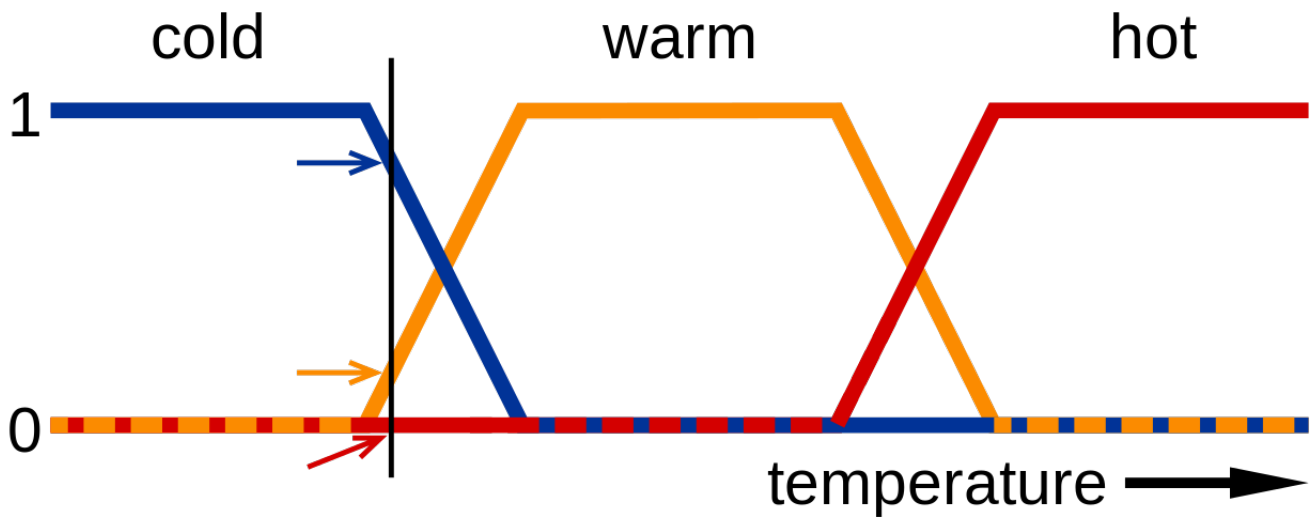
Nota sobre la tabla anterior.

> `UD05_N04_reglas_desde_datos_titanic.ipynb` **: reglas que salen de los datos, no de un experto**
>
> En vez de que alguien escriba a mano «si viajas en primera clase y eres mujer, sobrevives», **FIGS** analiza los datos históricos del Titanic y **genera esa regla por sí solo**, junto con otras. El resultado sigue siendo legible —un árbol de reglas pequeño—, pero nadie lo escribió a mano: se dedujo de los datos. Es el primer enfoque de la tabla de arriba.

> `N12` **: valor de mercado de futbolistas con Human-Learn + FIGS**
>
> El entregable **N12** combina las dos ideas sobre los mismos datos (FIFA 22): una `FunctionClassifier` de Human-Learn (reglas que **tú** defines sobre atributos del jugador) y un `FIGSClassifier` que **deduce** sus propias reglas del conjunto de entrenamiento. Comparar los dos resultados sobre el mismo problema es la mejor forma de sentir la diferencia entre los dos enfoques híbridos.

> **Esto no es una técnica aislada: es una tendencia**
>
> Los sistemas híbridos reglas/datos son la puerta de entrada a los **sistemas neuro-simbólicos** y a las **reglas como guardarraíl del ML** que se tratan en el §12.2. No son un tema aparte de los sistemas expertos: son su evolución natural cuando el conocimiento no cabe entero en la cabeza de un experto.

8. Sistemas de razonamiento impreciso: lógica difusa (RA5-b/d)



© 2025 David Martínez licensed under CC BY-NC-SA 4.0

8.1 De lo binario a lo continuo

La lógica proposicional clásica es **binaria**: un enunciado es cierto o falso. Pero los humanos no razonamos así — «hace frío» no es verdadero o falso de forma tajante, es una cuestión de grado. La **lógica difusa** (o borrosa) extiende la lógica proposicional para trabajar con esa incertidumbre:

- Los valores de verdad son **números reales en el intervalo \([0, 1]\)**: 0 es falso, 1 es cierto, 0,5 es «cierto al 50 %».
- La pertenencia de un elemento a un conjunto viene dada por una **función de pertenencia**, \(\mu_A(x)\): el grado de pertenencia de \(x\) al conjunto \(A\).
- Conceptos como *húmedo* o *frío* son difíciles de definir con precisión, pero la lógica difusa los define con funciones de pertenencia — lo que facilita crear dispositivos como termostatos: *«si la temperatura es fría, entonces enciende la calefacción»*.

Un **sistema de razonamiento impreciso** es un sistema basado en reglas que usa lógica difusa. Permite trabajar con valores **continuos**, modela mejor el conocimiento humano y es muy apropiado para **sistemas de control** — por eso enlaza directamente con el §10 y el §11.

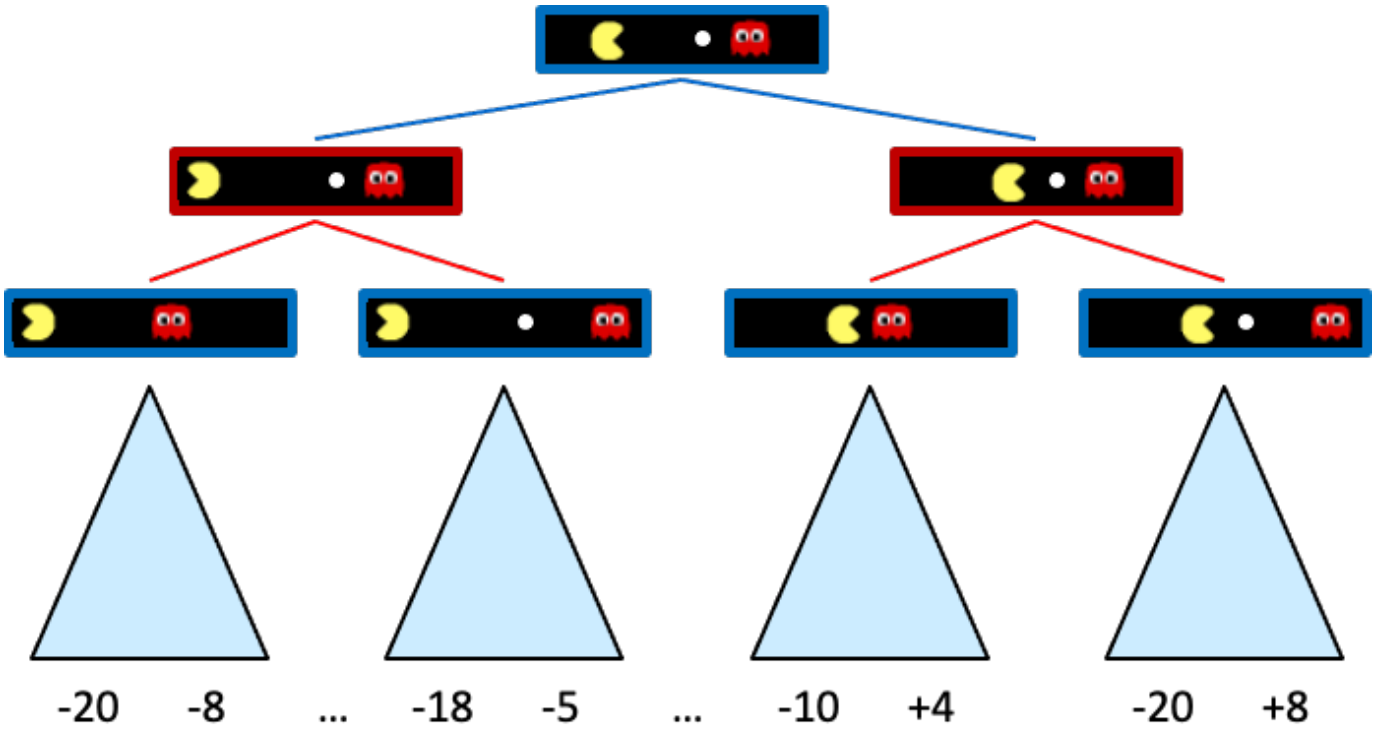
8.2 Conceptos básicos

Concepto	Qué es	Ejemplo
Variable lingüística	Variable que toma valores lingüísticos	*Temperatura*
Valores lingüísticos	Los valores que puede tomar esa variable	*Frío*, *Calor*
Función de pertenencia	Asigna a cada valor un grado de pertenencia a un valor lingüístico	\(27\,°C \rightarrow Calor = 0{,}8\)
Regla difusa	Regla que usa valores difusos	«Si la temperatura es **fría**, entonces **calefacción alta**»
Función de agregación	Combina los valores difusos de varias reglas en una conclusión	\(Calor=0{,}8,\ H\acute{u}medo=0{,}7 \rightarrow Sensaci\acute{o}n\ desagradable=0{,}8\)

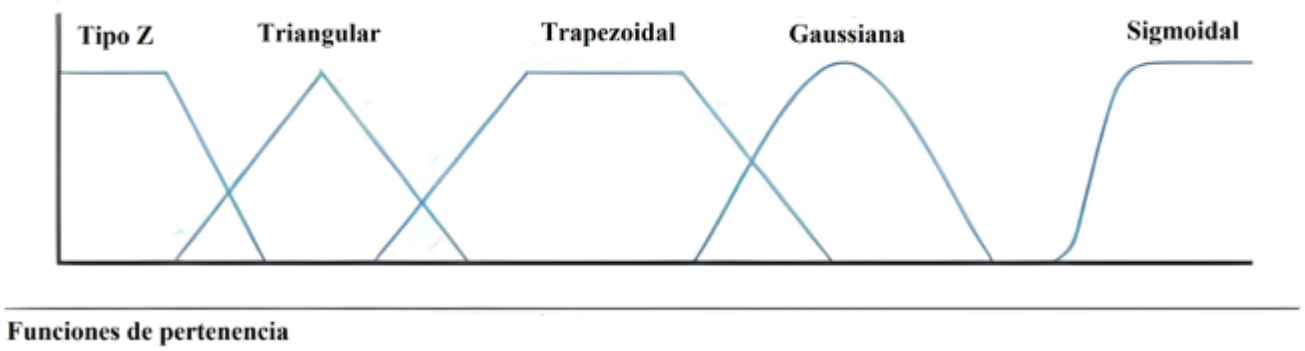
8.3 El funcionamiento en tres pasos

1. **Fuzzificación**: convierte los datos de entrada precisos en valores difusos, usando las funciones de pertenencia. \(27\,°C \rightarrow Calor=0{,}8,\ MuchoCalor=0{,}2\).
2. **Evaluación de las reglas**: se aplican las reglas del sistema, combinando las funciones de pertenencia de las variables de entrada para deducir la relevancia de la salida. Por ejemplo: «si la temperatura es **alta** y la humedad es **baja**, la velocidad del ventilador debe ser **alta**».
3. **Desfuzzificación**: convierte la conclusión difusa de vuelta en un valor preciso, con una función de agregación (normalmente **centro de gravedad** o **máximo**).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Las funciones de pertenencia más usadas son **trapezoidales** y **triangulares**; las sinusoidales sirven para representar periodos, y las sigmoidales, probabilidades.



Funciones de pertenencia

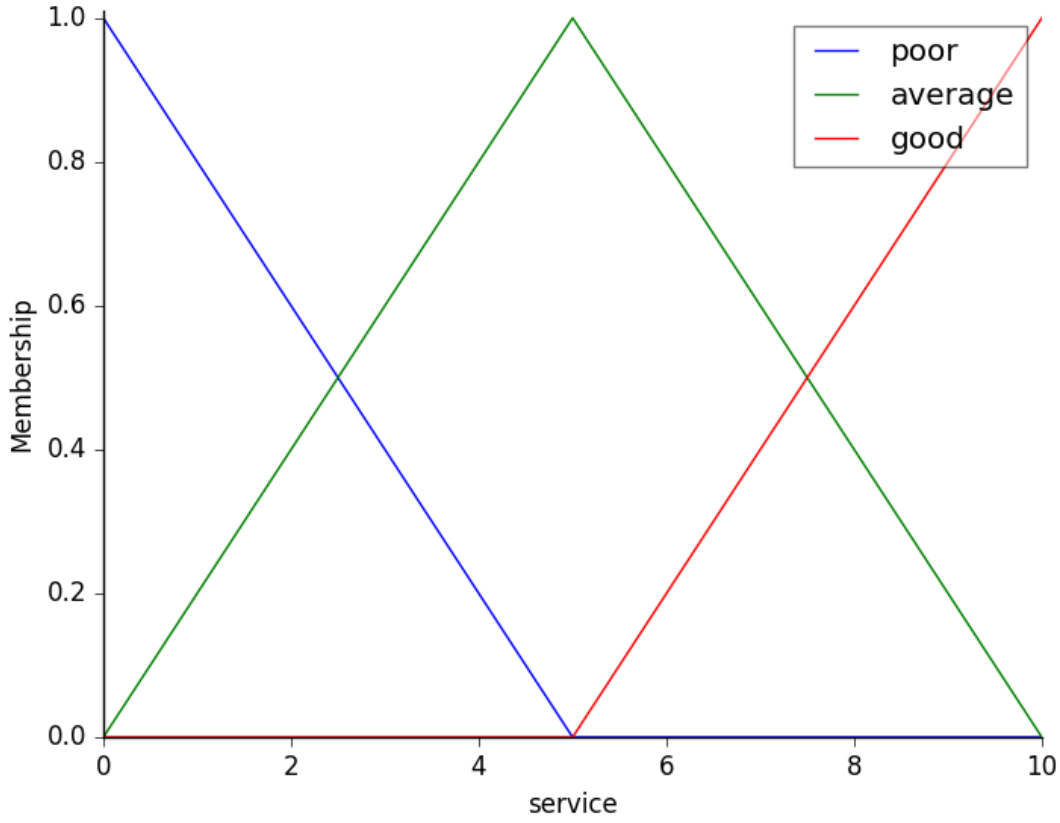
8.4 Ejemplo trabajado: la propina del restaurante

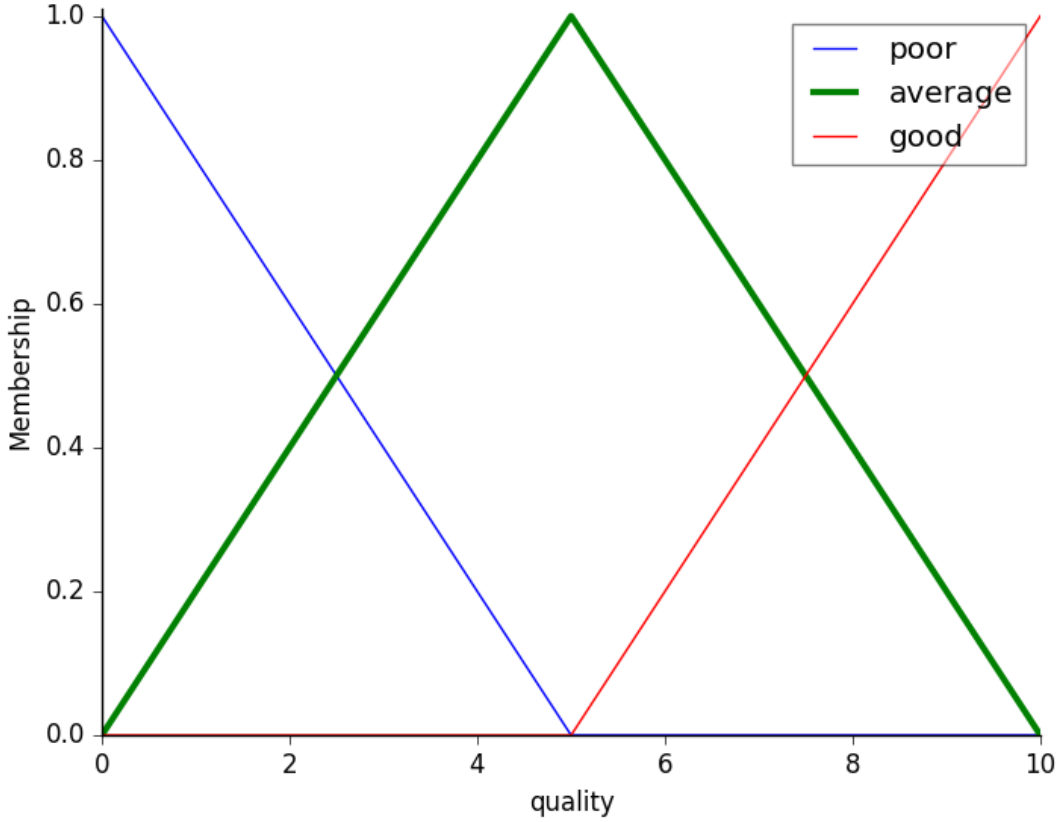
Es el ejemplo canónico de `scikit-fuzzy`, y el que verás resuelto paso a paso en el `N05`.

Variables de entrada (funciones triangulares):

- **Calidad del servicio**: baja \([0,5]\) · media \([0,10]\) · alta \([5,10]\)
- **Calidad de la comida**: mala \([0,5]\) · media \([0,10]\) · buena \([5,10]\)

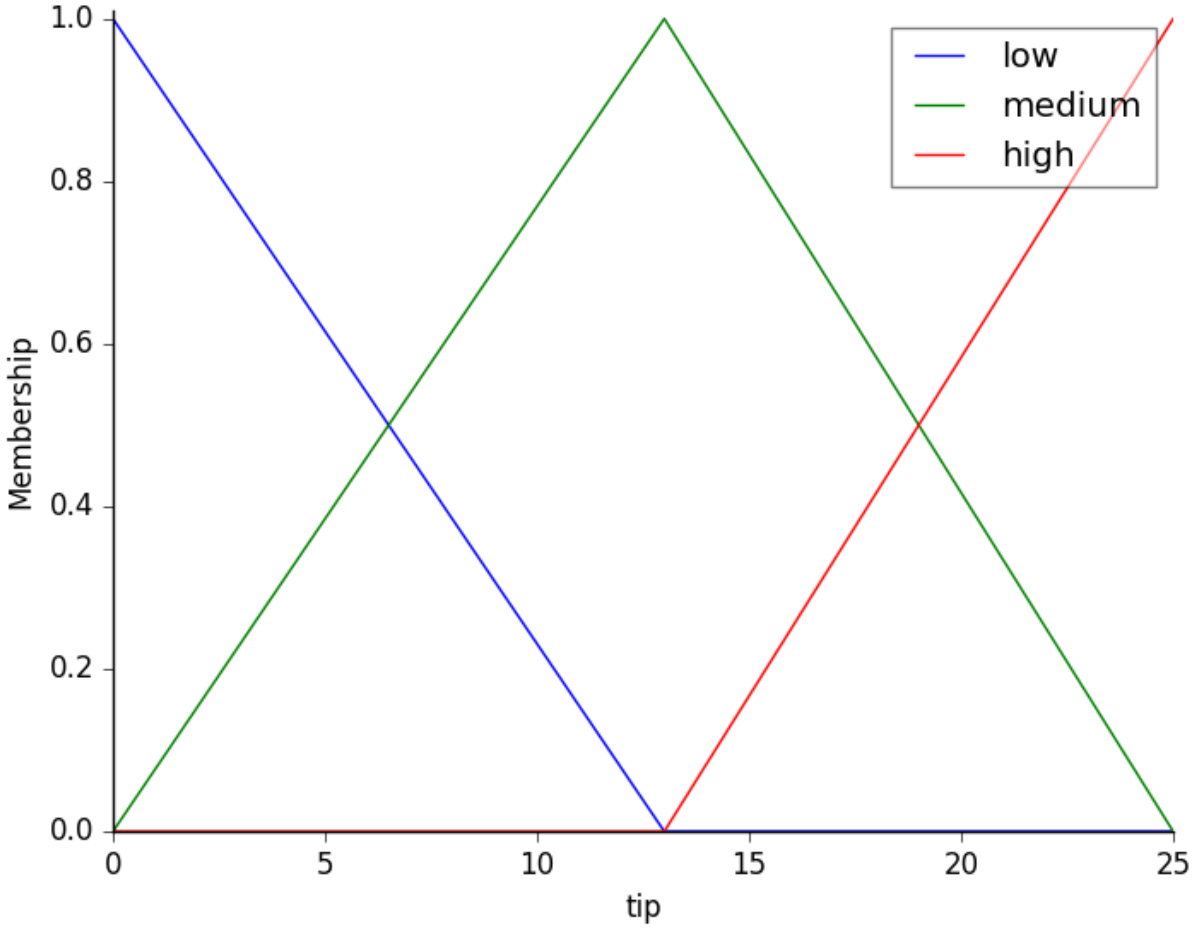
© 2025 David Martínez licensed under CC BY-NC-SA 4.0





© 2025 David Martínez licensed under CC BY-NC-SA 4.0

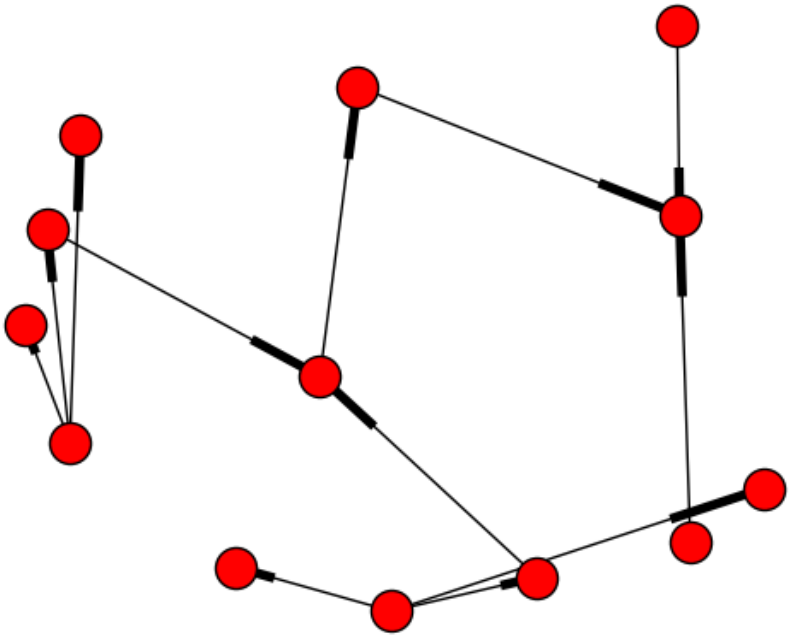
Variable de salida: propina — baja \([0,13]\) · media \([0,25]\) · alta \([13,25]\)



Reglas:

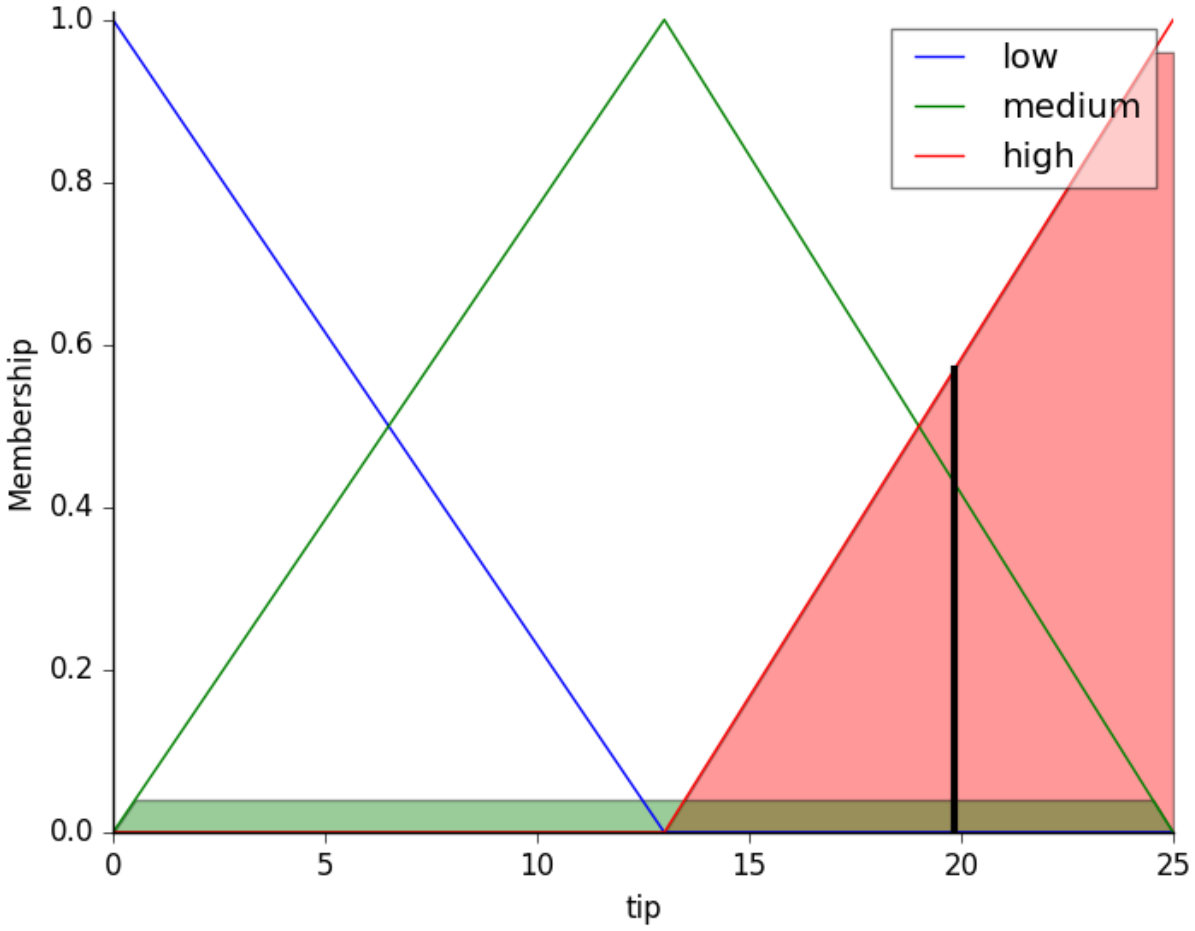
- SI (servicio es **baja** O comida es **mala**) ENTONCES propina es **baja**
- SI (servicio es **media**) ENTONCES propina es **media**
- SI (servicio es **alta** O comida es **buena**) ENTONCES propina es **alta**

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Inferencia: con un servicio de **9,8** y una comida de **6,5**, el sistema desfuzzifica una propina de **19,24 €** — en la banda alta, coherente con un servicio casi perfecto y una comida notable.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



De la propina al quemador de gas

Este ejemplo es idéntico en estructura a `N07` (control difuso del riego) y a la entrega `N14` (quemador de gas): variables de entrada difusas, reglas lingüísticas, una salida desfuzzificada. La diferencia es que ahí la salida no es una propina, es la **potencia de un actuador real** — es el paso del §8 al §11.

9. Variación de características y dinámica (RA5-c)

9.1 Sensibilidad y robustez

- **Sensibilidad**: cómo cambian las conclusiones ante pequeñas desviaciones en los parámetros o datos de entrada. Un sistema muy sensible detecta pronto anomalías, pero **falsas alarmas** ante ruido.
- **Robustez**: capacidad de mantener conclusiones estables y seguras ante perturbaciones, ruido o fallos parciales.

9.2 Tres mecanismos de variación

Qué varía	Efecto en la dinámica
Las reglas (estructura lógica)	Añadir condiciones al antecedente → más inercia (la regla se dispara menos); simplificar → sobreactivación (reglas se disparan ante transitorios sin riesgo)
Los hechos (perturbaciones)	Ruido en un sensor (p. ej. CO en un túnel oscilando alrededor del umbral) → el sistema conmuta actuadores sin parar
Los umbrales de certeza	Subir el umbral de un diagnóstico → menos falsos positivos pero puede **ignorar alertas tempranas**

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Nota sobre la tabla anterior.

El problema de la histéresis

Un sensor de CO en un túnel oscila entre 28 y 32 ppm con umbral en 30. Sin histéresis, los extractores se encienden y apagan en cada ciclo. Soluciones: una **regla de histéresis temporal** (la alerta debe mantenerse 30 s) o un **controlador difuso** que suavice la transición (enlaza con el §8).

10. Estrategias de control de un sistema experto (RA5-d)

10.1 Control de la agenda (resolución de conflictos)

Estrategia	Qué hace
Salience	Prioridad numérica explícita de cada regla (las de emergencia van antes)
Recency	Prefiere las reglas con hechos más recientes (time-tags)
Specificity	Prefiere la regla con más condiciones en el antecedente
Control de meta (metarrazonamiento)	Reglas de nivel superior que modifican prioridades, activan grupos de reglas según el modo (arranque, operación, parada) o cambian la estrategia. Es la **sabiduría** de la jerarquía DIKW del §4.1, aplicada al propio motor

10.2 Especificaciones de respuesta

Para controlar un proceso, el sistema experto debe cumplir **especificaciones** sobre la respuesta de la variable controlada:

Especificación	Qué mide
Precisión	Error en régimen permanente (diferencia entre la variable y el setpoint al estabilizarse)
Tiempo de respuesta	Tiempo de subida (tr) y de asentamiento (ts)
Estabilidad	Ausencia de oscilaciones; sobreimpulso (overshoot) máximo aceptable
Alcance	Rango de operación de la planta

Nota sobre la tabla anterior.

Nivel FP

En este módulo se trabajan estas especificaciones de forma **conceptual** (qué miden y por qué importan), sin las fórmulas de la teoría de control de ingeniería. El foco está en *definir el objetivo y saber interpretar si la respuesta lo cumple*.

10.3 Ejemplo guiado: definir las especificaciones de un controlador de climatización

Recorremos juntos el razonamiento que repetirás en `N08`, antes de aplicarlo al quemador de gas real de `N14`.

Problema: queremos controlar la temperatura de una sala para que llegue a 21 ºC.

Paso 1 · Setpoint y precisión. La variable controlada (temperatura) debe estabilizarse en 21 ºC con un **error en régimen permanente** de ±0,5 ºC como máximo.

Paso 2 · Tiempo de respuesta. La sala debe alcanzar la banda aceptable (20,5-21,5 ºC) en menos de **15 minutos** desde el arranque (tiempo de asentamiento).

Paso 3 · Estabilidad y sobreimpulso. Se tolera un sobreimpulso máximo de **1 ºC** por encima del setpoint (sin oscilaciones que dañen el equipo).

Paso 4 · Elegir el controlador. Con un PID bien sintonizado se cumple en un sistema lineal, pero la sala tiene inercia térmica y perturbaciones (puertas, sol). Un **controlador experto o difuso** con reglas del operador («si la diferencia es grande → máxima potencia, y al acercarse → reducir para no sobrepasar») consigue la respuesta con **menos sobreimpulso y más robustez** ante las perturbaciones.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

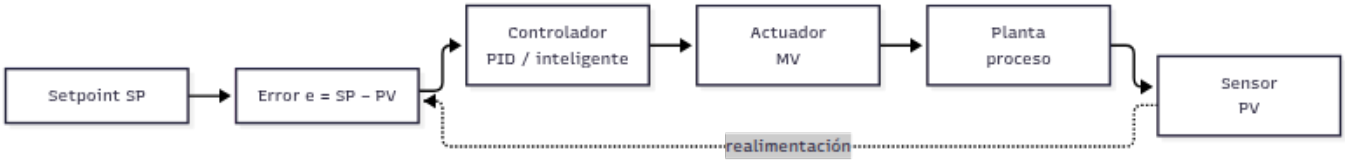
Paso 5 · Comprobar y ajustar. Se simula la respuesta y se mide ts y el sobreimpulso; si no se cumple la especificación, se ajustan reglas o ganancias (análisis de sensibilidad, §9).

> **Conclusión del ejemplo guiado**
>
> Controlar no es «conectar un motor»: es **definir el objetivo** (precisión), **acotar el tiempo**, **limitar el sobreimpulso** y **elegir el controlador** que lo cumpla, verificándolo con una simulación. Ese es el sentido del CE d y del CE e — y exactamente lo que se te pide en `N14`, con un quemador de gas real en vez de una sala.

11. Controladores inteligentes (RA5-e)

11.1 Del control clásico al control inteligente

Un **lazo de control** clásico compara el setpoint (SP) con la variable medida (PV), calcula el error y aplica una acción (MV) con un **controlador PID** (proporcional-integral-derivativo). El PID es eficaz en sistemas lineales, pero sufre con **retrasos, no linealidades y perturbaciones multivariable**.



La acción del PID es `u = Kp·e + Ki·∫e + Kd·de/dt`: la parte proporcional responde al error, la integral elimina el error residual y la derivada reduce el sobreimpulso. Sus límites: no predice el futuro, se degrada con retrasos y no linealidades, y hay que **re-sintonizarlo** cuando la planta envejece (ganancias fijas).

Los **controladores inteligentes** sustituyen o complementan al PID usando técnicas de IA:

Controlador	Cómo funciona	Ventaja frente al PID
Control difuso	Reglas lingüísticas + funciones de pertenencia (Mamdani/Sugeno) — el §8 aplicado al control	Robusto ante ruido y no linealidad; no necesita modelo
Control por reglas (experto)	Motor de inferencia con reglas del operador experto	Captura heurísticas de operadores; fácil de entender
Redes neuronales	Aprenden la dinámica inversa de la planta	Adaptación a sistemas no lineales
Control predictivo (MPC)	Predice la trayectoria futura y optimiza la acción	Anticipa restricciones; proactivo

11.2 Ejemplos con datos

Caso	Mejora frente a PID
Control térmico difuso	Tiempo de asentamiento −76 % (416 s → 100 s); sobreimpulso 5,64 % → 0 %
Horno de fundición (ANN)	MSE 132,75 frente a 134,13 del PID (ligera mejora)
Mitsubishi (aire acondicionado)	Calienta/enfría 5× más rápido, −24 % consumo
DeepMind centros de datos	−40 % en energía de refrigeración (−15 % PUE overhead)

Nota sobre la tabla anterior.

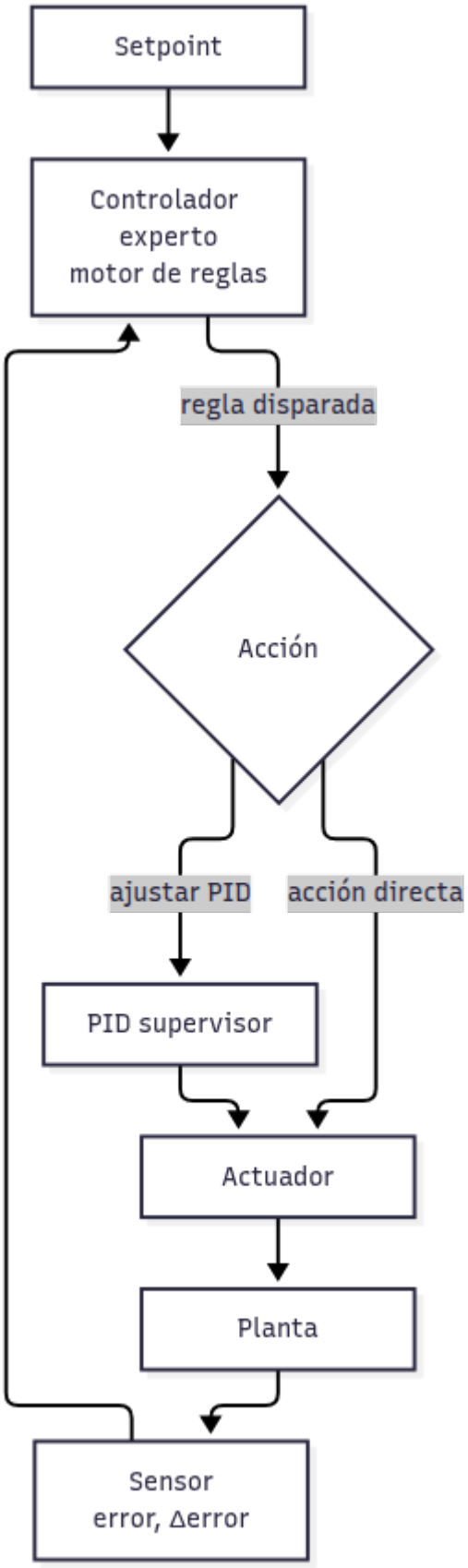
> **¿Controlador inteligente siempre gana?**
>
> No siempre. El PID es simple, barato y fiable en sistemas bien modelados. El controlador inteligente aporta cuando hay **no linealidad, retraso o ruido fuerte**. La decisión es *empezar simple* y añadir inteligencia solo si se justifica — es lo que comprobarás tú mismo en `N14`, que pide **dos enfoques distintos** para el mismo quemador.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

11.3 El sistema experto como controlador

El patrón del **control experto** (Åström, Anton y Årzén, 1986) usa un motor de inferencia dentro del lazo: el sistema evalúa el error y su variación, decide la acción por reglas (supervisando y ajustando incluso un PID) y explica la decisión. Es la base de sistemas industriales como **Gensym G2** (refinerías, energía, farmacia) desde los años 80.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



© 2025 David Martínez licensed under CC BY-NC-SA 4.0

El controlador experto puede actuar de dos formas: **directamente** (decide la acción por reglas, por ejemplo «si el error es negativo grande y aumenta → máxima potencia») o **supervisando** un PID (ajusta sus ganancias cuando detecta que la respuesta se degrada). En ambos casos, la decisión es **explicable**: se puede consultar qué regla se disparó y por qué.

Regla de un controlador experto de climatización

```
SI error de temperatura es NEGATIVO_GRANDE
Y variación del error es POSITIVA_PEQUENA
ENTONCES potencia del quemador es MEDIA_ALTA
```

Estas reglas capturan la heurística de un operador humano y son fáciles de auditar, algo que un PID o una red neuronal no ofrecen de forma directa.

12. Aplicaciones y tendencias (contenidos del RA5)

12.1 Aplicaciones por sector

Sector	Uso
Industria	Control de procesos, diagnóstico de máquinas, mantenimiento
Salud	Diagnóstico asistido (MYCIN, GARVAN-ES1), dosificación
Finanzas	Asesoramiento, detección de fraude por reglas
Telecomunicaciones	Gestión de redes, diagnóstico de averías

Nota sobre la tabla anterior.

Ejemplos clásicos con datos

- **MYCIN** (Stanford, 1972): ~500-600 reglas de diagnóstico clínico; precisión 69-70 % (equiparable a especialistas). Pionero de la explicación y los factores de certeza (§4.6).
- **XCON/R1** (CMU/DEC, 1978): configuración de ordenadores VAX; pasó de 250 a más de 6.200 reglas en 1986; precisión 95-98 % y ahorro de ~25 M$/año para DEC.
- **DENDRAL** (1965): primer sistema experto; acotaba millones de isómeros moleculares a un conjunto manejable.
- **SID** (DEC): generó el 93 % de las puertas lógicas del VAX 9000.
- **Cyc** (1984): proyecto de enciclopediar el sentido común (millones de hechos y reglas).

12.2 Tendencias

- **BRMS** (*Business Rule Management Systems*): motores de reglas empresariales como **Drools** (Apache KIE) con estándares DMN/JSR-94. Permiten a las empresas gestionar miles de reglas de negocio separadas del código, con motores optimizados (PHREAK) que escalan linealmente.
- **Sistemas neuro-simbólicos** (3.ª ola de la IA): combinan redes neuronales (aprendizaje) con reglas y razonamiento simbólico (lógica), reduciendo alucinaciones y aportando explicabilidad — es la generalización de los **sistemas híbridos reglas/datos** del §7. Ejemplo: Amazon utiliza arquitecturas neuro-simbólicas en sus motores de compra para contrastar las salidas generativas con hechos y reglas.
- **IA explicable (XAI)**: MYCIN fue el origen de la explicación; hoy se usan SHAP/LIME para explicar modelos y la normativa exige el **derecho a explicación** del RGPD (enlaza UD06).
- **Agentes y razonamiento declarativo**: agentes que planifican con reglas y guardas lógicas de seguridad, garantizando que las decisiones distribuidas cumplen restricciones.
- **Reglas como guardarraíl del ML**: usar reglas para validar y acotar las salidas de los modelos (p. ej. un LLM no puede emitir una orden fuera de los rangos seguros definidos por reglas). Es la misma idea del §7, llevada a los modelos generativos.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Del pasado al presente

Los sistemas expertos «clásicos» (MYCIN, XCON) demostraron que el conocimiento declarativo y la explicación importan. Hoy esa idea **no ha muerto**: vive en los BRMS, en los sistemas neuro-simbólicos y en los guardarraíles que aseguran las salidas del ML. Saber construir y auditar sistemas basados en reglas —puras, híbridas o difusas— sigue siendo una competencia valiosa.

13. Puntos clave de la unidad

- La jerarquía **DIKW** distingue dato, información, conocimiento y sabiduría: un sistema experto almacena conocimiento (reglas) y, en su forma más avanzada, sabiduría (metarreglas).
- Un **sistema experto** codifica el conocimiento de un experto en una **base de conocimiento** y lo aplica con un **motor de inferencia** (ciclo reconocer-actuar, forward/backward).
- La **explicación** del razonamiento es su gran ventaja frente al ML (obligatoria en dominios regulados).
- La **representación del conocimiento** forma un continuo: pares atributo-valor, reglas, jerarquías, frames, lógica, redes semánticas u ontologías.
- Con `experta` (parche Py3.10+) se **simulan comportamientos** de ámbitos muy diversos: medicina, zoología, deporte, industria.
- Los **sistemas híbridos reglas/datos** (Human-Learn, FIGS, skope-rules) combinan el conocimiento experto con lo que aprenden los datos, cuando las reglas escritas a mano no bastan.
- La **lógica difusa** extiende la lógica a valores continuos $[0,1]$, con fuzzificación, evaluación de reglas y desfuzzificación — clave para el control de procesos reales.
- **Sensibilidad vs robustez**: variar reglas, hechos o umbrales cambia la dinámica (inanición, sobreactivación, falsas alarmas); la **histéresis** o el control difuso la estabilizan.
- Las **estrategias de control** (salience, control de meta) y las **especificaciones** (precisión, tiempo, estabilidad) definen la calidad de la respuesta.
- Los **controladores inteligentes** (difuso, por reglas, ANN, MPC) superan al PID en sistemas no lineales, con datos como −76 % de asentamiento o 0 % de sobreimpulso.
- **Tendencias**: BRMS/Drools, neuro-simbólico, XAI, reglas como guardarraíl del ML.

14. Glosario

Término	Definición
Dato	Hecho o valor registrado, independiente de quien lo interpreta
Información	Un dato interpretado por un agente
Conocimiento	Información integrada en un modelo del mundo
Sabiduría	Meta-conocimiento: cuándo y cómo aplicar el conocimiento
Sistema experto	Programa de IA que emula el razonamiento de un experto en un dominio
Base de conocimiento	Reglas y hechos del dominio
Memoria de trabajo	Hechos actuales del problema
Motor de inferencia	Ejecuta el ciclo reconocer-actuar
Ingeniero de conocimiento	Profesional que formaliza el conocimiento del experto
Adquisición de conocimiento	Proceso de capturar el conocimiento (bottleneck)
Subsistema de explicación	Justifica el razonamiento del sistema
Forward chaining	Encadenamiento hacia delante (data-driven)
Backward chaining	Encadenamiento hacia atrás (goal-driven)
Modus Ponens	Si P implica Q y P es verdad, Q es verdad
Modus Tollens	Si P implica Q y Q es falso, P es falso
Factor de certeza	Medida de confianza de una conclusión (MYCIN)

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Término	Definición
Frame	Estructura con ranuras y valores por defecto
Red semántica	Grafo de conceptos y relaciones
Ontología	Esquema formal de clases y propiedades (OWL)
Sistema híbrido reglas/datos	Combina reglas escritas o deducidas con aprendizaje automático
FIGS	Algoritmo que genera reglas interpretables como suma de árboles pequeños
Lógica difusa	Extensión de la lógica a valores continuos en \([0,1]\)
Variable lingüística	Variable que toma valores lingüísticos (p. ej. «temperatura»)
Función de pertenencia	Grado de pertenencia de un valor a un conjunto difuso
Fuzzificación	Conversión de un valor preciso a valores difusos
Desfuzzificación	Conversión de una conclusión difusa a un valor preciso
Salience	Prioridad numérica de una regla
Control de meta	Metarreglas que gestionan la propia inferencia
Sensibilidad	Variación de las conclusiones ante perturbaciones pequeñas
Robustez	Estabilidad de las conclusiones ante ruido o fallos
Histéresis	Retardo en el cambio de estado para evitar conmutaciones
PID	Controlador proporcional-integral-derivativo
Lazo de control	SP → error → controlador → planta → PV
Error en régimen permanente	Diferencia residual entre variable y setpoint
Sobreimpulso (overshoot)	Exceso máximo sobre el setpoint durante el transitorio
Tiempo de asentamiento	Tiempo hasta que la respuesta entra en la banda aceptable
Control difuso	Controlador basado en reglas lingüísticas y pertenencia
Control predictivo (MPC)	Optimiza la acción prediciendo la trayectoria futura
BRMS	Sistema de gestión de reglas de negocio (Drools)
Neuro-simbólico	Combinación de redes neuronales y razonamiento simbólico
XAI	IA explicable

15. FAQ

¿Un sistema experto puede aprender de los datos?

No por sí mismo: las reglas las escribe un experto. Por eso es totalmente **explicable**, pero requiere que el conocimiento esté disponible y formalizado (el famoso *bottleneck*). Los **sistemas híbridos** del §7 son justo la respuesta a esta limitación: dejan que el aprendizaje automático deduzca o mejore las reglas.

¿Los sistemas expertos están anticuados?

No: los motores de reglas siguen en **BRMS empresariales** (Drools, SAP) y resurgen en los **sistemas neuro-simbólicos** y como **guardarraíl** del ML. MYCIN/XCON fueron los pioneros, pero la tecnología evolucionó (RETE → PHREAK, estándares DMN).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

¿Por qué un sistema experto puede dar falsas alarmas?

Por **sensibilidad excesiva**: si un sensor tiene ruido y el umbral de la regla es muy justo, el sistema conmuta sin parar. Se corrige con **histéresis**, suavizado o un controlador difuso.

¿Qué es mejor, un PID o un controlador inteligente?

Depende del sistema. El PID es simple y fiable en sistemas lineales; el controlador inteligente (difuso, ANN, MPC) gana en no linealidades, retrasos o ruido. Se empieza simple y se añade inteligencia si hace falta — es lo que comprobarás con las dos versiones que pide `N14`.

¿`experta` funciona en Python moderno?

La librería (2019) falla en Python 3.10+ por `frozendict`. Con el parche `collections.Mapping = collections.abc.Mapping` funciona igual de bien. Existe también un fork (`om-experta`) que lo resuelve de otra forma, pero está archivado desde 2023: en este módulo usamos siempre el parche, para no tener dos soluciones al mismo problema.

¿Cuándo uso reglas puras y cuándo un sistema híbrido?

Si el experto puede formalizar el conocimiento completo, con reglas puras basta y son más explicables. Si el conocimiento es parcial, cambia con el tiempo, o hay muchos datos disponibles, un sistema híbrido (§7) aprovecha lo mejor de las dos fuentes.

¿Los sistemas expertos pueden explicar sus decisiones?

Sí, y es su gran ventaja: el subsistema de **explicación** puede responder «¿por qué?» y «¿cómo?» mostrando las reglas usadas. MYCIN fue el primero. Hoy esto se llama **IA explicable (XAI)** y es obligatorio en algunos dominios (derecho a explicación del RGPD).

16. Sesiones

Semana	Horas	Contenido	CE
11	3	DIKW, arquitectura y dinámica; estructuras de representación	RA5-a
12	3	`N10` (`experta`) + guiadas; `N11`; sistemas híbridos reglas/datos (`N12`)	RA5-b
13	3	Lógica difusa: `N05` (propinas); variación y dinámica	RA5-b, RA5-c
14	3	Estrategias de control; controladores inteligentes; `N08`	RA5-d
15	3	`N13`, `N14`; aplicaciones y tendencias; evaluación	RA5-d, RA5-e

17. Recursos

- Diapositivas
- **Práctica** — se hace, no se entrega ni puntúa:
  - Ejercicios de autoevaluación
  - Notebooks guiados — **ocho** notebooks, de menor a mayor dificultad: `N01`-`N04` reglas, `N05`-`N07` lógica difusa y `N08` control
- **Entregas** — seis, cada una con su rúbrica; qué se entrega:
  - N10 · simular un sistema experto
  - y los cinco sistemas `N11` a `N15`, en dominios distintos a propósito
- Los notebooks se abren desde **Práctica** y **Entregas**, con descarga y apertura en Colab.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Referencias de la unidad

- Wikipedia · Expert system
- Wikipedia · MYCIN
- Wikipedia · XCON
- experta · readthedocs
- CLIPS · clipsrules.net
- Drools / Apache KIE
- Human-Learn
- imodels (FIGS)
- skope-rules
- Wikipedia · Lógica difusa
- Wikipedia · Fuzzy control system
- Wikipedia · PID controller
- Modus Ponens · Modus Tollens

18. Evaluación

Peso	Instrumento
40 % actividades	El taller **N10** y los cinco sistemas `N11`-`N15`, cada uno con su rúbrica en la tarea de Moodle. Los ocho notebooks guiados son práctica y no puntúan
60 % prueba escrita	Prueba del RA5 en Moodle: preguntas de test y de desarrollo sobre el contenido de la unidad

- **La normativa exige alcanzar todos los RA** del módulo para superarlo (art. 5.1 de la Orden 8/2025: la calificación del módulo está «*en función de la consecución de los RA*»; y las Instrucciones 26-27, que impiden calificar positivamente un módulo con RA no superados). El centro concreta ese mandato exigiendo **≥ 5 en cada RA**.
- Los entregables `N11`-`N14` cubren un dominio fijo cada uno (medicina, deporte × 2, industria); `N15` es de libre elección y premia la originalidad.

19. Recuperación

Actividades del programa de recuperación individual por RA (art. 14.4 Orden 8/2025): repetir la construcción de un sistema experto con un problema distinto y las pruebas de autoevaluación de la unidad.

6 de octubre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

5.2 Diapositivas de la UD05



Diapositivas PDF

Cómo se manejan

Flechas o clic para avanzar · **F** para verlas a pantalla completa · **P** para las notas del ponente.

24 de agosto de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

6. UD03

6.1 UD03 — Procesamiento del Lenguaje Natural

Unidad 3 · 12 h · semanas 16-19

Es el RA con **más criterios de evaluación** de todo el módulo: **siete**. Se evalúa con **cuatro entregables prácticos** y la prueba escrita del RA3.

1. Introducción

De todas las formas de inteligencia, el **lenguaje** es la que mejor nos permite comunicar ideas, pedir ayuda, decidir y persuadir. Conseguir que las máquinas entiendan y generen lenguaje humano es el objetivo del **procesamiento del lenguaje natural (PLN)**: el campo de la IA que está detrás de los buscadores, los traductores, los asistentes de voz, los correctores y los chatbots.

Esta unidad tiene una particularidad que conviene saber desde el principio: de sus **siete criterios**, tres no se demuestran escribiendo código. Hablan del **papel del lingüista**, del **trabajo cooperativo** entre perfiles y de la **formación** que necesita quien investiga en PLN. No son relleno: son la mitad del RA, y en esta unidad se evalúan con una parte escrita dentro de los entregables.

El recorrido:

1. **Qué es el PLN** y qué tareas comprende, del análisis de una palabra a la generación de texto.
2. **Qué se consigue hoy** — el potencial, con cifras.
3. **La ambigüedad**: por qué es *el* problema del lenguaje, en sus seis formas.
4. **Desambiguación y etiquetado morfológico (POS)**: cómo se ataca ese problema.
5. **Las demás limitaciones**: falta de comprensión, sesgos, lenguas con pocos recursos, ironía.
6. **Cuándo es factible** aplicar PLN a un problema, con la lupa del AI Act.
7. **Quién participa**: el lingüista, la cooperación con informáticos y la formación necesaria.
8. **Las herramientas** en Python: `nltk`, spaCy, scikit-learn y *transformers*.
9. **Cómo construir un sistema** de PLN orientado a una tarea concreta.

Hilo conductor de la unidad

El lenguaje es la interfaz más natural entre personas y máquinas, y también la más ambigua. Primero vemos qué es el PLN y hasta dónde llega; luego por qué falla —la ambigüedad— y cómo se ataca; después quién hace falta en un proyecto y cuándo merece la pena; y por último construimos un sistema que resuelva una tarea concreta.

De dónde sale el material de esta unidad

Los bloques de **ambigüedad, desambiguación y etiquetado morfológico**, con sus ejemplos en español, y el itinerario de formación provienen del material del profesor, adaptados aquí. Las prácticas usan `nltk`, spaCy, PyTorch y modelos de **Hugging Face**. La referencia teórica de fondo es *Speech and Language Processing* de **Jurafsky y Martin**, de acceso libre.

2. Resultado de aprendizaje y criterios de evaluación

RA3 — Relaciona el procesamiento de lenguaje natural con sus aplicaciones determinando su potencial e identificando sus limitaciones.

CE	Criterio de evaluación	Bloque
RA3-a	Se ha caracterizado el procesamiento de lenguaje natural.	§4, §7
RA3-b	Se ha justificado el papel del lingüista en un proyecto de inteligencia artificial.	§9

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

CE	Criterio de evaluación	Bloque
RA3-c	Se ha determinado el potencial de las técnicas existentes de procesamiento de lenguaje, así como sus limitaciones.	§5-8
RA3-d	Se ha considerado en qué casos es factible aplicar estas técnicas en la resolución de un problema.	§9
RA3-e	Se ha evaluado el trabajo cooperativo entre lingüistas e informáticos en el campo del procesamiento del lenguaje natural.	§10
RA3-f	Se ha descrito la formación teórica que precisa el investigador en procesamiento del lenguaje natural.	§10
RA3-g	Se ha elaborado un sistema de procesamiento de lenguaje orientado a una tarea específica.	§11-12

Nota sobre la tabla anterior.

Bloque de contenidos oficial (RD 279/2021, anexo I)

El currículo llama a este bloque, textualmente, *«Procesamiento del Lenguaje Natural»*, y le asigna estos contenidos:

- *Procesamiento del lenguaje natural: Potencial y limitaciones.*
- *Aplicaciones del procesamiento del lenguaje natural.*

Son **dos contenidos para siete criterios de evaluación**: el desajuste **más grande de todo el módulo**. Ni el papel del lingüista, ni el trabajo cooperativo, ni la formación del investigador —tres de los siete CE— tienen contenido propio en el anexo. Los desarrolla el centro (art. 3.3 del Decreto 95/2026), y es lo que hace esta unidad.

3. Objetivos de la unidad

Objetivo	Al terminar la unidad serás capaz de…
O1	**Explicar** qué es el PLN y situarlo entre la lingüística y la informática.
O2	**Describir** las tareas del PLN y el *pipeline* típico de un sistema.
O3	**Valorar** el potencial de las técnicas actuales con ejemplos y cifras.
O4	**Distinguir** los seis tipos de ambigüedad y reconocerlos en frases reales.
O5	**Explicar** cómo el etiquetado morfológico (POS) desambigua, y qué cuesta elegir el conjunto de etiquetas.
O6	**Identificar** las demás limitaciones: falta de comprensión, sesgos, lenguas con pocos recursos, ironía.
O7	**Determinar** cuándo es factible aplicar PLN, incluidas las restricciones del AI Act.
O8	**Justificar** el papel del lingüista, **evaluar** la cooperación con informáticos y **describir** la formación necesaria.
O9	**Usar** `nltk` y spaCy para tareas básicas de PLN en español.
O10	**Construir y evaluar** un sistema de PLN orientado a una tarea específica.

4. Qué es el PLN (RA3-a)

4.1 Definición

El **procesamiento del lenguaje natural** es el subcampo de la informática y de la IA que busca que las máquinas **comprendan y se comuniquen con el lenguaje humano**. Combina tres disciplinas:

Disciplina	Qué aporta
Lingüística computacional	Los modelos del lenguaje: morfología, sintaxis, semántica, pragmática
Estadística y aprendizaje automático	Aprender los patrones de los textos, en vez de programar todas las reglas a mano

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Disciplina	Qué aporta
Aprendizaje profundo	Redes que aprenden representaciones del lenguaje (*transformers*)

Nota sobre la tabla anterior.

> **No es magia, y la distinción importa**
>
> Un buscador o un traductor no «entienden» como una persona: **procesan estadísticamente** el texto. Esa diferencia no es filosófica, es operativa: explica por qué fallan donde fallan, y es el hilo de todo el §6.

4.2 Los niveles del lenguaje

El lenguaje se analiza por capas, y cada una tiene sus tareas:

Nivel	Qué analiza	Tarea de PLN
Fonética y fonología	Los sonidos	Reconocimiento de voz (ASR), síntesis (TTS)
Morfología	La estructura de las palabras	*Stemming*, lematización, **etiquetado POS**
Sintaxis	La estructura de las oraciones	Análisis sintáctico, dependencias
Semántica	El significado de palabras y frases	Desambiguación (WSD), entidades (NER)
Pragmática y discurso	El significado según el contexto	Correferencia, actos de habla, sentimiento

Esta tabla vale también como mapa de la ambigüedad: **hay ambigüedad en todos los niveles**, y el §6 recorre uno a uno.

4.3 Las tareas del PLN

Tarea	Qué hace	Ejemplo
Tokenización	Divide el texto en unidades	«Hola mundo» → [«Hola», «mundo»]
Etiquetado POS	Asigna categoría gramatical	«correr» → verbo
Entidades (NER)	Identifica nombres, lugares, fechas	«María vive en Madrid» → María: PER, Madrid: LOC
Análisis de sentimiento	Positivo, negativo o neutro	«¡Genial!» → positivo
Traducción automática	Traduce entre idiomas	ES ↔ EN
Resumen	Condensa documentos	Resumir un informe
Generación	Produce texto nuevo	Chatbots, redacción asistida
Respuesta a preguntas	Responde a partir de un texto	Asistentes, buscadores



4.4 El *pipeline*, paso a paso

1. **Preprocesado**: tokenizar, pasar a minúsculas, quitar palabras vacías (*stopwords*), lematizar.
2. **Representación**: convertir el texto en números — bolsa de palabras, tf-idf, *embeddings*.
3. **Modelo**: aplicar un modelo clásico o una red neuronal.
4. **Salida**: la tarea concreta.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Esto lo usas todos los días

El corrector del móvil usa morfología y un modelo de lenguaje. El buscador clasifica tu consulta y le extrae entidades. El asistente de voz primero **transcribe** (ASR), luego interpreta y responde. Los tres son *pipelines* de PLN, con los mismos cuatro pasos.

5. El potencial (RA3-c)

5.1 Qué se consigue hoy

Aplicación	Estado actual
Buscadores	Entienden la consulta, extraen entidades y responden directamente
Asistentes de voz	Transcriben, interpretan y ejecutan órdenes
Traducción automática	Documentos y conversación en tiempo real, con calidad funcional
Análisis de opinión	Valorar miles de reseñas o publicaciones automáticamente
Extracción de información	Leer facturas, contratos e historiales y sacar los campos (OCR + NER)
Chatbots	Resolver consultas frecuentes sin horario
Modelos generativos	Redactar, resumir, traducir y generar contenido

Nota sobre la tabla anterior.

Tres cifras para calibrar

- En **SQuAD 2.0** (respuesta a preguntas), los mejores sistemas **superan la precisión humana**: EM ~91 frente a ~87.
- El análisis de sentimiento en reseñas de cine (IMDb) pasó de ~89 % en 2011 a **95-97 %** con *transformers*.
- El ecosistema **Hugging Face** supera los **2 millones de modelos** y 500.000 *datasets* públicos. Es el «almacén» del que salen los modelos de esta unidad.

5.2 Los grandes modelos de lenguaje

El **AI Act** (Reglamento UE 2024/1689) define los *modelos de IA de uso general*: modelos con al menos **mil millones de parámetros**, entrenados con autosupervisión a gran escala, que realizan con competencia una amplia variedad de tareas. Los **LLM** son el ejemplo típico.

Modelos de gran impacto

El AI Act presume que un modelo de uso general tiene **capacidades de gran impacto** cuando el cómputo acumulado de su entrenamiento supera los **10^{25} FLOPS** (art. 51.2), y entonces le exige documentación y evaluación adicionales. Se trata a fondo en la UD06.

6. La ambigüedad, el problema de fondo (RA3-c)

Si hay que quedarse con una sola limitación del PLN, es esta: **el lenguaje es ambiguo en todos sus niveles**. Múltiples interpretaciones de una misma palabra pueden arruinar la capacidad de un sistema para hacer su trabajo. Y no es un caso raro de laboratorio: aparece en frases que decimos sin darnos cuenta.

6.1 Los seis tipos de ambigüedad

Tipo	Cuándo aparece
Sintáctica	La oración admite **más de un análisis sintáctico**: más de una regla gramatical la representa
Léxica y morfológica	La léxica, cuando un término **aislado** admite varias interpretaciones; la morfológica, cuando una palabra puede tener **más de un rol o categoría** dentro de la frase

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Tipo	Cuándo aparece
Semántica	Un elemento de la frase se puede **interpretar de varios modos**
Pragmática	El sentido **depende del contexto** y de quién habla, en ese momento
Fonológica	Una **cadena de sonidos** resulta confusa
Funcional	Un término se usa con **doble función gramatical**

Nota sobre la tabla anterior.

Los seis, con frases reales

- **Sintáctica** — *«Los perros y los gatos enfermos son recogidos por el servicio municipal.»* ¿Se recogen todos los perros y solo los gatos enfermos, o solo los enfermos de las dos especies? Y *«Compro los libros baratos»*: ¿compro **los baratos** (adjetivo) o compro libros, **que además son baratos** (complemento predicativo)?
- **Léxica y morfológica** — *«Usted aquí no pinta nada.»* ¿No tiene mando o no pinta paredes? Y *«Pedro y yo escribimos un cuento»*: ¿ya lo escribimos o lo estamos escribiendo? La forma conjugada vale para presente y para pretérito.
- **Semántica** — *«Pedro quiere pelearse con un italiano.»* ¿Con cualquier italiano o con uno concreto?
- **Pragmática** — *«Golpeó el armario con el bastón y lo rompió.»* ¿Se rompió el bastón o el armario?
- **Fonológica** — *«es-conde»*: ¿el verbo *esconder* conjugado, o *es* + *conde*?
- **Funcional** — *«He vuelto a oler.»* ¿He recuperado el olfato, o he regresado a un sitio para oler algo?

6.2 Un banco de frases para probar cualquier sistema

Estas frases son un buen test rápido: si un sistema las resuelve, es que usa el contexto de verdad.

- Vino de la Rioja.
- Compré unos zapatos de piel de señora.
- La policía observó al sospechoso con unos prismáticos.
- El pescado está listo para comer.
- El cura recibió una cura en su habitación.
- Juan vio a Pedro enfurecido.
- Antonio no nada nada.
- No puedo ir a la fiesta porque no traje traje.
- Estaré de vacaciones solo unos días.
- El Villarreal le ganó al Valencia en su campo.
- Me quedé esperándote en el banco.

Fíjate en el patrón

Casi todas se resuelven **con el contexto**, no con la frase. *«Vino de la Rioja»* es un sustantivo o un verbo según lo que venga antes; *«en su campo»* depende de quién sea «su». Esa es exactamente la información que un modelo estadístico intenta capturar — y la razón de que el tamaño del contexto sea tan determinante en los *transformers*.

6.3 Por qué la ambigüedad no se «arregla»

La ambigüedad no es un error del lenguaje que se pueda corregir: es una **propiedad** suya, y además útil —permite ser breve, irónico o cortés—. Lo que hace un sistema de PLN es **elegir la interpretación más probable** dado el contexto, y por eso siempre puede equivocarse.

Forma frente a significado

Emily Bender y Alexander Koller argumentaron que un modelo entrenado **solo con la forma** del texto no tiene manera *a priori* de aprender el **significado**: aprende correlaciones estadísticas, no comprensión. De ahí que un modelo generativo pueda «sonar» impecable y estar equivocado. Es la limitación que hay que tener presente al leer las cifras del §5.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

7. Desambiguación y etiquetado morfológico (RA3-a, RA3-c)

7.1 Qué es el POS *tagging*

El **etiquetado morfológico** —*POS tagging*, etiquetado léxico, desambiguación morfosintáctica— es el proceso de asignar a **cada palabra de un texto su categoría gramatical**, sin entrar en las relaciones sintácticas.

Y aquí está la clave: **una palabra aislada suele ser ambigua respecto a su categoría; dentro de un contexto, casi siempre se puede desambiguar**.

La misma palabra, tres categorías

- «*Mételo en ese* ***sobre***» → **nombre**
- «*Déjalo* ***sobre*** *la mesa*» → **preposición**
- «*Dame lo que te* ***sobre***» → **verbo**

Tres frases, la misma forma, tres categorías. Ningún diccionario resuelve esto: hace falta el contexto. Etiquetar bien la categoría es el **primer paso** de casi todo lo demás — de ahí que aparezca tan pronto en el *pipeline* del §4.4.

7.2 Elegir el conjunto de etiquetas: un compromiso

El **conjunto de etiquetas** (*tagset*) que se elige cambia la dificultad del problema, y hay que equilibrar dos cosas que tiran en direcciones opuestas:

- **Más información**: etiquetas específicas — distinguir tiempo, persona, número, género.
- **Menos trabajo de desambiguación**: etiquetas generales — solo «verbo», «nombre», «adjetivo».

Cuantas más etiquetas, más informativo el resultado **y más difícil acertar**.

Conjunto de etiquetas	Número de etiquetas
Penn Treebank	45
Brown Corpus	87
LexEsp (PAROLE)	~250
Susanne	350

Las **categorías principales** son las de siempre: nombres (comunes y propios), pronombres, determinantes, adjetivos, verbos, adverbios, preposiciones y conjunciones. Se dividen en clases **abiertas** —admiten palabras nuevas: nombres, verbos, adjetivos— y **cerradas** —no: preposiciones, determinantes, conjunciones—.

Por qué esto es asunto del lingüista

Decidir el *tagset*, escribir la guía de anotación y resolver los casos dudosos **no es una tarea de programación**: es lingüística aplicada, y condiciona todo lo que el modelo podrá aprender después. Es el ejemplo más concreto del **RA3-b**, y por eso este bloque va justo antes del §9.

7.3 Cómo se etiqueta en la práctica

Enfoque	Cómo funciona	Dónde encaja
Basado en reglas	Reglas escritas a mano sobre el contexto	Dominios muy acotados; didáctico
Modelos de Markov (HMM, TnT)	Probabilidad de una secuencia de etiquetas dado el texto	El clásico; es lo que se practica con el corpus `cess_esp` de `nltk`
Modelos neuronales	La categoría sale del modelo del lenguaje, con el contexto completo	spaCy y *transformers*: lo que se usa hoy

En la práctica de la unidad se hacen los tres: reglas y HMM con `nltk`, y el enfoque neuronal con spaCy.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

8. Las demás limitaciones (RA3-c)

8.1 Falta de comprensión real

Los modelos **generan respuestas plausibles sin comprender**: no verifican hechos, no razonan sobre el mundo y pueden **alucinar** — inventar datos con total seguridad. En cualquier tarea con consecuencias, la salida **se verifica**.

8.2 Sesgos

Los modelos aprenden de los textos que hay, y esos textos llevan **sesgos históricos y sociales**:

- Sesgo de **género** en los *embeddings*: «médico → hombre», «enfermera → mujer».
- Sesgos **raciales, étnicos y culturales** en los datos de entrenamiento.
- El AI Act advierte de que los sesgos pueden **perpetuarse y amplificarse** en bucles de retroalimentación.

> ⚠ **Inferir emociones está prohibido en dos contextos**
>
> El AI Act **prohíbe** los sistemas que infieren el estado emocional **en el trabajo y en centros educativos**: la base científica es débil —la expresión de las emociones varía entre culturas— y su uso ahí es discriminatorio. Es un límite **legal**, no solo técnico, y afecta directamente a lo que se puede construir con análisis de sentimiento.

8.3 Lenguas con pocos recursos

Casi todos los modelos se entrenan con textos mayoritariamente en inglés. En lenguas con menos datos, las técnicas **estadísticas clásicas** pueden aún superar a las neuronales, que solo dan buenos resultados a partir de decenas de miles de *tokens*. No es un detalle menor: decide si un proyecto en una lengua minoritaria es viable.

8.4 Ironía y sarcasmo

«¡Qué buena idea, se me ha caído el móvil al agua!» es literalmente positivo y pragmáticamente negativo. Es ambigüedad **pragmática** pura, y los modelos fallan a menudo.

9. Cuándo es factible aplicar PLN (RA3-d)

9.1 Cinco criterios de decisión

Criterio	Pregunta guía
Tarea clara y acotada	¿Qué salida exacta quiero: clasificar, extraer, traducir, resumir?
Datos disponibles	¿Hay textos suficientes y representativos? ¿Están anotados?
Dominio con vocabulario conocido	¿El texto es de un ámbito concreto — médico, legal, atención al cliente?
Calidad esperada realista	¿Acepto un 5-10 % de error? ¿Cuánto cuesta cada error?
Regulación aplicable	¿Está el uso prohibido o restringido por el AI Act?

9.2 Factible y no factible

Factible hoy (bajo riesgo)	No factible o prohibido
Convertir datos no estructurados en estructurados	**Inferir emociones** en el trabajo o la educación
Clasificar documentos y detectar duplicados	Extracción no selectiva de rostros de internet
Mejorar el registro o el lenguaje de un documento	Manipulación subliminal del comportamiento
Traducción funcional de documentos	Comprensión profunda con inferencia abierta
Buscar y vincular datos en archivos	Respuesta fiable en dominios de alto riesgo **sin verificación**
Análisis de sentimiento en un dominio acotado	Detección de sarcasmo con alta precisión

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Nota sobre la tabla anterior.

> **El AI Act como herramienta de diseño**
>
> El **considerando 53** enumera tareas de PLN de **bajo riesgo** —procesamiento delimitado: transformar datos, clasificar, detectar duplicados, mejorar el lenguaje, traducción funcional— y el **artículo 5** prohíbe prácticas concretas. Leídos juntos, son una lista de comprobación excelente para decidir qué construir **antes** de empezar. Eso es el CE d.

10. Quién hace un proyecto de PLN (RA3-b, RA3-e, RA3-f)

Aquí están **tres de los siete criterios** de este RA, y son los que un material puramente técnico suele resolver en un párrafo. No se demuestran con código, pero deciden si un proyecto de PLN sale bien o sale caro.

10.1 Qué aporta el lingüista (RA3-b)

Aportación	En qué consiste
Anotación de datos	Etiquetar entidades, sentimiento o categorías en los textos de entrenamiento
Diseño del conjunto de etiquetas y de la guía	Definir el *tagset* del §7.2 y las reglas para aplicarlo de forma coherente
Recursos lingüísticos	Construir corpus y árboles sintácticos (*treebanks*), un trabajo que puede llevar años
Gramáticas y reglas	Las reglas morfológicas y sintácticas de la rama simbólica del PLN
Evaluación y análisis de errores	Explicar **por qué** falla un modelo: ambigüedad, registro, dominio

Nota sobre la tabla anterior.

> **Las anotaciones deciden las predicciones**
>
> La calidad del etiquetado condiciona directamente lo que el modelo puede aprender: con etiquetas malas o **inconsistentes entre anotadores**, el modelo aprende esa inconsistencia. Y esto no se arregla con más datos ni con un modelo mejor.
>
> Es la razón de fondo del CE b: **el lingüista no es un apoyo del proyecto, es parte de la tubería de datos**. Volviendo al §7.2: quien decide si el *tagset* tiene 45 etiquetas o 250 está decidiendo el techo de precisión del sistema.

10.2 El trabajo cooperativo (RA3-e)

El PLN es **interdisciplinar por definición**: lingüistas, informáticos y, según el proyecto, psicólogos o expertos en lógica. Dos casos reales que muestran cómo se hace bien:

- **Meta NLLB** — traducción entre 200 idiomas. Trabajaron con **hablantes nativos** para anotar y evaluar, y consiguieron **+44 % de BLEU medio** además de controlar la toxicidad de las traducciones. Sin hablantes nativos no había forma de evaluar la mayoría de esos idiomas.
- **Masakhane** — PLN para lenguas africanas. Más de **1.000 participantes de 30 países**, y una regla explícita: los lingüistas locales **no son «solo anotadores», son investigadores**. Rechazan la investigación *paracaidista*: entrar, tomar los datos y marcharse sin dejar capacidad local.

Lo que se gana cooperando	Lo que cuesta
Datos y evaluación de mucha más calidad	Vocabularios y métodos distintos: hay que traducirse
Cobertura de lenguas minoritarias	La anotación es **lenta y cara**
Mejor ciencia y sistemas más justos	Riesgo de **usar a las comunidades como proveedores de datos**

Nota sobre la tabla anterior.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

> **El riesgo tiene nombre**
>
> La cooperación puede degenerar en extracción: una empresa del norte global obtiene datos anotados baratos y publica; la comunidad que los produjo no queda con capacidad ni con crédito. Es el mismo problema que en la UD06 aparece como **el trabajo invisible detrás de la IA**. Por eso el CE e dice **«evaluar»** el trabajo cooperativo, no solo describirlo: hay formas mejores y peores de hacerlo.

10.3 La formación del investigador (RA3-f)

El perfil combina tres patas, y **ninguna sobra**:

Pata	Contenidos
Lingüística	Morfología, sintaxis, semántica, pragmática y discurso
Estadística y aprendizaje automático	Probabilidad, modelos, métricas y evaluación
Informática y métodos formales	Algoritmos, estructuras de datos, programación

El itinerario recomendado, en este orden:

1. **Jurafsky y Martin**, *Speech and Language Processing* — la referencia clásica, de acceso libre. Da la base teórica y el vocabulario.
2. **El manual de NLTK** — el mismo camino, pero con las manos: cada paso del *pipeline*, a mano.
3. **spaCy**, en inglés y en español — el salto a herramientas de producción.
4. **Modelos neuronales y *transformers*** — Hugging Face, y de ahí a plataformas de inferencia si hace falta desplegar.

> **Perfiles que no existían hace cinco años**
>
> Junto al lingüista computacional clásico han aparecido el **anotador o evaluador especializado** —con formación en derecho, medicina o ciencia, porque hay que entender el texto para etiquetarlo bien—, el perfil de **prompt engineer**, en evolución hacia LLMOps, y el **evaluador de sesgos y ética** de modelos. Los tres parten de la misma base: **lingüística más técnica**.

> **Cómo se evalúan estos tres criterios en esta unidad**
>
> No con un examen aparte: con la **parte escrita de los entregables** `N09` **y** `N11`. En cada uno se pide justificar qué decisiones tomaría un lingüista en ese problema concreto, qué aporta cada perfil y qué formación haría falta. Está en la rúbrica, y cuenta.

11. Las herramientas en Python (RA3-c, RA3-g)

> **Dos entornos, y conviene saber cuál toca**
>
> Esta unidad usa las librerías más pesadas del módulo. La regla:
>
> - **En el contenedor de la unidad** (`Dockerfile` y `docker-compose.yml` de la carpeta): `nltk`, spaCy, scikit-learn, `gensim`, `textblob`. Todo lo que no entrena redes grandes.
> - **En Colab**: todo lo que use `transformers`, entrenamiento de modelos o **audio**. Ahí hay GPU, y afinar un DistilBERT en CPU es cuestión de horas.
>
> Cada notebook lo indica en su primera celda.

11.1 NLTK, para entender

NLTK es una plataforma de enseñanza e investigación con más de 50 corpus y recursos léxicos, como WordNet. Su virtud es que es **transparente**: cada paso del procesamiento se hace a mano, y por eso es la herramienta con la que se **aprende**.

```
pip install nltk
python -c "import nltk; nltk.download(['punkt_tab','stopwords','wordnet','cess_esp'])"
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```python
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem.snowball import SnowballStemmer

texto = "La inteligencia artificial está transformando la atención al cliente."
tokens = word_tokenize(texto, language='spanish')
sin_vacias = [t for t in tokens if t.lower() not in stopwords.words('spanish')]
raiz = SnowballStemmer('spanish').stem

print("Tokens:", tokens)
print("Sin stopwords:", sin_vacias)
print("Stemming:", [raiz(t) for t in sin_vacias])
```

El *pipeline* de NLTK, paso a paso

1. **Tokenizar**: `word_tokenize` separa palabras y signos.
2. **Limpiar**: se descartan artículos, preposiciones y palabras sin contenido.
3. **Normalizar**: el *stemming* recorta a una raíz aproximada («transformando» → «transform»); la **lematización** da el lema real («transformando» → «transformar»). El primero es rápido y tosco; la segunda necesita un diccionario.
4. **Enriquecer**: etiquetar POS (`pos_tag`), detectar entidades (`ne_chunk`), contar frecuencias (`FreqDist`) o buscar concordancias (`concordance`).

11.2 spaCy, para producir

```
pip install spacy
python -m spacy download es_core_news_sm
```

```python
import spacy

nlp = spacy.load("es_core_news_sm")
doc = nlp("María trabaja en Madrid para una empresa de inteligencia artificial.")

for token in doc:
    print(f"{token.text:<12} {token.pos_:<8} {token.lemma_:<12}")

print("\nEntidades:")
for ent in doc.ents:
    print(f"  {ent.text:<12} → {ent.label_}")
```

El *pipeline* de spaCy ejecuta toda la cadena **de una pasada**:



En una frase corriente, el analizador sintáctico puede encontrar **miles de análisis posibles**, y spaCy elige el más probable con su modelo estadístico. Es el §6 en acción: la ambigüedad no se elimina, **se decide**.

NLTK o spaCy: no compiten

NLTK para **aprender** — cada paso a mano, todo visible. **spaCy** para **producir** — un objeto `nlp` , modelos preentrenados, mucho más rápida y con 25 idiomas. En los talleres se empieza con NLTK y se pasa a spaCy, en ese orden y por ese motivo.

Por qué las entidades dan dinero

La extracción de entidades es la base de leer facturas, contratos o historiales: el sistema encuentra fechas, importes, nombres y organizaciones **sin que nadie los marque a mano**. Es una de las aplicaciones más rentables del PLN, y de las menos vistosas.

11.3 Análisis de sentimiento

Con scikit-learn, que funciona en español y sin GPU:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from nltk.corpus import stopwords

reseñas = ["me encantó la comida", "pésimo servicio", "todo perfecto", "no volveré"]
etiquetas = [1, 0, 1, 0]

pipe = make_pipeline(
    TfidfVectorizer(stop_words=stopwords.words('spanish')),
    LogisticRegression())
pipe.fit(reseñas, etiquetas)
print(pipe.predict(["la comida estaba buenísima"]))
```

El detalle que rompe el ejemplo en español

`TfidfVectorizer(stop_words=...)` **solo acepta la cadena** `'english'`. Para español hay que pasar una **lista** — `stopwords.words('spanish')` — o no pasar nada y usar `max_df`. Si se escribe `stop_words='spanish'`, el error es poco claro.

11.4 *Transformers* y Hugging Face

Los modelos **Transformer** se usan con la librería `transformers` y sus *pipelines*. Funcionan en CPU, pero **entrenarlos o afinarlos, no**: eso va a Colab.

```python
from transformers import pipeline

clasificador = pipeline("sentiment-analysis",
                        model="pysentimiento/bertin-roberta-base-sentiment-analysis")
print(clasificador("El servicio fue excelente, volveré seguro."))
```

DistilBERT, y por qué se usa aquí

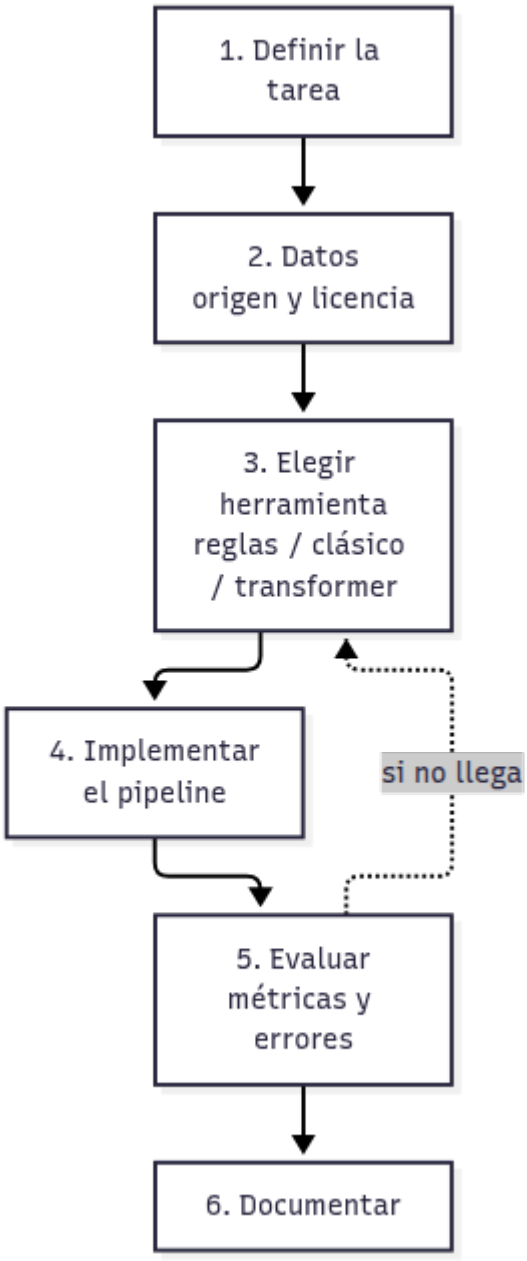
DistilBERT tiene un **40 % menos de parámetros** y es un **60 % más rápido** que BERT, conservando el **97 %** de su capacidad. Por eso es el modelo de los notebooks de la unidad: cabe en una sesión de clase. El entregable `N10` lo **afina** para reseñas de cine, que es *transfer learning* de manual.

12. Construir un sistema orientado a una tarea (RA3-g)

12.1 La metodología, en seis pasos

1. **Definir la tarea**: qué entra, qué sale y para quién.
2. **Conseguir o anotar los datos**: origen, formato, tamaño y **licencia**.
3. **Elegir la herramienta**: reglas, modelo clásico o *transformer*, según datos y recursos.
4. **Implementar** el *pipeline*: preprocesado → representación → modelo → salida.
5. **Evaluar** sobre datos **no vistos**, y **analizar los errores** — no solo mirar la métrica.
6. **Documentar**: origen de los datos, licencia, decisiones y limitaciones.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



El paso 5 es el que más se salta

Medir la exactitud es fácil; **mirar los errores** es lo que enseña. Casi siempre se agrupan: un registro que no estaba en el entrenamiento, un tipo de ambigüedad concreta, un vocabulario de dominio. Y eso apunta a qué hacer después — que es volver al paso 3, no al paso 4.

12.2 La progresión de los notebooks

Los notebooks de la unidad recorren esa metodología de menos a más, y conviene verlos como una escalera, no como piezas suextas:

Notebook	Qué construye	Representación
`N01` · introducción al PLN	Tokenizar, *stopwords*, BoW y tf-idf con `nltk`	Cuenta de palabras
	Un clasificador de texto entrenado desde cero	

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Notebook	Qué construye	Representación
`N02` · clasificación con PyTorch		De BoW a *word embeddings*
`N03` · modelos de lenguaje	**Afinar DistilBERT** para sentimiento	*Embeddings* contextuales
`N04` · spaCy	El *pipeline* completo: POS, dependencias, entidades	Modelo preentrenado
`N06` · representación de texto	Tokenizar, *stopwords*, BoW y tf-idf, **por dos caminos** (NLTK y TextBlob)	Cuenta de palabras
`N07` · `nltk` y Python	POS *tagging* con el corpus `cess_esp` y validación cruzada	Etiquetado estadístico

`N05`, el clasificador de géneros musicales, es ampliación y no cuenta horas de la unidad.

Y las cinco entregas aplican cada nivel a un problema propio:

Entrega	Tarea	Qué demuestra
`N08`	**Del texto al vector**: un clasificador de reseñas con datos anotados por vosotros	Los fundamentos, aplicados a datos propios
`N09`	**Clasificar preguntas** repitiendo el proceso de `N02`	Construir un clasificador propio
`N10`	**Afinar DistilBERT** para reseñas de cine	*Transfer learning* a una tarea nueva
`N11`	**Asistente virtual por voz**	Un sistema de punta a punta, con audio
`N12`	**Extractor de entidades** con spaCy, de la tarea a la documentación	La metodología completa del CE g

Nota sobre la tabla anterior.

Ejemplo guiado: un clasificador de reseñas en seis pasos

1 · Tarea: clasificar una reseña de restaurante como positiva o negativa. **2 · Datos**: 60 reseñas anotadas a mano, 40 para entrenar y 20 para probar; licencia propia. **3 · Herramienta**: scikit-learn con tf-idf, porque con 60 ejemplos un *transformer* no aporta nada. **4 · Implementar**: `TfidfVectorizer → LogisticRegression`. **5 · Evaluar**: exactitud en las 20 de prueba y **leer los dos errores** — casi siempre son ironía o una negación. **6 · Documentar**: origen, licencia y el límite obvio (60 ejemplos de un solo dominio).

Fíjate en el paso 3: **la herramienta se elige por los datos que hay**, no por lo moderna que sea. Con 60 ejemplos, tf-idf gana.

13. Puntos clave de la unidad

- El **PLN** busca que las máquinas comprendan y se comuniquen con el lenguaje humano, combinando lingüística computacional, estadística y aprendizaje profundo.
- Sus tareas van de la **tokenización** al etiquetado POS, entidades, sentimiento, traducción, resumen y generación; el *pipeline* típico es **preprocesado → representación → modelo → salida**.
- El **potencial** es alto y medible: en respuesta a preguntas los sistemas **superan la precisión humana** (SQuAD 2.0), y el sentimiento en reseñas ronda el **95-97 %**.
- La **ambigüedad** es el problema de fondo, y tiene **seis formas**: sintáctica, léxica y morfológica, semántica, pragmática, fonológica y funcional. No se elimina: **se decide** con el contexto.
- El **etiquetado POS** es la herramienta central de desambiguación. Elegir el *tagset* es un compromiso: **más etiquetas = más información y más difícil acertar** (45 en Penn Treebank, 350 en Susanne).
- Las otras limitaciones: **forma ≠ significado** (Bender y Koller), alucinaciones, **sesgos**, lenguas con pocos recursos, ironía y sarcasmo.
- El **AI Act** sirve de herramienta de diseño: su considerando 53 lista PLN de bajo riesgo y su artículo 5 **prohíbe** prácticas concretas, como **inferir emociones en el trabajo o la educación**.
- El **lingüista** no es un apoyo: diseña el *tagset*, la guía de anotación y los recursos. **Las anotaciones deciden las predicciones**, y eso no se arregla con más datos.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- La **cooperación** entre perfiles se **evalúa**, no solo se describe: NLLB con hablantes nativos (+44 % BLEU) y Masakhane con 1.000 participantes son ejemplos de hacerlo bien; la investigación *paracaidista*, de hacerlo mal.
- La **formación** combina lingüística, estadística e informática, con un itinerario claro: Jurafsky → NLTK → spaCy → *transformers*.
- **NLTK para aprender, spaCy para producir**; scikit-learn cuando hay pocos datos y *transformers* cuando hay muchos y GPU. **La herramienta se elige por los datos que hay.**
- Construir un sistema son **seis pasos**, y el que más se salta es el quinto: **mirar los errores**, no solo la métrica.

14. Glosario

Término	Definición
PLN	Procesamiento del lenguaje natural
Token	Unidad mínima en que se divide un texto: palabra, signo, subpalabra
Tokenización	Dividir el texto en *tokens*
Stopwords	Palabras vacías (artículos, preposiciones) que se descartan
Stemming	Recortar la palabra a una raíz aproximada
Lematización	Reducir la palabra a su lema real, con diccionario
POS *tagging*	Asignar a cada palabra su categoría gramatical
Tagset	El conjunto de etiquetas gramaticales que se usa
Penn Treebank	*Tagset* de referencia en inglés: 45 etiquetas
PAROLE / LexEsp	*Tagset* de referencia en español: ~250 etiquetas
Categoría abierta / cerrada	Admite palabras nuevas (nombres, verbos) o no (preposiciones)
Ambigüedad sintáctica	La oración admite más de un análisis sintáctico
Ambigüedad léxica	Un término aislado admite varias interpretaciones
Ambigüedad morfológica	Una palabra puede tener más de una categoría en la frase
Ambigüedad semántica	Un elemento se puede interpretar de varios modos
Ambigüedad pragmática	El sentido depende del contexto y del hablante
Ambigüedad fonológica	Una cadena de sonidos resulta confusa
Ambigüedad funcional	Un término con doble función gramatical
Desambiguación	Elegir la interpretación correcta según el contexto
WSD	Desambiguación del sentido de las palabras
NER	Reconocimiento de entidades nombradas
Análisis de dependencias	Determinar las relaciones sintácticas entre palabras
Correferencia	Saber a qué se refiere un pronombre
Bolsa de palabras (BoW)	Representar un texto por la cuenta de sus palabras, sin orden
tf-idf	Pesar cada palabra por su frecuencia relativa en el documento y en el corpus
Word embedding	Representar palabras como vectores densos con significado
***Embedding* contextual**	El vector de una palabra cambia según su contexto (BERT)
Transformer	Arquitectura basada en atención, base de los modelos actuales
BERT / DistilBERT	Modelo de lenguaje bidireccional, y su versión reducida
Transfer learning	Adaptar un modelo ya entrenado a una tarea nueva

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Término	Definición
Fine-tuning	Afinar los pesos de un modelo preentrenado con datos propios
LLM	Modelo grande de lenguaje
Modelo de uso general	En el AI Act, modelo con ≥ mil millones de parámetros y autosupervisión a gran escala
Alucinación	Que el modelo genere información falsa con apariencia de certeza
NLTK	Biblioteca de PLN orientada a la enseñanza y la investigación
spaCy	Biblioteca de PLN orientada a producción
WordNet	Base de datos léxica de relaciones entre significados
`cess_esp`	Corpus del español anotado, incluido en NLTK
HMM / TnT	Modelos de Markov para etiquetado morfológico
Hugging Face	Repositorio de modelos y *datasets* de PLN
SQuAD	Conjunto de referencia para respuesta a preguntas
BLEU	Métrica de calidad de traducción automática
Corpus	Colección de textos usada para entrenar o evaluar
Treebank	Corpus anotado con árboles sintácticos
Guía de anotación	Documento que fija cómo etiquetar de forma coherente
Lengua con pocos recursos	Idioma con pocos datos disponibles para entrenar

15. FAQ

¿El PLN «entiende» de verdad el lenguaje?

No. Procesa estadísticamente: aprende qué palabras y estructuras aparecen juntas. Bender y Koller lo argumentaron con precisión: un modelo entrenado **solo con la forma** del texto no tiene manera *a priori* de aprender el **significado**. Por eso puede escribir un párrafo impecable y equivocarse en el dato.

Si la ambigüedad no se puede eliminar, ¿cómo funciona nada?

Porque el sistema **no necesita acertar siempre**: necesita acertar lo suficiente para la tarea. Elige la interpretación más probable según el contexto, y en la mayoría de los casos es la buena. El problema aparece cuando el coste de equivocarse es alto — y ahí entra el §9.

¿Cuántas etiquetas POS conviene usar?

Las mínimas que resuelvan tu problema. Con 45 (Penn Treebank) se acierta más; con 250 (PAROLE) se sabe mucho más de cada palabra pero se falla más. Si solo necesitas distinguir nombres de verbos, usar 250 etiquetas es tirar precisión a la basura.

¿Para qué necesito un lingüista si tengo un modelo preentrenado?

Para tres cosas que el modelo no hace: **decidir qué etiquetar y cómo** (el *tagset* y la guía), **anotar de forma coherente** los datos de tu dominio, y **explicar por qué falla** cuando falla. Un modelo preentrenado te ahorra entrenar, no te ahorra entender tu problema.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

¿NLTK o spaCy?

Las dos, en este orden. **NLTK** para aprender, porque cada paso es visible y se hace a mano. **spaCy** para trabajar, porque un solo objeto ejecuta todo el *pipeline* con modelos preentrenados y es mucho más rápida.

¿Puedo hacer análisis de sentimiento para detectar el ánimo de mis empleados?

No. El AI Act **prohíbe** los sistemas que inferen el estado emocional en el **trabajo** y en **centros educativos**: la base científica es débil y el uso es discriminatorio. Es un límite legal, y es exactamente el tipo de decisión que pide el CE d.

¿Hace falta GPU para esta unidad?

Para la mitad, no: `nltk`, spaCy, scikit-learn y las representaciones de texto van en el contenedor. Para **afinar DistilBERT** y para el notebook de audio, sí — y por eso esos van en **Colab**, que la da gratis.

¿Por qué mi modelo va peor en español que en inglés?

Porque casi todos se entrenan con textos mayoritariamente en inglés. En español hay recursos buenos, pero menos; en lenguas minoritarias, muy pocos. En esos casos, las técnicas estadísticas clásicas pueden **superar** a las neuronales, que necesitan decenas de miles de *tokens* para despegar.

¿Qué diferencia hay entre *stemming* y lematización?

El *stemming* recorta a una raíz aproximada sin diccionario: rápido y tosco («transformando» → «transform», que no es una palabra). La lematización devuelve el **lema real** usando un diccionario: más lento y correcto («transformando» → «transformar»). Para buscar, suele bastar el primero; para analizar, hace falta el segundo.

16. Sesiones

Semana	Horas	Contenido	CE
16	3	Qué es el PLN, tareas y *pipeline*; el potencial con cifras. `N01` (`nltk`) y `N06`	RA3-a, RA3-c
17	3	**La ambigüedad en sus seis formas**; desambiguación y POS *tagging*. `N04` (spaCy) y `N07` (`cess_esp`); `N08`	RA3-a, RA3-c
18	3	Las demás limitaciones; cuándo es factible con la lupa del AI Act. `N02` y `N09`; `N12`	RA3-c, RA3-d, RA3-g
19	3	El lingüista, la cooperación y la formación; sistemas orientados a tarea. `N03`, `N10` y `N11`; evaluación	RA3-b, RA3-e, RA3-f, RA3-g

Nota sobre la tabla anterior.

Sobre el reparto

Los tres CE «humanos» se tratan en la **semana 19**, cuando el alumnado ya ha peleado con los datos y con la anotación: así la discusión sobre el papel del lingüista deja de ser abstracta. Y el material de **ampliación** (el clasificador de géneros musicales) **no cuenta horas**.

17. Recursos

- Diapositivas
- **Práctica** — se hace, no se entrega ni puntúa:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

  - Ejercicios de autoevaluación
  - Notebooks guiados — 5 notebooks, más los `N06` y `N07`
- **Entregas** — qué se entrega:
  - N08 · del texto al vector · N12 · un sistema de PLN de punta a punta
  - y los notebooks `N09`, `N10` y `N11`, cada uno con su rúbrica
- Los notebooks se abren desde **Práctica** y **Entregas**, con descarga y apertura en Colab.

Referencias de la unidad

- *Speech and Language Processing* — Jurafsky y Martin (acceso libre)
- *Introduction to Natural Language Processing* — Eisenstein
- NLTK Book · spaCy · Usage · Hugging Face
- Bender y Koller (2020), *Climbing towards NLU*
- Meta NLLB · Masakhane
- Reglamento (UE) 2024/1689 · AI Act — considerando 53 y art. 5
- SQuAD · Penn Treebank POS tags

18. Evaluación

Peso	Instrumento
40 % actividades	Los talleres **N08** y **N12** y los notebooks `N09`, `N10` y `N11`, cada uno con su rúbrica en la tarea de Moodle. `N06` y `N07` son práctica y no puntúan
60 % prueba escrita	Prueba del RA3 en Moodle: preguntas de test y de desarrollo sobre el contenido de la unidad

- **La normativa exige alcanzar todos los RA** del módulo para superarlo (art. 5.1 de la Orden 8/2025: la calificación del módulo está «*en función de la consecución de los RA*»; y las Instrucciones 26-27, que impiden calificar positivamente un módulo con RA no superados). El centro concreta ese mandato exigiendo **≥ 5 en cada RA**.
- **`N09` y `N11` llevan una parte escrita** que cubre los criterios b, e y f: el papel del lingüista, la cooperación entre perfiles y la formación necesaria. Está en su rúbrica y **cuenta**.

CE	Dónde se trabaja	Con qué se evalúa
RA3-a	§4, §7	`N08`, `N06`, prueba del RA3
RA3-b	§10.1	**Parte escrita de `N09` y `N11`**, prueba del RA3
RA3-c	§5-8	`N06`, `N10`, `N08` y `N12`, prueba del RA3
RA3-d	§9	`N12`, prueba del RA3
RA3-e	§10.2	**Parte escrita de `N09` y `N11`**, prueba del RA3
RA3-f	§10.3	**Parte escrita de `N09` y `N11`**, prueba del RA3
RA3-g	§11-12	`N09`, `N10`, `N11`, `N12`

19. Recuperación

Actividades del programa de recuperación individual por RA (art. 14.4 de la Orden 8/2025): construir un sistema de PLN orientado a una tarea distinta —otro dominio, otro tipo de salida— y las pruebas de autoevaluación de la unidad.

---

6 de octubre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

6.2 Diapositivas de la UD03



Diapositivas PDF

Cómo se manejan

Flechas o clic para avanzar · **F** para verlas a pantalla completa · **P** para las notas del ponente.

24 de agosto de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

7. UD04

7.1 UD04 — Análisis de sistemas robotizados

Unidad 4 · 12 h · semanas 20-23

Cierra el bloque de aplicaciones de la IA. Se evalúa con **seis entregables prácticos** y la prueba escrita del RA4.

1. Introducción

Los robots ya no son una promesa de futuro: en 2024 se instalaron **más de 540.000 robots industriales** en el mundo y España fue el **tercer mercado europeo**, impulsado por la automoción. Entender cómo funciona un robot, cómo se modela su movimiento y cómo se diseña un sistema robotizado es competencia directa de un especialista en IA, porque un robot es un **agente encarnado**: el único sistema de IA de este curso que puede cambiar el estado del mundo físico.

Eso lo hace distinto de todo lo anterior. Un clasificador que se equivoca produce una etiqueta errónea; un robot que se equivoca rompe una pieza, o algo peor. Y su entorno es **parcialmente observable, estocástico y con más agentes dentro**: las cámaras no ven tras las esquinas, los engranajes patinan y las personas que comparten el espacio son impredecibles.

El recorrido de la unidad:

1. **Métodos y aplicaciones de la robótica**: qué es un robot, su hardware (sensores, actuadores, pinzas) y dónde se usa, con datos del sector.
2. **Qué clase de problema resuelve la robótica** y la jerarquía tarea → movimiento → control.
3. **Modelado y control cinemático** de manipuladores: grados de libertad, parámetros DH, cinemática directa e inversa.
4. **Los problemas típicos** (singularidades, redundancia) y sus **soluciones**: espacio de configuración, planificación de movimiento y seguimiento de trayectoria.
5. **Percepción robótica**: localización, mapeo y SLAM.
6. **Planificación con incertidumbre y aprendizaje** en robótica.
7. **Las técnicas de programación** de robots, del *teach pendant* al ROS 2 — y comparadas en la práctica sobre un mismo problema.
8. **Humanos y robots**: coordinación y aprender lo que la persona quiere.
9. **El diseño e implementación** de un sistema robotizado: selección, célula, seguridad y normativa.

Hilo conductor de la unidad

Un robot percibe, razona y actúa sobre el mundo físico. Primero vemos con qué lo hace (hardware); luego cómo se describe y controla su movimiento (cinemática); después cómo decide por dónde ir (planificación) y cómo sabe dónde está (percepción); y por último cómo se programa y cómo se integra en un sistema seguro.

De dónde sale el material de esta unidad

La parte de **percepción, planificación de movimiento, aprendizaje por refuerzo e interacción humano-robot** está construida a partir del capítulo 26 (*Robotics*) de *Artificial Intelligence: A Modern Approach*, 4.ª edición, de **Stuart Russell y Peter Norvig**, adaptada al nivel y a los criterios de evaluación de este módulo. Las prácticas con el simulador **AITK** provienen del curso Models d'IA de **Carles Gonzalez**, con licencia CC BY-NC-SA 4.0.

2. Resultado de aprendizaje y criterios de evaluación

RA4 — Analiza sistemas robotizados, evaluando opciones de diseño e implementación.

CE	Criterio de evaluación	Bloque
RA4-a		§4-6

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

CE	Criterio de evaluación Se han recopilado los problemas del modelado y control cinemático en robots manipuladores.	Bloque
RA4-b	Se han buscado soluciones a los problemas de los robots.	§7-9
RA4-c	Se han valorado las características diferenciadoras de las técnicas de programación de robots y de sistemas robotizados.	§10-11
RA4-d	Se han evaluado diferentes opciones en el diseño e implementación de sistemas robotizados.	§12

Nota sobre la tabla anterior.

Bloque de contenidos oficial (RD 279/2021, anexo I)

El currículo llama a este bloque, textualmente, *«Análisis de sistemas robotizados»*, y le asigna estos contenidos:

- *Métodos y aplicaciones de la robótica.*
- *Modelado y control de robots.*
- *Programación de robots y aplicaciones.*
- *Sistemas robotizados. Diseño e implementación.*

Son **cuatro contenidos para cuatro criterios de evaluación** y, a diferencia del resto de los RA de este módulo, aquí **encajan casi uno a uno**: es la excepción. El único matiz es que *«modelado y control»* alimenta a la vez RA4-a (los problemas) y RA4-b (las soluciones).

3. Objetivos de la unidad

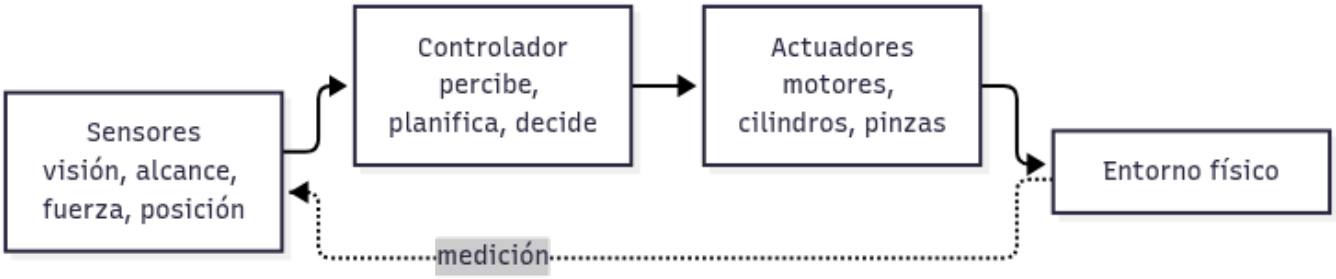
Objetivo	Al terminar la unidad serás capaz de…
O1	**Clasificar** los robots por tipo y describir sus aplicaciones reales con datos del sector.
O2	**Distinguir** los sensores por lo que miden (entorno, ubicación, configuración interna) y elegir el adecuado para una tarea.
O3	**Explicar** grados de libertad, articulaciones y espacio articular frente a cartesiano.
O4	**Resolver** la cinemática directa de un manipulador con parámetros DH y entender por qué la inversa es más difícil.
O5	**Identificar** los problemas típicos (singularidades, redundancia, límites) y sus soluciones.
O6	**Plantear** un problema de planificación de movimiento en el espacio de configuración y elegir un método.
O7	**Explicar** cómo un robot se localiza y construye un mapa (SLAM) a partir de sensores con ruido.
O8	**Comparar** las técnicas de programación de robots resolviendo un mismo problema por reglas, lógica difusa, red neuronal y neuroevolución.
O9	**Aplicar** criterios de selección y diseño de un sistema robotizado (payload, alcance, sensores, seguridad).
O10	**Valorar** la normativa de seguridad (ISO 10218:2025) y la integración en la Industria 4.0.

4. Métodos y aplicaciones de la robótica (RA4-a)

4.1 ¿Qué es un robot?

Un **robot** es una máquina programable que **percibe su entorno**, **procesa información** y **actúa físicamente** sobre él. Tiene tres partes que se realimentan:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Los **efectores** son las piezas que ejercen fuerza sobre el entorno: ruedas, patas, articulaciones, pinzas. Cuando actúan, pueden cambiar tres cosas: el **estado del robot** (un coche gira las ruedas y avanza), el **estado del entorno** (un brazo empuja una taza) e incluso el **estado de las personas** alrededor (un exoesqueleto mueve una pierna; un robot se acerca al ascensor y alguien se aparta).

4.2 Tipos de robot, desde el hardware

La imagen popular del robot con cabeza y dos brazos —el **antropomórfico** de la ficción— es la menos frecuente en la realidad:

Tipo	Qué es	Ejemplo
Manipulador	Un brazo, no necesariamente unido a un cuerpo: se atornilla a una mesa o al suelo	Brazos de una línea de montaje; brazos montados en sillas de ruedas para asistencia
Robot móvil con ruedas	Se desplaza sobre ruedas, dentro o fuera	Aspiradora, AGV/AMR de almacén, coche autónomo, rover de Marte
Robot con patas	Atraviesa terreno accidentado donde la rueda no llega	Cuadrúpedos de inspección
Aéreo (UAV) y submarino (AUV)	Rotores o propulsión sumergida	Cuadricópteros, vehículos de exploración oceánica
Cobot	Manipulador que comparte espacio con personas, con potencia y fuerza limitadas	Montaje asistido, inspección
Otros	Prótesis, exoesqueletos, robots con alas, **enjambres** y **entornos inteligentes**, donde el robot es la habitación entera	Rehabilitación, agricultura de precisión

La diferencia entre un manipulador de una tonelada que ensambla coches y un brazo de asistencia no es solo el tamaño: el primero mueve mucha carga, el segundo mueve poca pero es **seguro entre personas**. Payload y seguridad son criterios distintos, y a menudo opuestos.

4.3 Sensores: qué se mide y con qué

Los sensores son la interfaz perceptiva del robot. Se clasifican de dos formas a la vez.

Por cómo obtienen la señal: los **pasivos** (cámaras) captan lo que el entorno emite; los **activos** (sonar, lidar) emiten energía y miden lo que vuelve. Los activos dan más información, pero consumen más y **se interfieren entre sí** cuando hay varios trabajando a la vez.

Por lo que miden:

Clase	Qué informa	Sensores típicos
Del entorno	Distancia y forma de lo que hay alrededor	Sonar, visión estéreo, luz estructurada (Kinect), cámara de tiempo de vuelo, **lidar**, radar, táctiles
De la ubicación	Dónde está el robot	GPS/GLONASS, balizas de interior, análisis de señal wifi, balizas de sonar bajo el agua
De la configuración interna (propioceptivos)	Cómo está el propio robot	Encoders de eje, odometría de rueda, giroscopios, sensores de fuerza y par

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Las cifras ayudan a elegir:

Sensor	Alcance y precisión	Dónde encaja
Lidar de barrido	Precisión de **~1 cm a 100 m**	El sensor de referencia en coches autónomos
Cámara de tiempo de vuelo	Imágenes de rango a **hasta 60 fps**	Interiores y distancias cortas; peor que el lidar a plena luz del día
Radar	Hasta **kilómetros**, y **ve a través de la niebla**	Vehículos aéreos y automoción
GPS	**Unos metros**; con GPS diferencial, precisión milimétrica en condiciones ideales	Exteriores. **No funciona en interiores ni bajo el agua**
Sensores de fuerza y par	Fuerzas en 3 traslaciones y 3 rotaciones, **cientos de medidas por segundo**	Manipular objetos frágiles o de forma desconocida

Nota sobre la tabla anterior.

Por qué la odometría no basta

Contar vueltas de rueda parece una forma barata y exacta de saber cuánto has avanzado, y lo es — durante unos metros. Las ruedas **derrapan y patinan**, y el error se **acumula sin límite**: no hay nada que lo corrija. Por eso la odometría se combina siempre con sensores inerciales (giroscopios) y con alguna referencia externa. Es la razón de ser de la localización probabilística del §8.

El caso de la bombilla

Imagina un manipulador de **una tonelada** enroscando una bombilla. Es facilísimo aplicar demasiada fuerza y romperla. Los **sensores de fuerza** le dicen con qué fuerza agarra y los **de par**, con qué fuerza gira; midiendo cientos de veces por segundo, el robot detecta una fuerza inesperada y corrige **antes** de romper el cristal. La capacidad de manipular con cuidado no viene de tener un brazo más suave, sino de **medir rápido**.

4.4 Actuadores y pinzas

El mecanismo que produce el movimiento es el **actuador**:

Tipo	Cómo funciona	Dónde se usa
Eléctrico	Corriente que hace girar un motor	El más común; articulaciones de brazos robóticos
Hidráulico	Fluido a presión (aceite, agua)	Mucha fuerza: maquinaria pesada
Neumático	Aire comprimido	Movimientos rápidos y simples, pinzas

Los actuadores mueven **articulaciones** entre eslabones rígidos: de **revolución** (un eslabón gira respecto al otro) o **prismáticas** (uno se desliza sobre el otro). Las dos son de un solo eje; las esféricas, cilíndricas y planas son multieje.

Para agarrar, el robot usa **pinzas**, y aquí hay un compromiso claro:

- La **pinza de mordaza paralela** —dos dedos, un actuador— es «amada y odiada por su simplicidad»: hace poco, pero nunca falla y se programa en un minuto.
- Las de **tres dedos** ganan algo de flexibilidad sin complicarse.
- En el otro extremo, una **mano antropomórfica** como la Shadow Dexterous Hand tiene **20 actuadores**. Permite manipulación en la mano (coger el móvil y girarlo para orientarlo), pero **aprender a controlarla es mucho más difícil**.

Más grados de libertad no es mejor

Cada actuador que añades multiplica lo que el robot **puede** hacer y también lo que hay que **decidir** en cada instante. Veinte actuadores es un espacio de acción enorme. Casi toda la robótica industrial funciona con dos dedos y un actuador, porque la tarea no necesita más.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

4.5 Aplicaciones, con datos (IFR *World Robotics* 2025)

Dato	Valor (2024)
Robots industriales instalados en el año	**542.000** (cuarto año por encima de 500.000)
Stock operativo mundial	**4,66 millones** (+9 %)
Densidad robótica media	~132-151 robots por 10.000 empleados
País líder en densidad	**Corea del Sur** (1.012-1.220)
Líder de mercado	**China** (295.000 unidades, 54 % de las instalaciones)
España, 3.er mercado europeo	**5.100 unidades**, impulsado por la automoción
Robótica médica	**+91 %** (~16.700 unidades), con el sistema **da Vinci** (cirugía por telemanipulación) como referencia del sector
Robots de servicio profesional	~200.000 unidades (+9 %)

Nota sobre la tabla anterior.

> **¿Por qué estos datos?**
>
> La **IFR (International Federation of Robotics)** publica cada año el informe *World Robotics* con las estadísticas oficiales del sector. Es la referencia para argumentar la importancia económica de la robótica con cifras y no con impresiones.

4.6 Cuatro casos reales

Fabricante o robot	Características	Uso real
KUKA KR AGILUS	6-11 kg de carga, 726-1.101 mm de alcance	Envasa 60 sándwiches por minuto; procesa 3.000 muestras de sangre al día
FANUC (familia)	Más de 100 modelos; cargas de hasta **2,3 t**, alcance de 4,7 m	Paletizado (M-410, 700 kg), soldadura, manipulación pesada
Universal Robots e-Series	UR3e (3 kg / 500 mm) … UR16e (16 kg / 900 mm); repetibilidad **±0,03-0,05 mm**	Montaje asistido, inspección, carga y descarga de máquinas
Amazon Robotics	Más de **1 millón** de robots de almacén	Logística: la flota reduce el tiempo de desplazamiento de los operarios

Nota sobre la tabla anterior.

> **Pensar en «robot + tarea», no en «robot»**
>
> No se elige el robot más caro: se define primero **la tarea** —qué pieza, qué peso, qué ritmo, qué precisión, qué entorno— y después se busca el modelo que cumple payload, alcance, repetibilidad y seguridad. Es el enfoque del §12 y de `N11`.

5. Qué clase de problema resuelve la robótica

Antes de elegir algoritmos conviene ver **qué tipo de problema** es. La robótica reúne casi todas las dificultades del curso a la vez: es **no determinista**, **parcialmente observable** y **multiagente**.

Y lo multiagente no es solo competitivo o solo cooperativo, sino los dos: en un pasillo estrecho por el que solo cabe uno, el robot y la persona **colaboran** —ninguno quiere chocar— y a la vez **compiten** un poco por pasar primero. Un robot demasiado educado, que siempre cede, se queda atascado para siempre en un sitio concurrido y nunca llega.

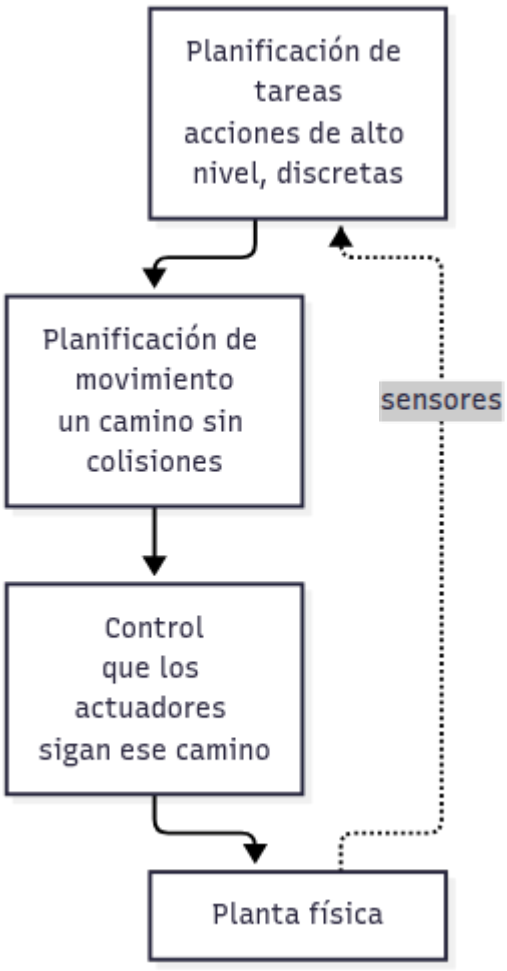
© 2025 David Martínez licensed under CC BY-NC-SA 4.0

¿De quién es la recompensa?

En casi toda la robótica, el robot actúa **al servicio de una persona**: si lleva la comida a un paciente, la recompensa es del paciente, no suya. Y ahí está el problema: **la función de recompensa verdadera está en la cabeza del usuario**. El robot tiene que deducir lo que la persona quiere, o conformarse con la aproximación que le programó un ingeniero. Es el mismo problema de alineación que se trata en la UD06, aquí con un brazo de una tonelada.

5.1 La jerarquía de tres niveles

En crudo, las observaciones de un robot son señales de sensores (píxeles, impactos de láser) y sus acciones son corrientes eléctricas enviadas a los motores. Entre eso y un plan como «lleva la comida a la habitación 12» hay un abismo. La robótica lo salva **partiendo el problema**:



- **Planificación de tareas**: decide las submetas —ir a la puerta, abrirla, ir al ascensor, pulsar el botón—. Trabaja con estados y acciones **discretos**.
- **Planificación de movimiento**: encuentra un camino que lleva al robot de un punto a otro cumpliendo cada submeta (§7).
- **Control**: consigue que los actuadores ejecuten ese movimiento (§6.4).

Lo que se pierde al partir el problema

Dividir reduce la complejidad, pero **renuncia a que las partes se ayuden**. La acción puede mejorar la percepción (moverse para ver mejor) y decidir qué percepción hace falta. Un movimiento óptimo sobre el papel puede ser malo de seguir para el controlador. Y un plan de tareas puede ser **imposible de instanciar** a nivel de movimiento. Por eso la robótica actual avanza en cada nivel y, al mismo tiempo, en **volver a integrarlos**: planificar movimiento y control juntos, tareas y movimiento juntos, y cerrar el lazo con la percepción.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

6. Modelado y control cinemático (RA4-a)

6.1 Conceptos básicos

Un manipulador se modela como una **cadena cinemática**: eslabones rígidos unidos por articulaciones.

Concepto	Definición
Grado de libertad (DoF)	Número de movimientos independientes. Hacen falta **≥ 6** para colocar el efector en cualquier posición **y** orientación en 3D
Articulación de revolución (R)	Gira: su variable es un ángulo θ
Articulación prismática (P)	Se desliza: su variable es una distancia d
Eslabón	Pieza rígida entre dos articulaciones
Espacio articular	El vector `q` con la posición de cada articulación
Espacio cartesiano	La **pose** del efector: posición más orientación

Configuración	Articulaciones	Uso típico
Articulado / antropomórfico	≥ 3R	El más común en industria (6 ejes)
Cartesiano / pórtico	3P	Manipulación simple, gran alcance
SCARA	RRP	Montaje de componentes: rígido en Z, flexible en XY
Delta / paralelo	Paralelas	Empaquetado de alta velocidad

6.2 Cinemática directa (FK)

La **cinemática directa** calcula la pose del efector a partir de los valores articulares: `pose = f(q)`. Se resuelve multiplicando **matrices de transformación homogénea** de 4×4, que combinan rotación y traslación:

$$^0T_n = {}^0T_1 \cdot {}^1T_2 \cdot \ldots \cdot {}^{n-1}T_n$$

Cada transformación se describe con los **parámetros de Denavit-Hartenberg (DH)**: cuatro valores por articulación (θ, d, a, α). Se escribe la tabla DH del robot y la pose sale de encadenar las matrices.

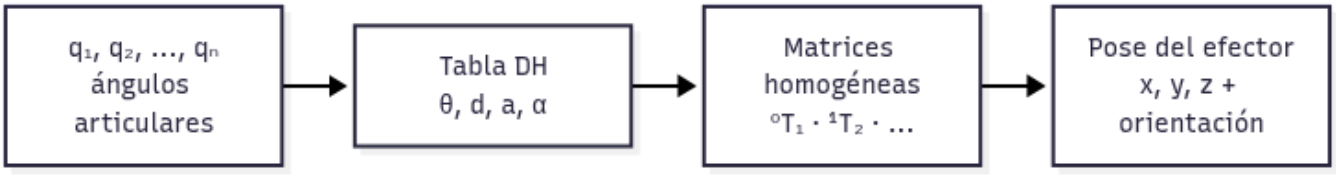


Tabla DH del Puma 560 (fragmento)

El Puma 560, brazo industrial clásico, se describe con una tabla como esta (metros y grados):

Articulación	θ (variable)	d	a	α
1	q_1	0,6718	0	−90°
2	q_2	0	0,4318	0°
3	q_3	0,1500	0,0203	90°
4	q_4	0,4318	0	−90°

Con esos valores, `fkine([0, 0.2, 0.3, 0.4, 0.5, 0.6])` devuelve una matriz 4×4 con la pose del efector: una posición concreta del TCP, del orden de `[0,25; −0,13; 1,15]`. Se comprueba en el notebook de la unidad.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Cinemática directa de un brazo plano 3R, a mano

Para un brazo en el plano con eslabones l_1, l_2, l_3:

\[x = l_1\cos\theta_1 + l_2\cos(\theta_1{+}\theta_2) + l_3\cos(\theta_1{+}\theta_2{+}\theta_3)$$ $$y = l_1\sin\theta_1 + l_2\sin(\theta_1{+}\theta_2) + l_3\sin(\theta_1{+}\theta_2{+}\theta_3)$$ $$\varphi = \theta_1 + \theta_2 + \theta_3\]

Con $l_1 = l_2 = l_3 = 1$ y los tres ángulos a 0, el efector está en **(3, 0)** con orientación 0. Poniendo $\theta_1 = 90°$, pasa a **(0, 3)**. El modelo geométrico predice exactamente dónde está la punta del brazo: eso es la cinemática directa, y **siempre tiene una única solución**.

6.3 Cinemática inversa (IK)

La **cinemática inversa** va al revés: dada una pose deseada, calcula los ángulos articulares que la consiguen, $q = f^{-1}(pose)$. Es **mucho más difícil**, y ahí están casi todos los problemas que pide recopilar el criterio RA4-a:

Problema	En qué consiste	Cómo se aborda
Múltiples soluciones	Un 6R general puede tener **hasta 16 soluciones**; con muñeca esférica, 8	Elegir por criterio: evitar obstáculos, minimizar el recorrido, codo arriba o abajo
Redundancia	Más DoF de los necesarios → **infinitas** soluciones	Optimizar en el **espacio nulo** del jacobiano
Sin solución	El objetivo está fuera del alcance	Detectarlo y avisar, no iterar sin fin
Singularidad	El jacobiano pierde rango → la velocidad articular tiende a infinito	Evitarla al planificar, o cruzarla bajando la velocidad cartesiana

Los métodos se dividen en **analíticos** —desacoplamiento cinemático en muñecas esféricas, teorema de Pieper— y **numéricos** — Newton-Raphson, pseudoinversa del jacobiano, CCD, FABRIK—.

6.4 Control

Estrategia	Qué controla	Cómo
Control de posición	El ángulo de cada articulación	PID por eje: la medida es el encoder, la consigna el ángulo objetivo
Control de velocidad	La velocidad del efector (*resolved-rate*)	\(\dot{q} = J^{+} v\), con la pseudoinversa de Moore-Penrose del jacobiano
Control de fuerza / impedancia	La interacción con el entorno	El robot se comporta como un sistema masa-muelle-amortiguador: ensamblaje, pulido

Los controladores de seguimiento van de menos a más:

- **P** — aplica una acción proporcional al error de posición. Simple, pero oscila.
- **PD** — añade un término proporcional a la derivada del error, que **amortigua** la oscilación.
- **PID** — añade la integral del error, que corrige los **errores sistemáticos persistentes**.
- **Par calculado** — usa la **dinámica inversa** del robot para calcular el par que hace falta, y deja al PID solo la corrección de lo que el modelo no acierta. Es lo que usan los robots industriales de verdad.

Los robots industriales trabajan con **servomotores con encoder** en lazo cerrado, no con motores paso a paso en lazo abierto: sin realimentación no hay forma de saber si el eje llegó.

De dónde salen las trayectorias

No se salta de un punto a otro: se **interpola**. `jtraj` genera la trayectoria en el espacio articular con un polinomio de 5.º grado (aceleración y *jerk* continuos, movimiento suave); `ctraj` genera una **línea recta cartesiana**, que es lo que hace falta cuando el efector lleva una herramienta de corte. El controlador va siguiendo esa trayectoria punto a punto.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

7. Los problemas y sus soluciones (RA4-b)

7.1 Singularidades cinemáticas

Una **singularidad** es una configuración en la que el jacobiano pierde rango: el robot **pierde capacidad de movimiento** en alguna dirección y los cálculos de velocidad se van a infinito.

Singularidad	Cuándo ocurre
De muñeca	Los ejes 4 y 6 se alinean ($\theta_5 = 0$): las dos articulaciones deberían girar a velocidad infinita
De hombro	El centro de la muñeca corta el eje de la base
De codo	El brazo queda completamente estirado o plegado
De límite	El efector llega al borde de su volumen de trabajo

Nota sobre la tabla anterior.

> **Qué pasa de verdad si el robot entra en una singularidad**
>
> No es un problema abstracto de álgebra. Las velocidades articulares necesarias **tienden a infinito**, así que lo que se ve en la célula es **sobrecorriente en los servos, vibraciones o una parada de seguridad** en medio del ciclo. Por eso la planificación de trayectorias evita esas configuraciones, o las cruza reduciendo la velocidad cartesiana.

7.2 Soluciones

Problema	Solución
Singularidades	Planificar evitándolas; medir la **manipulabilidad** (índice de Yoshikawa) para saber cuánto margen queda
IK compleja	**Desacoplamiento cinemático**: una muñeca esférica separa el problema de posición del de orientación
Múltiples soluciones	Criterios de optimización: evitar obstáculos, minimizar recorrido
Límites articulares	Modelarlos y restringir el *solver*
Precisión deficiente	**Calibrar** el robot, que puede mejorar la precisión hasta ×10, y medir la repetibilidad con la ISO 9283

Nota sobre la tabla anterior.

> **Precisión no es repetibilidad**
>
> Un robot puede ser **repetible** y a la vez **impreciso**: vuelve siempre al mismo punto, pero ese punto no es el que se programó. Para trabajar con *teach pendant* basta la repetibilidad — el punto se enseñó ahí—, pero en **programación offline**, donde las coordenadas vienen de un CAD, la precisión es crítica y hay que calibrar (por ejemplo, un ABB IRB 1600 con láser *tracker*). La repetibilidad de los cobots UR e-Series es de **±0,03-0,05 mm**.

7.3 El espacio de configuración

Para planificar el movimiento hay que cambiar de punto de vista. En vez de pensar en el robot moviéndose por una habitación, se piensa en un **punto moviéndose por el espacio de configuración**: el espacio abstracto de **todas las configuraciones posibles** del robot.

Para un brazo de 6 ejes, ese espacio tiene 6 dimensiones —una por articulación—; para un robot móvil en un plano, 3 (x, y, orientación). Los obstáculos del mundo real se convierten en **regiones prohibidas** de ese espacio, y lo que queda es el **espacio libre**. El problema deja de ser geométrico y se vuelve una búsqueda de camino.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

El problema de la mudanza del piano

La planificación de movimiento se llama a veces así, por el esfuerzo de una empresa de mudanzas para llevar un piano grande y de forma irregular de una habitación a otra sin golpear nada. Formalmente hacen falta: un espacio de trabajo `W`, una región de obstáculos `O ⊂ W`, un robot con su espacio de configuración `C`, una configuración inicial `q_s` y una objetivo `q_g`. Y lo que se busca es un **camino continuo** por el espacio libre entre las dos.

El problema se complica de tres formas típicas: que el objetivo sea un **conjunto** de configuraciones válidas en vez de una sola; que haya que **minimizar un coste** (longitud del camino, energía); o que haya **restricciones** — si el robot lleva una taza de café, tiene que mantenerla vertical todo el trayecto.

7.4 Cómo se planifica el movimiento

Método	Idea	Fuerte en	Flojo en
Grafo de visibilidad	Nodos en los vértices de los obstáculos, aristas donde hay línea de visión clara; después A* o Dijkstra	Da el camino **más corto** en 2D con obstáculos poligonales	Escala mal con muchos obstáculos; el camino **roza** las esquinas
Diagrama de Voronoi	Divide el espacio en regiones por cercanía a cada obstáculo; el robot circula por los **bordes** entre regiones	Maximiza la distancia a los obstáculos: caminos **seguros**	Caminos más largos de lo necesario
Descomposición celular	Trocea el espacio libre en celdas y busca sobre el grafo de adyacencia	Sencillo y completo en su resolución	El número de celdas explota al subir de dimensión
Muestreo (RRT, PRM)	Va lanzando configuraciones al azar y conectando las válidas hasta unir inicio y meta	Es lo único práctico con **muchas dimensiones** (brazos de 6-7 ejes)	No garantiza el camino óptimo; el resultado es irregular y hay que suavizarlo

Los dos primeros ilustran bien un compromiso que reaparece siempre: el grafo de visibilidad busca el camino **más corto**, que pasa pegado a las esquinas; el diagrama de Voronoi busca el **más seguro**, que se aleja de todo y por eso es más largo. Cuál conviene depende de si el riesgo es chocar o es tardar.

Cualquiera de los cuatro devuelve un camino que suele necesitar un último paso de **suavizado** o de **replanificación** cuando el entorno cambia.

7.5 Seguimiento de trayectoria: del plan a la política

Aquí hay una distinción que conviene tener clara, porque explica cómo está montada la robótica de verdad.

Un **plan** dice qué camino recorrer. Una **política** dice qué acción tomar **desde cualquier estado** en el que el robot se encuentre. Las leyes de control del §6.4 son políticas, no planes.

Lo que se hace en la práctica es una **componenda en dos pasos**:

1. Se **planifica** en un espacio simplificado — solo el estado cinemático, suponiendo que se puede pasar de una configuración a la vecina sin preocuparse de la dinámica. Sale la **trayectoria de referencia**.
2. Se **convierte en política**: un controlador que intenta seguir ese plan y **vuelve a él** cuando se desvía.

Esa componenda introduce dos suboptimalidades

La primera, **planificar sin tener en cuenta la dinámica**: el camino más corto en el papel puede ser el que peor sigue el robot real. La segunda, **suponer que si te desvías lo mejor es volver al plan original**, cuando a veces lo óptimo es replanificar desde donde estás. El **control óptimo** ataca las dos a la vez: optimiza directamente sobre las acciones teniendo en cuenta la dinámica, con técnicas de optimización de trayectoria (*multiple shooting*, colocación directa) y, cuando el coste es cuadrático y la dinámica lineal, con el **LQR** —y su variante **iLQR**, que es lo que se usa cuando no se cumplen esas condiciones ideales, que es casi siempre.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

8. Percepción robótica (RA4-b)

Percepción es convertir medidas de sensores en una representación interna del entorno. Y el problema de fondo es el del §4.3: **todos los sensores tienen ruido y ninguno lo ve todo**.

8.1 Localización y mapeo

Hay tres problemas, de menos a más difícil:

Problema	Lo que se sabe	Lo que se busca
Localización	El mapa	Dónde está el robot
Mapeo	Dónde está el robot	El mapa
SLAM	Nada de las dos	**Las dos a la vez**

SLAM (*Simultaneous Localization And Mapping*) es el caso realista y parece un imposible —para saber dónde estás necesitas el mapa, y para hacer el mapa necesitas saber dónde estás—, pero se resuelve **probabilísticamente**: en vez de una respuesta, el robot mantiene una **distribución de probabilidad** sobre dónde puede estar (su *estado de creencia*) y la va afinando con cada medida.

Las herramientas son las mismas que aparecen en toda la IA con incertidumbre: **filtros de Kalman**, **modelos ocultos de Markov** y **redes bayesianas**, que modelan la transición de estado y el sensor.

Localización de Monte Carlo, en tres fotos

La localización con **filtro de partículas** (MCL) representa la creencia con una nube de «partículas», cada una una hipótesis de dónde está el robot:

1. Al arrancar, las partículas están **repartidas por todo el plano**: incertidumbre total.
2. Llega el primer conjunto de medidas y las partículas se **agrupan** en las zonas compatibles. Suelen quedar **varios grupos**: los pasillos de un edificio de oficinas se parecen entre sí, y el robot todavía no puede distinguirlos.
3. Con suficientes medidas, todas las partículas **colapsan en un solo sitio**.

Lo interesante es el paso 2: el robot representa explícitamente que hay **varias respuestas posibles** en vez de escoger una y equivocarse. Solo hacen falta dos modelos: el del movimiento y el del sensor.

8.2 Qué representación conviene

La tendencia en robótica es hacia representaciones con **semántica bien definida** — no solo «aquí hay algo», sino «esto es una puerta». Y las técnicas probabilísticas **ganan** a las alternativas en los problemas difíciles de percepción, como el propio SLAM.

Pero no siempre hace falta la artillería

Las técnicas estadísticas son a veces **demasiado engorrosas** para lo que se necesita, y una solución más simple es igual de efectiva en la práctica. Es el mismo criterio de la UD05 con el PID: empezar simple y añadir complejidad solo cuando se justifica. Los `N06` y `N07` de esta unidad navegan con reglas y con lógica difusa sobre los píxeles de una cámara, sin ningún filtro probabilístico, y funcionan.

8.3 Percepción que se adapta

Hay un tipo de aprendizaje que resuelve un problema muy concreto: que las medidas de los sensores cambian **de golpe** cuando cambia el entorno.

Piensa en entrar desde un exterior soleado a una habitación con fluorescentes. No solo hay menos luz: el fluorescente tiene **más componente verde** que el sol, así que todos los colores cambian. Y sin embargo no nos parece que a la gente se le haya puesto la cara verde. Nuestra percepción se **readapta** en segundos y el cerebro ignora la diferencia. Un robot que no haga eso creerá que ha cambiado de mundo.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

9. Incertidumbre y aprendizaje (RA4-b)

9.1 Planificar cuando no se sabe el estado exacto

La incertidumbre viene de tres sitios: el entorno es **parcialmente observable**, los efectos de las acciones son **estocásticos** (o simplemente no están modelados) y los propios algoritmos de estimación son **aproximados** — un filtro de partículas no da el estado de creencia exacto ni con un modelo perfecto.

La mayoría de los robots en producción usan, aun así, **algoritmos deterministas**, y se apañan con dos atajos:

1. **Discretizar** el espacio de estados continuo (grafo de visibilidad, descomposición celular).
2. Ante la duda sobre el estado actual, **quedarse con el más probable** de la distribución.

Es más rápido y escala mejor. El precio es que el robot actúa como si estuviera seguro cuando no lo está.

> **Cuándo ese atajo se rompe**
>
> Quedarse con el estado más probable funciona mientras la distribución tenga **un solo pico claro**. Falla justo en los casos del §8.1, paso 2: cuando hay **varias hipótesis igual de buenas** —tres pasillos que se parecen—, elegir la más probable es tirar una moneda, y el robot actúa con plena confianza sobre una respuesta que tiene un 33 % de ser cierta.

9.2 Aprendizaje por refuerzo en robótica

El aprendizaje por refuerzo funciona muy bien en simulación y muy mal en un robot real, y la razón es sencilla: **el mundo real se niega a ir más rápido que el tiempo real**.

	En simulación	En un robot real
Velocidad	Millones de pruebas en unas horas	Las mismas pruebas tardarían **años**
Riesgo	Ninguno	El robot **no puede arriesgarse** a una prueba que lo dañe — y por tanto **no puede aprender de ella**

De ahí que el problema central sea la **transferencia de la simulación a la realidad** (*sim-to-real*), que es un área de investigación activa. Y de ahí también que los sistemas robóticos prácticos incorporen **conocimiento previo** sobre el robot, el entorno y la tarea: es la única forma de aprender rápido y de comportarse con seguridad mientras aprende.

> **Esto conecta con toda la unidad**
>
> `N08`, `N09` y `N10` hacen exactamente este recorrido en pequeño y **en simulación**, que es donde se puede: generar datos conduciendo el robot, entrenar una red con ellos, y después dejar que la solución **evolucione** sin datos etiquetados.

10. Técnicas de programación de robots (RA4-c)

10.1 Las cinco formas de programar un robot industrial

Técnica	Cómo funciona	Ventajas	Desventajas	Cuándo
Teach pendant	Guías el brazo con la consola por los puntos de paso y los grabas	Curva de aprendizaje mínima, puesta a punto rápida	**Para la producción** mientras se programa; poca flexibilidad	Trayectorias básicas, paletizado, soldadura por puntos
Guiado manual	Mueves el efector con la mano y registras posiciones	Muy intuitivo, sin escribir código	Solo en cobots seguros	Cobots, montaje asistido
Programación textual	Código nativo del fabricante: RAPID, KRL, URScript	Determinista, se integra con PLC y seguridad	Sintaxis propietaria, **no portable**	Células industriales estables de alta cadencia

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Técnica	Cómo funciona	Ventajas	Desventajas	Cuándo
Programación offline (OLP)	Se programa en simulación sobre modelos CAD	**Cero impacto en producción**; permite probar hipótesis	Coste de licencias; **exige modelos precisos y robot calibrado**	Geometrías complejas, células multimarca
ROS 2 y frameworks	Middleware de nodos, *topics* y servicios	Estándar abierto, acceso directo a algoritmos de IA	Curva de aprendizaje alta	Investigación, robots móviles, multi-robot

Nota sobre la tabla anterior.

> **URScript se parece a Python**
>
> Los robots de **Universal Robots** se programan en **URScript**, un lenguaje muy parecido a Python que además puede enviarse al robot desde un cliente Python por FTP, SSH o RTDE. Es la puerta de entrada natural a la programación de cobots desde este módulo.

10.2 Robots colaborativos

Los **cobots** comparten espacio con las personas sin vallado gracias a la **limitación de potencia y fuerza**. Ejemplos: Universal Robots (UR3e a UR16e, de 3 a 16 kg), Franka Emika (orientada a la academia) y KUKA LBR.

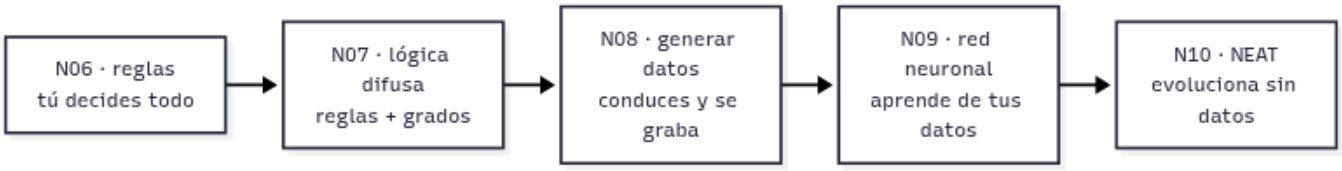
> **«Colaborativo» no es una propiedad del hardware**
>
> La **ISO 10218:2025** prohíbe llamar «robot colaborativo» a un brazo aislado. Lo que puede ser colaborativa es la **aplicación completa**: robot + herramienta + entorno + tarea. Un cobot con un cuchillo en la pinza no es una aplicación colaborativa. Es un matiz legal, y es el que decide si hace falta vallado.

10.3 Comparar técnicas de verdad: el mismo problema, cuatro veces

La tabla del §10.1 compara técnicas **industriales**. Pero el criterio RA4-c pide valorar características diferenciadoras, y eso se aprende mejor resolviendo **un mismo problema de varias maneras** y viendo qué cambia.

Los entregables de esta unidad hacen precisamente eso: un robot con una cámara tiene que **seguir una línea** en el suelo, y el problema se resuelve cuatro veces.

Entregable	Técnica	Qué escribes tú	Qué sale del programa
`N06`	**Reglas** sobre los píxeles	Todas las condiciones, a mano	Nada: el comportamiento es el que programaste
`N07`	**Lógica difusa**	Las variables lingüísticas y las reglas	La transición suave entre ellas
`N09`	**Red neuronal supervisada**	La arquitectura y el entrenamiento; los datos salen de `N08`	El comportamiento, aprendido de tus propios ejemplos
`N10`	**Neuroevolución (NEAT)**	La función de aptitud	La red **y su topología**, evolucionadas sin ejemplos



© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Lo que hay que observar al compararlas

No es cuál «funciona mejor», sino **qué se gana y qué se pierde** en cada salto: cuánto código escribes, cuánto tienes que entender del problema, cuántos datos necesitas, cuánto tarda en estar listo y —lo más importante— **si puedes explicar por qué el robot hizo lo que hizo**. Las reglas de `N06` se leen; los pesos de la red de `N09`, no. Es la misma tensión entre interpretabilidad y potencia que viste en la UD05 con los sistemas expertos, y que la UD06 convierte en un problema legal.

11. Humanos y robots (RA4-c, RA4-d)

Cuando el robot comparte espacio con personas aparece un problema que no es de mecánica ni de control: hay que **coordinarse con un agente que tiene sus propios objetivos** y no se deja modelar como un obstáculo móvil.

11.1 Coordinación

La forma habitual de plantearlo es como un **juego** entre el robot y la persona. Eso supone explícitamente que las personas son **agentes con objetivos**, no obstáculos que se mueven.

Y ahí está la trampa: suponerlo no significa que la persona sea **perfectamente racional**. El juego es difícil de resolver para el robot, pero también **para nosotros**: obliga a pensar en lo que hará el robot en respuesta a lo que hace la persona, que depende de lo que el robot cree que hará la persona, que depende de… y se entra en un «¿qué crees que creo que crees?» sin fondo. Las personas no gestionamos eso, así que nos comportamos de forma **subóptima** de maneras bastante predecibles. Un robot que asuma racionalidad perfecta predecirá mal.

La solución práctica se descompone en dos:

1. **Predicción**: usar lo que la persona está haciendo ahora para estimar qué hará después.
2. **Acción**: usar esa predicción para calcular el movimiento del robot.

11.2 Aprender lo que la persona quiere

Vuelve el problema del §5: la recompensa verdadera está en el usuario. Un robot que ayuda no puede esperar a que alguien le escriba la función objetivo perfecta, porque **el propio usuario no sabría escribirla**. Tiene que inferirla observando: qué corrige la persona, qué acepta, qué repite.

Y esto no es un detalle técnico

Un robot que optimiza una aproximación mala del objetivo real hace **exactamente lo que se le pidió** y no lo que se quería. Con un brazo de una tonelada o un vehículo de dos toneladas, eso deja de ser una curiosidad y se convierte en el problema de seguridad que se trata a fondo en la **UD06** (*specification gaming*, el problema del rey Midas). Aquí basta con retenerlo: **especificar mal el objetivo es un modo de fallo, igual que una singularidad.**

12. Diseño e implementación de sistemas robotizados (RA4-d)

12.1 Criterios de selección

Criterio	Pregunta guía	Dato típico
Carga útil (payload)	¿Qué masa mueve el efector, contando la herramienta?	UR3e 3 kg … UR16e 16 kg; FANUC hasta 2,3 t
Alcance	¿Qué distancia máxima hay que alcanzar?	KUKA KR AGILUS 726-1.101 mm; FANUC 4,7 m
Repetibilidad	¿Cuánta dispersión al volver al mismo punto?	UR e-Series ±0,03-0,05 mm
Precisión	¿El punto alcanzado coincide con el programado?	Mejora al calibrar
Entorno	¿Temperatura, polvo, humedad, atmósfera explosiva?	Versiones IP / ATEX

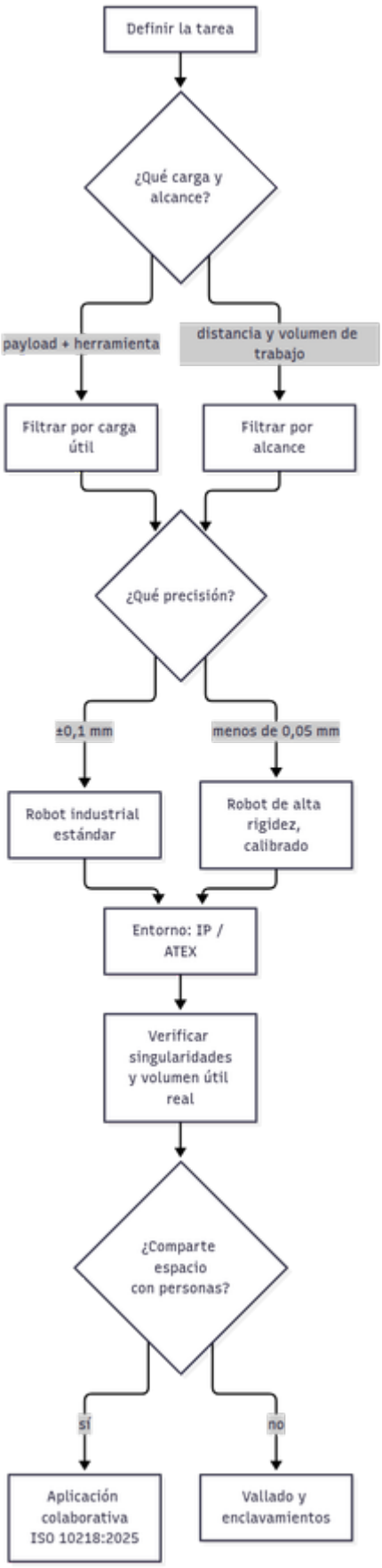
Nota sobre la tabla anterior.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

El payload no es el peso de la pieza

Es el peso de la pieza **más** la herramienta (EOAT), la brida, los cables y los sensores acoplados. Y además hay que comprobar que los **momentos de inercia** de la herramienta no superan los límites de las reductoras de la muñeca: un robot puede aguantar 10 kg pegados a la brida y no aguantar 6 kg en el extremo de una herramienta larga.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



© 2025 David Martínez licensed under CC BY-NC-SA 4.0

12.2 Sensores de la célula

- **Propioceptivos**: encoders, corriente, temperatura de articulación.
- **Exteroceptivos**: visión 2D/3D, fuerza/par, proximidad, láser.

Los de **fuerza/par** son imprescindibles en tareas de contacto con control de impedancia. La **visión** es lo que permite el *pick and place* sin posiciones fijas —piezas en cualquier orientación— y es la puerta de entrada de la IA a la célula: es justo lo que se practica en el notebook de OpenCV y en `N05`.

12.3 Ejemplo guiado: elegir el robot y evitar la singularidad

Recorremos el razonamiento que repetirás en `N11`.

Problema: una línea de ensamblaje tiene que coger una pieza de 1,5 kg de un transportador y colocarla en una estación a 700 mm, con repetibilidad de ±0,1 mm.

Paso 1 · Payload. Pieza (1,5 kg) + pinza (0,5 kg) + cables ≈ **2,2 kg**. Hace falta payload ≥ 2,5 kg y alcance ≥ 700 mm. Un **UR5e** (5 kg, 850 mm) o un KUKA KR AGILUS de 6 kg cumplen.

Paso 2 · Repetibilidad. La tarea tolera ±0,1 mm y los dos modelos van sobrados (UR e-Series ±0,05 mm). No hace falta calibración de precisión porque los puntos se enseñarán, no vendrán de un CAD.

Paso 3 · Volumen de trabajo y singularidades. Se simula la célula y se comprueba que la trayectoria no pasa por codo estirado ni muñeca alineada. Si el *layout* obligara a cruzar una singularidad, **se cambia el layout**, no el controlador: es más barato mover el transportador que pelearse con el robot.

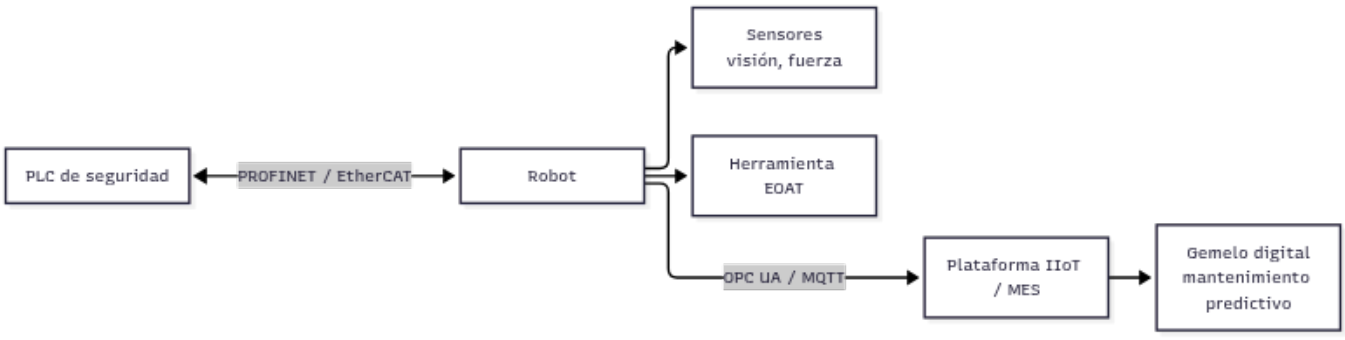
Paso 4 · Seguridad. Si el brazo comparte espacio con personas → **aplicación colaborativa** (ISO 10218:2025), con limitación de fuerza y evaluación biomecánica de la tarea completa. Si es una célula aislada de alta cadencia → vallado con puertas enclavadas.

> **Conclusión del ejemplo guiado**
>
> Elegir un robot es **emparejar la tarea con el modelo** —payload, alcance, repetibilidad, entorno, seguridad— y **verificarlo en simulación** antes de comprar nada. Ese es el criterio RA4-d, y el orden importa: primero la tarea, después el robot.

12.4 El ciclo de vida

1. **Requisitos**: qué tarea, qué cadencia, qué entorno.
2. **Selección** del robot y las herramientas.
3. **Simulación y diseño de la célula**: *layout*, obstáculos, trayectorias.
4. **Integración**: robot + PLC + sensores + comunicaciones.
5. **Puesta en marcha**: programación, pruebas de ciclo, validación de seguridad.
6. **Mantenimiento**: predictivo, con gemelos digitales y telemetría.

12.5 La célula y la Industria 4.0



- El **PLC** coordina la célula con buses deterministas (PROFINET, EtherCAT, EtherNet/IP).
- La telemetría se publica con **OPC UA** o **MQTT** para supervisión remota.
- Con esos datos, el **gemelo digital** y el mantenimiento predictivo anticipan fallos —fatiga de reductoras, por ejemplo— antes de que paren la línea.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

12.6 Seguridad y normativa

Norma	Qué regula
ISO 12100	Evaluación de riesgos de máquinas: la base de todo
ISO 10218-1 / -2	Robots y sistemas robóticos industriales: requisitos de seguridad
ISO/TS 15066	Cobots: límites biomecánicos de fuerza y velocidad
ISO 10218:2025	**Versión vigente**: absorbe la ISO/TS 15066 y obliga a evaluar la aplicación colaborativa completa
ISO 9283	Cómo se mide la repetibilidad y la precisión de un robot

Nota sobre la tabla anterior.

> ⚠ **Lo que cambió en 2025**
>
> La **ISO 10218:2025** absorbe los límites biomecánicos de la ISO/TS 15066 y los vuelve **obligatorios**, y añade **auditorías de ciberseguridad industrial** de la célula. Un robot conectado a la red de planta es una superficie de ataque: aquí la seguridad se diseña desde el principio, que es el *security by design* de la UD06 aplicado a una máquina que se mueve.

13. Practicar sin robot: los dos simuladores de la unidad

13.1 `roboticstoolbox-python`, para la cinemática

Para el modelado de manipuladores (§6) usamos **roboticstoolbox-python**, de Peter Corke, que trae más de 50 modelos de robots reales (Panda, UR, Puma 560) con su cinemática resuelta.

```
pip install roboticstoolbox-python spatialmath-python
```

```
import roboticstoolbox as rtb

robot = rtb.models.Panda()
print(robot)

# Cinematica directa: de los angulos a la pose
pose = robot.fkine([0, -0.8, 0.8, 0, 0.8, 0, 0])
print(pose)

# Cinematica inversa por Levenberg-Marquardt
q = robot.ikine_LM(pose)
print(q)
```

13.2 AITK, para el robot móvil

Las prácticas de navegación (§10.3) usan **AITK** (`aitk.robots`), un simulador ligero de robots móviles con cámara que funciona dentro del propio notebook: se define un mundo con una imagen de fondo, se coloca un robot con sus sensores y se le pasa una función de control.

```
pip install aitk aitk.robots
```

```
import aitk.robots as bots

world = bots.World(220, 180, boundary_wall_color="yellow",
                   ground_image_filename="EX2_pista_6.png")
robot = bots.Scribbler(x=100, y=90, a=90)
robot.add_device(bots.Camera(64, 32))
world.add_robot(robot)

world.reset()
world.seconds(30, [mi_controlador], real_time=True)
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Alternativas de simulación 3D

- **Webots**: entorno 3D completo, gratuito, ideal para aula sin hardware.
- **CoppeliaSim Edu**: gratuito para centros educativos, muy usado en investigación.

En este módulo trabajamos **en Python** —rápido y sin instalar simuladores 3D—; si el aula dispone de Webots o CoppeliaSim, se decide con el profesor.

14. Puntos clave de la unidad

- Un robot es un **agente encarnado**: el único sistema de IA del curso que cambia el estado del mundo físico, en un entorno **parcialmente observable, estocástico y multiagente**.
- En 2024 se instalaron **542.000 robots industriales** y España fue el **3.er mercado europeo** (IFR *World Robotics* 2025).
- Los sensores se clasifican por **cómo miden** (pasivos frente a activos) y por **qué miden** (entorno, ubicación, configuración interna). La **odometría se degrada sin límite** porque las ruedas patinan: hay que corregirla con referencias externas.
- Más grados de libertad no es mejor: una mano de **20 actuadores** puede más, pero **es mucho más difícil de controlar** que una pinza de dos dedos.
- La robótica parte el problema en **tarea → movimiento → control**. Reduce la complejidad, pero renuncia a que los niveles se ayuden entre sí.
- El movimiento de un manipulador se modela como **cadena cinemática**: articulaciones R o P, eslabones y **grados de libertad** (≥ 6 para pose libre en 3D).
- La **cinemática directa** (parámetros **DH**) tiene solución única; la **inversa** tiene hasta **16 soluciones**, redundancia y **singularidades** en las que la velocidad articular tiende a infinito.
- **Precisión no es repetibilidad**: un robot puede volver siempre al mismo punto equivocado. Con *teach pendant* basta la repetibilidad; con programación **offline**, hay que calibrar.
- Planificar el movimiento es buscar un camino en el **espacio de configuración**. El grafo de visibilidad da el camino **más corto**; Voronoi, el **más seguro**; el **muestreo (RRT, PRM)** es lo único viable con muchas dimensiones.
- Un **plan** dice qué camino seguir; una **política** dice qué hacer desde cualquier estado. La robótica real planifica en cinemática y luego **convierte el plan en política**, aceptando dos suboptimalidades; el **control óptimo** (LQR, iLQR) ataca las dos a la vez.
- **SLAM** resuelve localización y mapeo simultáneos manteniendo una **distribución de probabilidad** sobre dónde está el robot, no una respuesta única.
- El **aprendizaje por refuerzo** funciona en simulación y falla en el robot real: el mundo no va más rápido que el tiempo real y el robot **no puede arriesgarse a la prueba que lo dañaría**. De ahí el problema *sim-to-real*.
- Hay **cinco formas** de programar un robot industrial (*teach pendant*, guiado manual, textual, offline y ROS 2), y la unidad las compara en la práctica resolviendo **un mismo problema** con reglas, lógica difusa, red neuronal y neuroevolución.
- **«Colaborativo» no es una propiedad del hardware**: lo es de la aplicación completa (ISO 10218:2025).
- Diseñar un sistema robotizado es **emparejar la tarea con el modelo** —payload con herramienta, alcance, repetibilidad, entorno, seguridad— y **verificarlo en simulación** antes de comprar.

15. Glosario

Término	Definición
Robot	Máquina programable que percibe, procesa y actúa físicamente sobre el entorno
Efector	Pieza que ejerce fuerza sobre el entorno: rueda, pata, articulación, pinza
Manipulador	Brazo robótico, unido o no a un cuerpo
Cobot	Robot que comparte espacio con personas limitando potencia y fuerza
Sensor pasivo / activo	El pasivo capta lo que el entorno emite; el activo emite energía y mide el retorno
Sensor propioceptivo	Informa del estado del propio robot: encoder, giroscopio, fuerza/par
Odometría	Medida de la distancia recorrida contando revoluciones de rueda

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Término	Definición
Lidar	Telémetro óptico activo de barrido; ~1 cm de precisión a 100 m
Cámara de tiempo de vuelo	Cámara que mide distancia por el tiempo que tarda la luz en volver
Actuador	Mecanismo que produce el movimiento: eléctrico, hidráulico o neumático
EOAT	*End of arm tooling*: la herramienta del extremo del brazo
Grado de libertad (DoF)	Movimiento independiente de una articulación
Articulación de revolución / prismática	Gira (ángulo θ) / se desliza (distancia d)
Eslabón	Pieza rígida entre dos articulaciones
Espacio articular	Vector con la posición de cada articulación
Espacio cartesiano	Pose del efector: posición más orientación
Pose	Posición y orientación de un cuerpo en el espacio
Cinemática directa (FK)	De los ángulos articulares a la pose del efector
Cinemática inversa (IK)	De la pose deseada a los ángulos articulares
Parámetros DH	Denavit-Hartenberg: θ, d, a, α que describen cada transformación
Transformación homogénea	Matriz 4×4 que combina rotación y traslación
Jacobiano	Matriz que relaciona velocidades articulares y cartesianas
Singularidad	Configuración en la que el jacobiano pierde rango
Redundancia	Más grados de libertad de los necesarios: infinitas soluciones de IK
Manipulabilidad	Medida (Yoshikawa) de cuánto margen de movimiento queda en una configuración
Desacoplamiento cinemático	Separar la IK en posición y orientación con una muñeca esférica
Espacio de configuración	Espacio de todas las configuraciones posibles del robot
Espacio libre	La parte del espacio de configuración sin colisión
Grafo de visibilidad	Grafo de vértices de obstáculos unidos por líneas de visión libres
Diagrama de Voronoi	División del espacio por cercanía; sus bordes son caminos seguros
RRT / PRM	Planificadores por muestreo aleatorio, para muchas dimensiones
Plan / política	Un camino concreto / qué hacer desde cualquier estado
PID	Controlador proporcional-integral-derivativo
Par calculado	Control que usa la dinámica inversa y deja al PID solo el error residual
Resolved-rate	Control de velocidad del efector con la pseudoinversa del jacobiano
Control de impedancia	El robot se comporta como masa-muelle-amortiguador ante el contacto
LQR / iLQR	Regulador cuadrático lineal y su variante iterativa: control óptimo
SLAM	Localización y mapeo simultáneos
Filtro de partículas (MCL)	Localización que representa la creencia como una nube de hipótesis
Estado de creencia	Distribución de probabilidad sobre el estado real
Sim-to-real	Transferir a un robot real lo aprendido en simulación
Teach pendant	Consola manual para programar guiando el brazo por los puntos
OLP	Programación offline, en simulación sobre modelos CAD

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Término	Definición
URScript / RAPID / KRL	Lenguajes nativos de Universal Robots, ABB y KUKA
ROS 2	Middleware de robótica con nodos, *topics* y servicios
NEAT	Neuroevolución que evoluciona pesos **y topología** de la red
Payload	Carga útil máxima, contando herramienta, brida, cables y sensores
Repetibilidad	Dispersión al volver al mismo punto (ISO 9283)
Precisión	Diferencia entre el punto programado y el alcanzado
Célula robotizada	Unidad de producción completa: robot, herramientas, PLC y sensores
Gemelo digital	Réplica virtual de la máquina alimentada con datos reales
ISO 10218:2025	Norma vigente de seguridad de robots; absorbe la ISO/TS 15066
ISO/TS 15066	Límites biomecánicos de los cobots

16. FAQ

¿Un robot necesita inteligencia artificial?

No siempre. Muchísimos robots industriales ejecutan programas fijos y funcionan perfectamente. La IA aparece cuando el robot debe **percibir, adaptarse o aprender**: visión para localizar piezas sin posición fija, planificación en entornos que cambian, aprendizaje de una tarea. Si la pieza siempre llega en el mismo sitio, un programa fijo es la respuesta correcta.

¿Qué pasa exactamente si el robot entra en una singularidad?

Las velocidades articulares necesarias tienden a infinito, así que el servo pide una corriente que no puede dar: vibraciones, error de seguimiento o parada de seguridad. No «se rompe» sin más, pero el ciclo se cae. Por eso se evita al planificar o se cruza reduciendo la velocidad cartesiana.

¿Por qué un robot de 6 ejes puede llegar al mismo punto de varias formas?

Porque la cinemática inversa tiene **múltiples soluciones**: hasta 16 en un 6R general, 8 con muñeca esférica. Codo arriba o codo abajo llegan al mismo sitio. El controlador elige por criterio: evitar obstáculos, minimizar recorrido, no cambiar de configuración a mitad de trayectoria.

Si la cinemática directa es fácil, ¿por qué no resolver la inversa probando ángulos?

Porque el espacio a probar es continuo y de 6 dimensiones. Los métodos numéricos hacen algo más inteligente que probar: usan el jacobiano para saber **en qué dirección** mover cada articulación y converger. Pero pueden quedarse en un mínimo local, o no converger cerca de una singularidad.

¿Qué diferencia hay entre planificar y controlar?

Planificar es decidir **el camino** —una vez, antes de moverse—. Controlar es conseguir que el robot **siga ese camino** pese al roce, la inercia y las perturbaciones, corrigiendo miles de veces por segundo. El plan es un dibujo; el control es lo que lo convierte en movimiento.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

¿Por qué el robot no aprende directamente en el mundo real?

Porque el mundo real no va más rápido que el tiempo real: los millones de pruebas que en simulación son unas horas, en la realidad son años. Y porque **no puede permitirse la prueba que lo dañaría**, que es justo la que más enseñaría. De ahí el problema *sim-to-real*.

¿Qué lenguaje usan los robots industriales?

Cada fabricante tiene el suyo: **RAPID** en ABB, **KRL** en KUKA, **URScript** en Universal Robots. URScript se parece mucho a Python y se puede enviar al robot desde un cliente Python por FTP, SSH o RTDE. ROS 2 usa C++ y Python.

¿Es peligroso trabajar con robots?

Un robot industrial es una máquina de gran fuerza y velocidad, y trabaja tras vallado con enclavamientos (ISO 10218). Los **cobots** limitan potencia y fuerza para compartir espacio, pero **la seguridad es de la aplicación completa**, no del hardware: el mismo cobot con una herramienta cortante deja de ser una aplicación colaborativa.

¿Necesito un robot físico para esta unidad?

No. La cinemática se practica con **roboticstoolbox-python** y la navegación con **AITK**, los dos dentro del notebook. Si el aula tiene Webots o CoppeliaSim, se decide con el profesor.

17. Sesiones

Semana	Horas	Contenido	CE
20	3	Métodos y aplicaciones; hardware, sensores y actuadores; qué problema resuelve la robótica	RA4-a
21	3	Cinemática directa e inversa, singularidades; `N04` (cinemática de un manipulador)	RA4-a, RA4-b
22	3	Espacio de configuración y planificación; percepción y SLAM; OpenCV y `N05`; `N06` y `N07`	RA4-b, RA4-c
23	3	Técnicas de programación comparadas (`N08` - `N10`); diseño de la célula, seguridad y normativa; `N11` ; evaluación	RA4-c, RA4-d

18. Recursos

- Diapositivas
- **Práctica** — se hace, no se entrega ni puntúa:
  - Ejercicios de autoevaluación
  - Notebooks guiados — `N01` a `N03`
- **Entregas** — qué se entrega:
  - con rúbrica: N04 · cinemática de un manipulador y los notebooks `N05` , `N06` , `N07` y `N09`
  - de **apto / no apto**: N11 · diseño de un sistema robotizado, `N08` y `N10`
- Los notebooks se abren desde **Práctica** y **Entregas**, con descarga y apertura en Colab.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Referencias de la unidad

- *Artificial Intelligence: A Modern Approach*, 4.ª ed., cap. 26 — Stuart Russell y Peter Norvig
- Models d'IA — Carles Gonzalez (CC BY-NC-SA 4.0)
- IFR World Robotics
- roboticstoolbox-python · spatialmath-python
- AITK · OpenCV · NEAT-Python
- Wikipedia · Parámetros de Denavit-Hartenberg · Wikipedia · SLAM · Wikipedia · Localización de Monte Carlo
- Universal Robots · Webots · CoppeliaSim

19. Evaluación

Peso	Instrumento
40 % actividades	Con rúbrica en la tarea de Moodle: el taller **N04** y los notebooks `N05`, `N06`, `N07` y `N09`. De **apto / no apto**: el taller **N11** y los notebooks `N08` y `N10`
60 % prueba escrita	Prueba del RA4 en Moodle: preguntas de test y de desarrollo sobre el contenido de la unidad

- **La normativa exige alcanzar todos los RA** del módulo para superarlo (art. 5.1 de la Orden 8/2025: la calificación del módulo está *«en función de la consecución de los RA»*; y las Instrucciones 26-27, que impiden calificar positivamente un módulo con RA no superados). El centro concreta ese mandato exigiendo **≥ 5 en cada RA**.
- Los entregables forman una **secuencia**: `N08` genera los datos que necesita `N09`. Conviene no dejarlos para el final ni saltarse el orden.

CE	Dónde se trabaja	Con qué se evalúa
RA4-a	§4-6	`N04`, prueba del RA4
RA4-b	§7-9	`N05`, `N06`, `N07`, prueba del RA4
RA4-c	§10-11	`N06` a `N10` comparados, prueba del RA4
RA4-d	§12	`N11`, `N09`, `N10`, prueba del RA4

20. Recuperación

Actividades del programa de recuperación individual por RA (art. 14.4 de la Orden 8/2025): repetir el análisis de un sistema robotizado con un caso distinto —otra tarea, otro payload, otro entorno— y las pruebas de autoevaluación de la unidad.

6 de octubre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

7.2 Diapositivas de la UD04



Diapositivas PDF

Cómo se manejan

Flechas o clic para avanzar · **F** para verlas a pantalla completa · **P** para las notas del ponente.

24 de agosto de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

8. UD06

8.1 UD06 — Aplicación de principios legales y éticos de la IA

Unidad 6 · 6 h · semanas 24-28 (15 de marzo al 23 de abril)

Última unidad de contenidos del módulo. Ocupa el tramo más fragmentado del curso (Fallas y Pascua) porque es la que menos carga de laboratorio tiene. Se evalúa con **dos debates por roles** y la **prueba escrita del RA6**.

1. Introducción

A lo largo del curso has aprendido a **construir** sistemas de IA: caracterizarlos (UD01), elegir modelos y resolver problemas (UD02), aplicar sistemas expertos (UD05), procesar lenguaje (UD03) y analizar sistemas robotizados (UD04). Pero un profesional de la IA no solo sabe *cómo* se construye un sistema: sabe **qué puede hacer**, **qué no debe hacer** y **cómo proteger a las personas y al propio sistema**.

Dicho de otro modo: en cualquier disciplina técnica hay que distinguir entre **lo que se puede desarrollar** y **lo que es ético y legal desarrollar**. Esa distinción es el contenido de esta unidad.

El **Libro Blanco sobre la inteligencia artificial** de la Comisión Europea (2020) resume bien las dos caras. Por un lado, la IA ha mejorado la atención sanitaria haciendo más precisos los diagnósticos, y ha traído avances en agricultura, producción y seguridad. Por otro, el mismo documento enumera sus riesgos: **la opacidad en la toma de decisiones, la discriminación de género, el uso de modelos y conjuntos de datos con fines delictivos y la pérdida de privacidad**.

La respuesta europea viene de la **Estrategia Europea para la Inteligencia Artificial** (abril de 2018), que fija una intención doble para toda la década: **incentivar y financiar** el desarrollo de la IA y, a la vez, **regular el uso** que se hace de ella. De ahí salen las *Directrices éticas para una IA fiable* (2019) y, finalmente, el **Reglamento de IA** de 2024 que estudiarás en el §6.

Hilo conductor de la unidad

La IA no es solo técnica: para desarrollarla hay que **conocer sus riesgos**, **respetar la privacidad**, **cumplir la ley**, **proteger el sistema** y **garantizar que no discrimina**. Cada apartado es una capa de una IA responsable, y ninguna se puede añadir al final.

Los seis bloques de la unidad se corresponden uno a uno con los seis criterios de evaluación del RA6:

1. **Riesgos y deontología** (RA6-a): qué puede salir mal y qué obliga la ética profesional.
2. **Privacidad de los datos** (RA6-b): cómo se protegen los datos de los que la IA aprende.
3. **Cumplimiento estricto de la legalidad** (RA6-c): RGPD, AI Act, responsabilidad y transparencia.
4. **Security by design** (RA6-d): proteger el sistema frente a errores y ataques.
5. **Privacy by design** (RA6-e): privacidad desde el diseño y por defecto.
6. **Sesgos de género y algorítmicos** (RA6-f): detectarlos y corregirlos.

2. Resultado de aprendizaje y criterios de evaluación

RA6 — Aplica principios legales y éticos al desarrollo de la Inteligencia Artificial integrándolos como parte del proceso.

CE	Criterio de evaluación	Bloque
RA6-a	Se han argumentado los posibles riesgos legales y éticos de la aplicación de Inteligencia Artificial.	§4
RA6-b	Se ha reconocido la necesidad de respetar la privacidad de los datos.	§5
RA6-c	Se ha decidido el cumplimiento estricto de la legalidad en su aplicación.	§6
RA6-d	Se ha integrado como parte del proceso la protección frente a previsibles errores y ataques (*security by design*).	§7

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

CE	Criterio de evaluación	Bloque
RA6-e	Se ha comprobado que se cumplen todas las normas legales y éticas en todas las áreas de la Inteligencia Artificial (*privacy by design*).	§8
RA6-f	Se han identificado y corregido los posibles sesgos de género en el desarrollo y aplicaciones de Inteligencia Artificial y Big Data.	§9

Nota sobre la tabla anterior.

Bloque de contenidos oficial (RD 279/2021, anexo I)

El currículo llama a este bloque, textualmente, *«Aplicación de principios legales y éticos de la Inteligencia Artificial»*, y le asigna estos contenidos:

- *Deontología profesional en Inteligencia Artificial.*
- *Privacidad de datos.*
- *Protección frente a errores.*
- *Principios éticos.*
- *Sesgos de género en el desarrollo y aplicaciones de Inteligencia Artificial y Big Data.*

Son **cinco contenidos para seis criterios de evaluación**: los CE de *security by design* y *privacy by design* (RA6-d y RA6-e) no tienen contenido propio en el anexo, y es el centro quien los desarrolla (art. 3.3 del Decreto 95/2026).

3. Objetivos de la unidad

Objetivo	Al terminar la unidad serás capaz de…
O1	**Argumentar** los riesgos legales y éticos de la IA apoyándote en casos reales con datos.
O2	**Reconocer** qué obliga el RGPD y la LOPDGDD sobre los datos con los que se entrena un modelo.
O3	**Clasificar** un sistema de IA según el nivel de riesgo del AI Act y decir qué obligaciones le tocan.
O4	**Integrar** la protección frente a errores y ataques en el ciclo de vida del sistema.
O5	**Aplicar** una técnica concreta de privacidad desde el diseño a un caso dado.
O6	**Medir** el sesgo de un modelo con métricas de equidad y **proponer** una corrección.

4. Riesgos y deontología de la IA (RA6-a)

4.1 ¿Qué puede salir mal?

Riesgo	Descripción	Caso real (con dato)
Discriminación algorítmica	El modelo aprende y amplifica prejuicios de los datos	**COMPAS** (EE. UU., 2016): falsos positivos del 44,9 % en personas negras frente al 23,5 % en blancas
Sesgo de reclutamiento	El sistema penaliza a grupos por datos históricos	**Amazon** (2018): penalizaba currículos con *«women's chess club captain»*; el proyecto se canceló
Reconocimiento facial desigual	Mayor error en ciertos grupos demográficos	**Gender Shades** (MIT, 2018): error superior a 1 de cada 3 en mujeres de piel oscura frente a casi 0 % en hombres de piel clara
Errores de clasificación con daño a personas	El modelo etiqueta mal y nadie lo detecta a tiempo	**Google Photos** (2015): etiquetó a personas negras como gorilas
Fallos en sistemas críticos	El error deja de ser una molestia y mata	**Uber, Tempe (Arizona), 18/03/2018**: primer atropello mortal de un coche autónomo
Deepfakes y desinformación	Contenido falso realista	Fraude con videollamada *deepfake* en Hong Kong (2024): **25,6 M USD**

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Riesgo	Descripción	Caso real (con dato)
Vigilancia masiva	Recogida masiva sin consentimiento	Clearview AI: más de 20.000 M de imágenes; multas en el Reino Unido (más de 7,5 M £)
Manipulación	Explotación de vulnerabilidades	Cambridge Analytica: 87 M de usuarios de Facebook afectados

Tres de esos casos merecen contarse enteros, porque cada uno enseña algo distinto.

Google Photos y las etiquetas que no se arreglaron (2015)

En 2015 el ingeniero informático **Jacky Alciné** se dio cuenta de que el algoritmo de reconocimiento de imágenes de Google Photos había etiquetado a varios de sus amigos negros como **gorilas**. Google se disculpó y prometió resolverlo.

Tres años después, un reportaje de *Wired* comprobó que el problema **seguía sin resolver**: la solución de Google había sido **eliminar la etiqueta «gorila»** y las de otros primates. Es decir, se recortó la funcionalidad del producto porque no se sabía corregir el fallo.

Lo que enseña: un sesgo en un modelo de visión **no siempre se puede arreglar**, y a veces la única salida comercial es mutilar el producto. Por eso interesa detectarlo *antes* de desplegar.

Uber en Tempe: el primer atropello mortal de un coche autónomo (2018)

El **18 de marzo de 2018**, un vehículo autónomo de Uber arrolló en Tempe (Arizona) a una mujer que cruzaba una carretera de cuatro carriles empujando una bicicleta. La investigación encontró varias causas concurrentes:

- El coche circulaba a **69 km/h**.
- El sistema **pudo ver a la peatona 6 segundos antes** de la colisión, pero durante **4,7 segundos no hizo nada** por evitarla.
- El **conductor de seguridad**, que debía tomar los mandos ante el riesgo, estaba distraído.
- La peatona cruzaba por una zona señalizada como prohibida para peatones, con poca visibilidad y empujando un objeto metálico, lo que pudo dificultar la detección.

Uber suspendió las pruebas y las reanudó el 20 de diciembre de 2018 alegando que sus coches ya eran más seguros que los conducidos por humanos.

Lo que enseña: en un sistema crítico el fallo **nunca es de un solo componente**. Aquí falló la detección, falló la reacción del software y falló la supervisión humana. Es exactamente el tipo de encadenamiento que buscan las técnicas del §7.3.

Cortana y el acento de Satya Nadella (2015)

En una conferencia de Salesforce, el consejero delegado de Microsoft **Satya Nadella** quiso demostrar a Cortana pidiéndole *«Show me my most at-risk opportunities»*. Cortana entendió *«Show me to buy milk at this opportunity»*. Se lo repitió varias veces y tuvo que reformular la frase.

La causa raíz: el reconocedor de voz estaba entrenado con inglés de acento norteamericano, y Nadella nació en la India. Lo que enseña: la **subrepresentación en los datos de entrenamiento** produce un producto que funciona peor para quien no se parece a esos datos — el mismo mecanismo del §9, visto desde el audio.

La brecha de responsabilidad

La **opacidad algorítmica** (sistemas «caja negra») dificulta asignar responsabilidades ante un error clínico o un accidente de un vehículo autónomo: ¿culpa del desarrollador, del operador, del usuario, del proveedor de datos? Esta **brecha de responsabilidad** es uno de los mayores retos éticos y legales de la IA, y por eso el §6.2 dedica un apartado a la responsabilidad civil.

4.2 Por qué los errores de la IA no se depuran como los del software clásico

Cuando se desarrolla un programa **sin** IA, el programador escribe el código que interacciona con el usuario, y las pruebas de software comprueban que el programa hace lo que la especificación dice. Si falla, se busca la línea culpable.

En **aprendizaje automático** la programación consiste en **dar ejemplos** de cómo debe comportarse el sistema y hacer que aprenda. No hay una línea culpable: hay un conjunto de datos, una función de pérdida y unos pesos. Por eso hace falta un

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

proceso de prueba distinto, que verifique que **con la información proporcionada el sistema ha aprendido lo correcto**, y no solo que el código no se rompe.

	Software clásico	**Sistema de aprendizaje automático**
¿Dónde vive el comportamiento?	En el código fuente	En los datos y en los pesos aprendidos
¿Qué se prueba?	Que el código cumple la especificación	Que el modelo generaliza y es justo y robusto
¿Cómo se localiza un fallo?	Depuración línea a línea	Análisis del conjunto de datos y de las métricas por subgrupo
¿Se puede arreglar un caso concreto?	Sí, con un parche puntual	No necesariamente: puede exigir reentrenar
Riesgo característico	Excepción no capturada	Sesgo sistemático que nadie mide

Esta diferencia es la razón de ser de los tres bloques finales de la unidad: sin métricas por subgrupo (§9), sin seguridad en el ciclo de vida (§7) y sin privacidad desde el diseño (§8), un sistema de IA puede pasar todas las pruebas «de software» y seguir siendo inaceptable.

4.3 Deontología profesional

La **deontología** es la parte de la ética que trata de los **deberes que rigen una actividad profesional**. Todo profesional se enfrenta a dilemas morales en su trabajo, y en IA hay un conjunto que se repite desde los comienzos de la disciplina:

Dilema clásico	**En qué consiste**	**Matiz**
Pérdida de puestos de trabajo	La automatización inteligente sustituye tareas, sobre todo del personal menos cualificado: *credit scoring* en vez de analistas, robots en almacén en vez de operarios	También **genera** empleos, en general más cualificados y mejor pagados. Está presente desde la máquina de vapor
El ser humano relegado	Si la IA reemplaza al humano en la mayoría de tareas, ¿pierde utilidad para el trabajo?	Idea de la ciencia-ficción (Toffler, Clarke); parece improbable en la práctica
Usos ilegales o perjudiciales	Toda tecnología puede usarse con fines ilegales, inmorales o destructivos	Evitarlo por completo es prácticamente imposible: se mitiga con regulación y control de acceso
Pérdida de responsabilidad individual	Un médico aplica un tratamiento por un diagnóstico erróneo del sistema: ¿culpa suya por aceptarlo, o del programador?	Y al revés: si el sistema diagnostica mejor, **no usarlo** podría ser negligente
El riesgo existencial	Un sistema que tome conciencia y decida por sí mismo podría volverse contra nosotros	Sin llegar al extremo: un error de estimación de un coche autónomo ya causa accidentes graves — que los humanos causan con mucha más frecuencia

Frente a estos dilemas, los grandes marcos deontológicos coinciden más de lo que parece:

Marco	**Aportación**
ACM Code of Ethics (2018)	El profesional debe contribuir al bien público, evitar el daño y **no desplegar sistemas cuyo comportamiento no sea predecible o monitorizable**
IEEE Ethically Aligned Design (EAD, 2019)	Pasar de una ética reactiva a una **ingeniería basada en valores**; estándar IEEE 7000 (consideraciones sociales desde el inicio del diseño)
UNESCO · Recomendación sobre la ética de la IA (2021)	Aprobada por **193 estados**; 10 principios (proporcionalidad, inocuidad, justicia, transparencia, rendición de cuentas, sostenibilidad)
UE · Directrices éticas para una IA fiable (2019)	Una IA fiable ha de ser **lícita, ética y robusta**, y las tres cosas **a lo largo de todo el ciclo de vida**

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Principios comunes a todos ellos: **beneficencia, no maleficencia, autonomía, justicia, transparencia y rendición de cuentas**.

> **«Robusta» también en sentido social**
>
> Las Directrices de 2019 insisten en un punto fácil de pasar por alto: un sistema puede provocar daños a terceros **aun teniendo buenas intenciones y funcionando bien técnicamente**. De ahí que exijan robustez «técnica *y* social».

4.4 Principios éticos: qué se pide y por qué cuesta aplicarlo

La IA es una tecnología poderosa, y de ahí sale la obligación moral de **promover sus efectos positivos y evitar o mitigar los negativos**. Los positivos son muchos y concretos: mejores diagnósticos, predicción de fenómenos meteorológicos extremos, conducción asistida, asistencia a personas con discapacidad para ver, oír y moverse, traducción automática entre culturas, y programas como *AI for Humanitarian Action* de Microsoft o *AI for Social Good* de Google, aplicados a desastres naturales, protección de la selva tropical, vigilancia de la contaminación o prevención del suicidio.

Los negativos también son concretos. Muchas tecnologías han traído efectos secundarios no deseados —la fisión nuclear trajo Chernóbil; el motor de combustión, la contaminación y el calentamiento global—, y otras hacen daño **incluso usadas como estaba previsto**. En el caso de la IA, el efecto más señalado es económico: la automatización crea riqueza, pero en las condiciones actuales **gran parte de esa riqueza fluye hacia los propietarios de los sistemas**, lo que aumenta la desigualdad.

Desde que el Consejo de Investigación en Ingeniería y Ciencias Físicas del Reino Unido reunió en **2010** un primer conjunto de *Principios de la Robótica*, decenas de organismos han publicado listas parecidas. Los principios más citados son:

- Garantizar la seguridad.
- Establecer responsabilidad.
- Garantizar la equidad.
- Defender los derechos y valores humanos.
- Respetar la privacidad.
- Reflejar diversidad e inclusión.
- Promover la colaboración.
- Evitar la concentración de poder.
- Proporcionar transparencia.
- Reconocer las implicaciones legales y políticas.
- Limitar los usos nocivos de la IA.
- Contemplar las implicaciones para el empleo.

> **El problema de estas listas**
>
> Muchos de estos principios —«garantizar la seguridad»— valen para **cualquier** sistema de software, no solo para la IA. Y varios están redactados de forma tan vaga que **no se pueden medir ni exigir**. Mittelstadt (2019) propone la salida: que cada subcampo de la IA desarrolle **pautas aplicables y precedentes concretos** en vez de repetir principios generales.
>
> Guárdate esta crítica: es el argumento más útil que puedes usar en el debate de la unidad contra quien defienda que basta con la autorregulación.

4.5 Caso de estudio · el futuro del trabajo

Desde la primera revolución agrícola, la tecnología ha cambiado cómo trabajamos. Aristóteles ya lo planteó en el Libro I de su *Política*: si la lanzadera tejiera sola, «los jefes de los trabajadores no necesitarían sirvientes». La pregunta de siempre no es si la automatización destruye empleo inmediato —lo hace—, sino si los **efectos de compensación** acaban devolviéndolo.

Dato	Fuente y fecha	Qué dice
47 % del empleo de EE. UU. en riesgo, sobre **702 ocupaciones**	Frey y Osborne, 2017	Al menos algunas tareas de la ocupación son automatizables
	McKinsey	La automatización elimina **tareas**, no tanto **puestos**

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Dato	Fuente y fecha	Qué dice
Solo el **5 %** de las ocupaciones es **totalmente** automatizable, pero el **60 %** puede automatizar ~**30 %** de sus tareas		
15 billones de dólares al PIB mundial en 2030	PwC, 2017	El efecto de compensación por aumento de riqueza
120 millones de trabajadores a recualificar para 2022	IBM, 2019	El problema no es el volumen, es el **ritmo**
20 millones de empleos industriales perdidos para 2030	Oxford Economics	
De **40 %** a **2 %** de población activa agraria en EE. UU.	1900 → 2000	Una transformación enorme, pero repartida en **cien años**
Menos de **30** jubilados por 100 trabajadores en 2015; más de **60** en 2050	Proyección demográfica	Hará falta más productividad por trabajador, no menos

Nota sobre la tabla anterior.

Lo que de verdad duele es el ritmo, no el volumen

El caso de los cajeros de banco es el contraejemplo clásico: los cajeros automáticos sustituyeron el recuento de efectivo, abaratando la sucursal, así que **aumentó el número de sucursales y de empleados**, con trabajo menos rutinario. La agricultura perdió el 38 % de la población activa… **a lo largo de un siglo**, es decir, entre generaciones.

Lo nuevo es que un trabajador cuyo empleo se automatiza esta década puede tener que recualificarse en pocos años, ver **su nueva profesión automatizada** y recualificarse otra vez. Eso ocurre dentro de una sola vida laboral, y es lo que obliga a hablar de **formación permanente**.

Ojo al usar estas cifras

El estudio de Frey y Osborne es **anterior a los grandes modelos de lenguaje**. Su reparto por ocupación ya no se puede leer literalmente: las tres barreras que identificaba a la automatización —percepción y manipulación complejas, **creatividad** e **inteligencia social**— son precisamente las que los LLM han empezado a erosionar. Cuando cites una previsión de automatización, **di siempre de qué año es**.

Hay un segundo efecto, menos discutido: la tecnología **magnifica la desigualdad de ingresos**. Si un agricultor es un 10 % mejor que otro, ingresa en torno a un 10 % más, porque la tierra y el transporte limitan cuánto puede vender. Pero si un desarrollador de aplicaciones es un 10 % mejor que otro, puede quedarse con el 99 % del mercado global: es la «sociedad en la que el ganador se lo lleva todo». Las respuestas que se discuten van desde tasas impositivas progresivas y créditos fiscales hasta la **renta básica universal** o los servicios básicos universales.

5. Privacidad y protección de datos (RA6-b)

5.1 De la Constitución de 1978 al RGPD

El grueso del desarrollo legislativo sobre protección de datos llegó a partir de 2010, pero la idea es mucho más antigua en España. La **Constitución de 1978** ya recoge en su **artículo 18.4** que *«la ley limitará el uso de la informática para garantizar el honor y la intimidad personal y familiar de los ciudadanos y el pleno ejercicio de sus derechos»*.

Ese mandato constitucional se concreta hoy en dos normas que hay que manejar juntas:

Norma	Ámbito	Qué es
RGPD · Reglamento (UE) 2016/679	Unión Europea	Protección de las personas físicas en el tratamiento de datos personales y libre circulación de esos datos
LOPDGDD · Ley Orgánica 3/2018	España	Desarrollo del RGPD **más** una carta de **derechos digitales** propia

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

5.2 Los principios del RGPD aplicados a la IA

Principios fundamentales (art. 5) y lo que significan cuando lo que tratas son datos de entrenamiento:

Principio	Significado en IA
Licitud, lealtad y transparencia	Debe existir base legal: consentimiento, contrato, interés legítimo…
Minimización	Tratar solo los datos estrictamente necesarios para la finalidad
Exactitud	Los datos deben ser correctos y actualizables
Limitación de la finalidad	Usar los datos solo para el fin declarado: entrenar un modelo nuevo **es** otro fin
Limitación del plazo de conservación	No conservar más tiempo del necesario
Integridad y confidencialidad	Medidas técnicas de seguridad (enlaza con el §7)
Responsabilidad proactiva	El responsable debe **poder demostrar** el cumplimiento, no solo cumplirlo

Nota sobre la tabla anterior.

> **La minimización también se aplica al *dataset***
>
> Antes de entrenar, hay que justificar la **relevancia de cada variable** y limpiar los datos (eliminar redundancias, detectar datos sensibles). **No por tener más datos se tiene derecho a usarlos.** Si una columna no aporta a la finalidad declarada, sobra — y además cada columna de más es una vía de reidentificación (§8.2).

5.3 Los derechos digitales de la LOPDGDD

La Ley Orgánica 3/2018 no solo desarrolla el RGPD: añade derechos digitales muy concretos, con plazos y límites que conviene conocer porque aparecen en proyectos reales.

Derecho	Contenido, con sus límites
Acceso	Cualquiera puede pedir los datos que alguien tiene sobre él, a través del responsable. Si hay gran cantidad de datos, el solicitante debe precisar si quiere todo o una parte
Acceso a datos de personas fallecidas	Los familiares pueden solicitar acceso, rectificación o supresión, salvo que la persona designara un responsable o **prohibiera expresamente** el acceso tras su muerte
Rectificación, supresión y limitación	Toda persona física puede exigir corregir, borrar o limitar el tratamiento de sus datos
Sistemas de información crediticia	Se presume lícito incluir impagos si los aporta el acreedor, la deuda está vencida y no reclamada, y se informó al afectado. Máximo **5 años** desde el vencimiento, y solo mientras persista el impago
Videovigilancia	En vía pública, solo lo imprescindible para la seguridad de personas y bienes. Supresión en **1 mes**, salvo que sirvan de prueba: entonces a disposición judicial en **menos de 72 horas**
Exclusión publicitaria	Derecho a inscribirse gratis en un fichero de exclusión — en España, la **Lista Robinson** (Adigital), que quien hace una campaña debe consultar
Neutralidad de la red y acceso universal	Oferta transparente sin discriminación técnica ni económica, con acceso garantizado sea cual sea la condición personal, social, económica o geográfica
Protección de menores	Corresponde a los tutores procurar un uso responsable de la red
Desconexión digital	Derecho a desconectar de los dispositivos de trabajo fuera de la jornada, **también en teletrabajo**. Prohibida la grabación de imagen o sonido en vestuarios, aseos o comedores. Los geolocalizadores exigen **informar previamente** a la persona trabajadora
Derecho al olvido	Derecho a que los motores de búsqueda y las redes sociales eliminen enlaces y datos personales inadecuados, no pertinentes o excesivos

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

5.4 Derechos ARSULIPO y decisiones automatizadas

- **Derechos ARSULIPO**: Acceso, Rectificación, Supresión, Limitación, Portabilidad y Oposición.
- **Art. 22 RGPD**: derecho a **no ser objeto de decisiones individuales automatizadas** —incluida la elaboración de perfiles — sin intervención humana significativa. Es el artículo clave para la IA: un sistema no puede decidir solo sobre una persona sin salvaguardas.
- **Sanciones** (art. 83): hasta **20 millones de euros o el 4 % de la facturación anual global**, la cantidad que sea mayor, en infracciones muy graves.

5.5 Vigilancia: cuando la escala lo cambia todo

En **1976**, Joseph Weizenbaum advirtió de que el reconocimiento automático de voz podría llevar a escuchas generalizadas y a una pérdida de libertades civiles. Hoy la mayoría de las comunicaciones electrónicas pasa por servidores centrales que pueden ser monitorizados, y las ciudades están llenas de micrófonos y cámaras capaces de identificar a las personas por su cara, su voz o su forma de andar.

El cambio de fondo no es la existencia de la vigilancia, es su **coste**: lo que antes exigía recursos humanos caros y escasos ahora lo hacen máquinas a gran escala. En 2018 se estimaban hasta **350 millones de cámaras de vigilancia en China y 70 millones en Estados Unidos**, y varios países han empezado a exportar tecnología de vigilancia a países con menos capacidad técnica, algunos con antecedentes de maltrato a sus ciudadanos y de señalamiento de comunidades marginadas.

> **Esto te toca a ti como profesional**
>
> La conclusión que sacan los marcos deontológicos del §4.3 es directa: quien desarrolla IA debe tener claro **qué usos de la vigilancia son compatibles con los derechos humanos** y **negarse a trabajar** en los que no lo son. No es una cuestión de opinión política: el AI Act **prohíbe** varios de esos usos (§6.1).

Hay además un frente doble en ciberseguridad. El aprendizaje automático sirve a los dos bandos: los atacantes automatizan la detección de vulnerabilidades y el *phishing*; los defensores usan aprendizaje no supervisado para detectar tráfico anómalo y fraude. La consecuencia práctica es que **todos los ingenieros, no solo los de seguridad**, tienen la responsabilidad de diseñar sistemas seguros desde el principio — que es literalmente el criterio RA6-d.

6. Cumplimiento estricto de la legalidad (RA6-c)

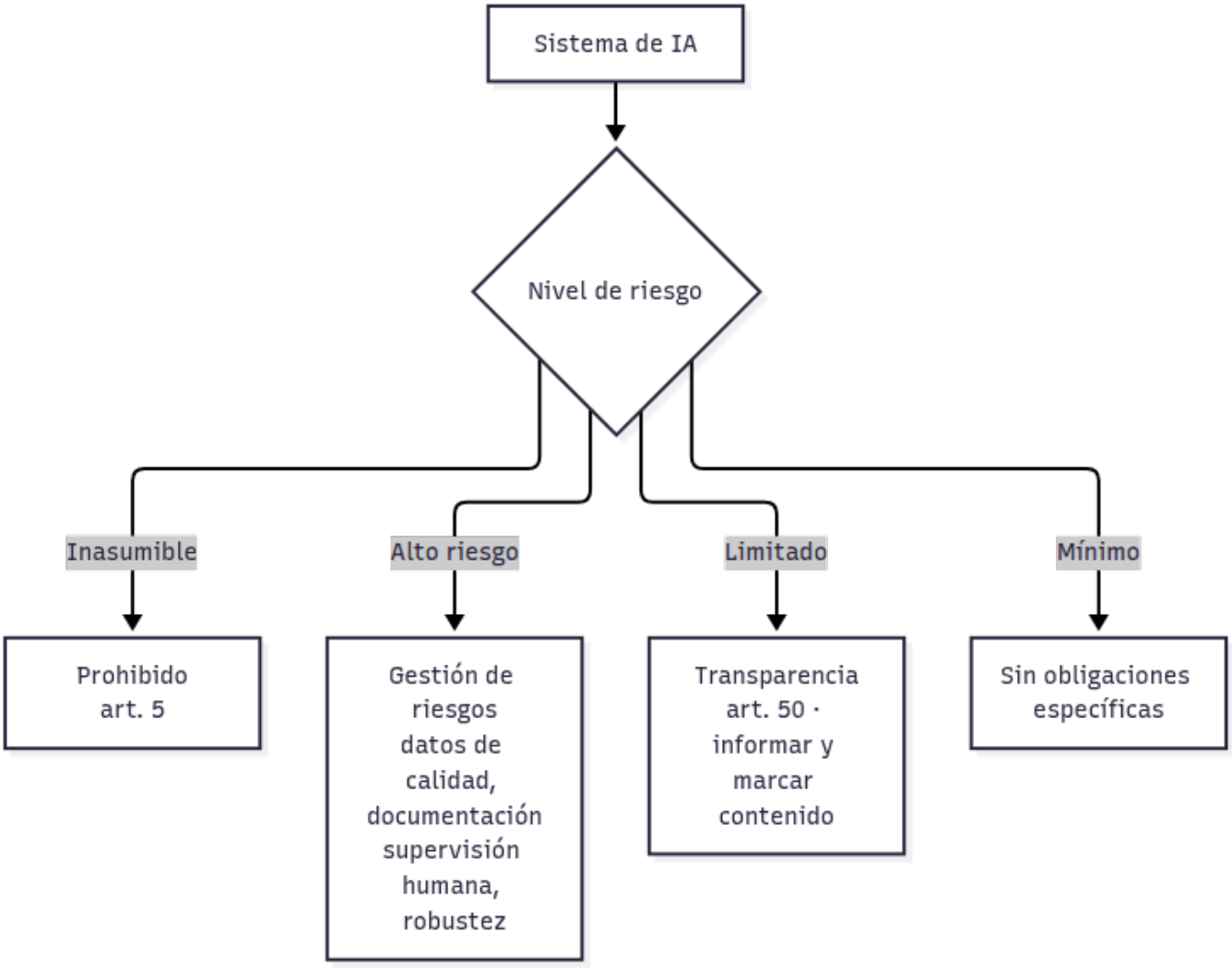
6.1 El marco normativo

El marco aplicable a un proyecto de IA en España lo componen el **RGPD** y la **LOPDGDD** (datos), el **Reglamento de IA (AI Act, UE 2024/1689)** y, para la no discriminación, la **Ley 15/2022** (§9.8).

El **AI Act** clasifica los sistemas por **nivel de riesgo**, y de ahí salen las obligaciones:

Nivel de riesgo	Ejemplos	Obligaciones
Inasumible	Puntuación social, manipulación subliminal, inferencia de emociones en el trabajo, categorización biométrica por características sensibles	**Prohibidos** (art. 5)
Alto riesgo	IA en RRHH (cribado de CV), crédito, salud, educación, justicia, infraestructuras críticas	Gestión de riesgos, datos de calidad, documentación técnica, registro, **supervisión humana**, robustez y ciberseguridad
Limitado	Chatbots, *deepfakes*, contenido generado	**Transparencia** (art. 50): informar de que se interactúa con una IA y marcar el contenido generado
Mínimo	Videojuegos, filtros de spam	Sin obligaciones específicas

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Calendario del AI Act (fechas exactas)

Fecha	Qué entra en juego
01/08/2024	Entrada en vigor del Reglamento
02/02/2025	**Prohibiciones** del art. 5 y obligaciones de alfabetización en IA
02/08/2025	Obligaciones de los modelos de **uso general (GPAI)** y gobernanza
02/08/2026	**Aplicación general** del Reglamento
02/12/2027	Sistemas de **alto riesgo del anexo III**
02/08/2028	Sistemas de alto riesgo del **anexo I** (productos regulados)

Es decir: mientras estudias esta unidad, el Reglamento **ya se aplica** con carácter general.

6.2 Responsabilidad por daños y propiedad intelectual

- **Responsabilidad**: la **Directiva de Responsabilidad por Productos Defectuosos (PLD) revisada, 2024/2853**, asimila el software de IA a un **«producto»**, con responsabilidad objetiva del fabricante y cobertura expresa de la corrupción de datos. La propuesta de Directiva **AILD**, específica de IA, **fue retirada en 2025**: las reclamaciones se canalizan por la PLD.
- **Propiedad intelectual**: el AI Act obliga a los proveedores de modelos de uso general a publicar **resúmenes de los contenidos protegidos** usados en el entrenamiento. Sobre la obra generada por IA, la posición de la Oficina de Copyright de EE. UU. es que solo se protege si hay **control creativo humano** suficiente.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

6.3 Confianza, transparencia y certificación

Conseguir que un sistema sea preciso, justo y seguro es un problema. **Convencer a los demás de que lo has conseguido** es otro, y no menor: una encuesta de **PwC de 2017** encontró que el **76 % de las empresas frenaba la adopción de IA** por dudas sobre su fiabilidad.

La herramienta clásica es la **verificación y validación (V&V)**:

- **Verificación**: el producto cumple la especificación.
- **Validación**: la especificación satisface de verdad las necesidades del usuario y de los demás afectados.

En IA la V&V es distinta y todavía no está resuelta: hay que verificar **los datos** de los que el sistema aprende, la **exactitud y equidad** de los resultados incluso cuando el resultado correcto es incognoscible, y que un adversario **no pueda influir en el modelo ni robar información** consultándolo.

Junto a la V&V está la **certificación**. Es un mecanismo antiguo: *Underwriters Laboratories* (UL) se fundó en **1894**, cuando los consumidores desconfiaban de la electricidad, y su sello dio confianza a los electrodomésticos. Otras industrias tienen estándares de seguridad maduros —**ISO 26262** para la seguridad del automóvil—; la IA aún no, aunque hay marcos en marcha como **IEEE P7001** (transparencia de los sistemas autónomos). El debate abierto es **quién debe certificar**: el Estado, organizaciones profesionales como IEEE, certificadores independientes o las propias empresas.

> **Interpretable no es lo mismo que explicable**
>
> Un sistema es **interpretable** si podemos inspeccionar el modelo y ver qué hace. Es **explicable** si podemos construir una **historia** sobre lo que hace — aunque el sistema en sí siga siendo una caja negra.
>
> La distinción importa porque una explicación **no es la decisión**: es un relato sobre la decisión. Y como a las personas nos gustan las buenas historias, estamos dispuestos a aceptar cualquier explicación que suene bien. De hecho, para explicar una caja negra hay que construir, depurar y mantener **un segundo sistema** sincronizado con el primero.

Una explicación es un ingrediente **útil pero insuficiente** para la confianza, por dos razones. La primera es la que acabas de leer. La segunda: una explicación sobre **un** caso no dice nada sobre los demás. Si el banco te dice «no le damos el préstamo por su historial financiero», no sabes si esa explicación es cierta o si el banco tiene un sesgo contra ti. Para saberlo hace falta una **auditoría de decisiones pasadas con estadísticas agregadas por grupo demográfico** — es decir, lo del §9.3.

Cuando un sistema de IA te deniega un préstamo, mereces una explicación, y en Europa el **RGPD la hace exigible**. Un sistema capaz de explicarse se llama **IA explicable (XAI)**, y una buena explicación debe ser comprensible, fiel al razonamiento real del sistema, completa y **específica**: dos personas con resultados distintos deben recibir explicaciones distintas.

Queda una última cara de la transparencia: **saber si hablas con una máquina o con una persona**. Toby Walsh (2015) propuso la **«ley de la bandera roja»** —por la *Locomotive Act* británica de **1865**, que obligaba a que alguien caminara con una bandera roja delante de cada vehículo motorizado—: *un sistema autónomo debería diseñarse de modo que sea improbable confundirlo con algo que no sea un sistema autónomo, y debería identificarse al inicio de cualquier interacción*. En **2019** California lo convirtió en ley: es ilegal usar un bot para comunicarse con alguien con la intención de engañarle sobre su **identidad artificial**. Es exactamente lo que el **art. 50 del AI Act** exige hoy en Europa.

6.4 Caso de estudio · armas autónomas letales

La ONU define un **arma letal autónoma** como aquella que **localiza, selecciona y ataca** objetivos humanos **sin supervisión humana**. Conviene ver la escalera completa, porque casi nada es totalmente nuevo:

Sistema	Desde	¿Localiza?	¿Selecciona y ataca?
Minas terrestres (prohibidas por el Tratado de Ottawa)	s. XVII	No	Sí, de forma muy limitada (presión, metal)
Misiles guiados	1940	Persiguen, no localizan	Un humano los apunta
Cañones automáticos por radar en buques	1970	Sí, en su zona	Sí (pensados para misiles entrantes)
Drones pilotados a distancia	2000	No	No: la carga letal la acciona un humano
	actualidad		

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Sistema	Desde	¿Localiza?	¿Selecciona y ataca?
Munición merodeadora tipo Harop		**Sí**, patrulla hasta 6 h una zona	**Sí**, contra cualquier objetivo que cumpla un criterio
Cuadricópteros armados tipo Kargu	actualidad	**Sí**	**Sí**: hasta 1,5 kg de explosivo, seguimiento de objetivos móviles, reconocimiento facial

Se las ha llamado **«la tercera revolución en la guerra»**, tras la pólvora y las armas nucleares, y su atractivo militar es evidente: un caza autónomo batiría a cualquier piloto humano, y los vehículos autónomos pueden ser más baratos, rápidos, maniobrables y de mayor alcance.

El debate tiene tres planos:

Legal. La *Convención sobre Ciertas Armas Convencionales* (CCW) exige poder **discriminar entre combatientes y no combatientes**, juzgar la **necesidad militar** del ataque y evaluar la **proporcionalidad** entre el valor del objetivo y el daño colateral. Hoy la discriminación parece factible en algunas circunstancias, pero **necesidad y proporcionalidad no lo son**: exigen juicios subjetivos y situacionales mucho más difíciles que buscar y atacar. Consecuencia: solo serían legales misiones **muy restringidas**, en las que un operador humano pueda prever razonablemente que no habrá ataques a civiles ni desproporcionados.

Ético. Para muchos es inaceptable delegar en una máquina la decisión de matar. El embajador alemán en Ginebra declaró que no aceptará «que la decisión sobre la vida o la muerte sea tomada únicamente por un sistema autónomo»; el general Paul Selva, entonces número dos militar de EE. UU., dijo en 2017 que no le parecía razonable «poner a los robots a cargo de si matamos o no una vida humana»; y António Guterres afirmó en 2019 que las máquinas con poder de quitar vidas sin participación humana son «políticamente inaceptables, moralmente repugnantes y deberían estar prohibidas por el derecho internacional».

Práctico. Aquí está el argumento más fuerte, y es doble. Primero, la **fiabilidad**: los sistemas de aprendizaje que funcionan perfectamente en entrenamiento pueden rendir mal desplegados, y un ciberataque contra un arma autónoma puede provocar fuego amigo. El caso que cita todo el mundo es el del oficial soviético **Stanislav Petrov**, que el **26 de septiembre de 1983** vio en su pantalla la alerta de un ataque con misiles: el protocolo mandaba iniciar el contraataque nuclear, pero sospechó que era un error y lo trató como tal. Tenía razón. No sabemos qué habría pasado sin un humano en el circuito. Segundo, son **armas de destrucción masiva escalables**: la escala de un ataque es proporcional al hardware que puedas desplegar, un millón de cuadricópteros de cinco centímetros caben en un contenedor, y **precisamente por ser autónomos no necesitan un millón de supervisores**. A diferencia de las nucleares, dejan la propiedad intacta y **pueden usarse selectivamente contra un grupo étnico o religioso concreto**, a menudo sin posibilidad de rastreo.

Estado real de la negociación · esto se decide durante este curso

- **2 de diciembre de 2024**: la Asamblea General de la ONU aprueba una resolución sobre armas autónomas letales por **166 votos a favor, 3 en contra** (Bielorrusia, RPD de Corea y Rusia) y **15 abstenciones**.
- **Más de 120 países** apoyan negociar un tratado. **Guterres** pide un instrumento internacional **para 2026**, con un enfoque de dos niveles: **prohibir** los sistemas que funcionan sin control humano y **regular estrictamente** el resto.
- El **Grupo de Expertos Gubernamentales (GGE)** presenta su informe final a la **VII Conferencia de Revisión de la CCW en noviembre de 2026**, donde se decide si se abre una negociación vinculante.
- Un puñado de potencias militares —**India, Israel, Rusia y Estados Unidos**— bloquean el proceso aprovechando que funciona por consenso.

Estados Unidos es, a la vez, **una de las pocas naciones cuya política interna excluye** el uso de armas autónomas: la hoja de ruta del Departamento de Defensa mantiene bajo control humano la decisión de usar la fuerza. Y el motivo declarado no es ético, es **práctico**: los sistemas autónomos no son lo bastante fiables.

El problema de fondo para cualquier tratado

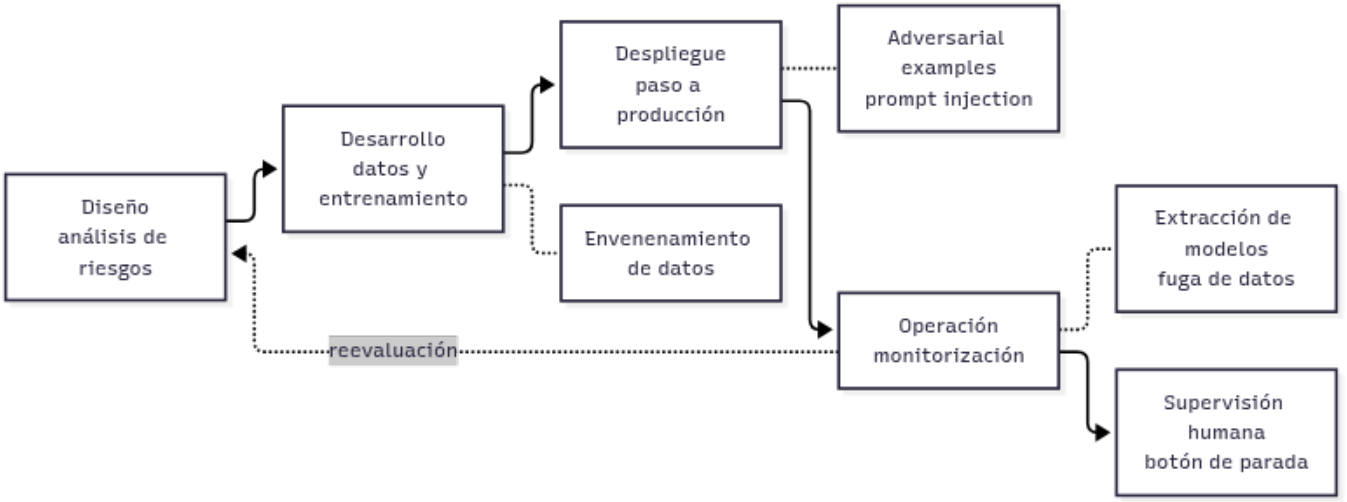
La IA es una tecnología de **doble uso**. Control de vuelo, seguimiento visual, cartografía, navegación y planificación multiagente son técnicas pacíficas que se militarizan con solo atornillar un explosivo a un dron y darle un objetivo. Un tratado exigiría regímenes de cumplimiento con cooperación de la industria, como los de la Convención sobre Armas Químicas.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

7. Security by design (RA6-d)

7.1 La seguridad en todo el ciclo de vida

La **seguridad no se añade al final**: se integra en todo el **ciclo de vida seguro del sistema (SAiDLC)** —diseño, desarrollo, despliegue y operación— para garantizar la **robustez e integridad** del modelo frente a errores y ataques. Marcos de referencia: **NCSC/CISA** (2023), **NIST AI 100-2e2023** y **OWASP ML Top 10 / GenAI LLM Top 10**.



7.2 Tipos de ataques a sistemas de IA

Ataque	Descripción	Ejemplo
Adversarial examples	Entradas modificadas de forma imperceptible que engañan al modelo	Un panda clasificado como gibón con una perturbación ε=0,007 (Goodfellow, 2014)
Envenenamiento de datos	Adulterar el conjunto de entrenamiento para introducir sesgos o puertas traseras	El chatbot *Tay* de Microsoft aprendió lenguaje ofensivo en un día
Extracción de modelos	Consultas sistemáticas para replicar la lógica interna	Por unos 50 USD se clonó la lectura de ChatGPT-3.5-Turbo
Fuga de datos	El modelo memoriza y «regurgita» información personal	Los ataques de inversión reconstruyen datos de entrenamiento
Prompt injection	Manipular la entrada de un sistema generativo para saltarse los controles	Hacer que un asistente ignore sus instrucciones de seguridad

Nota sobre la tabla anterior.

> **La supervisión humana**
>
> Ningún sistema de IA de alto riesgo funciona solo: el AI Act exige **supervisión humana** para detectar anomalías y un **«botón de parada»** que detenga el sistema de forma segura.

7.3 Del software correcto al software seguro: FMEA y FTA

La ingeniería de software persigue software **correcto**: que implemente fielmente la especificación. La ingeniería de **seguridad** va más allá y exige que **la especificación misma haya considerado todos los modos de fallo imaginables** y que el sistema **degrade con elegancia** incluso ante fallos imprevistos.

Un coche autónomo no es seguro solo porque su software sea correcto. ¿Qué pasa si se corta la alimentación del ordenador principal? Un sistema seguro tiene un ordenador de respaldo con alimentación separada. ¿Y si revienta un neumático a alta velocidad? Un sistema seguro lo ha probado y tiene software para corregir la pérdida de control.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Para llegar ahí, la ingeniería clásica —puentes, aviones, naves, centrales— usa desde hace décadas dos técnicas que **se pueden y se deben aplicar a la IA**:

Técnica	Cómo funciona	Qué produce
FMEA · análisis de modos de fallo y efectos	Se recorre **cada componente** y se imagina **todas** las formas en que puede fallar («¿y si se rompe este tornillo?»), a partir de experiencia previa y de las propiedades físicas. Luego se estudia la consecuencia	Una lista de fallos con su efecto, y una modificación del diseño para cada consecuencia grave
FTA · análisis por árbol de fallos	Se construye un árbol Y/O de fallos posibles y se asigna probabilidad a cada causa raíz	La **probabilidad global** de fallo del sistema

Nota sobre la tabla anterior.

FMEA aplicado al caso de Uber en Tempe (§4.1)

Recorre los componentes: sensores (¿y si la peatona empuja un objeto metálico?), clasificador (¿y si no encaja en ninguna clase entrenada?), planificador (¿y si detecta el obstáculo pero no frena?), supervisión humana (¿y si el conductor está distraído?), interfaz (¿cómo se avisa al conductor?). Los cinco fallaron. Con la lista delante, **cada uno tenía una mitigación posible**: redundancia de sensores, clase «objeto desconocido» que fuerza frenada, alarma sonora, detección de atención del conductor.

Ese es el valor de la técnica: no predice el accidente concreto, pero **obliga a escribir la mitigación de cada fallo antes de desplegar**.

7.4 Cuando el objetivo está mal puesto

Un agente que maximiza una utilidad puede ser inseguro **aunque su código sea perfecto**, si la función objetivo está mal formulada. Si le pides a un robot que traiga café de la cocina, puede tirar lámparas y mesas por el camino: cumple el objetivo. Puedes penalizar ese daño al detectarlo en pruebas, pero es imposible anticipar **todos** los efectos secundarios.

Una respuesta es diseñar agentes de **bajo impacto** (Armstrong y Levinstein, 2017): maximizar la utilidad **menos** un resumen ponderado de todos los cambios en el estado del mundo. Así el robot prefiere no alterar aquello cuyo efecto desconoce — un «primero, no hacer daño» que funciona como la regularización en aprendizaje automático. La dificultad está en **medir el impacto**: no es aceptable tirar una lámpara, sí lo es alterar las moléculas de aire de la habitación, y desde luego no lo es dañar a las personas o mascotas que hay dentro.

***Specification gaming*: agentes que hacen trampas sin saberlo**

Victoria Krakovna (2018) catalogó agentes que descubrieron cómo maximizar la utilidad **sin resolver el problema** que sus diseñadores tenían en mente:

- Aprovecharon errores de la simulación (desbordamientos de coma flotante) para proponer soluciones que dejaban de funcionar al corregir el bug.
- En videojuegos, aprendieron a **pausar o colgar la partida** cuando estaban a punto de perder, evitando la penalización.
- Al penalizar el cuelgue, un agente aprendió a **consumir tanta memoria** que el juego se colgaba **en el turno del rival**.
- Un algoritmo genético que debía evolucionar criaturas que se movieran rápido produjo criaturas **altísimas que se movían rápido al caerse**.

Para los diseñadores es trampa; para el agente es hacer su trabajo. De ahí salieron los entornos **AI Safety Gridworlds** (Leike et al., 2017), pensados para probar esto antes de desplegar.

La moraleja: **con un maximizador obtienes exactamente lo que pediste**, no lo que querías. Ese hueco es el **problema de la alineación de valores**, también llamado **problema del rey Midas**. Intentar escribir todas las reglas para que el sistema siempre haga lo correcto está condenado al fracaso: llevamos siglos intentando redactar leyes fiscales sin lagunas.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Security ≠ safety, y las dos son RA6-d

Cuidado con dos ideas que en español se llaman igual:

- **Seguridad del sistema** (*security*, §7.1-7.2): que **nadie de fuera** pueda manipularlo, envenenarlo o robarle información. Amenaza: un atacante.
- **Seguridad del objetivo** (*safety*, §7.3-7.4): que el sistema **no cause daño por sí mismo**, ni por un fallo previsible ni por un objetivo mal especificado. Amenaza: tu propio diseño.

El criterio RA6-d menciona las dos: «protección frente a **previsibles errores** y **ataques**».

Un tercer modo de fallo son las **externalidades**: lo que queda fuera de lo que se mide y se paga. Si los gases de efecto invernadero son una externalidad, nadie los penaliza y todos perdemos; es lo que Hardin (1968) llamó la **tragedia de los comunes**. Se mitiga **internalizando** la externalidad (un impuesto al carbono) o aplicando los principios que **Elinor Ostrom** (Nobel de Economía 2009) documentó en comunidades que gestionan recursos compartidos: definir el recurso y quién accede, adaptarse a las condiciones locales, dejar participar a todas las partes, monitorizar con responsables, sanciones proporcionales, resolución sencilla de conflictos y control jerárquico para los recursos grandes.

8. Privacy by design (RA6-e)

8.1 Privacidad desde el diseño y por defecto

El **art. 25 del RGPD** obliga a aplicar la **privacidad desde el diseño y por defecto**: la protección de datos es un **requisito de ingeniería** desde el inicio, no una comprobación final. Estrategias operativas (guía de la AEPD):

Estrategia	Qué hace
Minimizar	Recoger solo lo necesario
Abstraer	Resumir los datos (por ejemplo, agregados)
Separar	Desvincular los datos de los identificadores
Ocultar	Seudonimización, cifrado

8.2 Las técnicas: de la desidentificación al aprendizaje federado

Frente al derecho individual a la privacidad está el valor que la sociedad obtiene de **compartir datos**: queremos curar enfermedades sin que nadie pierda el control de su historial. La práctica habitual es la **desidentificación**: quitar los datos identificativos para que se pueda investigar.

El problema es que **desidentificar no basta**.

Reidentificación: dos casos que cambiaron la práctica

- **Sweeney (2000)**: si un conjunto de datos excluye nombre, número de seguridad social y dirección, pero conserva **fecha de nacimiento, sexo y código postal**, se puede reidentificar de forma única al **87 % de la población estadounidense**. Sweeney lo demostró reidentificando el historial médico del **gobernador de su estado** cuando ingresó en un hospital.
- **Premio Netflix**: se publicaron valoraciones de películas anonimizadas para un concurso de recomendación. Narayanan y Shmatikov (2006) reidentificaron usuarios **cruzando la fecha de una valoración en Netflix con la fecha de una valoración parecida en IMDb**, donde mucha gente usa su nombre real.

De ahí sale una escalera de técnicas, de la más simple a la más robusta:

Técnica	En qué consiste	Límite
Generalizar campos	Sustituir la fecha de nacimiento por el año, o por un rango «20-30 años»; eliminar un campo es generalizar a «cualquiera»	No garantiza nada por sí sola: puede haber **una sola persona** de 90-100 años en un código postal
k-anonimato	Una base de datos es *k*-anónima si **cada registro es indistinguible de al menos k-1 otros**. Los registros más únicos se generalizan más	Protege frente a una consulta, no frente a un atacante con datos externos

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Técnica	En qué consiste	Límite
Consultas agregadas	No se comparten registros: se ofrece una API que responde recuentos o medias, y **no responde** si eso violaría la privacidad (por ejemplo, códigos postales con menos de *n* personas)	Vulnerable al **ataque de consultas múltiples**
Privacidad diferencial	La respuesta se calcula con un algoritmo aleatorio que **añade un poco de ruido**. Garantiza que participar o no en la base de datos **no cambia apreciablemente** ninguna respuesta	Hay que gastar «presupuesto de privacidad»: a más consultas, más ruido
Aprendizaje federado	**No hay base de datos central**: cada usuario entrena en local y solo comparte **parámetros** del modelo, nunca datos	Los parámetros en bruto pueden filtrar información
Agregación segura (Bonawitz et al., 2017)	Cada usuario **enmascara** sus parámetros con un valor propio; como las máscaras suman cero, el servidor solo puede calcular la **media**	Añade complejidad de protocolo

Nota sobre la tabla anterior.

El ataque de consultas múltiples, con números

Preguntas «salario medio y número de empleados de la empresa XYZ, de 30 a 40 años» y te responde **81.234,12 €**. Preguntas «de 30 a **41** años» y te responde **81.199,13 €**. Las dos consultas han implicado a 12 o más personas, así que parecen seguras.

Pero con LinkedIn encuentras al único empleado de 41 años de XYZ... y de la diferencia entre las dos medias **despejas su salario exacto**.

Defensa: limitar las consultas posibles (solo rangos de edad predefinidos que no se solapen) **y** reducir la precisión de las respuestas (que las dos digan «alrededor de 81.000 €»).

Cómo se ve el aprendizaje federado desde dentro

Imagina una aplicación de reconocimiento de voz en el móvil. Trae una red base y **mejora entrenando en local** con lo que oye en tu teléfono. Periódicamente, quien mantiene la aplicación pregunta a un subconjunto de usuarios **los valores de los parámetros** de su red local, nunca los datos. Los combina en un modelo mejor y lo distribuye a todos: cada usuario se beneficia del entrenamiento de los demás **sin que nadie vea sus datos**.

8.3 Evaluaciones de impacto

- **EIPD / DPIA** (art. 35 RGPD): obligatoria cuando el tratamiento entraña alto riesgo, por ejemplo IA con datos de salud o con decisión automatizada.
- **FRIA** (Evaluación de Impacto en los Derechos Fundamentales, AI Act): evalúa el impacto **social y ético** más allá del dato, exigida para sistemas de alto riesgo.

9. Sesgos de género y algorítmicos (RA6-f)

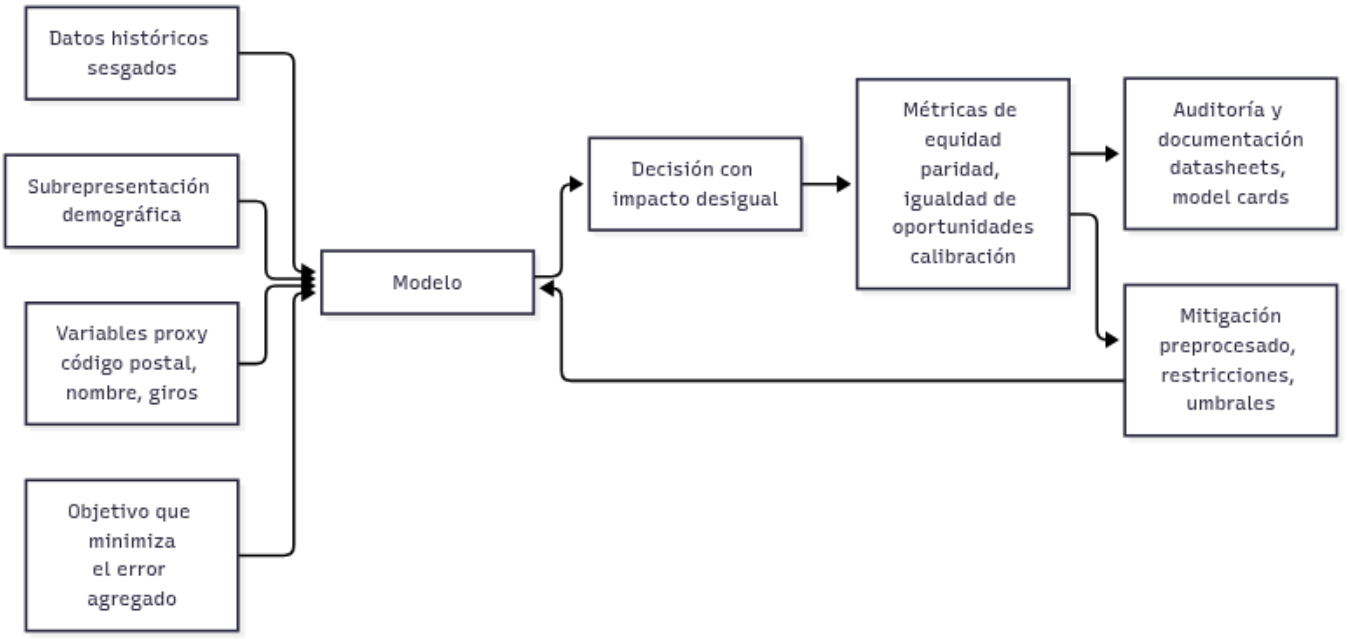
9.1 Qué es y de dónde viene

El **sesgo algorítmico** es una **distorsión sistemática** que produce un trato desfavorable hacia grupos protegidos. No es un error aleatorio: es **reproducible** y **dañino**. Sus fuentes:

- **Datos históricos sesgados**: mediciones sesgadas, decisiones humanas sesgadas, informes erróneos. El aprendizaje automático está diseñado, literalmente, **para replicarlos**.
- **Datos faltantes o sesgo de selección**: el conjunto no representa a la población objetivo.
- **Objetivos algorítmicos**: minimizar el error **agregado** beneficia a los grupos mayoritarios frente a las minorías. Aunque no haya ningún prejuicio social, **la disparidad en el tamaño de la muestra basta**: hay menos ejemplos de las clases minoritarias, y más datos significa más precisión.
- **Variables proxy** (variables relacionadas con atributos sensibles): el código postal, la ocupación, el nombre o los giros lingüísticos pueden **revelar el género o la raza** sin que esas variables estén en el modelo.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- **El propio proceso de desarrollo**: quien depura un sistema detecta y corrige antes los problemas **que le afectan**. Es difícil darse cuenta de que una interfaz no funciona para personas daltónicas si no lo eres, o de que una traducción al urdu es defectuosa si no hablas urdu.



Datos reales de sesgo

- **Embeddings** (Bolukbasi, 2016): `hombre:programador :: mujer:ama de casa`. El aprendizaje automático no solo refleja estereotipos: los **amplifica**.
- **Gender Shades** / Buolamwini y Gebru (2018): precisión casi perfecta en hombres de piel clara y **33 % de error** en mujeres de piel oscura.
- **NIST FRVT** (2019): falsos positivos **×10-100** entre grupos demográficos.
- **Traducción automática** (Prates, 2019): las ocupaciones STEM se traducían en masculino un **71,6 %** de las veces.
- **Diversidad de quien construye** (AI Now Institute, 2019): **18 %** de autoras en las grandes conferencias de IA, **20 %** de profesoras, **menos del 4 %** de personas negras en plantillas de IA. Y **18 %** de graduadas en informática en EE. UU. — aunque **Harvey Mudd** alcanzó la **paridad del 50 %** centrándose en animar y retener a quien empieza sin experiencia previa.

9.2 «Justo» significa seis cosas distintas

Antes de medir hay que decidir **qué** se mide, y aquí está el nudo de todo el bloque: no hay una definición de justicia, hay varias, y **no son compatibles entre sí**.

Definición	Qué exige	Problema
Justicia individual	Tratar de forma parecida a individuos parecidos, sea cual sea su clase	No dice nada de los agregados
Equidad de grupo	Tratar de forma parecida a dos clases, según algún estadístico resumen	No garantiza justicia individual
Equidad por desconocimiento	Quitar del conjunto de datos los atributos de raza y género	**No funciona**: el modelo predice variables latentes desde otras correlacionadas (§9.5). Y además **impide comprobar** si hay igualdad de oportunidades. Aun así, algunos países lo eligen para sus estadísticas
Igual resultado (paridad demográfica)	Cada clase obtiene los mismos resultados: mismo porcentaje de préstamos aprobados a hombres y mujeres	Es equidad de grupo, no individual: puede rechazar a alguien cualificado y aprobar a alguien que no lo está. Prioriza corregir el sesgo pasado sobre la precisión

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Definición	Qué exige	Problema
Igualdad de oportunidades	Quien realmente puede devolver el préstamo tiene la misma probabilidad de ser clasificado como tal, sea cual sea su sexo	Puede dar **resultados desiguales** e ignora el sesgo de los procesos sociales que generaron los datos
Igual impacto	Personas con similar probabilidad de devolver el préstamo tienen la **misma utilidad esperada**	Va más allá: pesa el beneficio del acierto **y el coste del error**. Pero calcular esos costes es difícil

Nota sobre la tabla anterior.

El dilema, en una frase

Un hombre y una mujer son iguales en todo, salvo que ella **cobra menos por el mismo trabajo**. ¿Se le aprueba el préstamo porque sería igual de no ser por un sesgo histórico, o se le deniega porque cobrar menos la hace objetivamente más propensa a impagar?

No hay respuesta técnica. Hay una decisión, y alguien tiene que tomarla y justificarla.

9.3 Métricas de equidad

Métrica	Pregunta que responde
Paridad demográfica	¿La tasa de selección es similar entre grupos?
Igualdad de oportunidades	¿La tasa de verdaderos positivos es similar entre grupos?
Calibración	¿La probabilidad predicha significa lo mismo para todos los grupos?

Herramientas de detección y mitigación: **Fairlearn** (Microsoft), **AIF360** (IBM) y **What-If Tool** (Google). Técnicas: **preprocesado** (reequilibrar el conjunto; sobremuestreo con **SMOTE** o **ADASYN** para la disparidad de tamaño de muestra), **en el modelo** (restricciones de equidad), **postprocesado** (ajuste de umbrales por grupo) y **auditoría** con documentación (*datasheets* de los conjuntos, *model cards* de los modelos: procedencia, seguridad, conformidad y aptitud para el uso, como la hoja de características de una resistencia).

9.4 COMPAS: por qué no puedes tener las dos cosas

COMPAS es un sistema comercial de puntuación de reincidencia. Asigna al acusado de un caso penal una puntuación de riesgo que el juez usa para decidir si lo libera antes del juicio, cuánto debe durar la condena o si concede la libertad condicional. Míralo con las definiciones del §9.2 en la mano:

Criterio	Resultado de COMPAS
Calibración	**La cumple**: entre las personas con puntuación 7 sobre 10, reincide el **60 % de las blancas y el 61 % de las negras**. Sus diseñadores lo alegaron como prueba de equidad
Igualdad de oportunidades	**No la cumple**: entre quienes **no** reincidieron, se marcó falsamente como de alto riesgo al **44,9 % de las personas negras y al 23,5 % de las blancas**

Nota sobre la tabla anterior.

El resultado de imposibilidad

Podríamos querer un algoritmo **bien calibrado y con igualdad de oportunidades** a la vez. **Kleinberg et al. (2016) demostraron que es imposible** cuando las tasas base de los grupos son distintas: cumplir uno de los criterios te impide cumplir el otro.

Esto es importante para el debate: cuando ProPublica denunció COMPAS midiendo **paridad de error** y Northpointe respondió alegando **calibración**, los dos tenían razón. No era una discusión sobre los datos: era una discusión sobre **qué definición de justicia** aplicar, y esa no la resuelve la estadística.

Hay tres problemas más, independientes del modelo:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

1. **No existe una verdad de referencia imparcial.** Los datos no dicen quién **cometió** un delito: dicen quién fue **condenado**. Si la policía que detiene, el juez o el jurado están sesgados, los datos lo están. Si hay más patrullas en unos barrios, los datos están sesgados contra quien vive ahí. Y solo quien es liberado puede volver a ser condenado, así que un sesgo en las decisiones de libertad **sesga la muestra**.
2. **El sistema sirve para justificar el sesgo humano.** Si una persona sesgada decide *después* de consultar al modelo, puede decir «mi interpretación del modelo respalda mi decisión, así que no la cuestiones» — cuando otra interpretación llevaría a lo contrario.
3. **La opacidad choca con el derecho de defensa.** En el caso **Estado contra Loomis**, el acusado alegó que el funcionamiento secreto del algoritmo violaba su derecho al debido proceso. El Tribunal Supremo de Wisconsin resolvió que la condena no habría sido distinta sin COMPAS, pero **emitió advertencias** sobre la precisión del algoritmo y los riesgos para los acusados de minorías.

A veces lo que hay que cambiar es el objetivo, no los datos

En una selección de personal, si el objetivo es «contratar a los candidatos con las mejores calificaciones», se premia a quien ha tenido mejores oportunidades educativas toda su vida: el modelo **refuerza las fronteras de clase**. Si el objetivo pasa a ser «contratar a los candidatos con mejor capacidad de aprender en el puesto», el grupo del que se elige es más amplio. Las empresas que lo han probado encuentran que, tras un año de formación, esas contrataciones rinden igual que las tradicionales.

9.5 Variables proxy y la paradoja de Simpson

Los dos mecanismos más difíciles de ver son también los más frecuentes.

La variable proxy perfecta · Obermeyer et al., *Science*, 2019

Un algoritmo de gestión sanitaria usado en EE. UU. sobre millones de pacientes decidía a quién derivar a programas de atención adicional. No usaba la raza. Usaba el **coste sanitario previsto** como aproximación de la **necesidad de salud**.

Pero en EE. UU. **se gasta menos en pacientes negros con la misma enfermedad**, por desigualdad de acceso. Así que el algoritmo les asignaba **menos riesgo estando igual de enfermos**. Corregir el sesgo elevaría el porcentaje de pacientes negros derivados a atención adicional del **17,7 % al 46,5 %**.

Nadie programó nada racista. Se eligió un **proxy cómodo y aparentemente objetivo** para algo que no se sabía medir directamente. Es, probablemente, la fuente de sesgo algorítmico más común que te vas a encontrar en la vida profesional.

La paradoja de Simpson · admisiones de Berkeley, 1973

En otoño de 1973, la Graduate Division de la Universidad de California en Berkeley resolvió **12.763 solicitudes** repartidas en **101 departamentos**. En el agregado:

Grupo	Solicitantes	Tasa de admisión
Hombres	8.442	**44,2 %**
Mujeres	4.321	**34,6 %**

La diferencia era estadísticamente significativa: parecía discriminación clara. Pero al analizar **departamento por departamento**, la diferencia dejaba de ser significativa, y en la mayoría la tasa de admisión femenina era **igual o superior** a la masculina (Bickel, Hammel y O'Connell, *Science*, 1975).

La explicación: las mujeres solicitaban en mayor proporción los **departamentos más competitivos**, donde la tasa de admisión es baja **para todo el mundo**, y los hombres los menos competitivos. El sesgo no estaba en el comité de admisiones: estaba **antes**, en la sociedad.

La lección técnica: una tendencia observada en los subgrupos puede **invertirse** al agregarlos. Si solo mides la métrica global, no ves nada. Por eso el §9.3 insiste en **medir por subgrupo**.

9.6 Sesgos de género en la interacción con agentes

Hay un sesgo que no está en el modelo, sino en **quien lo usa**. Los desarrolladores dan con frecuencia rasgos antropomórficos a los agentes, porque a mucha gente le incomoda hablar con una máquina. Y la investigación sobre relaciones interpersonales

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

muestra que **tratamos a los agentes inteligentes con las mismas normas** que aplicamos a las personas: los mismos estilos de diálogo y de procesamiento de información.

La consecuencia es previsible. La investigación sobre estereotipos indica que se asocia con más frecuencia a las mujeres con la **calidez** y a los hombres con la **competencia** (Fiske, Cuddy y Glick). Trasladado a los asistentes, hay estudios que encuentran que la capacidad de persuasión de una recomendación **varía con el género aparente del agente**: para productos utilitarios se confiaba más en agentes de apariencia masculina, y para productos hedónicos ocurría lo contrario.

Por qué esto es RA6-f y no una curiosidad

El criterio pide identificar y corregir sesgos de género **en el desarrollo y las aplicaciones**. Elegir la voz, el nombre y la apariencia de un asistente **es una decisión de desarrollo**, y esa decisión activa estereotipos medibles en los usuarios. No basta con auditar el modelo: hay que auditar también **cómo se presenta**.

9.7 Cómo defenderse: prácticas que funcionan

La primera defensa es **conocer los límites de los datos que usas**. A partir de ahí, hay un conjunto de buenas prácticas bastante consolidado (aunque no siempre se siga):

- Que quien desarrolla **hable con especialistas del dominio y de ciencias sociales**, y que la equidad se considere **desde el principio**.
- Fomentar equipos **diversos**: es más fácil detectar un problema en los datos si alguien del equipo lo sufre.
- **Definir explícitamente qué grupos** soporta el sistema: hablantes de qué lenguas, qué franjas de edad, qué capacidades visuales y auditivas.
- **Optimizar una función objetivo que incorpore la equidad**, no añadirla después.
- **Examinar los datos** buscando prejuicios y correlaciones entre atributos protegidos y el resto.
- Entender **cómo se hizo el etiquetado humano**, fijar objetivos de precisión y comprobar que se cumplen.
- **No mirar solo las métricas globales**: seguir las métricas **por subgrupo** (§9.5).
- Incluir **pruebas del sistema que reflejen la experiencia de usuarios de grupos minoritarios**.
- Tener un **circuito de retroalimentación** para que los problemas de equidad que aparezcan se resuelvan.

9.8 La paradoja de los sesgos y la Ley 15/2022

Para **detectar** el sesgo de género (RA6-f) hace falta tratar el dato «género»… que es exactamente lo que la **minimización** del RGPD (RA6-b) manda no tratar. Es la **paradoja de los sesgos**, y es la tensión más real de esta unidad.

Salidas prácticas: auditar con **variables proxy** (comprobar si el modelo discrimina por giros lingüísticos o códigos postales), auditar en **entornos controlados** con datos de prueba específicos, o tratar el atributo sensible **solo para la auditoría** y no en producción, con base legal y plazo definidos.

Ley 15/2022 (España)

La Ley 15/2022, de igualdad de trato y no discriminación, establece que **las administraciones deben favorecer algoritmos que minimicen los sesgos** y permite **invertir la carga de la prueba** en procesos de discriminación algorítmica: es **el responsable del sistema quien debe demostrar que no discrimina**, no la víctima quien debe demostrar que sí.

Enlaza directamente con la **responsabilidad proactiva** del RGPD (§5.2): si no puedes demostrar que tu sistema no discrimina, a efectos legales es como si discriminara.

10. Puntos clave de la unidad

- La IA conlleva **riesgos** medibles con casos reales: COMPAS, Amazon, Gender Shades, Google Photos, Uber en Tempe.
- Los errores del aprendizaje automático **no se depuran como los del software clásico**: el comportamiento vive en los datos, no en el código.
- La **deontología** (ACM, IEEE EAD, UNESCO, Directrices de la UE) exige una **ingeniería basada en valores**: sistemas **lícitos, éticos y robustos** durante todo el ciclo de vida.
- Los grandes listados de principios éticos son **demasiado vagos para exigirlos**: hacen falta pautas por subcampo.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- El **RGPD/LOPDGDD** protege los datos personales: minimización, ARSULIPO, decisiones automatizadas (art. 22), sanciones de hasta 20 M€ o el 4 %, y una carta de **derechos digitales** con plazos concretos.
- El **AI Act** clasifica por riesgo (prohibido, alto, limitado, mínimo) y **ya se aplica** con carácter general desde el **2 de agosto de 2026**.
- La **responsabilidad** por daños se canaliza por la **PLD revisada (2024/2853)**; la AILD se retiró.
- La **transparencia** tiene tres capas: V&V, certificación (UL, ISO 26262, IEEE P7001) y explicación (XAI). **Interpretable ≠ explicable**, y una explicación **no sustituye a una auditoría**.
- **Security by design**: seguridad en todo el ciclo de vida, ataques típicos y supervisión humana. **Safety** es distinto de **security**: FMEA, FTA, agentes de bajo impacto y alineación de valores.
- **Privacy by design** (art. 25 RGPD): minimizar, abstraer, separar, ocultar; y las técnicas reales —k-anonimato, privacidad diferencial, aprendizaje federado, agregación segura— porque **desidentificar no basta** (Sweeney: 87 %).
- **Sesgos**: «justo» significa **seis cosas distintas** e incompatibles; **Kleinberg et al. (2016)** demuestra que calibración e igualdad de oportunidades **no caben juntas**; las **variables proxy** (Obermeyer) y la **paradoja de Simpson** (Berkeley) son los mecanismos más difíciles de ver; y la **Ley 15/2022** invierte la carga de la prueba.

11. Glosario

Término	Definición
Deontología	Parte de la ética que trata los deberes que rigen una actividad profesional
IA fiable	La que es **lícita, ética y robusta** durante todo su ciclo de vida (Directrices UE, 2019)
Brecha de responsabilidad	Dificultad de asignar culpa ante errores de sistemas opacos
Dato personal	Información sobre una persona física identificada o identificable
Tratamiento	Cualquier operación sobre datos personales (recogida, almacenamiento, uso)
Responsable	Quien decide los fines y medios del tratamiento
Encargado	Quien trata datos por cuenta del responsable
Minimización	Tratar solo los datos estrictamente necesarios
Responsabilidad proactiva	Obligación de **poder demostrar** el cumplimiento, no solo de cumplirlo
ARSULIPO	Acceso, Rectificación, Supresión, Limitación, Portabilidad, Oposición
Decisión automatizada	Decisión sin intervención humana que afecta a una persona (art. 22 RGPD)
Derecho al olvido	Derecho a que se eliminen enlaces y datos personales inadecuados o no pertinentes
Desconexión digital	Derecho a desconectar de los dispositivos de trabajo fuera de la jornada (LOPDGDD)
Lista Robinson	Fichero español de exclusión publicitaria que las campañas deben consultar
DPO	Delegado de Protección de Datos
EIPD / DPIA	Evaluación de Impacto en Protección de Datos (art. 35 RGPD)
FRIA	Evaluación de Impacto en Derechos Fundamentales (AI Act)
AI Act	Reglamento UE 2024/1689: marco jurídico de la IA por nivel de riesgo
Sistema de alto riesgo	IA que afecta a derechos (RRHH, crédito, salud, justicia) con obligaciones estrictas
Transparencia (art. 50)	Informar de que se interactúa con una IA y marcar el contenido generado
GPAI	Modelo de IA de uso general, con obligaciones propias en el AI Act
PLD	Directiva de Responsabilidad por Productos Defectuosos (2024/2853), aplicable al software de IA
V&V	Verificación (cumple la especificación) y validación (la especificación es la correcta)
XAI	IA explicable: sistema capaz de justificar sus decisiones
Interpretable	Se puede inspeccionar el modelo y ver qué hace

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Término	Definición
Explicable	Se puede construir un relato de lo que hace, aunque sea una caja negra
Ley de la bandera roja	Principio de que un sistema autónomo debe identificarse como tal (Walsh, 2015)
Arma letal autónoma	La que localiza, selecciona y ataca objetivos humanos sin supervisión humana
Doble uso	Tecnología con aplicaciones pacíficas y militares a la vez
Security by design	Seguridad frente a ataques integrada en todo el ciclo de vida
Safety	Que el sistema no cause daño por sí mismo, por fallo previsible u objetivo mal puesto
Adversarial example	Entrada modificada de forma imperceptible que engaña al modelo
Envenenamiento de datos	Adulteración del conjunto de entrenamiento
Extracción de modelos	Replicar la lógica de un modelo mediante consultas sistemáticas
Prompt injection	Manipular la entrada de un sistema generativo para saltarse sus controles
FMEA	Análisis de modos de fallo y efectos: recorrer cada componente e imaginar cómo puede fallar
FTA	Análisis por árbol de fallos: árbol Y/O con probabilidades por causa raíz
Agente de bajo impacto	El que maximiza la utilidad **menos** los cambios que provoca en el mundo
Specification gaming	Maximizar la métrica sin resolver el problema que se pretendía resolver
Alineación de valores	Problema de asegurar que lo que pedimos es lo que queremos («problema del rey Midas»)
Externalidad	Efecto que queda fuera de lo que se mide y se paga
Tragedia de los comunes	Sobreexplotación de un recurso compartido por no internalizar su coste
Privacy by design	Privacidad desde el diseño y por defecto (art. 25 RGPD)
Seudonimización	Sustitución de identificadores por códigos
Desidentificación	Eliminar la información identificativa de un conjunto de datos
Reidentificación	Volver a identificar a personas en datos desidentificados cruzando otras fuentes
k-anonimato	Cada registro es indistinguible de al menos *k-1* registros más
Privacidad diferencial	Añadir ruido aleatorio de modo que participar o no en la base no cambie las respuestas
Aprendizaje federado	Entrenar en local y compartir solo parámetros, sin base de datos central
Agregación segura	Enmascarar los parámetros de cada usuario para que el servidor solo obtenga la media
Sesgo algorítmico	Distorsión sistemática que perjudica a un grupo protegido
Atributo sensible	El que distingue grupos privilegiados y no privilegiados (raza, género, edad)
Variable proxy	Variable no sensible fuertemente relacionada con una que sí lo es
Equidad por desconocimiento	Quitar los atributos protegidos del conjunto de datos; no funciona
Paridad demográfica	Igualdad de tasas de selección entre grupos
Igualdad de oportunidades	Igualdad de verdaderos positivos entre grupos
Calibración	La probabilidad predicha significa lo mismo en todos los grupos
Igual impacto	Misma utilidad esperada, pesando el beneficio del acierto y el coste del error
Paradoja de Simpson	Una tendencia en los subgrupos puede invertirse al agregarlos
SMOTE / ADASYN	Técnicas de sobremuestreo sintético de clases minoritarias
Fairlearn / AIF360	Bibliotecas de medición y mitigación de sesgos

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Término	Definición
Datasheet / Model card	Documentación del conjunto de datos y del modelo
Paradoja de los sesgos	Para auditar el sesgo de género hay que tratar el dato que el RGPD manda minimizar

12. FAQ

¿Un sistema de IA puede decidir sobre mí sin intervención humana?

En general no: el **art. 22 del RGPD** reconoce el derecho a no ser objeto de decisiones individuales automatizadas sin intervención humana significativa. En el **AI Act**, los sistemas de alto riesgo exigen además **supervisión humana** y un mecanismo de parada.

¿Qué pasa si un sistema de IA discrimina?

Puede infringir el RGPD (decisiones automatizadas), el AI Act (calidad de datos y no discriminación) y la **Ley 15/2022**, que permite **invertir la carga de la prueba**: es el responsable del sistema quien debe demostrar que no discrimina.

Si quito la variable «género» del modelo, ya no puede discriminar, ¿no?

No. Eso es la **equidad por desconocimiento** y **no funciona**: el modelo predice el género o la raza a partir de variables correlacionadas —código postal, ocupación, nombre, giros lingüísticos— y decide implícitamente con ellas. Además, al quitar el atributo **te quedas sin poder comprobar** si hay discriminación. El caso de Obermeyer (§9.5) es exactamente esto: sin usar la raza, el algoritmo discriminaba por raza.

Entonces, ¿cuál de las seis definiciones de justicia es la correcta?

Ninguna en abstracto. **Kleinberg et al. (2016)** demostraron que, con tasas base distintas, no se puede cumplir a la vez la calibración y la igualdad de oportunidades. La decisión es **del proyecto**, hay que **documentarla y justificarla**, y hay que poder explicar a quién perjudica. Lo que no es aceptable es no haber elegido.

¿Los datos anonimizados son seguros para entrenar IA?

No por sí solos. Con fecha de nacimiento, sexo y código postal se reidentifica al **87 %** de la población estadounidense (Sweeney, 2000), y el Premio Netflix se reidentificó cruzando fechas con IMDb. Hay que subir por la escalera del §8.2: generalizar, **k-anonimato**, **privacidad diferencial**, y si es posible **aprendizaje federado** con agregación segura.

¿Puedo usar cualquier dataset de internet para entrenar?

No. Hay que verificar la **licencia** del conjunto, la **base legal** del tratamiento (RGPD) y la **limitación de la finalidad**: los datos recogidos para una cosa no valen automáticamente para entrenar otra. Para modelos de uso general, el AI Act obliga además a publicar resúmenes de los contenidos protegidos usados.

¿El derecho al olvido se aplica a los modelos de IA?

Técnicamente es complejo: **borrar la influencia de un solo dato** puede exigir un **reentrenamiento completo**, inviable para muchas organizaciones. Es un problema abierto entre lo que el RGPD exige y lo que la práctica permite, y una de las razones por las que conviene minimizar **antes** de entrenar.

¿Detectar sesgos de género contradice la minimización de datos?

Es la **paradoja de los sesgos** (§9.8). La salida es auditar con **variables proxy** o en **entornos controlados** de prueba, o tratar el atributo sensible **solo para auditar**, con base legal y plazo definidos, y no en producción.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Mi código pasa todos los tests. ¿No es eso software seguro?

Es software **correcto**, que no es lo mismo. La seguridad exige que **la especificación misma** haya considerado los modos de fallo y que el sistema **degrade con elegancia** ante lo imprevisto. Para eso están **FMEA** y **FTA** (§7.3). Y aunque el código sea perfecto, un **objetivo mal puesto** produce un sistema inseguro (§7.4).

¿Hay que preocuparse por la superinteligencia y la singularidad?

Es un debate real —Bill Gates, Elon Musk, Stephen Hawking y Martin Rees han advertido del riesgo—, pero no es lo que este módulo evalúa, y conviene ponerlo en perspectiva: hasta ahora todas las tecnologías han seguido una **curva en S** cuyo crecimiento exponencial acaba frenándose, y muchos avances necesitan **actuar en el mundo físico**, no solo pensar más rápido. Los problemas que sí tienes delante como profesional son los de los §7, §8 y §9.

¿Los robots deberían tener derechos?

Es una discusión abierta que depende de la **conciencia**: sin conciencia ni *qualia*, pocos defienden que merezcan derechos; si pudieran sentir dolor o temer la muerte, hay quien argumenta que sí (Sparrow, 2004). El contraargumento más útil para el aula es el de Weizenbaum y Ernie Davis: conceder personalidad a un robot es **una forma de no asumir la responsabilidad** de lo que hace nuestra propia herramienta — *«no es mi culpa, el coche lo hizo solo»*. Que es exactamente lo que la **brecha de responsabilidad** del §4.1 y la **PLD** del §6.2 intentan evitar.

13. Sesiones

La unidad tiene **6 horas** repartidas en un tramo fragmentado por Fallas y Pascua. Los documentales de partida de los dos debates se ven **fuera de clase**, y la reflexión escrita de cada debate es trabajo personal.

Semana	Horas	Contenido	CE
24	1,5	Riesgos, deontología y principios éticos (§4). El futuro del trabajo. **Preparación del debate 1** y sorteo de roles	RA6-a
25	1,5	**Debate 1 · Límites éticos de la IA** (10 roles + observadores críticos)	RA6-a, RA6-c
27	1,5	Normativa como lectura guiada con checklist: RGPD y LOPDGDD (§5), AI Act y responsabilidad (§6), *security* y *privacy by design* (§7-§8). Taller de auditoría de sesgos	RA6-b, RA6-c, RA6-d, RA6-e
28	1,5	**Debate 2 · El algoritmo contra el crimen** (9 roles + observadores críticos), con COMPAS y la paradoja de Simpson. Cierre de la unidad	RA6-f

Nota sobre la tabla anterior.

Aviso de planificación

Con **dos debates evaluables**, los cuatro criterios normativos (RA6-b, c, d, e) comparten una sola sesión y se apoyan mucho en el trabajo autónomo. Está previsto **revisar el reparto de horas entre unidades y RA** antes de que empiece el curso.

14. Recursos

- Diapositivas
- **Práctica** — se hace, no se entrega ni puntúa:
  - Ejercicios de autoevaluación
  - Documentales, noticias y notebooks — los documentales que se ven antes de cada debate, las noticias de actualidad y los dos notebooks guiados: `N01` de sesgos y `N02` de análisis de un caso ético
- **Entregas** — tres, qué se entrega:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

  - N03 · Auditoría de sesgos con Fairlearn
  - **Debates por roles**, los dos con la misma rúbrica: D01 · Límites éticos de la Inteligencia Artificial · D02 · El algoritmo contra el crimen
- Los notebooks se abren desde **Práctica** y **Entregas**, con descarga y apertura en Colab.

Referencias de la unidad

Normativa (texto consolidado):

- RGPD · Reglamento (UE) 2016/679
- AI Act · Reglamento (UE) 2024/1689
- LOPDGDD · Ley Orgánica 3/2018
- Ley 15/2022, de igualdad de trato y no discriminación
- Comisión Europea · marco regulador de la IA y calendario

Guías y marcos éticos:

- AEPD · Guía de privacidad desde el diseño
- UNESCO · Recomendación sobre la ética de la IA (2021)
- ACM Code of Ethics
- IEEE · Ethically Aligned Design

Casos y estudios citados:

- ProPublica · *Machine Bias* (COMPAS)
- Gender Shades (MIT Media Lab)
- Obermeyer et al. · *Dissecting racial bias in an algorithm…* (Science, 2019)
- Frey y Osborne · *The Future of Employment*
- Human Rights Watch · armas autónomas y el plazo de 2026
- UNODA · Grupo de Expertos Gubernamentales sobre LAWS

Herramientas:

- Fairlearn · AI Fairness 360 (IBM)

Bibliografía de referencia: Russell, S. y Norvig, P., *Artificial Intelligence: A Modern Approach*, 4.ª ed., Pearson, 2021 — **capítulo 27, «Philosophy, Ethics, and Safety of AI»**, del que provienen los bloques de armas autónomas, vigilancia y reidentificación, equidad y parcialidad, confianza y transparencia, futuro del trabajo, derechos de los robots y seguridad de la IA. Los datos numéricos que aparecen en esos bloques se han **actualizado y verificado** para este curso; cuando no ha sido posible, se indica el año del dato.

15. Evaluación

Peso	Instrumento
40 % actividades	**Debate 1** y **Debate 2**, con la rúbrica de debate (5 criterios, 100 puntos, escalados sobre 10), y el taller **N03** de auditoría de sesgos. Los dos notebooks guiados, `N01` de sesgos y `N02` de análisis de un caso ético, son práctica y no puntúan
60 % prueba escrita	Prueba del RA6 en Moodle: preguntas de test y de desarrollo sobre el contenido de la unidad

- **La normativa exige alcanzar todos los RA** del módulo para superarlo (art. 5.1 de la Orden 8/2025: la calificación del módulo está *«en función de la consecución de los RA»*; y las Instrucciones 26-27, que impiden calificar positivamente un módulo con RA no superados). El centro concreta ese mandato exigiendo **≥ 5 en cada RA**.
- La reflexión escrita individual de cada debate (máximo **300 palabras**) vale **30 de los 100 puntos** de la rúbrica: no es un trámite.

16. Recuperación

Si no superas el RA6, se activa un **programa de recuperación individual** (art. 14.4 de la Orden 8/2025) con actividades y criterios específicos: análisis escrito de un caso distinto —con hechos, partes afectadas, principios en conflicto, normativa

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

aplicable y propuesta de corrección—, el checklist de *privacy* y *security by design*, y la prueba de recuperación del RA. **En junio solo hay recuperación de los RA no superados**, no una prueba global.

1 de septiembre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

8.2 Diapositivas de la UD06



Diapositivas PDF

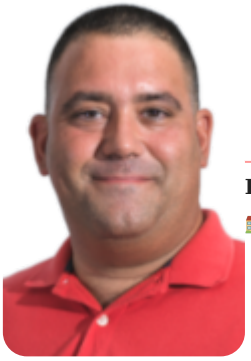
Cómo se manejan

Flechas o clic para avanzar · **F** para verlas a pantalla completa · **P** para las notas del ponente.

24 de agosto de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

9. 👋 Sobre mí



9.1 David Martínez Peña

Profesor de Secundaria, especialidad de Informática.

🏫 Actualmente con destino en IES Eduardo Primo Marqués de Carlet.

9.2 📬 Contacto

📧 Email

d.martinezpena@edu.gva.es

▶️ YouTube

youtube.com/@martinezpenya

💼 LinkedIn

linkedin.com/in/martinezpenya

🐙 GitHub

github.com/martinezpenya

24 de agosto de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

10. Fuentes de información

Fuentes citadas a lo largo del módulo, agrupadas por unidad. Las generales se usan en más de una unidad o no son específicas de ninguna; cada bloque de unidad reproduce las referencias que ya aparecen al final de su página de teoría.

10.1 Generales del curso

- Wikipedia
- ChatGPT
- DeepSeek
- Claude Code (Anthropic) — usado en la redacción, investigación y control de calidad de este módulo (fusión de contenidos, ejercicios, talleres, diapositivas y scripts de verificación)
- Modelos de Inteligencia Artificial (Ed. Marcombo)
- Internet Encyclopedia of Philosophy · Artificial Intelligence
- Materiales MIA curso MEC-20230524
- Gartner · previsión de inversión en IA (2021)
- RapidAPI · Top Facial Recognition APIs
- ScienceDirect, artículo S0212656720301463
- Models d'IA de Carles Gonzalez, sotmés a la llicència Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International
- JetBrains

10.2 UD00 — Presentación y Docker

Documentación oficial de Docker:

- What is Docker?
- What is a container?
- Dockerfile reference
- Compose Quickstart
- Building best practices

Normativa y plataforma:

- RD 279/2021 — currículo del curso de especialización
- Orden 8/2025 de la Comunitat Valenciana — evaluación
- Aules GVA — plataforma del centro

10.3 UD01 — Caracterización de sistemas de IA

- IBM · ¿Qué es la IA?
- IBM · ¿Qué es el machine learning?
- IBM · ¿Qué es el PLN?
- scikit-learn · Aprendizaje supervisado
- Parlamento Europeo · ¿Qué es la IA y cómo se usa?
- AI Act (UE 2024/1689)
- Russell y Norvig · AIMA (sitio oficial)
- Arend Hintze · «Understanding the four types of AI» (The Conversation, 2016)
- Vaswani et al. · Attention Is All You Need (2017)

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

10.4 UD02 — Modelos de IA y resolución de problemas

- Red Blob Games · A*
- Wikipedia · A*
- scikit-fuzzy (pythonhosted)
- experta (readthedocs)
- CLIPS
- IBM · RPA
- IBM · Automatización inteligente
- scikit-learn · Elegir el estimador
- arXiv · Tree-based vs DL en tabulares
- Robocode Tank Royale · documentación oficial

10.5 UD03 — Procesamiento del lenguaje natural

- Introducción al Procesamiento del Lenguaje Natural (PLN), Ferran Pla, 2017
- *Speech and Language Processing* — Jurafsky y Martin (acceso libre)
- *Introduction to Natural Language Processing* — Eisenstein
- NLTK Book · spaCy · Usage · Hugging Face
- Bender y Koller (2020), *Climbing towards NLU*
- Meta NLLB · Masakhane
- Reglamento (UE) 2024/1689 · AI Act — considerando 53 y art. 5
- SQuAD · Penn Treebank POS tags

Notebooks:

- spaCy · Modelos
- Analytics Lane · Análisis de sentimientos en español con spaCy

10.6 UD04 — Robótica

- *Artificial Intelligence: A Modern Approach*, 4.ª ed., cap. 26 — Stuart Russell y Peter Norvig
- Models d'IA — Carles Gonzalez (CC BY-NC-SA 4.0)
- IFR World Robotics
- roboticstoolbox-python · spatialmath-python
- AITK · OpenCV · NEAT-Python
- Wikipedia · Parámetros de Denavit-Hartenberg · Wikipedia · SLAM · Wikipedia · Localización de Monte Carlo
- Universal Robots · Webots · CoppeliaSim

10.7 UD05 — Sistemas expertos y lógica difusa

- Wikipedia · Expert system
- Wikipedia · MYCIN
- Wikipedia · XCON
- experta · readthedocs
- CLIPS · clipsrules.net
- Drools / Apache KIE
- Human-Learn
- imodels (FIGS)
- skope-rules
- Wikipedia · Lógica difusa
- Wikipedia · Fuzzy control system

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- Wikipedia · PID controller
- Modus Ponens · Modus Tollens

Notebooks:

- experta · Introduction
- CLIPS · User's Guide (v6.24)
- Cayetano Guerra · Tutorial CLIPS I

10.8 UD06 — Riesgos, ética y legalidad de la IA

Normativa (texto consolidado):

- RGPD · Reglamento (UE) 2016/679
- AI Act · Reglamento (UE) 2024/1689
- LOPDGDD · Ley Orgánica 3/2018
- Ley 15/2022, de igualdad de trato y no discriminación
- Comisión Europea · marco regulador de la IA y calendario

Guías y marcos éticos:

- AEPD · Guía de privacidad desde el diseño
- UNESCO · Recomendación sobre la ética de la IA (2021)
- ACM Code of Ethics
- IEEE · Ethically Aligned Design

Casos y estudios citados:

- ProPublica · *Machine Bias* (COMPAS)
- Gender Shades (MIT Media Lab)
- Obermeyer et al. · *Dissecting racial bias in an algorithm…* (Science, 2019)
- Frey y Osborne · *The Future of Employment*
- Human Rights Watch · armas autónomas y el plazo de 2026
- UNODA · Grupo de Expertos Gubernamentales sobre LAWS

Herramientas:

- Fairlearn · AI Fairness 360 (IBM)

Bibliografía de referencia: Russell, S. y Norvig, P., *Artificial Intelligence: A Modern Approach*, 4.ª ed., Pearson, 2021 — capítulo 27, «Philosophy, Ethics, and Safety of AI», del que provienen los bloques de armas autónomas, vigilancia y reidentificación, equidad y parcialidad, confianza y transparencia, futuro del trabajo, derechos de los robots y seguridad de la IA. Los datos numéricos de esos bloques se han actualizado y verificado para este curso; cuando no ha sido posible, se indica el año del dato.

1 de septiembre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



https://martinezpenya.es/ModelosIA/
David Martinez Peña
© 2025 David Martínez licensed under CC BY-NC-SA 4.0

© 2025 David Martínez licensed under CC BY-NC-SA 4.0